Statistical Machine Learning

Lecture 1 – Introduction and math preliminaries



UPPSALA
UNIVERSITET

Jens Sjölund
https://jsjol.github.io/
Department of Information Technology
Uppsala University

Course webpage



UPPSALA
UNIVERSITET

What is the course about?

Machine learning

"Machine learning is about learning, reasoning and acting based on data."

Three cornerstones:

1. Data.
2. Mathematical model.
3. Learning algorithm.

Focus of this course

Supervised machine learning

Methods for learning a predictive **model** $f(\mathbf{x}) = y$ of the relationship between

- features of possible relevance $\mathbf{x}$ (**input**)
- outcome of interest y (**output**)

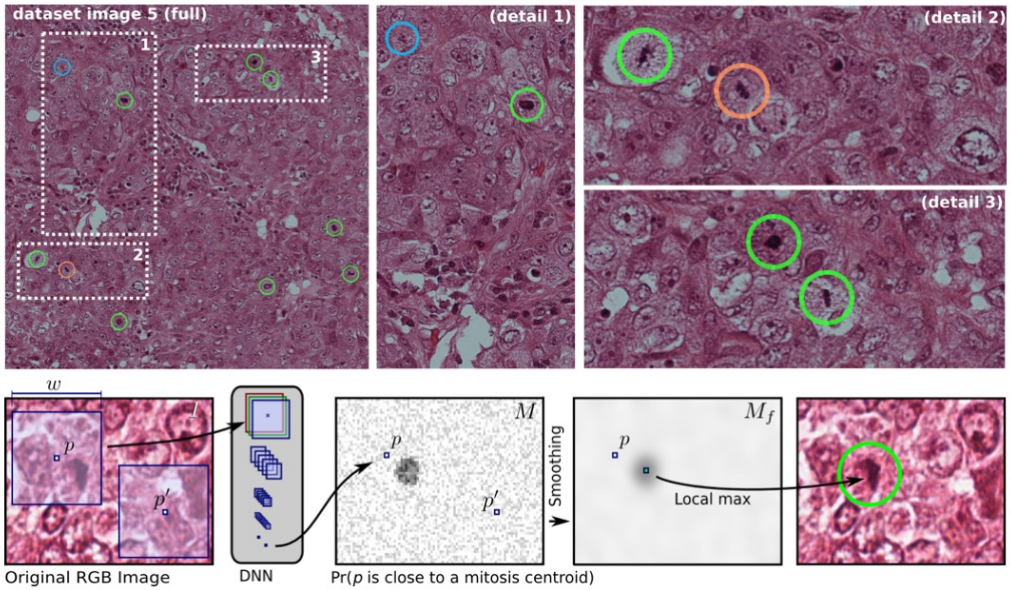
from observed **training data**

$$\mathcal{T} \stackrel{\text{def}}{=} \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_n, y_n)\}.$$

UPPSALA UNIVERSITET

ex) Cancer diagnosis

Systems for detecting cell divisions (mitosis) in histology images can be used to improve (or automate) cancer diagnosis.



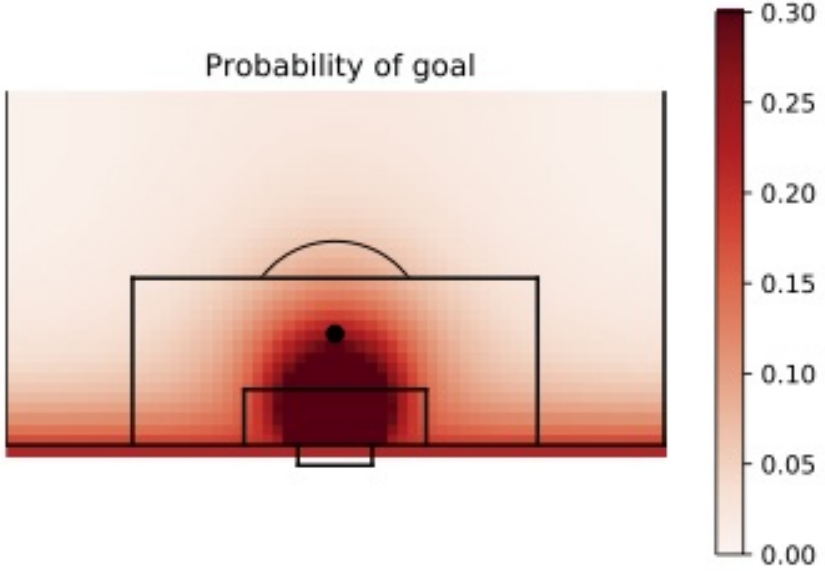
- Learn a model with
  input $\mathbf{x}$ = RGB histology image (pixel values)
  output y = number and locations (in the image) of mitosis detections
- Training data: Histology images labeled by experts.
- Uses deep neural networks to model f (Lectures 7 and 8).

D. C. Cireşan, A. Giusti, L. M. Gambardella and J. Schmidhuber. **Mitosis Detection in Breast Cancer Histology Images with Deep Neural Networks**. In *Medical Image Computing and Computer Assisted Intervention*, 411-418, 2013.

ex) Goals in football

- Predicting the probability a shot is a goal
- Learn a model with

  input $\mathbf{x} =$ location, header/foot, left/right foot, from cross/pass ... ...
  output $y =$ goal
- Logistic regression (Lecture 3) and Random forest models (Lecture 6).



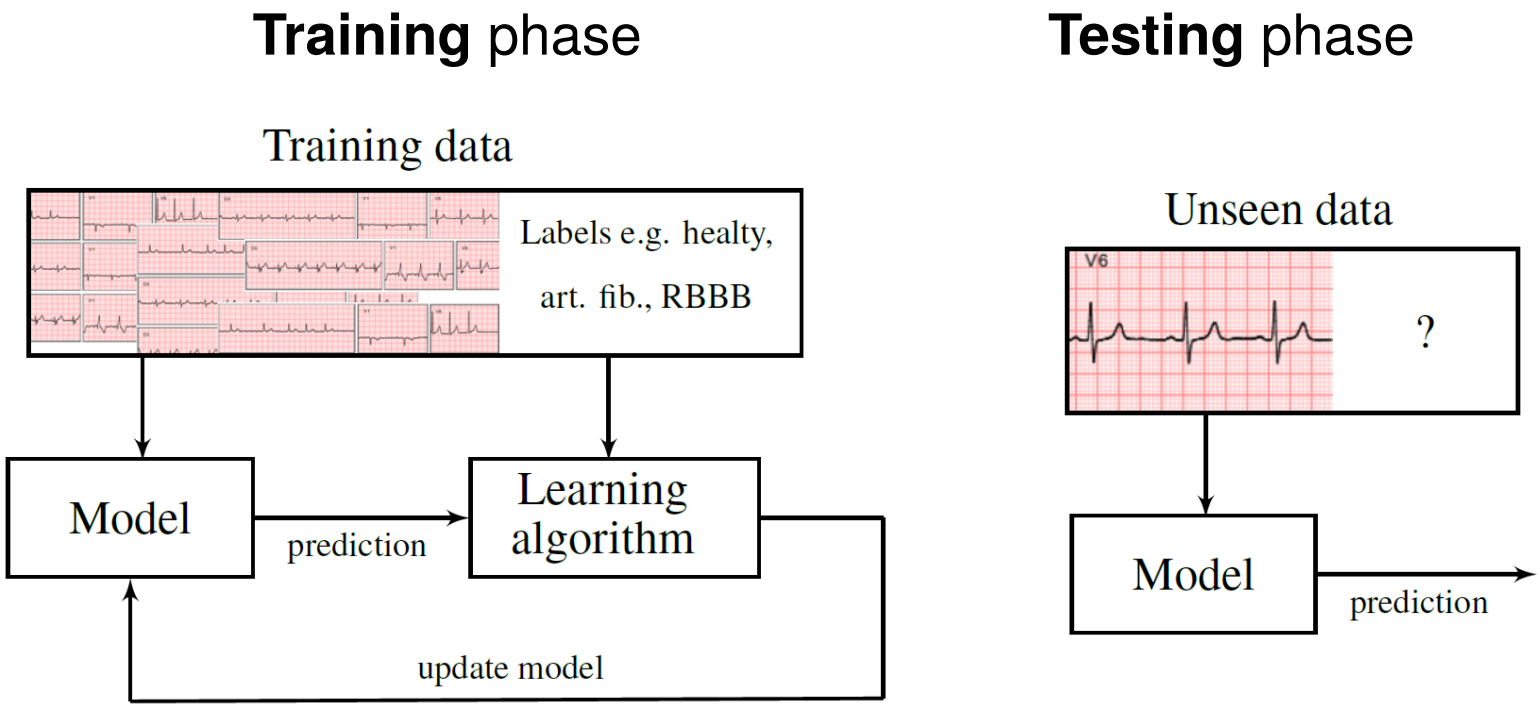
Statistical machine learning

Why the word "statistical" in the course title?

- Probability theory is used to define the models.
- Statistical tools are used to learn the models from training data.

Allows us to reason about the ***uncertainties*** in the data, models, predictions, etc.!

Training and testing



Key assumption: The unseen testing data comes from the same probability distribution as the training data.

Quantitative and qualitative variables

Both input variables ($\mathbf{x}$) and output variables (y) can be either **quantitative** or **qualitative**.

- **Quantitative** variables take on numerical values (real numbers, integer values, . . .).
- **Qualitative** variables take on values in one of K distinct classes, e.g. “true or false”, “disease type A, B or C”.

We will mostly use integer coding (like $\{1, 2, 3, 4\}$ or $\{0, 1\}$), but the coding (or labeling) of qualitative variables is arbitrary and unordered.

Regression vs. classification

We will distinguish between two types of problems:
regression and **classification**

Regression is when the output y is quantitative, e.g.

- Climate models ($y =$ "increase in global temperature")
- Socio-economic models ($y =$ "change in GDP", "happiness")
- Social media engagement ($y =$ "watch time of Youtube videos")

Classification is when the output y is qualitative, e.g.

- Spam filters ($y \in \{\texttt{spam}, \texttt{proper email}\}$).
- Face recognition, e.g. on your phone ($y \in \{\texttt{match}, \texttt{no match}\}$).

Models

Different machine learning methods use different **models** for the function f (even if the task at hand may be the same).

"All models are wrong, but some are useful" — *George Box*

Some methods use more ***flexible models*** than others, capable of capturing complex dependencies between inputs and outputs.

Why use a more restrictive model instead of a flexible one?

- Flexible models tend to be harder to **interpret**.
- Flexible models tend to be less **robust**.



UPPSALA
UNIVERSITET

Course information

Course elements

- 10 lectures
- 10 exercise sessions
- 1 project (3-4 students, written report)
- 1 computer lab (4h, no report)
- Everything distributed via the course webpage

Lecture outline

1. Introduction
- Introduction to statistical thinking
- Introduction to Python
2. Basic regression, linear regression
3. Classification, logistic regression
4. Classification, LDA, QDA, k-NN
5. Generalization performance
6. Tree-based methods, bagging, boosting
7. Deep neural networks
8. Model learning as numerical optimization
9. Limitations and new directions for supervised learning
10. Summary and outlook

You can post questions that you want us to discuss in the lectures in our Padlet.

Exercise sessions

10 exercise sessions in total:

- Solve problems before the session
- Ask questions in the discussion forum. We will check them every day.
- 5 pen-and-paper sessions.
- 5 computer-based sessions (using Python).
- Feel free to use your own laptop – Python is freely available.

Each session 3 times, 2 in parallel.

A great opportunity to discuss and ask questions!

Course webpage

- All information should be available on the course webpage.
- We are 300 people trying learn about a subject. Let's help each other as much as possible!
- If you have a question that someone else might also have, please **post it in the discussion forum**.
- If you absolutely have to contact us directly, then please contact **only one of the staff** via Studium. However, if we consider the question of potential interest to others, we will restate and answer it in the discussion forum.
- Please use the helpdesks to get help with the project.

Examination

Project:

- Solved in groups of 3 or 4 students (sign-up opens soon)
- Written report (deadline: see home page)
- Peer-review: read and review another group's report (anonymously)
- Material most relevant for the project presented in lectures 3–6, **but you can start working on the solution after lecture 3**
- We offer **helpdesks** if you have questions about the project.
- Graded U/G/VG. Grade VG is for projects of notable quality.

Lab:

- 4 h computer lab, solved in groups of 2 students, graded U/G
- There will be an announcement when you can sign up
- **The preparatory exercises need to be submitted before the first lab session!**

Written exam:

- Written pen-and-paper exam, graded as U, 3, 4, or 5.
- Old exams are available on the course home page

UPPSALA UNIVERSITET

Examination

Grading:

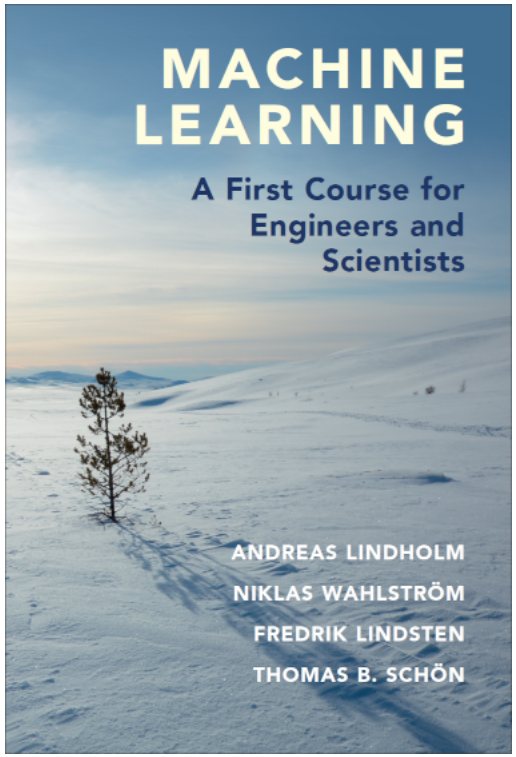
- To pass the course you need to pass the project, lab and exam.
- Grade VG on the project will increase your course grade by one step, but not from U to 3.

		Exam grade			
		U	3	4	5
Grade on project	U	U	U	U	U
	G	U	3	4	5
	VG	U	4	5	5
		Course grade			

Secret to passing: Revise foundations and get your hands dirty with problem-solving!

UPPSALA UNIVERSITET

Course literature



Available in print or for free via **smlbook.org**.
Possible to give feedback at book Github page.

Mathematical preliminaries

Discrete random variables

A **random variable** X can be either discrete or continuous.

- For a **discrete** random variable we write $p(x \mid \theta)$ for the probability of $X = x$ given the parameter(s) θ.
- Here, $p(x \mid \theta)$ is also called the probability mass function. It is nonnegative and sums to one, hence $0 \leq p(x \mid \theta) \leq 1$.

Example The outcome of a coin flip can be described by a random variable ($X = 1$ if heads, $X = 0$ if tails) that follows the Bernoulli distribution

$$p(x|\theta) = \begin{cases} \theta & , \quad \text{if } x = 1 \\ 1 - \theta & , \quad \text{if } x = 0 \end{cases}$$

If it's a fair coin, then $\theta = 0.5$.

Continuous random variables

A **random variable** X can be either discrete or continuous.

- For a **continuous** random variable, we write $p(x \mid \theta)$ for the probability density function (PDF).
- The PDF is nonnegative and **integrates** to unity, hence it is not bounded above in general.
- The probability is the area under the PDF, i.e. we have to integrate to get a **probability**, e.g.

$$P(X \leq x|\theta) = \int_{-\infty}^{x} p(z|\theta)dz.$$

Continuous random variables

Example If X is uniformly distributed on the interval $[1.1, 1.3]$, then

$$p(x) = \begin{cases} 5, & \text{if } x \in [1.1, 1.3] \\ 0, & \text{otherwise.} \end{cases}$$

The probability that $X \in [1.15, 1.25]$ is given by

$$P(X \in [1.15, 1.25]) = \int_{1.15}^{1.25} p(x)\, dx = 5 \cdot 0.1 = 0.5.$$

Mean and Variance

- If X is a random variable with PDF $p(x)$, then the **expected value** or **mean** of X is given by

$$\mu_X = \mathbb{E}[X] = \int x\, p(x) \mathrm{d}x.$$

- The **variance** of X is defined as

$$\mathrm{Var}[X] = \mathbb{E}[(x-\mu)^2] = \mathbb{E}[X^2] - \mathbb{E}[X]^2.$$

- The **standard deviation** of X is the square root of the variance

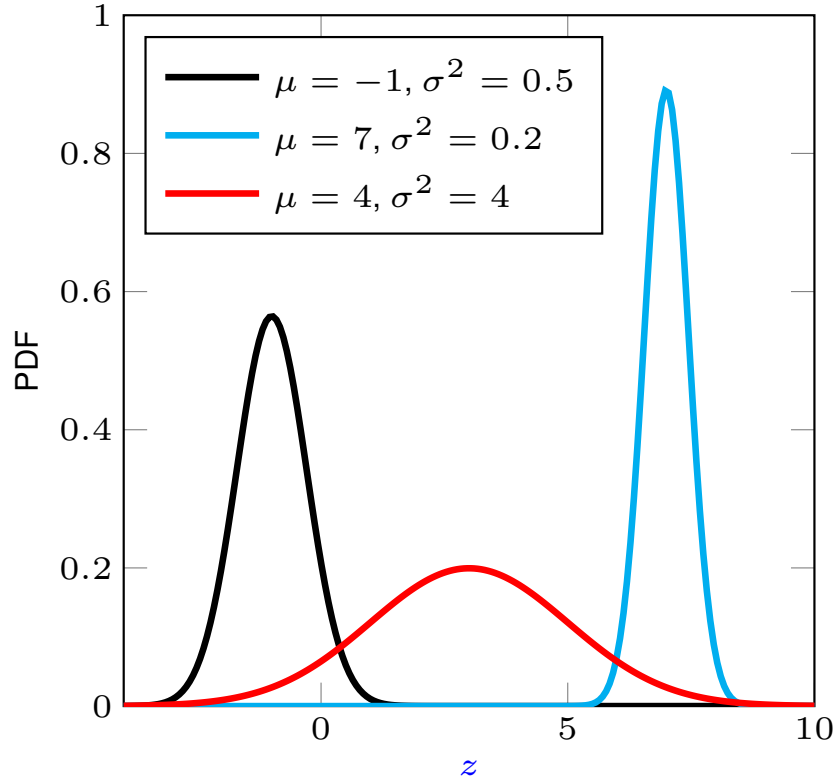
$$\sigma_X = \sqrt{V[X]}.$$

  Conversely, the variance $V[X]$ is often written as σ_X^2.

Normal (Gaussian) distribution

Probability density function for the scalar Gaussian distribution

$$p(x|\mu,\sigma^2) = \mathcal{N}(x \,|\, \mu, \sigma^2)$$
$$= \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$



- μ is the mean (expected value of the distribution)
- σ is the standard deviation
- σ^2 is the variance

$X \sim \mathcal{N}(\mu, \sigma^2)$ means that X is a Gaussian random variable with mean μ and variance σ^2. $\sim$ reads “distributed according to”.

Independent random variables

Example Assume we roll two dice and denote their outcomes X_1 and X_2. This can be represented as a multivariate (bivariate) random variable $X = (X_1, X_2)$ with a corresponding **joint distribution** $p(x_1, x_2)$.
Since the die rolls are iid, $p(x_1, x_2) = p(x_1)p(x_2) = \frac{1}{36}$.

In general, if X_1 and X_2 are **independent** random variables, then the joint distribution factorizes $p(x_1, x_2) = p(x_1)p(x_2)$ and

$$\mathbb{E}[X_1 X_2] = \mathbb{E}[X_1]\mathbb{E}[X_2].$$

In this course, we will often talk about variables that are independent and identically distributed (**iid**).

Covariance and correlation

- The **covariance** between two random variables X and Y is defined as

$$\begin{aligned}\mathrm{Cov}(X, Y) &= \mathbb{E}[(X - \mu_X)(Y - \mu_Y)^T] \\ &= \mathbb{E}[XY^T] - \mu_X \mu_Y^T.\end{aligned}$$

  If X and Y are vectors, then $\mathrm{Cov}(X, Y)$ is a matrix.

- The **correlation** between two random variables X and Y is defined as

$$\rho_{X,Y} = \frac{\mathrm{Cov}(X, Y)}{\sigma_X \sigma_Y}.$$

- Independent variables are uncorrelated, i.e. have zero correlation, but the converse does not hold in general.

Covariance and correlation

Example Assume we roll two dice and denote their outcomes X_1 and X_2, and their sum $Y = X_1 + X_2$. What is the covariance between X_1 and Y?

$$\mathbb{E}[X_1] = \mathbb{E}[X_2] = \sum_{k=1}^{6} k\, p(k) = \frac{1+2+3+4+5+6}{6} = 3.5$$

$$\mathbb{E}[Y] = \mathbb{E}[X_1] + \mathbb{E}[X_2] = 7 \quad \text{(linearity of expectations)}$$

$$\mathbb{E}[X_1 Y] = \mathbb{E}[X_1(X_1 + X_2)] = E[X_1^2] + \mathbb{E}[X_1]\mathbb{E}[X_2]$$

$$= \frac{1^2 + 2^2 + 3^2 + 4^2 + 5^2 + 6^2}{6} + 3.5^2 \approx 27.4$$

$$\text{Cov}(X_1, Y) = \mathbb{E}[X_1 Y] - \mathbb{E}[X_1]\mathbb{E}[Y] = 27.4 - 3.5 \cdot 7 = 2.9$$

Since $\text{Cov}(X_1, Y) \neq 0$, X_1 and Y are **not** independent.

Conditional distribution

The **conditional distribution** $p(x \mid y)$ is the probability distribution of X given that $Y = y$.

Example Assume we roll two dice and denote their outcomes X_1 and X_2, and their sum $Y = X_1 + X_2$. What is the conditional distribution of Y given that $X_1 = 3$?

Given that $X_1 = 3$, the sum $Y = 3 + X_2$. Hence,

$$p(y \mid X_1 = 3) = \begin{cases} 1/6, & \text{if } y \in \{4, \ldots, 9\} \\ 0, & \text{otherwise.} \end{cases}$$

Marginal distribution

The **marginal distribution** of a subset of random variables is the probability distribution when disregarding the other variables.

Example Assume we roll two dice and denote their outcomes X_1 and X_2, and their sum $Y = X_1 + X_2$. What is the marginal distribution of Y?

	1	2	3	4	5	6
1	2	3	4	5	6	7
2	3	4	5	6	7	8
3	4	5	6	7	8	9
4	5	6	7	8	9	10
5	6	7	8	9	10	11
6	7	8	9	10	11	12

$$p(y) = \begin{cases} 1/36, & y \in \{2, 12\} \\ 2/36, & y \in \{3, 11\} \\ 3/36, & y \in \{4, 10\} \\ 4/36, & y \in \{5, 9\} \\ 5/36, & y \in \{6, 8\} \\ 6/36, & y \in \{7\} \\ 0, & \text{otherwise.} \end{cases}$$

A few concepts to summarize lecture 1

Machine Learning: Deals with learning, reasoning and acting based on data.

Regression: Learning problem where the *output* is quantitative.

Classification: Learning problem where the *output* is qualitative.

Training and testing: We learn a predictive model on the training data and want to use on the test data.

Probability and statistics: The basis for most of machine learning.

Statistical Machine Learning

Lecture 2 – Linear regression, regularization



UPPSALA
UNIVERSITET

Sebastian Mair
https://smair.github.io/
Department of Information Technology
Uppsala University

Course webpage

Summary of Lecture 1 (I/II)

What is this course about? Supervised machine learning

In one sentence:

Methods for automatically learning (training, estimating, . . .)
a model for the relationship between

- the **input** $\mathbf{x}$, and
- the **output** y

from observed **training data**

$$\mathcal{T} := \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_n, y_n)\}.$$

Summary of Lecture 1 (II/II)

Regression vs. classification

- **Numerical** variables take on numerical values (real numbers, integer values, . . .).
- **Categorical** variables take on values in one of K distinct classes, e.g. “true or false”, “disease type A, B or C”.

Regression is when the output y is numerical.

Classification is when the output y is categorical.

Summary of Lecture 1 (II/II)

What maths do we need.

- **Calculus** Finding a parameter which minimizes the distance between two points.
- **Matrix algebra** For keeping track of sum of squares.
- **Probability theory** Estimating parameters. The normal distribution important.

One key idea that brings these together is maximum likelihood. Finding the model that is maximally likely (closest to data).

Outline – Lecture 2

Aim: To introduce linear regression and its regularized version.

Outline:

1. Summary of Lecture 1
2. Linear regression models
3. Maximum likelihood and least squares
4. Regularization
   - Ridge regression
   - LASSO

Linear regression is the foundation of statistics and (supervised) machine learning.

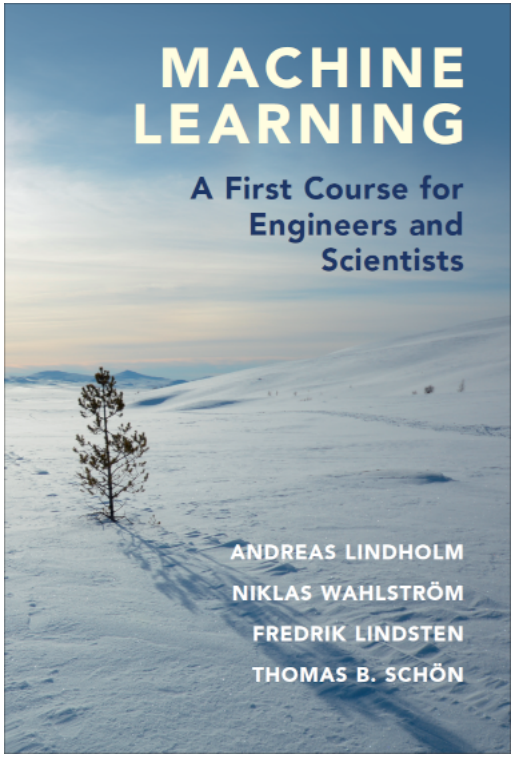
The course book

This course was developed over the last years.

As a result, a book based on these notes was published as a textbook with Cambridge University Press.

Book website: `smlbook.org`

Feedback can be provided via the Github page.



Please read as part of your studies.

Regression



- **Input variable** $\mathbf{x}$
- **Output variable** y

Regression: Learning a model explaining y from $\mathbf{x}$, when y is numerical, i.e.,

$$y = f(\mathbf{x}; \beta) + \epsilon.$$

Here, β are the **parameters** of the model.

(y categorical $\rightarrow$ classification)

UPPSALA UNIVERSITET

Numerical or categorical variables

Numerical or categorical?

17.31 kg, 22.37 kg, 51.34 kg
1 = brown hair, 2 = red hair, 3 = blonde hair
Adenine, Thymine, Cytosine, Guanine
1 bike, 2 bikes, 5 bikes

UPPSALA UNIVERSITET

Numerical or categorical variables

Numerical or categorical?

17.31 kg, 22.37 kg, 51.34 kg	Numerical
1 = brown hair, 2 = red hair, 3 = blonde hair	
Adenine, Thymine, Cytosine, Guanine	
1 bike, 2 bikes, 5 bikes	

UPPSALA UNIVERSITET

Numerical or categorical variables

Numerical or categorical?

17.31 kg, 22.37 kg, 51.34 kg	Numerical
1 = brown hair, 2 = red hair, 3 = blonde hair	Categorical
Adenine, Thymine, Cytosine, Guanine	
1 bike, 2 bikes, 5 bikes	

Numerical or categorical variables

Numerical or categorical?

17.31 kg, 22.37 kg, 51.34 kg	Numerical
1 = brown hair, 2 = red hair, 3 = blonde hair	Categorical
Adenine, Thymine, Cytosine, Guanine	Categorical
1 bike, 2 bikes, 5 bikes	

Numerical or categorical variables

Numerical or categorical?

17.31 kg, 22.37 kg, 51.34 kg	Numerical
1 = brown hair, 2 = red hair, 3 = blonde hair	Categorical
Adenine, Thymine, Cytosine, Guanine	Categorical
1 bike, 2 bikes, 5 bikes	Numerical

Numerical or categorical variables

Numerical or categorical?

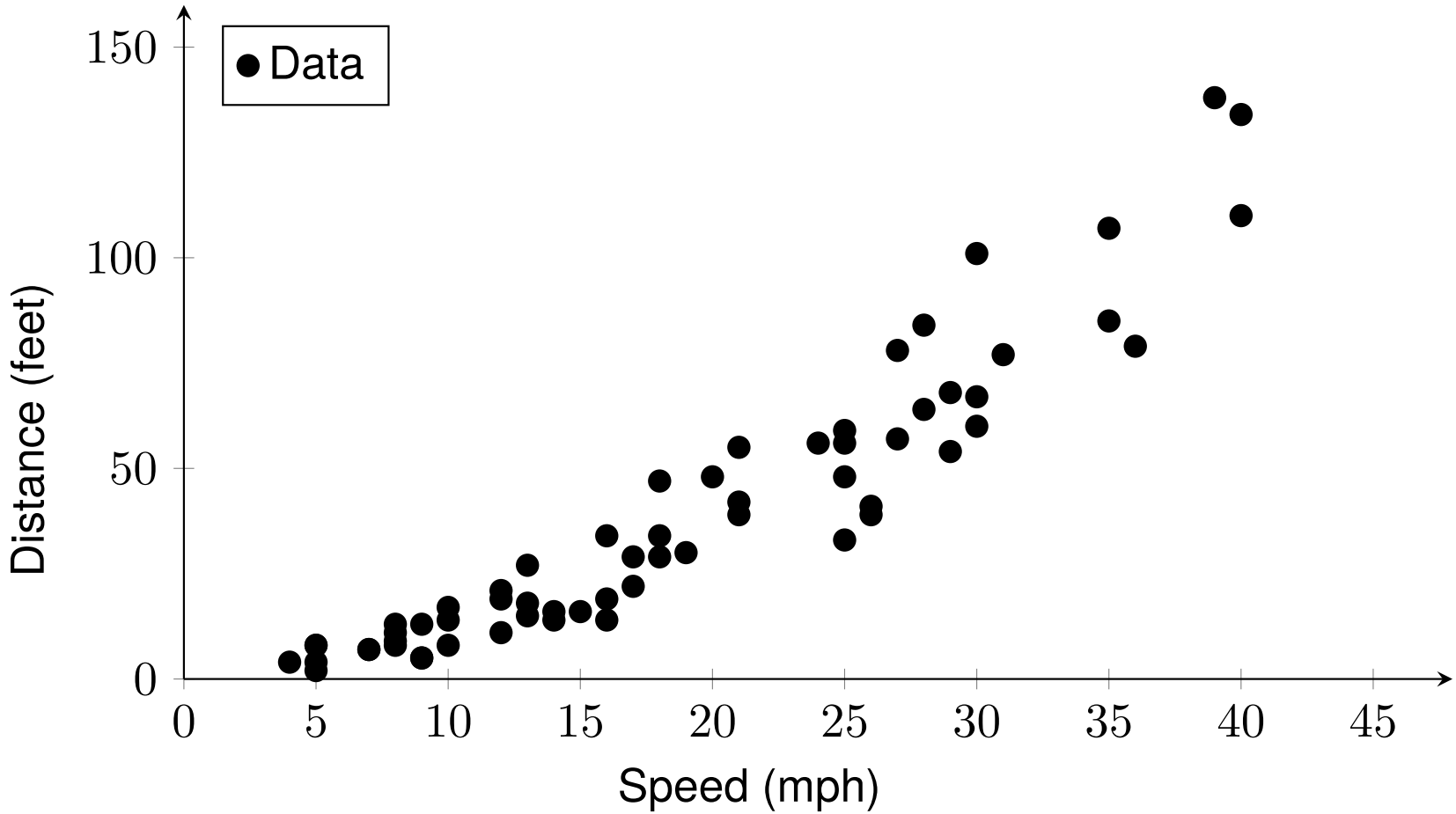
17.31 kg, 22.37 kg, 51.34 kg	Numerical
1 = brown hair, 2 = red hair, 3 = blonde hair	Categorical
Adenine, Thymine, Cytosine, Guanine	Categorical
1 bike, 2 bikes, 5 bikes	Numerical

Categorical output variable? $\rightarrow$ classification. (But can be done with regression!)

Categorical input variable? Still regression!

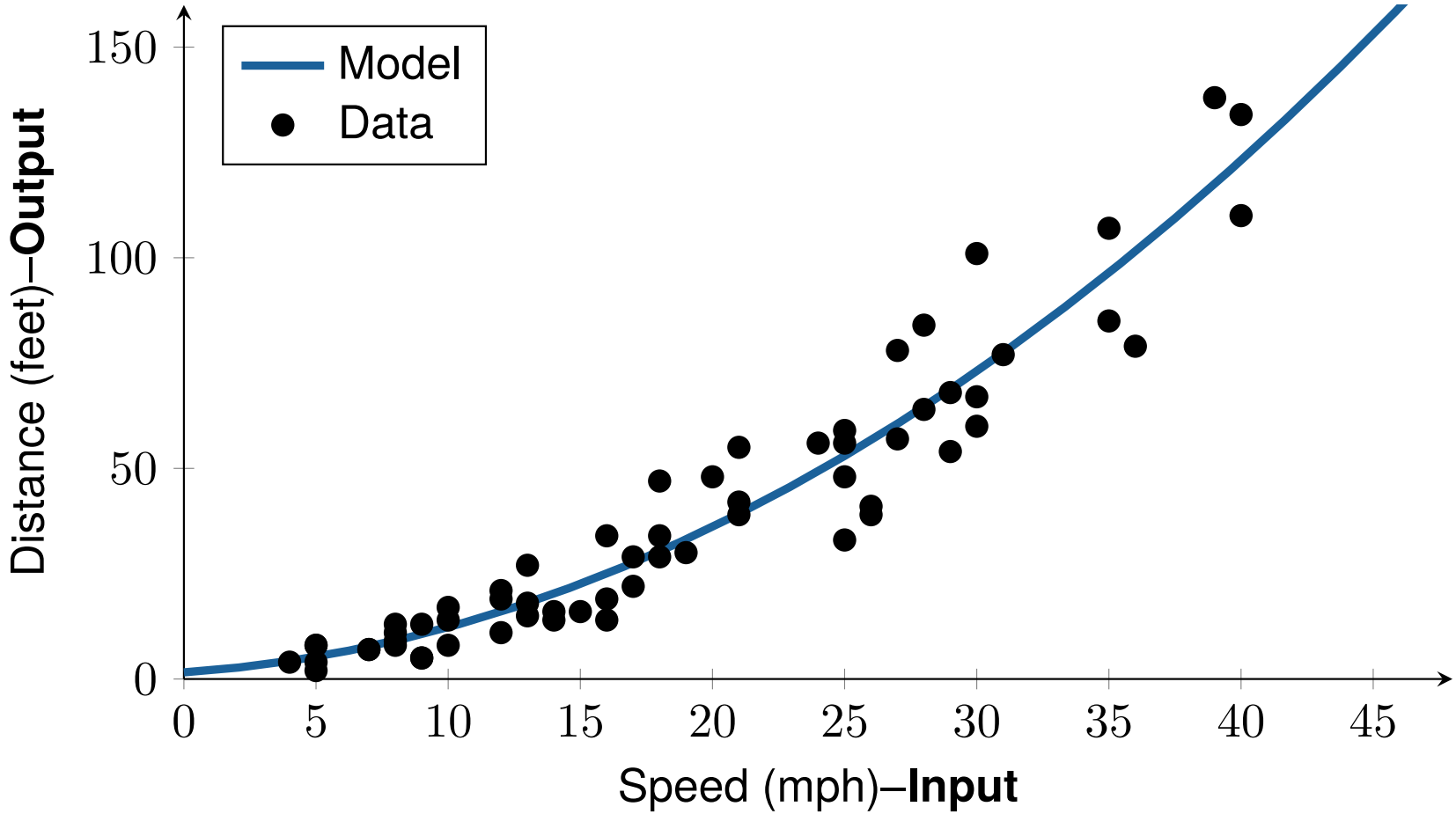
UPPSALA UNIVERSITET

Regression example: car stopping distances



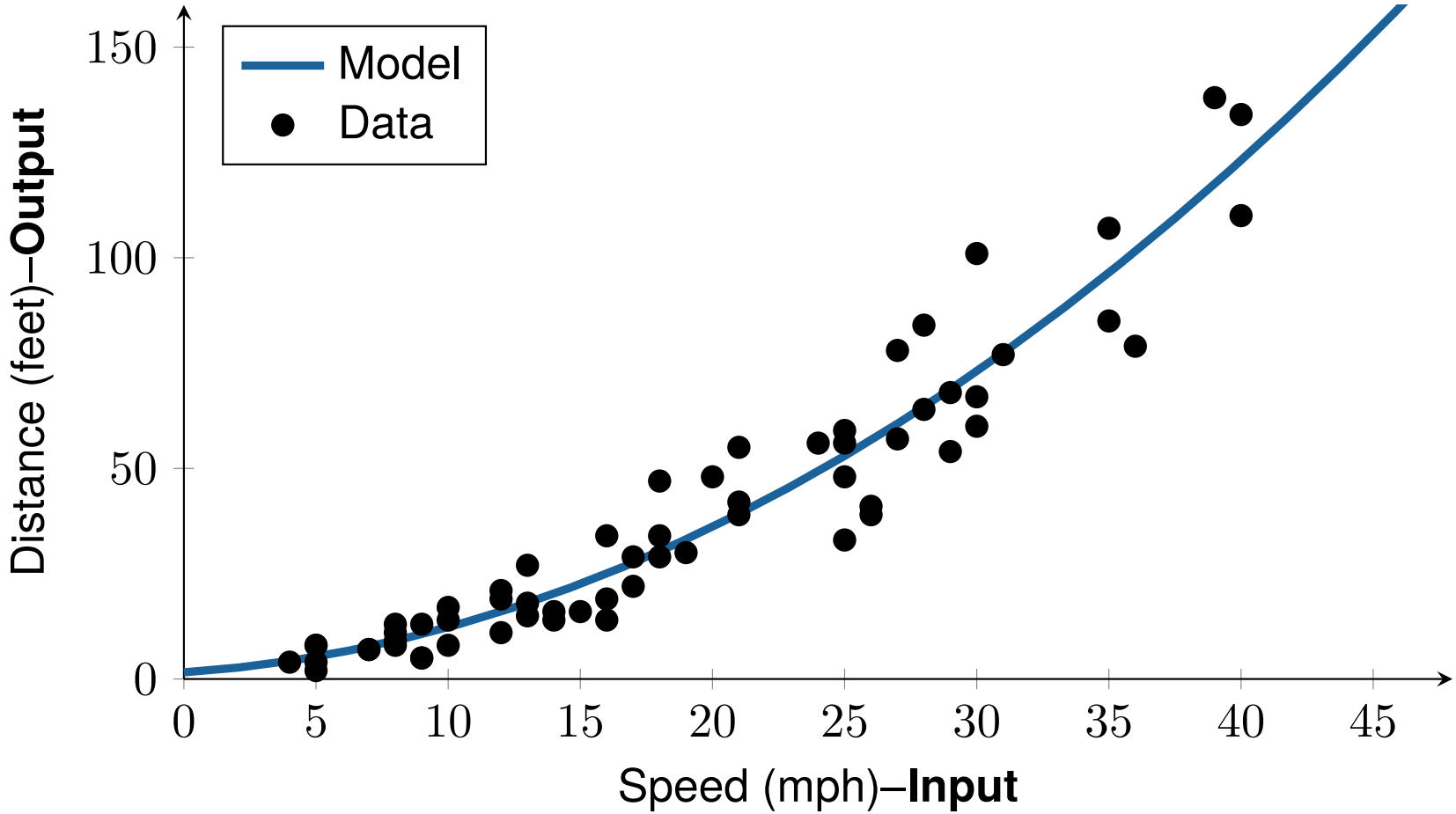
UPPSALA UNIVERSITET

Regression example: car stopping distances



UPPSALA UNIVERSITET

Regression example: car stopping distances



(in fact a linear regression model with nonlinear transformation of the input variables)

Regression example: Alpha Go zero



Regression example: Alpha Go zero





- Input: State of the game (19×19 grid, either black, white or blank)
- Output: Probability for the current player to win the game
- \+ *reinforcement learning*

Silver et al. **Mastering the game of Go with deep neural networks and tree search**, *Nature* 529, 484–489, 2016.

Regression example: Alpha Go zero



- Input: State of the game (19×19 grid, either black, white or blank)
- Output: Probability for the current player to win the game
- \+ *reinforcement learning*

Silver et al. **Mastering the game of Go with deep neural networks and tree search**, *Nature* 529, 484–489, 2016.



- Input: Same
- Output: Probability for the current player to win the game *and* what move to make

Silver et al. **Mastering the game of Go without human knowledge**, *Nature* 550, 354–359, 2017.

Silver et al. **A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play**, *Science*, 362(6419): 1140–1144, 2018.

Linear regression



"Linear regression = Regression with a linear model",

Output y is linear combination of k inputs $x_1, \ldots, x_k$ plus some noise/error ϵ,

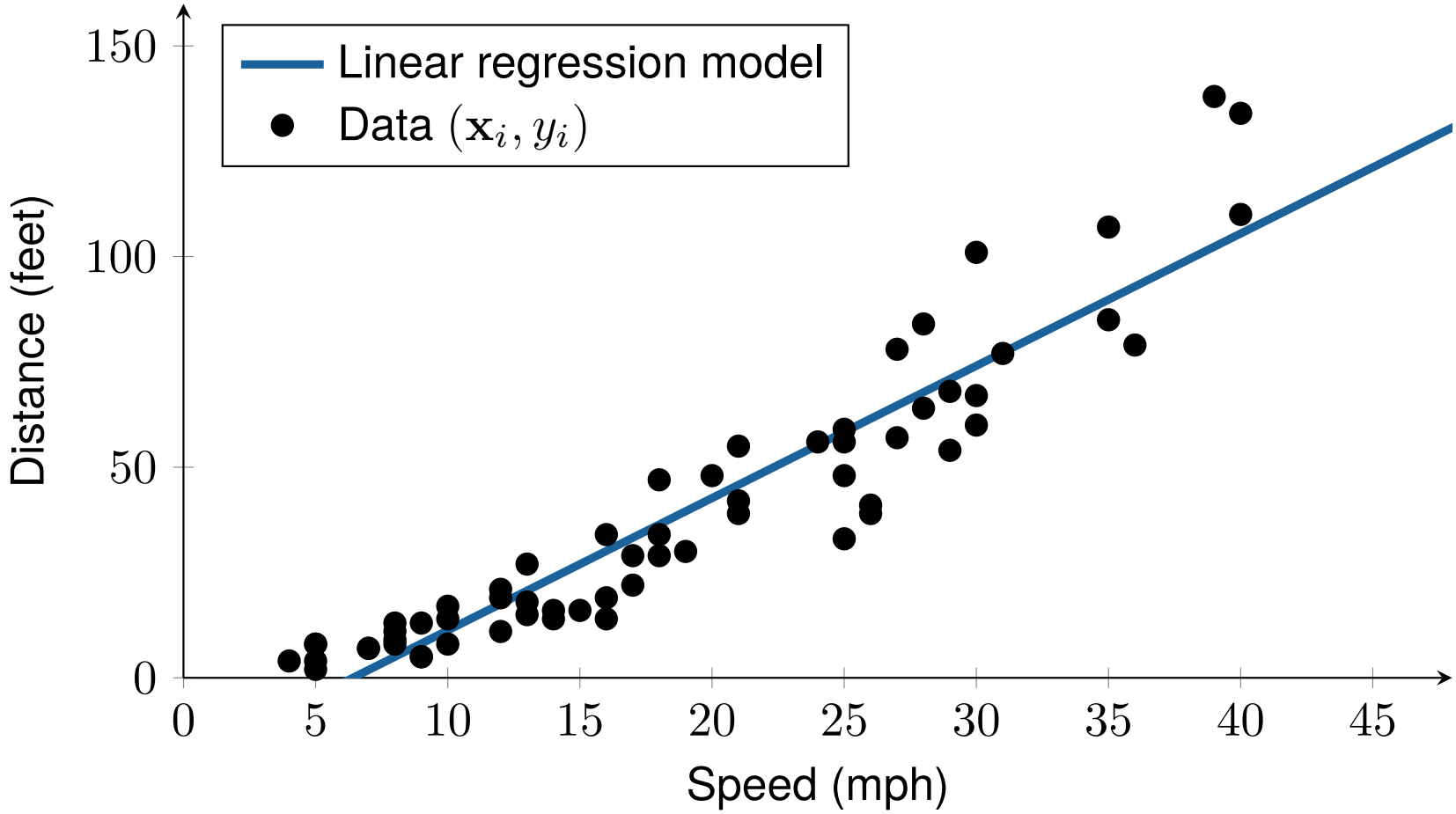
$$y = \underbrace{\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_k x_k}_{f(\mathbf{x};\beta)} + \epsilon.$$

Workflow (for most methods, not only linear regression):

1. Learn/train/estimate model from training data $\mathcal{T}$:
   find $\widehat{\beta}_0, \widehat{\beta}_1, \ldots, \widehat{\beta}_k$
2. Predict output for new test input $\mathbf{x}_\star$ using the model
   $\widehat{y} = \widehat{\beta}_0 + \widehat{\beta}_1 x_{\star 1} + \widehat{\beta}_2 x_{\star 2} + \cdots + \widehat{\beta}_k x_{\star k}$

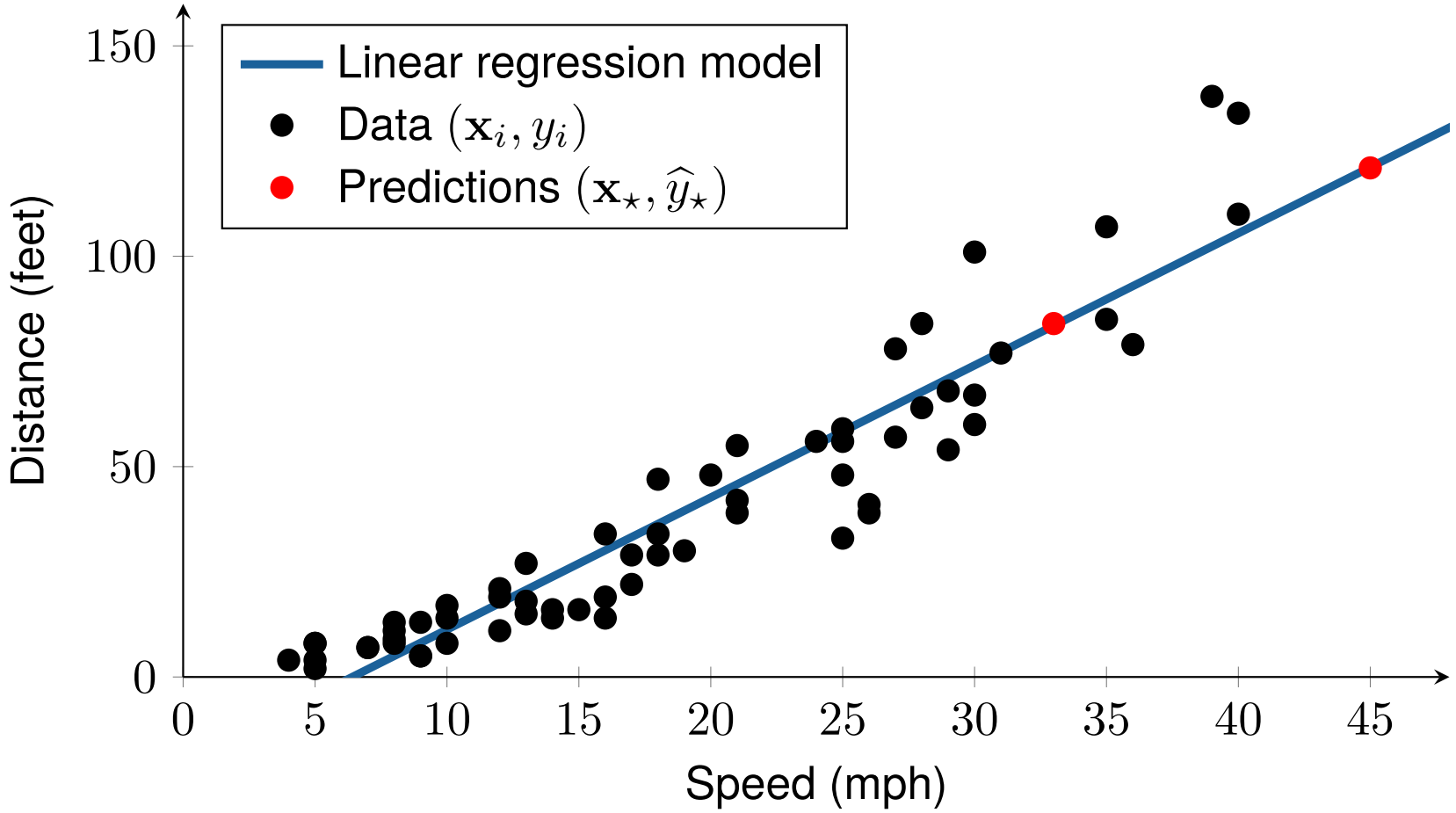
UPPSALA UNIVERSITET

Linear regression ($k = 1$)



UPPSALA UNIVERSITET

Linear regression ($k = 1$)



Learning the model from data



Linear regression model:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_k x_k + \epsilon$$

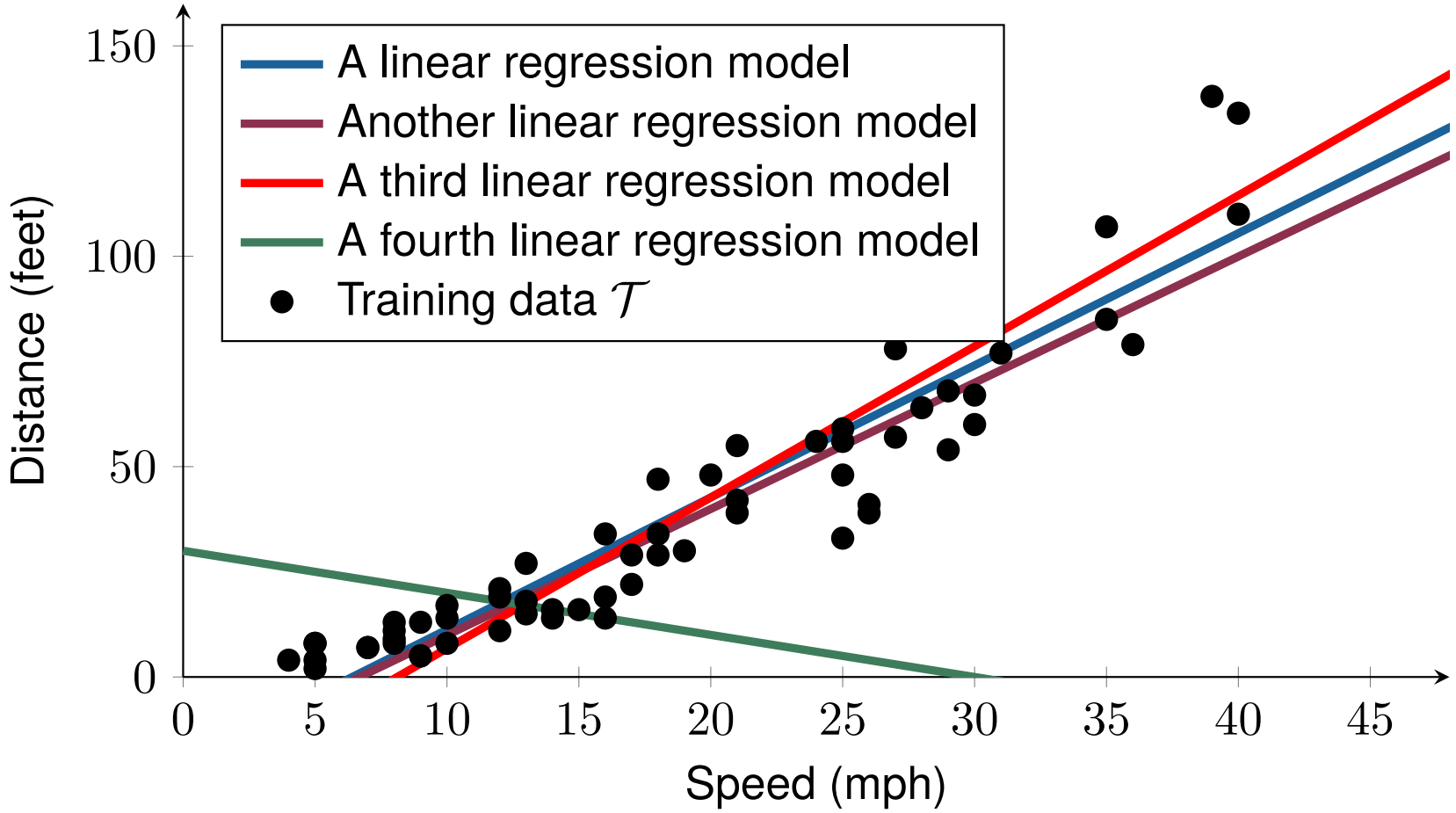
How to choose $\beta_0, \beta_1, \ldots, \beta_k$ (=β, column vector)?

Use **training data** $\mathcal{T} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$!

$$\mathbf{x}_i = \begin{bmatrix} x_{i1} \\ x_{i2} \\ \vdots \\ x_{ik} \end{bmatrix}, \quad \mathbf{X} = \begin{bmatrix} 1 & -\mathbf{x}_1^\mathsf{T} - \\ 1 & -\mathbf{x}_2^\mathsf{T} - \\ \vdots & \vdots \\ 1 & -\mathbf{x}_n^\mathsf{T} - \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}, \quad \boldsymbol{\epsilon} = \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{bmatrix}.$$

UPPSALA UNIVERSITET

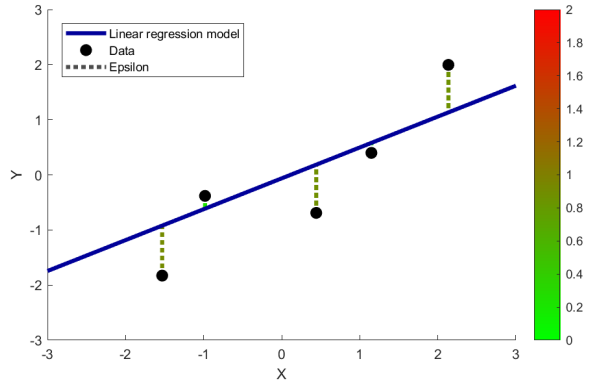
What is a good model?



UPPSALA UNIVERSITET

Learning using maximum likelihood

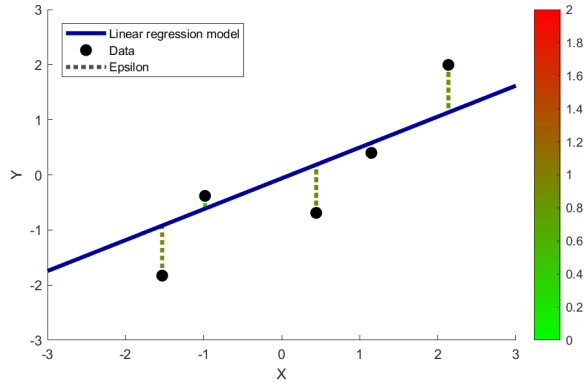
Learning a model from data is a matter of looking at the errors ϵ!

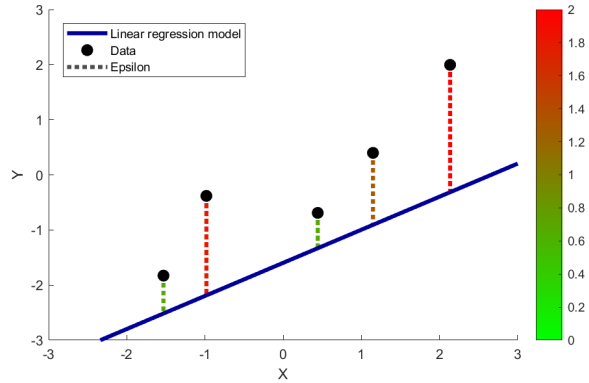


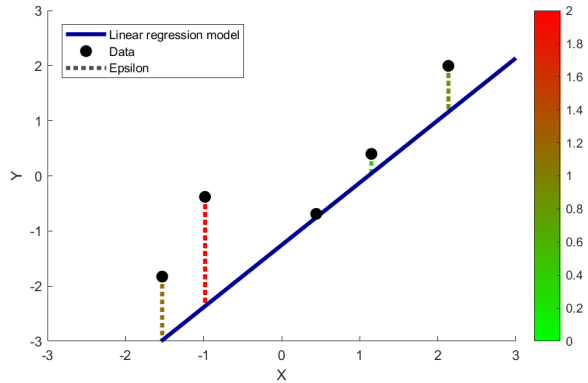
UPPSALA UNIVERSITET

Learning using maximum likelihood

Learning a model from data is a matter of looking at the errors ϵ!



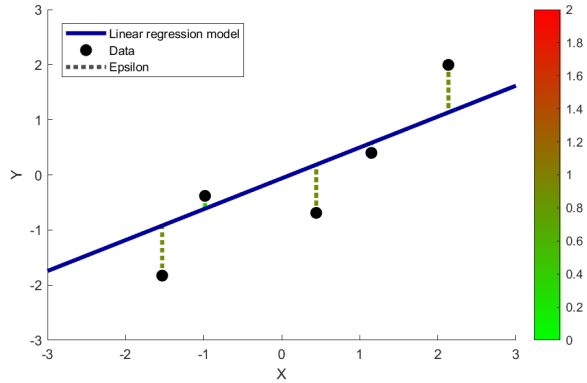


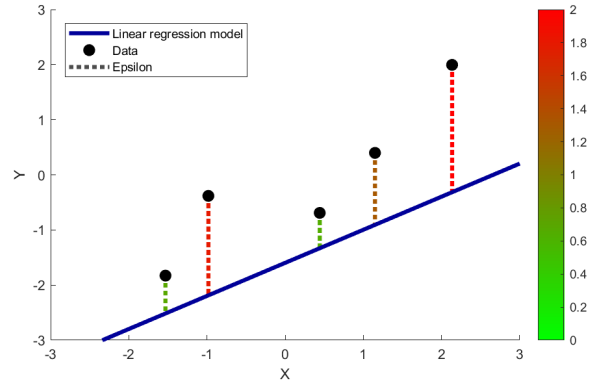


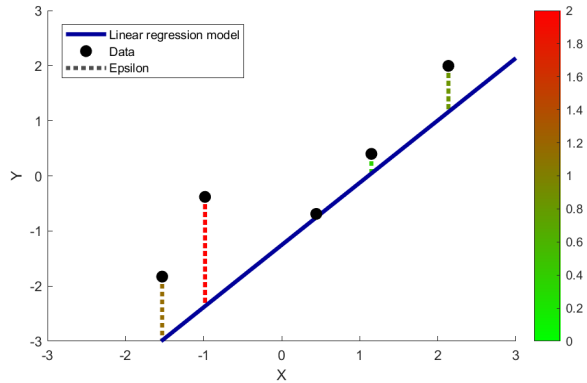
UPPSALA UNIVERSITET

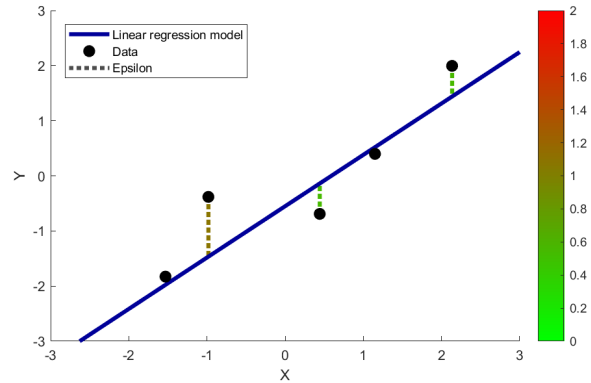
Learning using maximum likelihood

Learning a model from data is a matter of looking at the errors ϵ!





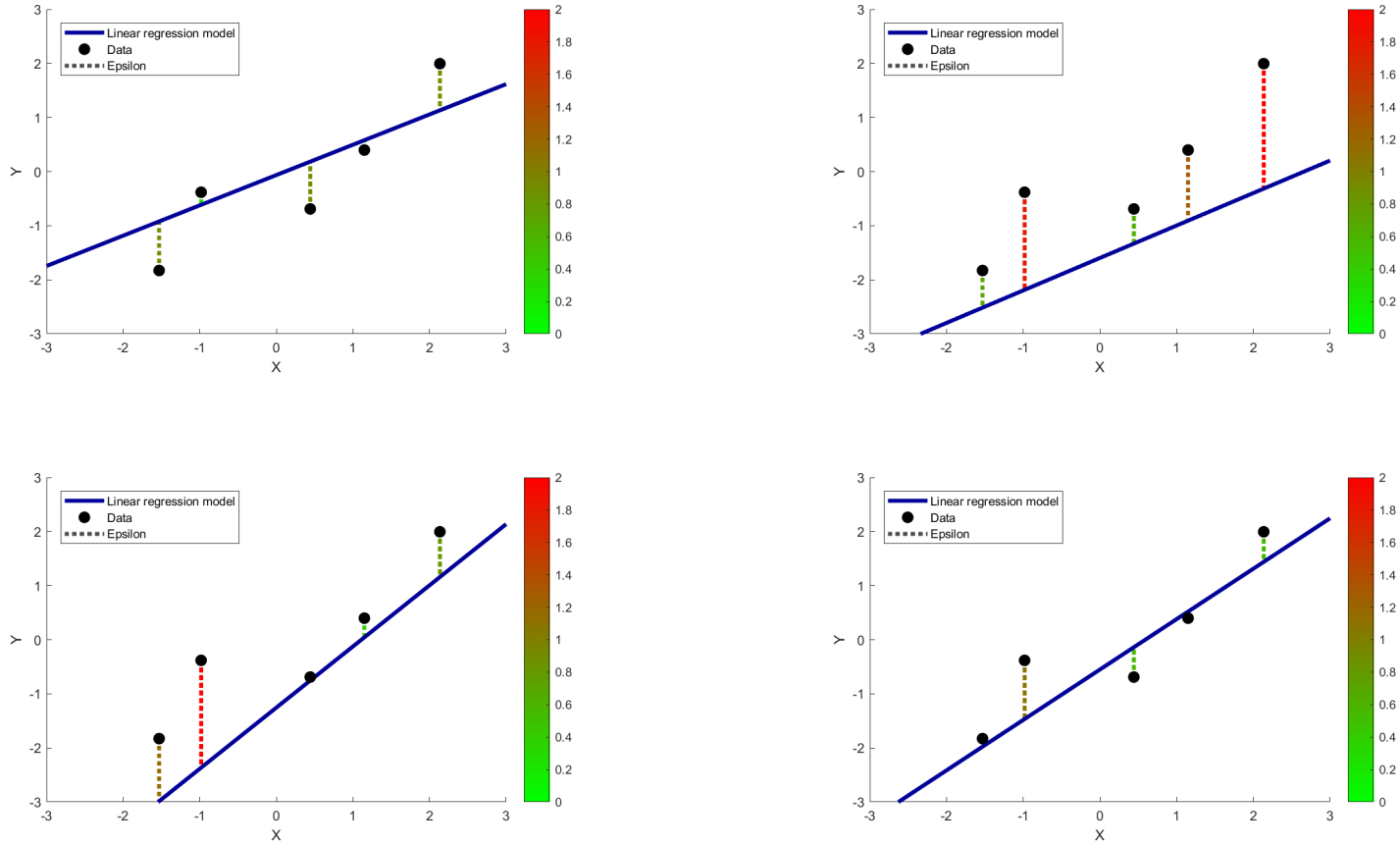




UPPSALA UNIVERSITET

Learning using maximum likelihood

Learning a model from data is a matter of looking at the errors ϵ!



Maximum likelihood: Think of ϵ (dotted) as random variables, and ***choose the model*** (solid) ***such that the resulting*** ϵ ***are as likely as possible***.

UPPSALA UNIVERSITET

Linear regression model in matrix form

Recall our linear regression model:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \sigma_\epsilon^2 I).$$

Assumptions (for now):

1. $\mathbf{y}$ – observed **random** variable.
2. $\boldsymbol{\beta}$ – unknown **deterministic** variable.
3. $\boldsymbol{X}$ – known **deterministic** variable.
4. $\boldsymbol{\epsilon}$ – unknown **random** variable.
5. σ_ϵ – unknown/known **deterministic** variable.

Learning using maximum likelihood



Using the **maximum likelihood principle**

$$\widehat{\beta} = \underset{\beta}{\text{argmax}}\, p(\mathbf{y} \mid \mathbf{X}; \beta)$$

and assuming $\epsilon \sim \mathcal{N}(0, \sigma_\epsilon^2)$ independently for each data point i

$$\Rightarrow p(y_i|\mathbf{x}_i; \beta) = \frac{1}{\sqrt{2\pi\sigma_\epsilon^2}} \exp\left(-\frac{1}{2\sigma_\epsilon^2}(\beta_0 + \beta_1 x_{i1} + \cdots + \beta_k x_{ik} - y_i)^2\right)$$

$$\Rightarrow p(\mathbf{y}|\mathbf{X}; \beta) = \prod_{i=1}^{n} p(y_i|x_i; \beta) \propto \exp\left(-\frac{1}{2\sigma_\epsilon^2}\sum_{i=1}^{n}(\beta_0 + \cdots + \beta_k x_{ik} - y_i)^2\right)$$

$$\Rightarrow \widehat{\beta} = \underset{\beta}{\text{argmin}} \sum_{i=1}^{n}(\beta_0 + \beta_1 x_{i1} + \cdots + \beta_k x_{ik} - y_i)^2 = \underset{\beta}{\text{argmin}} \underbrace{\|\mathbf{X}\beta - \mathbf{y}\|_2^2}_{\substack{\text{Loss function} \\ \text{induced by} \\ \text{maximum likelihood}}},$$

the **least squares** problem is achieved.

Least squares in matrix form

The least squares problem is given by

$$\widehat{\beta} = \underset{\beta}{\operatorname{argmin}} \, \|\mathbf{X}\beta - \mathbf{y}\|_2^2.$$

Least squares in matrix form

The objective of the least squares problem can be written as

$$V(\beta) = \|\mathbf{X}\beta - \mathbf{y}\|_2^2 = \beta^\mathsf{T}\mathbf{X}^\mathsf{T}\mathbf{X}\beta - 2\mathbf{y}^\mathsf{T}\mathbf{X}\beta + \mathbf{y}^\mathsf{T}\mathbf{y}.$$

Minimize by differentiating and setting

$$\frac{\partial V(\beta)}{\partial \beta} = 2\mathbf{X}^\mathsf{T}\mathbf{X}\beta - 2\mathbf{y}^\mathsf{T}\mathbf{X}$$

to zero. Therefore,

$$\mathbf{X}^\mathsf{T}\mathbf{X}\widehat{\beta} = \mathbf{X}^\mathsf{T}\mathbf{y}.$$

Least squares in matrix form

The least squares problem

$$\widehat{\beta} = \underset{\beta}{\operatorname{argmin}} \|\mathbf{X}\beta - \mathbf{y}\|_2^2$$

is solved by the **normal equations**

$$\widehat{\beta} = \left(\mathbf{X}^\mathsf{T}\mathbf{X}\right)^{-1}\mathbf{X}^\mathsf{T}\mathbf{y}.$$

Remember $\mathbf{X}^\mathsf{T}\mathbf{X}$ is like sum of squares (similar to co-variances of input variables) and $\mathbf{X}^\mathsf{T}\mathbf{y}$ is similar to co-variance between input and output.
For $k = 1$:

$$\widehat{\beta}_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

Linear regression: the key concepts

The linear regression model

$$y = \beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k + \epsilon$$

+

Maximum likelihood

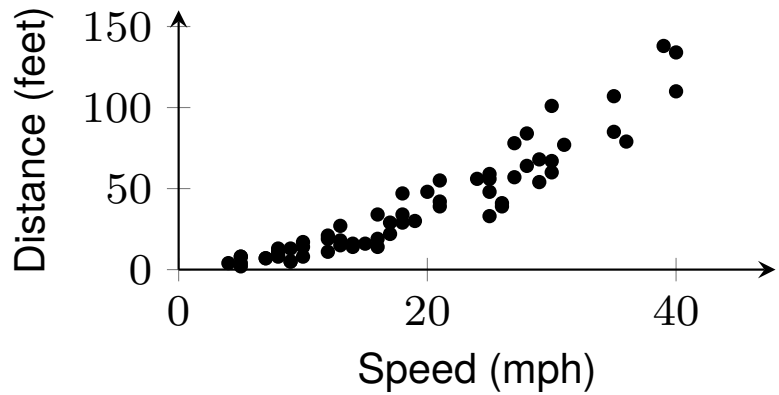
$$\epsilon \sim \mathcal{N}(0, \sigma_\epsilon^2) \text{ iid}$$

Our first learning tool

UPPSALA UNIVERSITET

Example

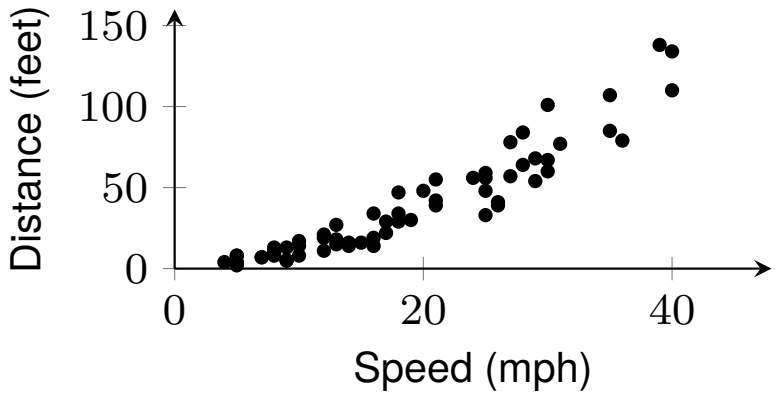
- x = Speed
- y = Distance



Example

- x = Speed
- y = Distance

$$y = \beta_0 + \beta_1 x + \epsilon, \;\; \epsilon \sim \mathcal{N}(0, \sigma_\epsilon^2)$$



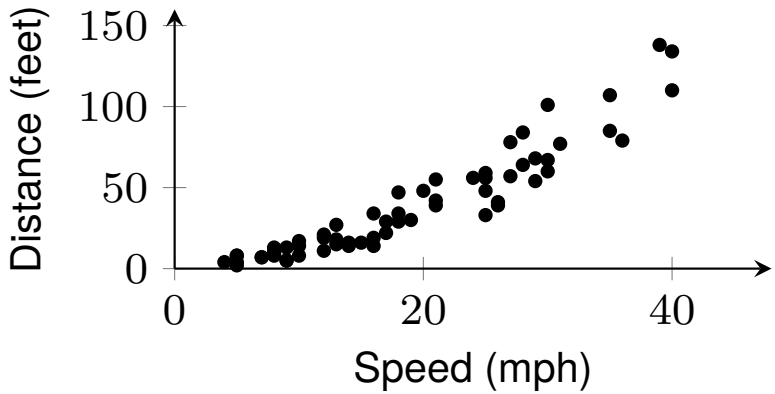
Example

- x = Speed
- y = Distance

$$y = \beta_0 + \beta_1 x + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma_\epsilon^2)$$

$$\beta = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix}$$

$$\mathbf{X} = \begin{bmatrix} 1 & 4 \\ 1 & 5 \\ 1 & 5 \\ 1 & 5 \\ 1 & 5 \\ 1 & 7 \\ 1 & 7 \\ 1 & 8 \\ \vdots & \vdots \\ 1 & 39 \\ 1 & 39 \\ 1 & 40 \end{bmatrix} \qquad \mathbf{y} = \begin{bmatrix} 4 \\ 2 \\ 4 \\ 8 \\ 8 \\ 7 \\ 7 \\ 8 \\ \vdots \\ 138 \\ 110 \\ 134 \end{bmatrix}$$



Example

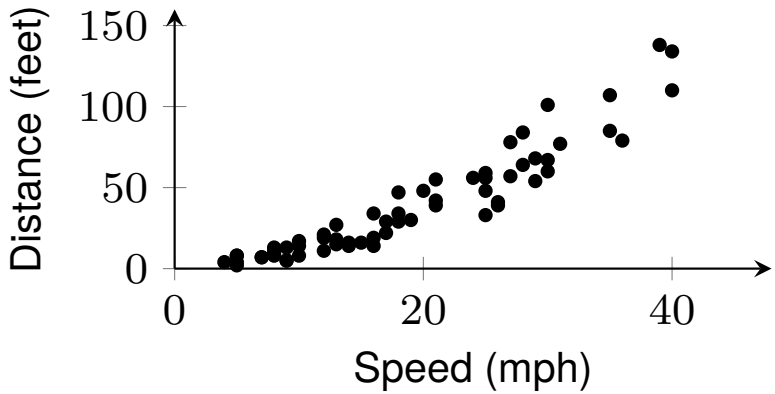
- x = Speed
- y = Distance

$$y = \beta_0 + \beta_1 x + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma_\epsilon^2)$$

$$\beta = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix}$$

$$\mathbf{X} = \begin{bmatrix} 1 & 4 \\ 1 & 5 \\ 1 & 5 \\ 1 & 5 \\ 1 & 5 \\ 1 & 7 \\ 1 & 7 \\ 1 & 8 \\ \vdots & \vdots \\ 1 & 39 \\ 1 & 39 \\ 1 & 40 \end{bmatrix} \qquad \mathbf{y} = \begin{bmatrix} 4 \\ 2 \\ 4 \\ 8 \\ 8 \\ 7 \\ 7 \\ 8 \\ \vdots \\ 138 \\ 110 \\ 134 \end{bmatrix}$$

The normal equations $\Rightarrow \widehat{\beta} = \begin{bmatrix} -20.1 \\ 3.1 \end{bmatrix}$



Example

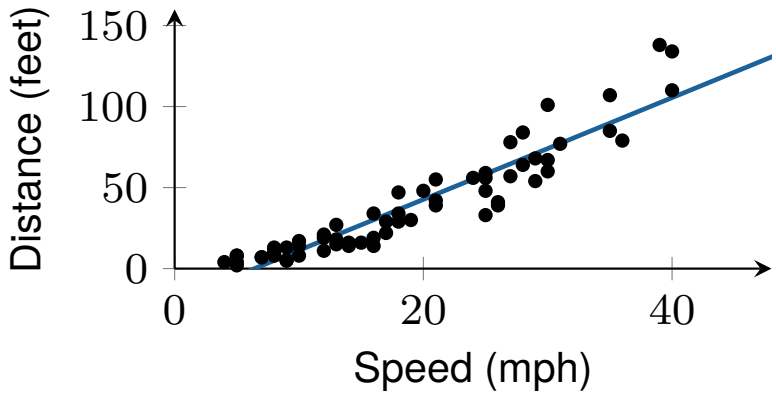
- x = Speed
- y = Distance

$$y = \beta_0 + \beta_1 x + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma_\epsilon^2)$$

$$\beta = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix}$$

$$\mathbf{X} = \begin{bmatrix} 1 & 4 \\ 1 & 5 \\ 1 & 5 \\ 1 & 5 \\ 1 & 5 \\ 1 & 7 \\ 1 & 7 \\ 1 & 8 \\ \vdots & \vdots \\ 1 & 39 \\ 1 & 39 \\ 1 & 40 \end{bmatrix} \qquad \mathbf{y} = \begin{bmatrix} 4 \\ 2 \\ 4 \\ 8 \\ 8 \\ 7 \\ 7 \\ 8 \\ \vdots \\ 138 \\ 110 \\ 134 \end{bmatrix}$$

The normal equations $\Rightarrow \widehat{\beta} = \begin{bmatrix} -20.1 \\ 3.1 \end{bmatrix}$



Example

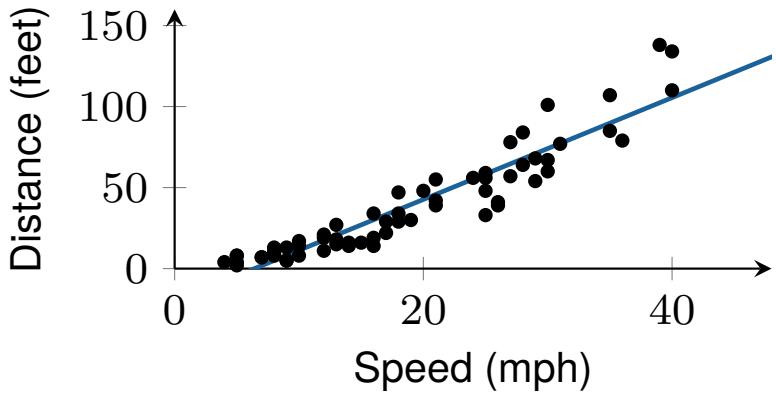
- x = Speed
- y = Distance

$$y = \beta_0 + \beta_1 x + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma_\epsilon^2)$$

$$\beta = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix}$$

$$\mathbf{X} = \begin{bmatrix} 1 & 4 \\ 1 & 5 \\ 1 & 5 \\ 1 & 5 \\ 1 & 5 \\ 1 & 7 \\ 1 & 7 \\ 1 & 8 \\ \vdots & \vdots \\ 1 & 39 \\ 1 & 39 \\ 1 & 40 \end{bmatrix} \qquad \mathbf{y} = \begin{bmatrix} 4 \\ 2 \\ 4 \\ 8 \\ 8 \\ 7 \\ 7 \\ 8 \\ \vdots \\ 138 \\ 110 \\ 134 \end{bmatrix}$$

The normal equations $\Rightarrow \widehat{\beta} = \begin{bmatrix} -20.1 \\ 3.1 \end{bmatrix}$



Use the model for predictions!

Transforming the inputs

"If the speed v is an input variable, why can't the kinetic energy ($\propto v^2$) be an input variable?"

We can make arbitrary nonlinear transformations to the input variables!

The model is still a linear regression model, since

$$y = \beta_0 + \beta_1 v + \beta_2 v^2 + \beta_3 \cos(v) + \beta_4 \arctan(v) + \epsilon$$

is equivalent to

$$\begin{aligned} y &= \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \beta_4 x_4 + \epsilon, \\ \text{with} \quad x_1 &= v \\ x_2 &= v^2 \\ x_3 &= \cos(v) \\ x_4 &= \arctan(v), \end{aligned}$$

where v = original input variable, x_i transformed input variables (features).

Example

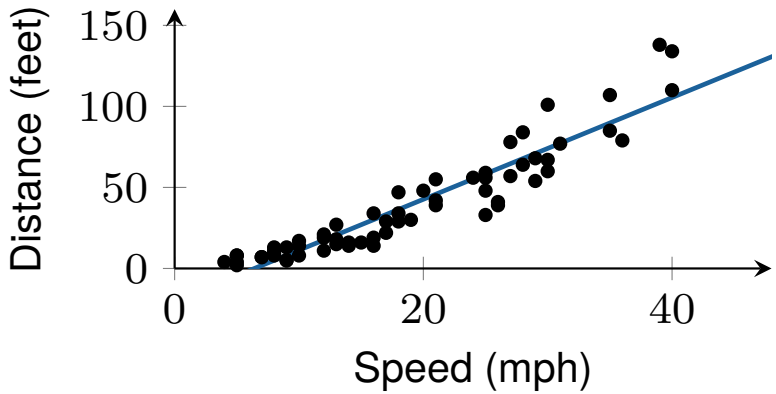
- x = Speed
- y = Distance

$$y = \beta_0 + \beta_1 x + \epsilon, \epsilon \sim \mathcal{N}(0, \sigma_\epsilon^2)$$

$$\beta = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix}$$

$$\mathbf{X} = \begin{bmatrix} 1 & 4 \\ 1 & 5 \\ 1 & 5 \\ 1 & 5 \\ 1 & 5 \\ 1 & 7 \\ 1 & 7 \\ 1 & 8 \\ \vdots & \vdots \\ 1 & 39 \\ 1 & 39 \\ 1 & 40 \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} 4 \\ 2 \\ 4 \\ 8 \\ 8 \\ 7 \\ 7 \\ 8 \\ \vdots \\ 138 \\ 110 \\ 134 \end{bmatrix}$$

The normal equations $\Rightarrow \widehat{\beta} = \begin{bmatrix} -20.1 \\ 3.1 \end{bmatrix}$



Example

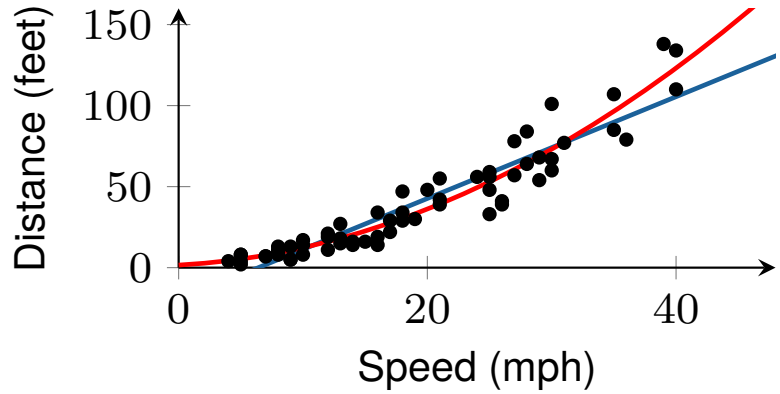
- x = Speed
- y = Distance

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \epsilon, \epsilon \sim \mathcal{N}(0, \sigma_\epsilon^2)$$

$$\beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}$$

The normal equations $\Rightarrow \widehat{\beta} = \begin{bmatrix} 1.58 \\ 0.42 \\ 0.066 \end{bmatrix}$

$$\mathbf{X} = \begin{bmatrix} 1 & 4 & 16 \\ 1 & 5 & 25 \\ 1 & 5 & 25 \\ 1 & 5 & 25 \\ 1 & 5 & 25 \\ 1 & 7 & 49 \\ 1 & 7 & 49 \\ 1 & 8 & 64 \\ \vdots & \vdots & \vdots \\ 1 & 39 & 1521 \\ 1 & 39 & 1521 \\ 1 & 40 & 1600 \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} 4 \\ 2 \\ 4 \\ 8 \\ 8 \\ 7 \\ 7 \\ 8 \\ \vdots \\ 138 \\ 110 \\ 134 \end{bmatrix}$$



Transforming the inputs

If the original input variable is v, we can use for instance:

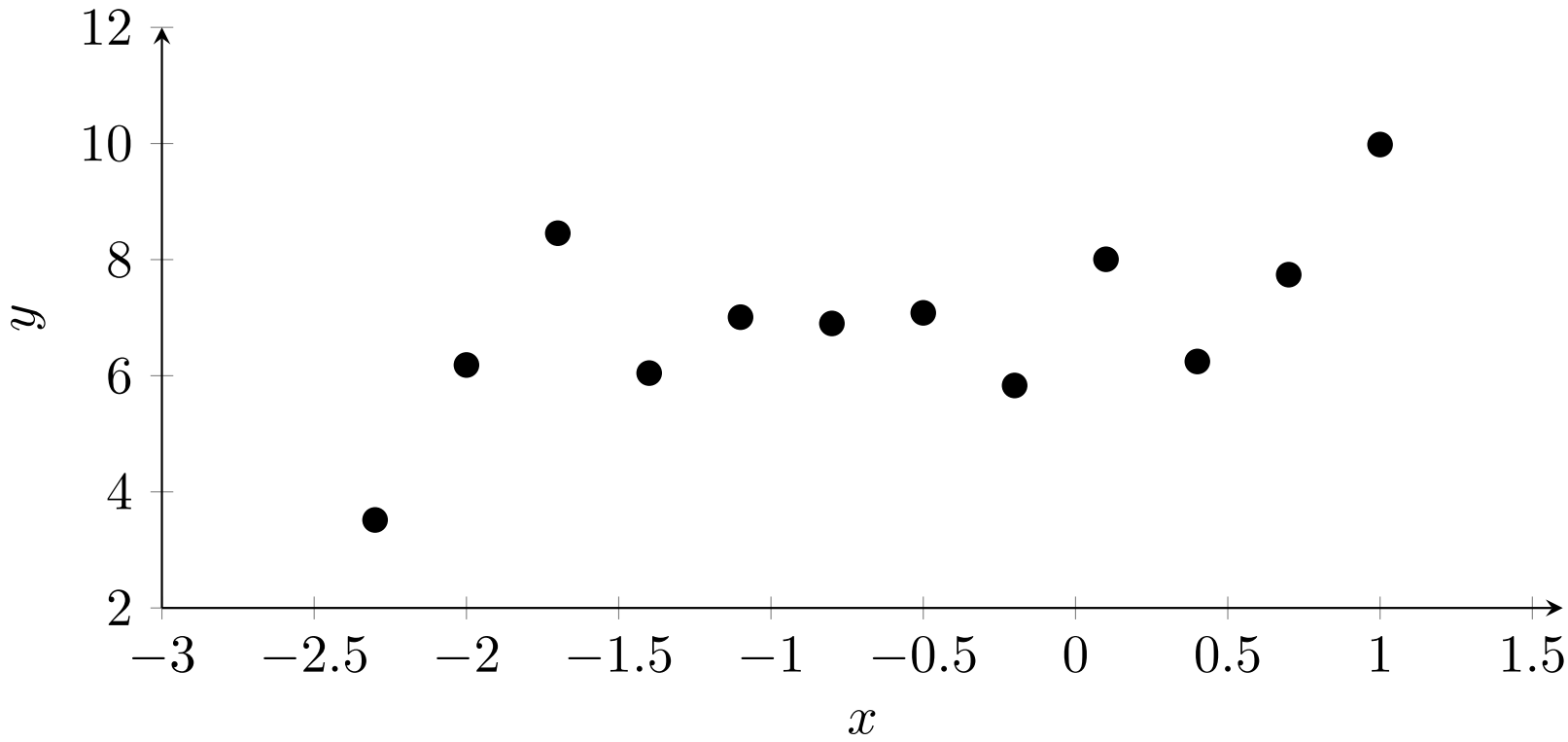
- a polynomial in v

$$y = \beta_0 + \beta_1 \underset{\underset{x_1}{||}}{v} + \beta_2 \underset{\underset{x_2}{||}}{v^2} + \beta_3 \underset{\underset{x_3}{||}}{v^3} + \cdots + \beta_p \underset{\underset{x_p}{||}}{v^p} + \epsilon$$

- radial basis function kernels (see book)
- ...

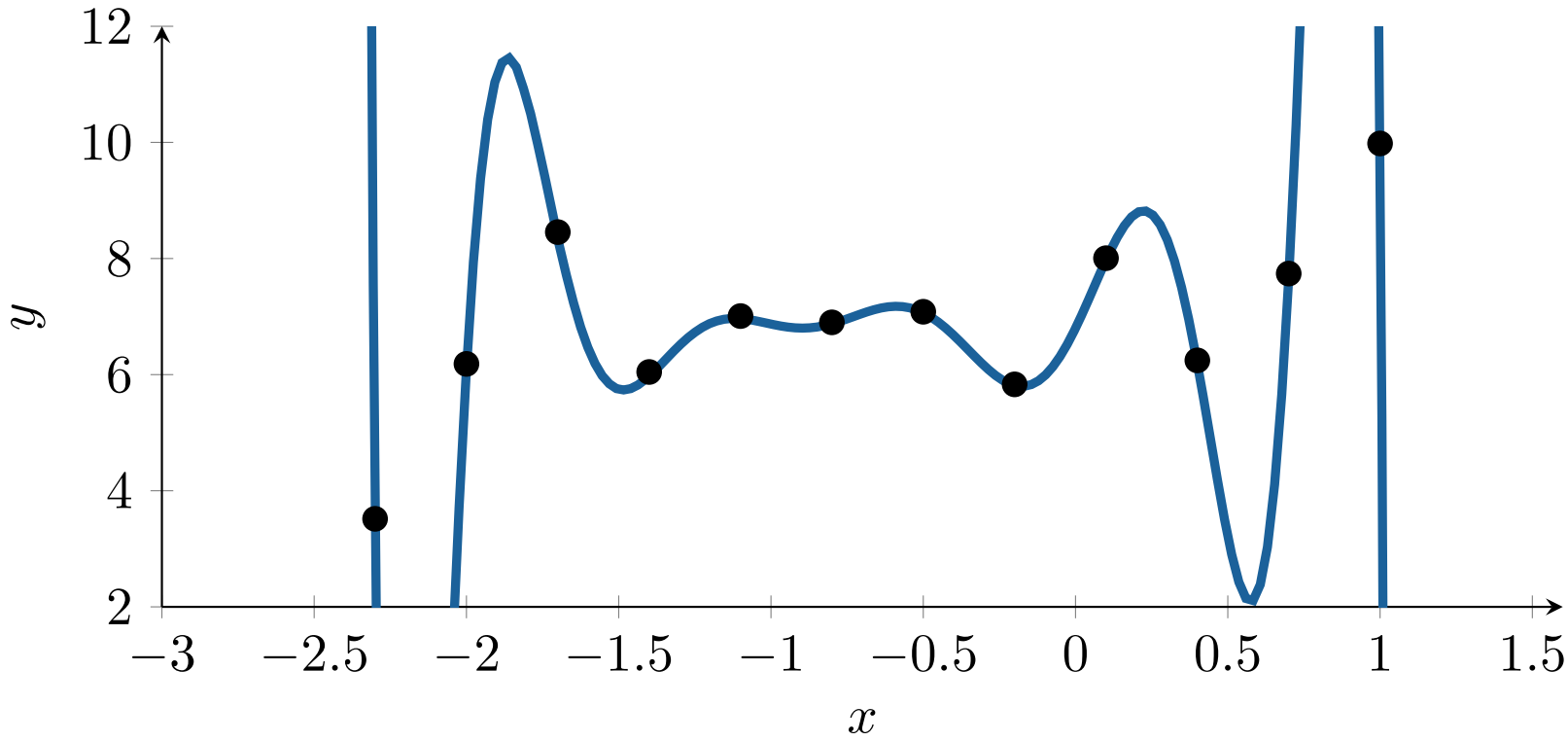
Is the model too flexible?

With a $p = n - 1$ degree polynomial, we can fit n data points perfectly.



Is the model too flexible?

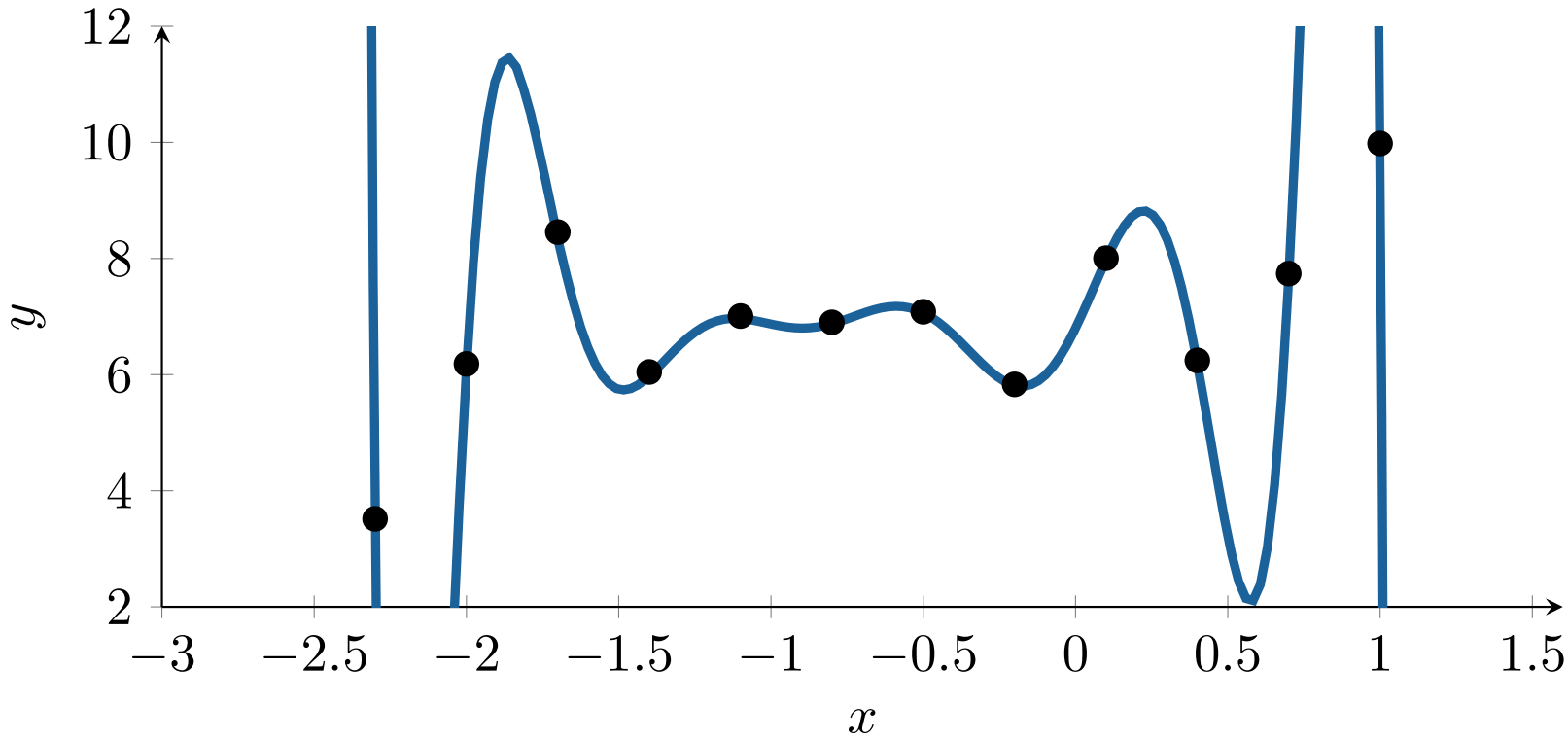
With a $p = n - 1$ degree polynomial, we can fit n data points perfectly.



Is this desired?

Is the model too flexible?

With a $p = n - 1$ degree polynomial, we can fit n data points perfectly.



Is this desired? **Overfit!**

Regularization



"Keep β small unless the data really convinces us otherwise"

Least squares

$$\widehat{\boldsymbol{\beta}} = \underset{\boldsymbol{\beta}}{\text{argmin}} \, \|\mathbf{X}\boldsymbol{\beta} - \mathbf{y}\|_2^2$$

Regularization



"Keep β small unless the data really convinces us otherwise"

Ridge regression = Least squares with ℓ_2 regularizer

$$\widehat{\boldsymbol{\beta}} = \underset{\boldsymbol{\beta}}{\text{argmin}} \, \|\mathbf{X}\boldsymbol{\beta} - \mathbf{y}\|_2^2 + \gamma \|\boldsymbol{\beta}\|_2^2$$

Regularization



"Keep β small unless the data really convinces us otherwise"

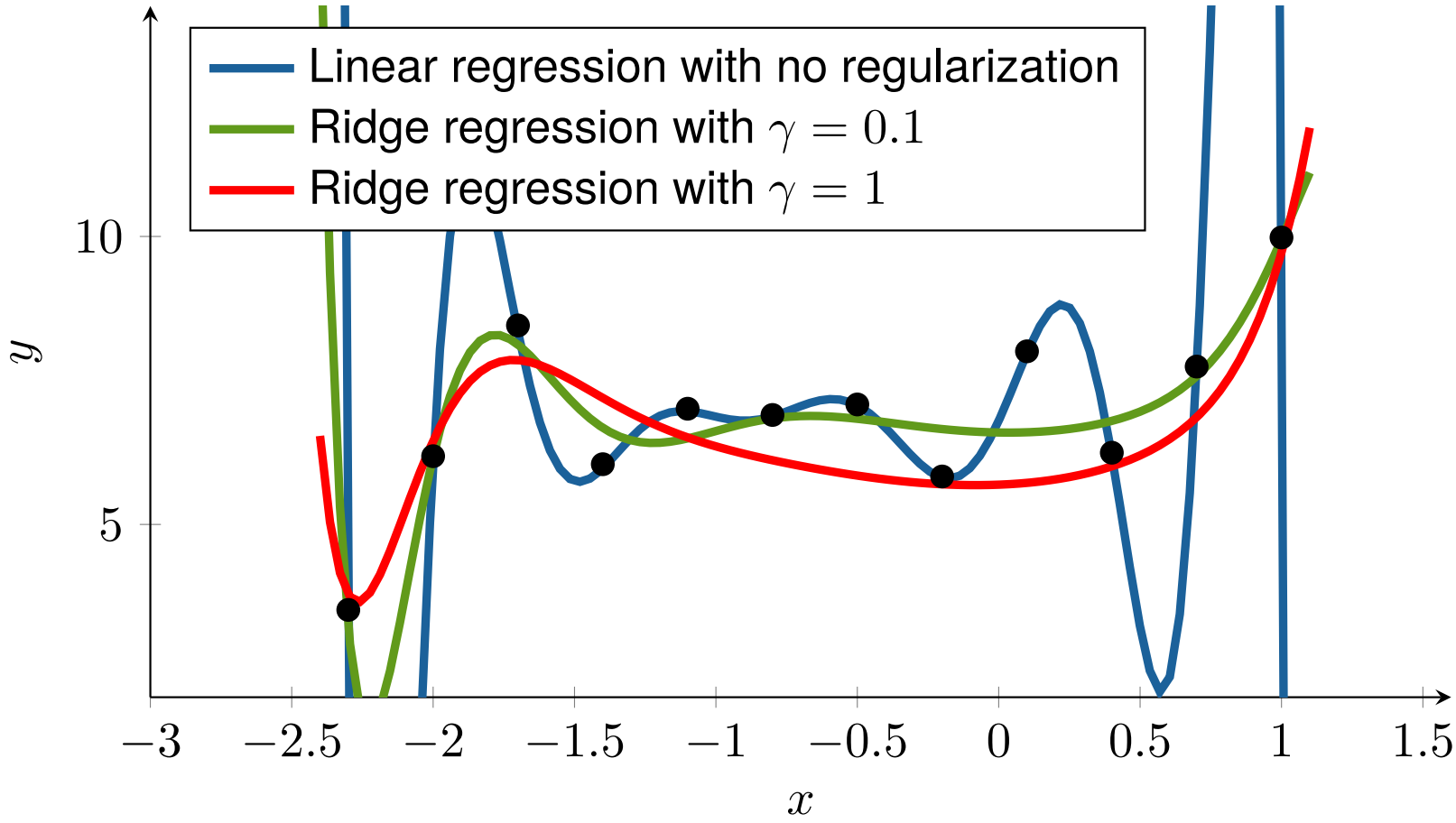
Ridge regression = Least squares with ℓ_2 regularizer

$$\widehat{\boldsymbol{\beta}} = \underset{\boldsymbol{\beta}}{\text{argmin}} \, \|\mathbf{X}\boldsymbol{\beta} - \mathbf{y}\|_2^2 + \gamma\|\boldsymbol{\beta}\|_2^2$$

$$\Rightarrow (\mathbf{X}^\mathsf{T}\mathbf{X} + \gamma\mathbf{I}_{p+1})\widehat{\boldsymbol{\beta}} = \mathbf{X}^\mathsf{T}\mathbf{y}$$

Here, $\gamma \geq 0$ is the regularization parameter.

Is the model too flexible?



Regularization can help us to avoid overfitting!

Regularization

Ridge regression

$$\widehat{\boldsymbol{\beta}} = \underset{\boldsymbol{\beta}}{\operatorname{argmin}} \, \|\mathbf{X}\boldsymbol{\beta} - \mathbf{y}\|_2^2 + \gamma\|\boldsymbol{\beta}\|_2^2$$

(has a closed-form solution for $\widehat{\boldsymbol{\beta}}$)

LASSO

$$\widehat{\boldsymbol{\beta}} = \underset{\boldsymbol{\beta}}{\operatorname{argmin}} \, \|\mathbf{X}\boldsymbol{\beta} - \mathbf{y}\|_2^2 + \gamma\|\boldsymbol{\beta}\|_1$$

(lacks a closed-form solution for $\widehat{\boldsymbol{\beta}}$)

Regularization can be used in many methods, not only linear regression!

Dummy variables for categorical inputs

For a categorical input with 2 different classes/levels/labels A and B:
Create a dummy variable

$$x = \begin{cases} 0 & \text{if A} \\ 1 & \text{if B} \end{cases}$$

$$\Rightarrow y = \beta_0 + \beta_1 x + \varepsilon = \begin{cases} \beta_0 + \varepsilon & \text{if A} \\ \beta_0 + \beta_1 + \varepsilon & \text{if B} \end{cases}$$

UPPSALA UNIVERSITET

Dummy variables for categorical inputs

For a categorical input with $k = 4$ different classes/levels/labels A, B, C, D:
Create $k - 1 = 3$ dummy variables

$$x_1 = \begin{cases} 1 & \text{if B} \\ 0 & \text{if not B} \end{cases}, \quad x_2 = \begin{cases} 1 & \text{if C} \\ 0 & \text{if not C} \end{cases}, \quad x_3 = \begin{cases} 1 & \text{if D} \\ 0 & \text{if not D} \end{cases}$$

$$\Rightarrow y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \varepsilon = \begin{cases} \beta_0 + \varepsilon & \text{if A} \\ \beta_0 + \beta_1 + \varepsilon & \text{if B} \\ \beta_0 + \beta_2 + \varepsilon & \text{if C} \\ \beta_0 + \beta_3 + \varepsilon & \text{if D} \end{cases}$$

A few concepts to summarize lecture 2

Regression is about learning a model that describes the relationship between an input variable $\mathbf{x}$ (both numerical and categorical) and a numerical output variable y.

Linear regression corresponds to regression with a linear model.

Maximum likelihood with a Gaussian iid assumption on ϵ
$\Rightarrow$ **least squares** and **normal equations**.

Nonlinear transformations can be applied to the input variables.

Overfitting is when the model adapts (too much) to noise in the data.

Regularization can help against overfitting.

Categorical variables are handled by dummy variables.

Statistical Machine Learning

Lecture 2 – Maximum likelihood refresher



UPPSALA
UNIVERSITET

Sebastian Mair
`https://smair.github.io/`
Department of Information Technology
Uppsala University

Course webpage

Maximum likelihood

Let $x_1, x_2, \ldots, x_n$ be a sample of n iid random variables $X_1, X_2, \ldots, X_n$ with pmf $\mathbf{P}_\theta(x)$ (or pdf $f_\theta(x)$) parametrized by $\theta \in \Theta$.

Goal: Estimate θ based on the sample.

Idea: Choose $\theta^\star$ that maximizes the joint probability of the observed data.

$$\theta^\star = \arg\max_{\theta \in \Theta} \mathbf{P}_\theta[X_1 = x_1, X_2 = x_2, \ldots, X_n = x_n].$$

Maximum likelihood

$$
\begin{aligned}
&\mathbf{P}_\theta[X_1 = x_1, X_2 = x_2, \ldots, X_n = x_n] \\
&\overset{\text{iid}}{=} \mathbf{P}_\theta[X_1 = x_1] \cdot \mathbf{P}_\theta[X_2 = x_2] \cdot \ldots \cdot \mathbf{P}_\theta[X_n = x_n] \\
&= \prod_{i=1}^{n} \mathbf{P}_\theta[X_i = x_i] =: \mathcal{L}(\theta).
\end{aligned}
$$

We call $\mathcal{L}(\theta)$ the **likelihood function**. Our estimate is now given by

$$
\theta^\star = \arg\max_{\theta \in \Theta} \ \mathcal{L}(\theta)
$$

Maximum likelihood

Instead of the product, we often maximize the the logarithmized version $\ell(\theta) = \log \mathcal{L}(\theta)$, called the **log-likelihood function**. Thus,

$$\theta^\star = \arg\max_{\theta \in \Theta} \ \ell(\theta)$$

Note: Using the probability $\mathbf{P}_\theta[X_i = x_i]$ does not makes sense in the continuous case as it is zero for all x_i. Instead, we use the pdf $f_\theta(x_i)$.

Maximum likelihood - Example: Poisson distribution

Assumption: The sample $x_1, x_2, \ldots, x_n$ is iid and follows a Poisson distribution.

Reminder: The pmf of a Poisson distribution with parameter $\lambda > 0$ is given by

$$\mathbf{P}(X = k) = \frac{\lambda^k}{k!} \cdot e^{-\lambda} \quad (k \in \mathbb{N}_0).$$

Maximum likelihood - Example: Poisson distribution

The likelihood function is given by

$$\mathcal{L}(\lambda) = \mathbf{P}_\lambda[X_1 = x_1, X_2 = x_2, \ldots, X_n = x_n] \overset{\text{iid}}{=} \prod_{i=1}^{n} \mathbf{P}_\lambda[X_i = x_i] = \prod_{i=1}^{n} \frac{\lambda^{x_i}}{x_i!} \cdot e^{-\lambda}$$

and the corresponding log-likelihood function is

$$\begin{aligned}
\ell(\lambda) = \log \mathcal{L}(\lambda) &= \sum_{i=1}^{n} \log\left(\frac{\lambda^{x_i}}{x_i!} \cdot e^{-\lambda}\right) = \sum_{i=1}^{n} \left[\log\left(\frac{\lambda^{x_i}}{x_i!}\right) + \log\left(e^{-\lambda}\right)\right] \\
&= \sum_{i=1}^{n} \left[\log\left(\lambda^{x_i}\right) - \log(x_i!) - \lambda\right] = \sum_{i=1}^{n} x_i \log(\lambda) - \sum_{i=1}^{n} \log(x_i!) - n\lambda \\
&= \log(\lambda) \sum_{i=1}^{n} x_i - \sum_{i=1}^{n} \log(x_i!) - n\lambda
\end{aligned}$$

Maximum likelihood - Example: Poisson distribution

$$\ell(\lambda) = \log(\lambda) \sum_{i=1}^{n} x_i - \sum_{i=1}^{n} \log(x_i!) - n\lambda$$

We can maximize this log-likelihood function by setting the derivative

$$\frac{\mathrm{d}}{\mathrm{d}\lambda} \ell(\lambda) = \frac{1}{\lambda} \sum_{i=1}^{n} x_i - n$$

to zero and re-arrange the terms

$$\frac{1}{\lambda} \sum_{i=1}^{n} x_i - n = 0 \implies \frac{1}{\lambda} \sum_{i=1}^{n} x_i = n \implies \sum_{i=1}^{n} x_i = n\lambda \implies \lambda = \frac{1}{n} \sum_{i=1}^{n} x_i = \bar{x}.$$

Thus, our estimate for λ is given by $\frac{1}{n} \sum_{i=1}^{n} x_i$.

Maximum likelihood - Example: Normal distribution

Assumption: The sample $x_1, x_2, \ldots, x_n$ is iid and follows a normal distribution.

Reminder: The pdf of a normal distribution with param. $\mu \in \mathbb{R}$ and $\sigma^2 > 0$ is given by

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2}\frac{(x-\mu)^2}{\sigma^2}\right)$$

Maximum likelihood - Example: Normal distribution

$$
\begin{aligned}
\ell(\mu, \sigma^2) = \log \mathcal{L}(\mu, \sigma^2) &= \log \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2} \frac{(x_i - \mu)^2}{\sigma^2} \right) \\
&= \sum_{i=1}^{n} \log \left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2} \frac{(x_i - \mu)^2}{\sigma^2} \right) \right] \\
&= \sum_{i=1}^{n} \underbrace{\log 1}_{=0} - \log \sqrt{2\pi\sigma^2} - \frac{1}{2} \frac{(x_i - \mu)^2}{\sigma^2} \\
&= \sum_{i=1}^{n} -\frac{1}{2} \log 2\pi - \log \sigma + -\frac{1}{2} \frac{(x_i - \mu)^2}{\sigma^2} \\
&= -\frac{n}{2} \log 2\pi - n \log \sigma - \frac{1}{2\sigma^2} \sum_{i=1}^{n} (x_i - \mu)^2.
\end{aligned}
$$

Maximum likelihood - Example: Normal distribution

For μ, we derive

$$\frac{\partial \ell(\mu, \sigma^2)}{\partial \mu} = \frac{\partial}{\partial \mu}\left[-\frac{1}{2\sigma^2}\sum_{i=1}^{n}(x_i - \mu)^2 \right] = -\frac{1}{2\sigma^2}\sum_{i=1}^{n}\frac{\partial}{\partial \mu}\left[(x_i - \mu)^2\right]$$
$$= -\frac{1}{\sigma^2}\sum_{i=1}^{n}(x_i - \mu)(-1) \stackrel{!}{=} 0.$$

Hence,

$$0 = \sum_{i=1}^{n}(x_i - \mu) \implies 0 = \sum_{i=1}^{n} x_i - n\mu \implies \mu = \frac{1}{n}\sum_{i=1}^{n} x_i.$$

Maximum likelihood - Example: Normal distribution

For σ^2, we derive

$$\begin{aligned}\frac{\partial \ell(\mu, \sigma^2)}{\partial \sigma} &= \frac{\partial}{\partial \sigma}\left[- n \log \sigma - \frac{1}{2\sigma^2} \sum_{i=1}^{n} (x_i - \mu)^2 \right] \\ &= -n \frac{\partial}{\partial \sigma} \log \sigma - \frac{1}{2} \frac{\partial}{\partial \sigma} \sigma^{-2} \sum_{i=1}^{n} (x_i - \mu)^2 \\ &= -\frac{n}{\sigma} - \frac{-2}{2} \sigma^{-3} \sum_{i=1}^{n} (x_i - \mu)^2 = -\frac{n}{\sigma} + \sigma^{-3} \sum_{i=1}^{n} (x_i - \mu)^2 \stackrel{!}{=} 0.\end{aligned}$$

Hence,

$$\frac{n}{\sigma} = \sigma^{-3} \sum_{i=1}^{n} (x_i - \mu)^2 \implies \sigma^2 = \frac{1}{n} \sum_{i=1}^{n} (x_i - \mu)^2.$$

Lecture 3 – Classification, logistic regression



UPPSALA
UNIVERSITET

Jens Sjölund
https://jsjol.github.io/
Department of Information Technology
Uppsala University

Course webpage

Summary of lecture 2 (I/III)

Regression is about learning a model that describes the relationship between an input variable $\mathbf{x}$ and a numerical output variable y

$$y = f(\mathbf{x}; \boldsymbol{\theta}) + \epsilon.$$

Linear regression is regression with a linear model

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_p x_p + \epsilon.$$

How to learn/train/estimate θ?

Use the **maximum likelihood** principle: assume $\varepsilon \sim \mathcal{N}(0, \sigma_\epsilon^2)$ iid
$\Rightarrow$ **least squares** & **normal equations**

$$\widehat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \ \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2, \qquad \widehat{\boldsymbol{\theta}} = (\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}\mathbf{y},$$

$$\mathbf{x}_i = \begin{bmatrix} x_{i1} \\ \vdots \\ x_{ip} \end{bmatrix}, \qquad \mathbf{X} = \begin{bmatrix} 1 & -\mathbf{x}_1^\mathsf{T}- \\ \vdots & \vdots \\ 1 & -\mathbf{x}_n^\mathsf{T}- \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}.$$

Summary of lecture 2 (II/III)

We can make **arbitrary nonlinear transformations** of the inputs, for example polynomials

$$y = \theta_0 + \theta_1 \underset{\| \; x_1}{v} + \theta_2 \underset{\| \; x_2}{v^2} + \theta_3 \underset{\| \; x_3}{v^3} + \cdots + \theta_p \underset{\| \; x_p}{v^p} + \epsilon$$

($v =$ original input variable, x_k transformed input variables or features)

Categorical input variables are handled by creating dummy variables.

Summary of lecture 2 (III/III)

Overfitting may occur when the model is **too flexible!**

Can be handled using **regularization**, which amounts to adding a term $R(\boldsymbol{\theta})$ to the cost function which controls the model flexibility,

$$\widehat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \underbrace{V(\boldsymbol{\theta}, \mathbf{X}, \mathbf{y})}_{\text{data fit}} + \lambda \underbrace{R(\boldsymbol{\theta})}_{\text{penalty}}$$

Ridge regression

$$\widehat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta}}{\text{argmin}} \ \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 + \gamma\|\boldsymbol{\theta}\|_2^2$$

LASSO

$$\widehat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta}}{\text{argmin}} \ \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 + \gamma\|\boldsymbol{\theta}\|_1$$

Outline – Lecture 3

Aim: To introduce the classification problem and derive a first useful classifier: logistic regression.

Outline:

1. Summary of Lecture 2
1. The classification problem
2. Logistic regression
3. Diagnostic tools for classification (via example)

Categorical outputs

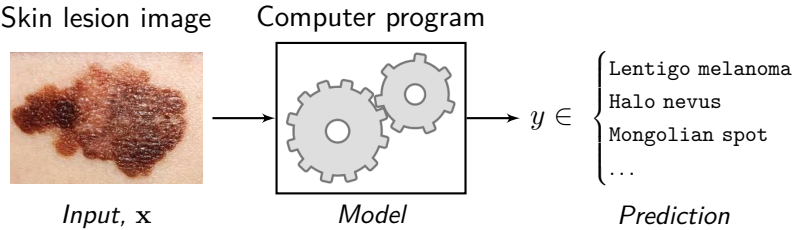
Many machine learning applications have **categorical outputs** y

- Face verification:
  $y \in \{\texttt{match}, \texttt{no match}\}$
- Identify the spoken vowel from an audio signal: $y \in \{\texttt{A}, \texttt{E}, \texttt{I}, \texttt{O}, \texttt{U}, \texttt{Y}\}$
- Diagnosis system for leukemia:
  $y \in \{\texttt{ALL}, \texttt{AML}, \texttt{CLL}, \texttt{CML}, \texttt{no leukemia}\}$
- Anomaly detection, e.g. in cybersecurity or predictive maintenance:
  $y \in \{\texttt{normal}, \texttt{anomalous}\}$

The classification problem

Classification: learn a **model** which, for each input data point $\mathbf{x}$ can predict its **class** $y \in \{1, \ldots, M\}$.

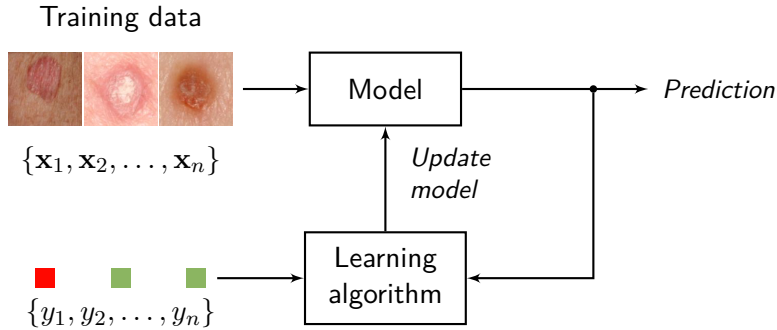
ex) Classifying skin lesions



Andre Esteva, Brett Kuprel, Roberto A. Novoa, Justin Ko, Susan M. Swetter, Helen M. Blau and Sebastian Thrun. **Dermatologist-level classification of skin cancer with deep neural networks**. *Nature*, 542:115–118, 2017.

Training a classifier

Supervised learning: The model is **learned** by adapting it to labeled **training data** $\{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$.



Classification

A **classification model** can be specified in terms of the **conditional class probabilities**

$$p(y = m \,|\, \mathbf{x}) \quad \text{for} \quad m = 1, \ldots, M,$$

encoding the probability for class m given (i.e. conditioned on) that we know the input $\mathbf{x}$.

More specifically (for a binary classification task, $M = 2$, $y = 1$ or $y = -1$) we learn a model $g(\mathbf{x})$ that describes the conditional class probabilities $p(y \,|\, \mathbf{x})$ ($=$ a **classifier**),

$$g(\mathbf{x}) = p(y = 1 \,|\, \mathbf{x}).$$

Why not linear regression?

Can we use linear regression for classification problems?

ex) Classifying e-mails as spam. Code the output as

$$y = \begin{cases} -1 & \text{if } \texttt{ham} \ (= \text{good email}), \\ 1 & \text{if } \texttt{spam}, \end{cases}$$

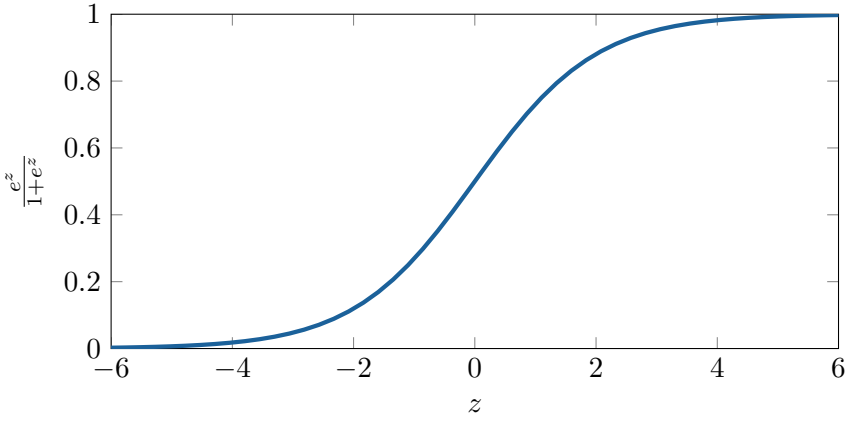
and learn a linear regression model. Classify as spam if $\hat{y} > 0$.

Why is this not a good idea?

- Sensitive to unequally sized classes in the training data.
- Difficult to generalize to $K > 2$ classes.

Logistic function (aka sigmoid function)

The function $f : \mathbb{R} \mapsto [0, 1]$ defined as $f(z) = \frac{e^z}{1+e^z}$ is known as the **logistic function**.



Classification – the multi-class case

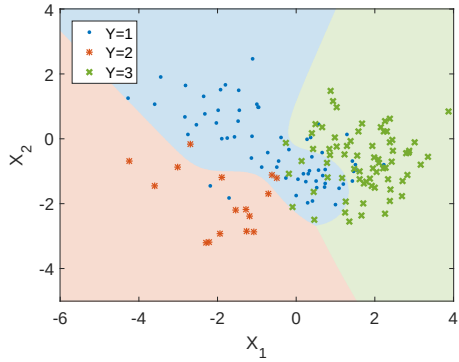
We return a vector valued classifier $\boldsymbol{g}(\mathbf{x})$, where

$$\begin{bmatrix} p(y=1\,|\,\mathbf{x}) \\ p(y=2\,|\,\mathbf{x}) \\ \vdots \\ p(y=M\,|\,\mathbf{x}) \end{bmatrix} \text{ is modeled by } \begin{bmatrix} g_1(\mathbf{x}) \\ g_2(\mathbf{x}) \\ \vdots \\ g_M(\mathbf{x}) \end{bmatrix} = \boldsymbol{g}(\mathbf{x}).$$

Each element $g_m(\mathbf{x})$ corresponds to the conditional class probability $p(y=m\,|\,\mathbf{x})$.

Decision boundaries

The input space can be segmented into M regions, separated by so-called **decision boundaries**.



Finding the decision boundary

The **decision boundary** is found by solving the equation $g(x) = 1 - g(x)$.

For logistic regression this corresponds to

$$\frac{e^{\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}}}{1 + e^{\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}}} = \frac{1}{1 + e^{\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}}}$$

which we can write as $e^{\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}} = 1$. Hence, we have the following expression for the decision boundary

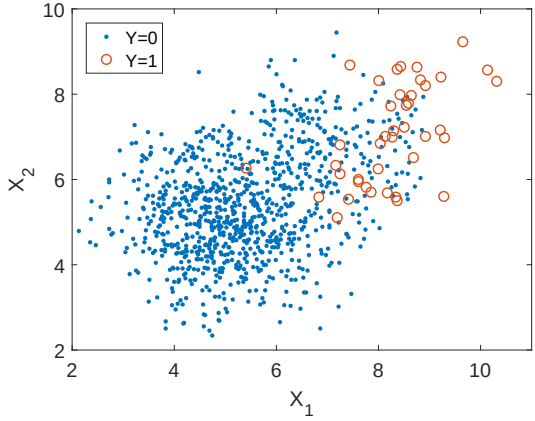
$$\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x} = 0.$$

Linear expression for the decision boundary!

ex) Logistic regression

Consider a *toyish* problem where we want to build a classifier for whether a person has a certain disease ($y = 1$) or not ($y = -1$) based on two biological indicators x_1 and x_2.

The training data consists of $n = 1,000$ labeled samples (right).



ex) Logistic regression

We then fit a logistic regression model

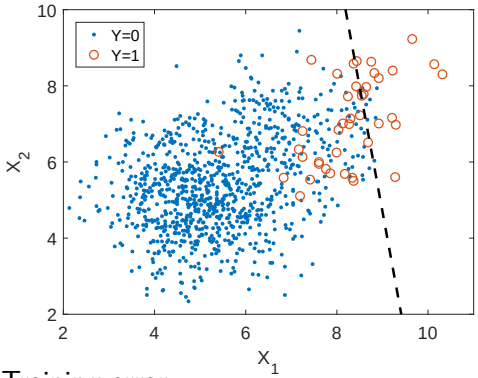
$$p(y = 1 \,|\, \mathbf{x}) = \frac{e^{\theta_0 + \theta_1 x_1 + \theta_2 x_2}}{1 + e^{\theta_0 + \theta_1 x_1 + \theta_2 x_2}}$$

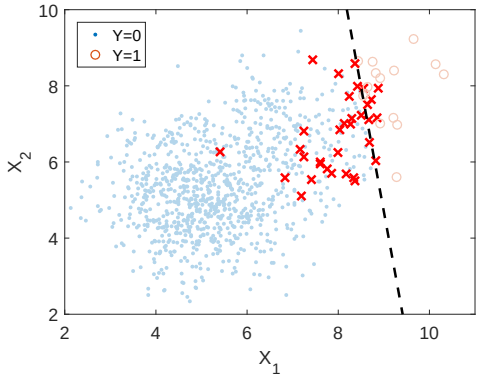
using maximum likelihood.

To evaluate the performance of a classifier, our default choice is the **misclassification rate**:

$$\frac{1}{n} \sum_{i=1}^{n} \underbrace{\mathbb{I}(\hat{y}_i \neq y_i)}_{\begin{cases} 1 & \text{if } \hat{y}_i \neq y_i \\ 0 & \text{if } \hat{y}_i = y_i \end{cases}}$$

ex) Logistic regression: training error



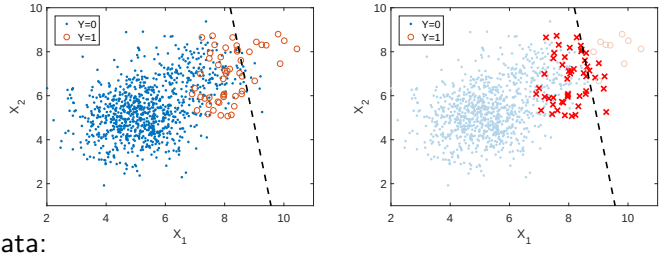


Training error:

$$\frac{1}{n}\sum_{i=1}^{n}\mathbb{I}(\hat{y}_i \neq y_i) = 3.3\%.$$

ex) Logistic regression: test error

To further test the classifier we evaluate it on ***previously unseen* test data**: $\{(\mathbf{x}_i', y_i')\}_{i=1}^{n_t}$.



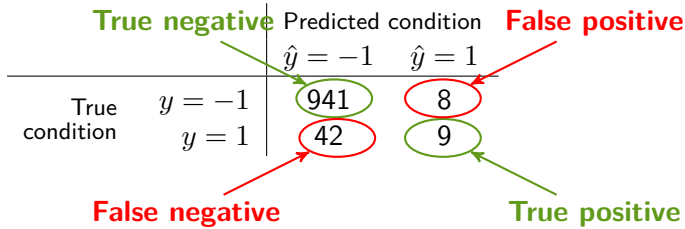
(Estimated) error on the test data:

$$\frac{1}{n_t}\sum_{i=1}^{n_t}\mathbb{I}(\hat{y}_i' \neq y_i') = 5.0\%$$

Error on the test data for the naive classifier $\hat{y} \equiv 0$: 5.1%.

ex) Logistic regression: confusion matrix

Instead of just looking at the misclassification rate it is better to compute the so-called **confusion matrix**.

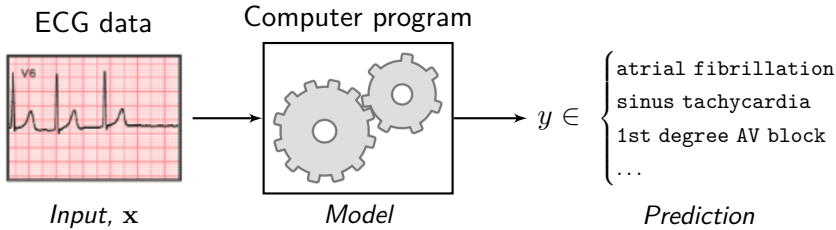


Out of 51 patients affected by the disease, only 9 are correctly classified!

The **True Positive Rate (TPR)** is just $9/51 \approx 17.7\%$.

A classification problem from our research

Aim: Automatic classification of Electrocardiography (ECG) data.



We are now at human level (medical doctors) performance.

Antonio H. Ribeiro, Manoel Horta, Gabriela Paixao, Derick Oliveira, Paulo R. Gomes, Jessica A. Canazart, Milton Pifano, Wagner Meira Jr., Thomas B. Schön and Antonio Luiz Ribeiro. **Automatic diagnosis of short-duration 12-lead ECG using a deep convolutional network**. In *Nature Communications*, 2020. To appear.

Confusion matrices for ECG classification

	Predicted Class	
Actual Class	1dAVb	Not 1dAVb
1dAVb	**24**	9
Not 1dAVb	2	**918**

	Predicted Class	
Actual Class	RBBB	Not RBBB
RBBB	**36**	0
Not RBBB	5	**912**

Actual Class	LBBB	Not LBBB
LBBB	**33**	0
Not LBBB	1	**919**

Actual Class	SB	Not SB
SB	**19**	3
Not SB	5	**926**

Actual Class	AF	Not AF
AF	**11**	2
Not AF	2	**938**

Actual Class	ST	Not ST
ST	**40**	2
Not ST	6	**905**

Conservative predictions

Common to start thinking about a classifier minimizing the total misclassification error.

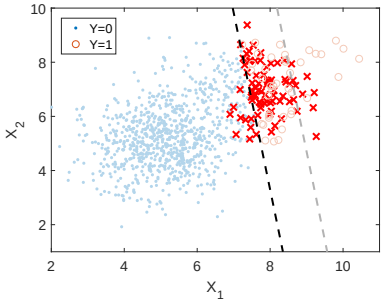
What if false negatives are "worse" than false positives?

Idea: Modify the prediction model

$$\hat{y}(\mathbf{x}_\star) = \begin{cases} 1 & \text{if } g(\mathbf{x}_\star) > r, \\ -1 & \text{otherwise,} \end{cases}$$

where $0 \leq r \leq 1$ is a user chosen threshold.

ex) Logistic regression, cont'd



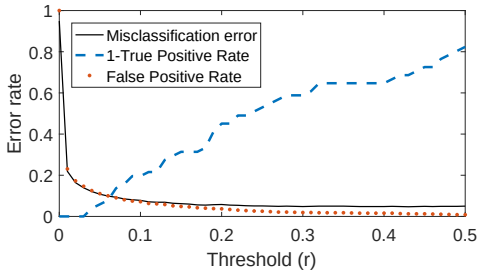
	$\hat{y} = -1$	$\hat{y} = 1$
$y = -1$	881	68
$y = 1$	10	41

Table: Confusion matrix ($r = 0.2$)

If we set the threshold at $r = 0.2$,

- ▲ The ***true positive rate*** is increased to $41/51 = 80.4\%$.
- ▼ However, the ***misclassification error*** is increased to 7.8%.

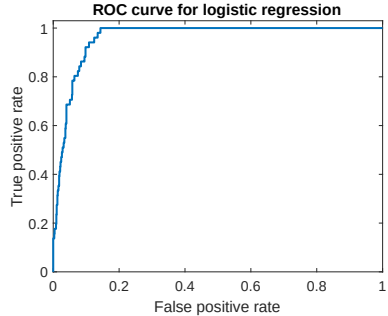
ex) Logistic regression: error rates



As we increase the threshold r from 0 to 0.5:

- ▲ The ***misclassification error*** decreases
- ▲ The number of ***non-diseased persons incorrectly classified as diseased*** **(False Positive Rate)** decreases.
- ▼ The number of ***diseased persons incorrectly classified as non-diseased*** **(1 – True Positive Rate)** increases!

ex) Logistic regression: ROC and AUC



- **ROC[1] curve**: plot of TPR vs. FPR.
- **Area Under Curve (AUC):** condensed performance measure for the classifier, taking all possible thresholds into account.
- AUC $\in [0, 1]$ where AUC $= 0.5$ corresponds to a random guess. [***ex)*** AUC $= 0.96$]

[1] For *Receiver Operating Characteristics*, which is a historical name.

A few concepts to summarize lecture 3

Classification: The problem of modeling a relationship between an (arbitrary) input $\mathbf{x}$ and a categorical output y.

Decision boundaries: Points in the input space where the classifier $\hat{y}(\mathbf{x})$ changes value.

Logistic regression: A model that uses a logistic function to model a binary output variable.

Confusion matrix: Table with predicted vs true class labels (for binary classification $\Rightarrow$ number of true negatives, true positives, false negatives, and false positives).

More on evaluating performance

Training/learning: Minimizing the average loss on training data. Loss functions differ between methods and the values can not directly be compared.

We would like to evaluate already trained models with respect to hyperparameters (e.g. regularization parameter γ) but also different methods (logistic regression, neural net, etc.).

Error function

$$E(\hat{y}(\mathbf{x})), y) = \begin{cases} \mathbb{I}\left(\widehat{y}(\mathbf{x}) = y\right) & \text{(classification)} \\ \left(\widehat{y}(\mathbf{x}) - y\right)^2 & \text{(regression)} \end{cases}$$

Note: Error function not necessarily the same as loss function.

More on evaluating performance

Training error

$$E_{\text{train}} = \frac{1}{n}\sum_{i=1}^{n} E(\widehat{y}(\mathbf{x}_i), y_i)$$

Measures how well a predictor $\widehat{y}$ performs on training data, but we are interested in new data. Let $p(\mathbf{x}, y)$ be the joint distribution over data.

Expected new data error

$$E_{\text{new}} = \mathbb{E}_\star\left[E(\widehat{y}(x_\star), y_\star)\right] = \int E(\widehat{y}(x_\star), y_\star)p(x_\star, y_\star)dx_\star dy_\star$$

Impossible to compute since $p(\mathbf{x}, y)$ is unknown, but minimizing E_{new} is our ultimate goal.

Maybe we can learn it?

Approximating integrals

By the law of large numbers, we can approximate integrals using samples:

$$\mathbb{E}\left[h(\mathbf{x})\right] = \int h(\mathbf{x})p(\mathbf{x})d\mathbf{x} \approx \frac{1}{n}\sum_{i=1}^{n} h(\mathbf{x}_i), \quad \mathbf{x}_i \overset{\text{i.i.d.}}{\sim} p(\mathbf{x}_i),\ i = 1, \ldots, n$$

With samples from $p(\mathbf{x}, y)$, we can estimate E_{new}!

Note: Important that samples come from real world, "in-production" distribution.

Training data are (or should be) n samples from $p(\mathbf{x}, y)$.

Approximate $E_{\text{new}} \overset{?}{\approx} E_{\text{train}}$? **NO!**

Training data is part of the predictor $\widehat{y}(\mathbf{x}; \mathcal{T})$!

Lecture 4 – Discriminant Analysis, k-Nearest Neighbors



UPPSALA
UNIVERSITET

Sebastian Mair
https://smair.github.io/
Department of Information Technology
Uppsala University

Course webpage

Summary of Lecture 3 (I/VI)

The classification problem amounts to modeling the relationship between the input $\mathbf{x}$ and a **categorical output** y, i.e., the output belongs to one out of M distinct **classes**.

A **classifier** is a prediction model $\widehat{y}(\mathbf{x})$ that maps any input $\mathbf{x}$ into a predicted class $\widehat{y} \in \{1, \ldots, M\}$.

Common classifier predicts each input as belonging to the **most likely class** according to the conditional probabilities

$$p(y = m \,|\, \mathbf{x}) \text{ for } m \in \{1, \ldots, M\}.$$

Summary of Lecture 3 (II/VI)

For binary (two-class) classification, $y \in \{-1, 1\}$, the **logistic regression model** is

$$p(y = 1 \,|\, \mathbf{x}) \approx g(\mathbf{x}) = \frac{e^{\theta^{\mathsf{T}}\mathbf{x}}}{1 + e^{\theta^{\mathsf{T}}\mathbf{x}}}.$$

The model parameters θ are found by maximum likelihood by (numerically) maximizing the log-likelihood function,

$$\log \ell(\theta) = -\sum_{i=1}^{n} \log\left(1 + e^{-y_i \theta^{\mathsf{T}}\mathbf{x}_i}\right).$$

Summary of Lecture 3 (III/VI)

Using the common classifier, we get the prediction model

$$\widehat{y}(\mathbf{x}) = \begin{cases} 1 & \text{if } g(\mathbf{x}) > 0.5 \\ -1 & \text{otherwise.} \end{cases} \quad \Leftrightarrow \widehat{\theta}^{\mathsf{T}}\mathbf{x} > 0$$

This attempts to **minimize the total misclassification error**.

More generally, we can use

$$\widehat{y}(\mathbf{x}) = \begin{cases} 1 & \text{if } g(\mathbf{x}) > r \\ -1 & \text{otherwise,} \end{cases}$$

where $0 \leq r \leq 1$ is a **user chosen threshold**.

Summary of Lecture 3 (IV/VI)

Confusion matrix:

		Predicted condition $\widehat{y}=0$	Predicted condition $\widehat{y}=1$	Total
True condition	$y=0$	**TN**	**FP**	**N**
	$y=1$	**FN**	**TP**	**P**
	Total	**N***	**P***	

For the classifier

$$\widehat{y}(\mathbf{x}) = \begin{cases} 1 & \text{if } g(\mathbf{x}) > r \\ -1 & \text{otherwise,} \end{cases}$$

the numbers in the confusion matrix will **depend on the threshold** r.

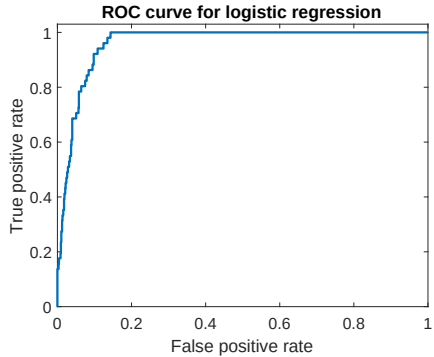
- Decreasing r $\Rightarrow$ **TN**, **FN** decrease and **FP**, **TP** increase.
- Increasing r $\Rightarrow$ **TN**, **FN** increase and **FP**, **TP** decrease.

Summary of Lecture 3 (V/VI)

Some commonly used performance measures are:

- True positive rate: $\mathbf{TPR} = \dfrac{\mathbf{TP}}{\mathbf{P}} = \dfrac{\mathbf{TP}}{\mathbf{FN} + \mathbf{TP}} \in [0, 1]$
- False positive rate: $\mathbf{FPR} = \dfrac{\mathbf{FP}}{\mathbf{N}} = \dfrac{\mathbf{FP}}{\mathbf{FP} + \mathbf{TN}} \in [0, 1]$
- Precision: $\mathbf{Prec} = \dfrac{\mathbf{TP}}{\mathbf{P^*}} = \dfrac{\mathbf{TP}}{\mathbf{FP} + \mathbf{TP}} \in [0, 1]$

Summary of Lecture 3 (VI/VI)



- **ROC: plot of TPR vs. FPR** as r ranges from 0 to 1.[1]
- **Area Under Curve (AUC):** condensed performance measure for the classifier, taking all possible thresholds into account.

[1] Possible to draw ROC curves based on other tuning parameters as well.

UPPSALA UNIVERSITET

Contents – Lecture 4

1. Summary of lecture 3
2. Generative models
3. Linear Discriminant Analysis (LDA)
4. Quadratic Discriminant Analysis (QDA)
5. A nonparametric classifier – k-Nearest Neighbors (kNN)

Generative models

A **discriminative model** describes how the output y is generated, i.e., $p(y \mid \mathbf{x})$.

A **generative model** describes how both the output y and the input $\mathbf{x}$ is generated via $p(y, \mathbf{x})$.

Predictions can be derived using laws of probability

$$p(y \mid \mathbf{x}) = \frac{p(y, \mathbf{x})}{p(\mathbf{x})} = \frac{p(y)p(\mathbf{x} \mid y)}{\int_y p(y, \mathbf{x})}.$$

Generative models for classification

We have built classifiers based on models $g_m(\mathbf{x}_\star)$ of the conditional probabilities $p(y = m \,|\, \mathbf{x})$.

A generative model for classification can be expressed as

$$p(y = m \,|\, x) = \frac{p(y = m)p(\mathbf{x} \,|\, y = m)}{\sum_{m=1}^{M} p(y = m)p(\mathbf{x} \,|\, y = m)}$$

- $p(y = m)$ denotes the **marginal** (prior) probability of class $m \in \{1, \ldots, M\}$.
- $p(\mathbf{x} \,|\, y = m)$ denotes the **conditional** probability density of $\mathbf{x}$ for an observation from class m.

Multivariate Gaussian density

The p-dimensional Gaussian (normal) probability density function with mean vector $\boldsymbol{\mu}$ and covariance matrix $\boldsymbol{\Sigma}$ is,

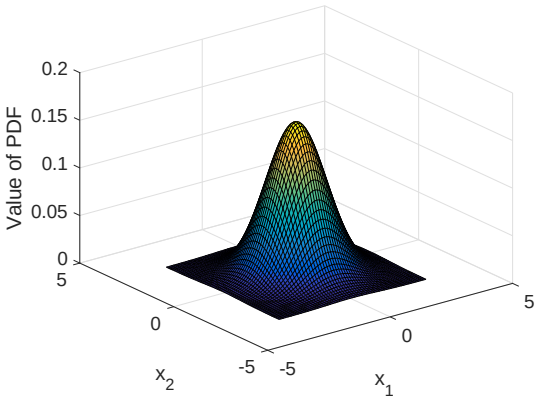
$$\mathcal{N}(\mathbf{x} \,|\, \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{p/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right),$$

where $\boldsymbol{\mu}$: $p \times 1$ vector and $\boldsymbol{\Sigma}$: $p \times p$ positive definite matrix.

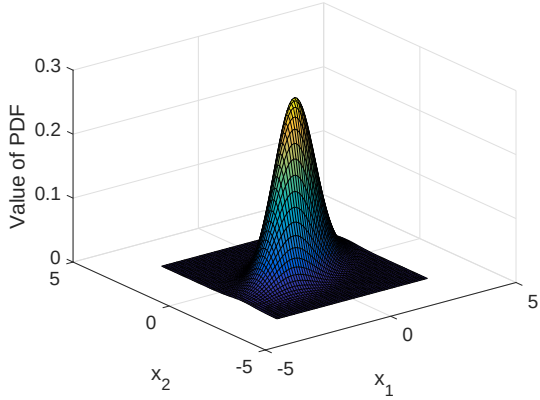
Let $\mathbf{x} = (x_1, \ldots, x_p)^{\mathsf{T}} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$,

- μ_j is the mean of x_j,
- Σ_{jj} is the variance of x_j,
- Σ_{ij} $(i \neq j)$ is the covariance between x_i and x_j.

Multivariate Gaussian density



$$\mathbf{\Sigma} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$



$$\mathbf{\Sigma} = \begin{pmatrix} 1 & 0.4 \\ 0.4 & 0.5 \end{pmatrix}$$

Linear discriminant analysis

We need

1. The prior class probabilities $\pi_m \triangleq p(y = m)$, $m \in \{1, \ldots, M\}$.
2. The conditional probability densities of the input $\mathbf{x}$, $f_m(\mathbf{x}) \triangleq p(\mathbf{x} \mid y = m)$, for each class m.

This gives us the model

$$g_m(\mathbf{x}) = \frac{\pi_m f_m(\mathbf{x})}{\sum_{m=1}^{M} \pi_m f_m(\mathbf{x})}.$$

Linear discriminant analysis

For the **first task**, a natural *estimator* is the proportion of training samples in the mth class

$$\widehat{\pi}_m = \frac{1}{n} \sum_{i=1}^{n} \mathbb{I}\{y_i = m\} = \frac{n_m}{n},$$

where n is the size of the training set and n_m the number of training samples of class m.

Linear discriminant analysis

For the **second task**, a simple model is to *assume* that $f_m(x)$ is a multivariate normal density with mean vector $\boldsymbol{\mu}_m$ and covariance matrix $\boldsymbol{\Sigma}_m$,

$$f_m(\mathbf{x}) = \frac{1}{(2\pi)^{p/2}|\boldsymbol{\Sigma}_m|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_m)^\mathsf{T}\boldsymbol{\Sigma}_m^{-1}(\mathbf{x}-\boldsymbol{\mu}_m)\right).$$

If we further *assume* that all classes share the same covariance matrix,

$$\boldsymbol{\Sigma} \stackrel{\text{def}}{=} \boldsymbol{\Sigma}_1 = \cdots = \boldsymbol{\Sigma}_M,$$

the remaining parameters of the model are

$$\boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \ldots, \boldsymbol{\mu}_M, \boldsymbol{\Sigma}.$$

Linear discriminant analysis

These parameters are naturally estimated as the (within class) sample means and sample covariance, respectively:

$$\widehat{\boldsymbol{\mu}}_m = \frac{1}{n_m} \sum_{i:y_i=m} \mathbf{x}_i, \qquad m = 1, \ldots, M,$$

$$\widehat{\boldsymbol{\Sigma}} = \frac{1}{n-M} \sum_{m=1}^{M} \sum_{i:y_i=m} (\mathbf{x}_i - \widehat{\boldsymbol{\mu}}_m)(\mathbf{x}_i - \widehat{\boldsymbol{\mu}}_m)^{\mathsf{T}}.$$

Modeling the class probabilities using the normal assumptions and these parameter estimates is referred to as ***Linear Discriminant Analysis (LDA)***.

Linear discriminant analysis, summary

The LDA classifier assigns a test input $\mathbf{x}$ to class m for which,

$$\widehat{\delta}_m(\mathbf{x}) = \mathbf{x}^\mathsf{T}\widehat{\mathbf{\Sigma}}^{-1}\widehat{\boldsymbol{\mu}}_m - \frac{1}{2}\widehat{\boldsymbol{\mu}}_m^\mathsf{T}\widehat{\mathbf{\Sigma}}^{-1}\widehat{\boldsymbol{\mu}}_m + \log\widehat{\pi}_m$$

is largest, where

$$\widehat{\pi}_m = \frac{n_m}{n}, \qquad m = 1, \ldots, M,$$

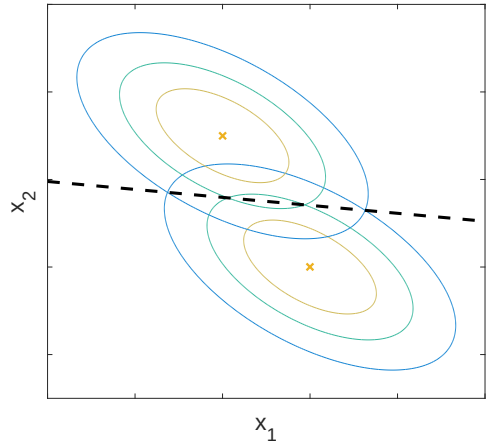
$$\widehat{\boldsymbol{\mu}}_m = \frac{1}{n_m}\sum_{i:y_i=m}\mathbf{x}_i, \qquad m = 1, \ldots, M,$$

$$\widehat{\mathbf{\Sigma}} = \frac{1}{n-M}\sum_{m=1}^{M}\sum_{i:y_i=m}(\mathbf{x}_i - \widehat{\boldsymbol{\mu}}_m)(\mathbf{x}_i - \widehat{\boldsymbol{\mu}}_m)^\mathsf{T}.$$

LDA is a ***linear classifier*** (its decision boundaries are linear).

ex) LDA decision boundary

Illustration of LDA decision boundary – the level curves of two Gaussian PDFs with **the same covariance matrix** intersect along a straight line, $\widehat{\delta}_1(\mathbf{x}) = \widehat{\delta}_2(\mathbf{x})$.



ex) Simple spam filter

We will use LDA to construct a ***simple*** spam filter:

- **Output:** $y \in \{\texttt{spam}, \texttt{ham}\}$
- **Input:** $\mathbf{x} = 57$-dimensional vector of "features" extracted from the email
  - Frequencies of 48 predefined words (`make`, `address`, `all`, ...)
  - Frequencies of 6 predefined characters (`;`, `(`, `[`, `!`, `$`, `#`)
  - Average length of uninterrupted sequences of capital letters
  - Length of longest uninterrupted sequence of capital letters
  - Total number of capital letters in the e-mail
- Dataset consists of $4,601$ emails classified as either `spam` or `ham` (split into 75 % training and 25 % testing)

UCI Machine Learning Repository — Spambase Dataset
(`https://archive.ics.uci.edu/ml/datasets/Spambase`)

ex) Simple spam filter

LDA confusion matrix (test data):

	$\widehat{y}=0$	$\widehat{y}=1$
$y=0$	667	23
$y=1$	102	358

Table: Threshold $r=0.5$

	$\widehat{y}=0$	$\widehat{y}=1$
$y=0$	687	3
$y=1$	237	223

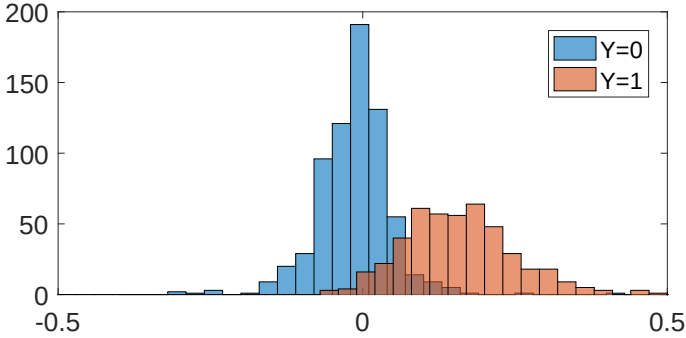
Table: Threshold $r=0.9$

ex) Simple spam filter

A **linear** classifier uses a **linear decision boundary** for separating the two classes as much as possible.

In $\mathbb{R}^p$ this is a $(p-1)$-dimensional hyperplane. How can we visualize the classifier?

One way: project the (labeled) test inputs onto the normal of the decision boundary.



Gmail's spam filter

Official Gmail blog July 9, 2015:

"... the spam filter now uses an artificial neural network to detect and block the especially sneaky spam—the kind that could actually pass for wanted mail." — Sri Harsha Somanchi, Product Manager

Official Gmail blog February 6, 2019:

"With TensorFlow, we are now blocking around 100 million additional spam messages every day" — Neil Kumaran, Product Manager, Gmail Security

Quadratic discriminant analysis

Do we have to assume a common covariance matrix?

No! Estimating a separate covariance matrix for each class leads to an alternative method, Quadratic Discriminant Analysis (QDA).

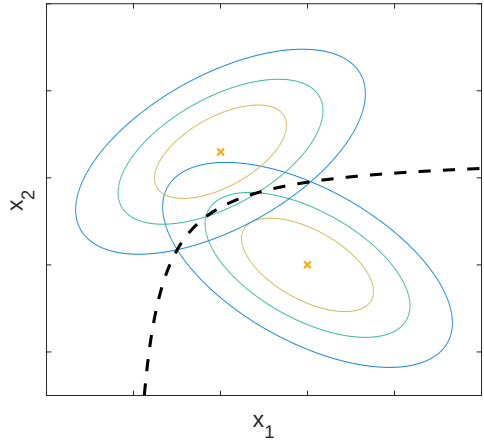
Whether we should choose LDA or QDA has to do with the **bias-variance trade-off**, i.e., the risk of **over- or underfitting**.

Compared to LDA, QDA...

- has more parameters.
- is more flexible (lower bias).
- has higher risk of overfitting (larger variance).

ex) QDA decision boundary

Illustration of QDA decision boundary – the level curves of two Gaussian PDFs with **different covariance matrices** intersect along a curved (quadratic) line.



Parametric and nonparametric models

So far we have looked at a few **parametric models**,

- linear regression,
- logistic regression,
- LDA and QDA,

all of which are parametrized by a ***fixed-dimensional parameter***.

Non-parametric models instead allow the flexibility of the model to grow with the amount of available data.

- ▲ can be ***very*** flexible (= low bias)
- ▼ can suffer from overfitting (high variance)
- ▼ can be compute and memory intensive

As always, the ***bias-variance trade-off*** is key also when working with non-parametric models.

k-NN

The **k-nearest neighbors (k-NN)** classifier is a simple non-parametric method.

Given training data $\mathcal{T} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, for a test input $\mathbf{x}_\star$,

1. Identify the set

$$R_\star = \{i : \mathbf{x}_i \text{ is one of the } k \text{ training data points closest to } \mathbf{x}_\star\}.$$

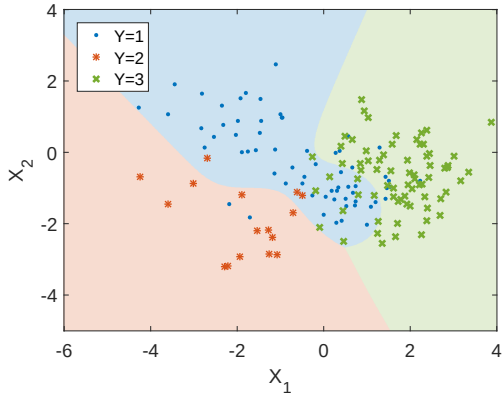
2. Classify $\mathbf{x}_\star$ according to a majority vote within the neighborhood $R_\star$, i.e., assign $\mathbf{x}_\star$ to class m for which

$$\sum_{i \in R_\star} \mathbb{I}\{y_i = m\}$$

is largest.

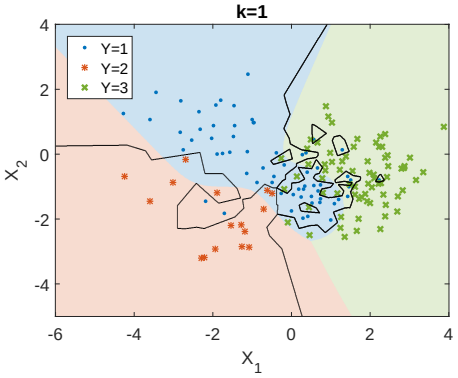
ex) k-NN on a toy model

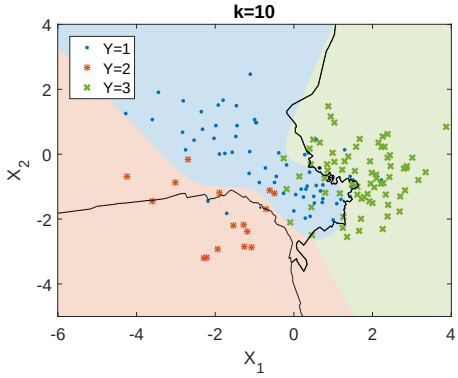
We illustrate the k-NN classifier on a synthetic example where the optimal classifier is known (colored regions).



ex) k-NN on a toy model

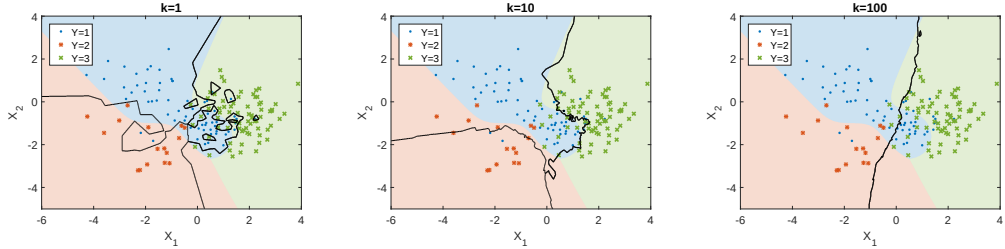
The decision boundaries for the k-NN classifier are shown as black lines.





UPPSALA UNIVERSITET

ex) k-NN on a toy model



The choice of the ***tuning parameter*** k controls the model flexibility:

- Small $k \Rightarrow$ small bias, large variance.
- Large $k \Rightarrow$ large bias, small variance.

A few concepts to summarize lecture 4

Generative model: A model that describes both the output y and the input $\mathbf{x}$.

Linear discriminant analysis (LDA): A classifier based on the assumption that the distribution of the input $\mathbf{x}$ is multivariate Gaussian for each class, with different means but the same covariance. LDA is a linear classifier.

Quadratic discriminant analysis (QDA): Same as LDA, but where a different covariance matrix is used for each class. QDA is *not* a linear classifier.

Non-parametric model: A model where the flexibility is allowed to grow with the amount of available training data.

k-NN classifier: A simple non-parametric classifier based on classifying a test input $\mathbf{x}$ according to the class labels of the k training samples nearest to x.

Lecture 5 – Cross-validation and the bias-variance trade-off



UPPSALA
UNIVERSITET

Jens Sjölund
https://jsjol.github.io/
Department of Information Technology
Uppsala University

Course webpage

Summary of Lecture 4 (I/III)

Linear Discriminant Analysis (LDA) models the conditional class probabilities as

$$p(y = m \,|\, \mathbf{x}) = \frac{p(\mathbf{x} \,|\, y = m)p(y = m)}{\sum_{j=1}^{M} p(\mathbf{x} \,|\, y = j)p(y = j)}.$$

where

- $p(y = m) = \pi_m$ is the *prior* probability of class m.
- $p(\mathbf{x} \,|\, y = m) = \mathcal{N}(\mathbf{x} \,|\, \boldsymbol{\mu}_m, \boldsymbol{\Sigma})$ is the probability density of $\mathbf{x}$ for an observation that comes from the mth class.

The **parameters** are: $\pi_1, \ldots, \pi_M, \boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_M, \boldsymbol{\Sigma}$

Summary of Lecture 4 (II/III)

The parameters are estimated as the class frequencies and (within class) sample means and covariances, respectively,

$$\widehat{\pi}_m = \frac{n_m}{n} \qquad m = 1, \ldots, M,$$

$$\widehat{\boldsymbol{\mu}}_m = \frac{1}{n_m} \sum_{i:y_i=m} \mathbf{x}_i, \qquad m = 1, \ldots, M,$$

$$\widehat{\boldsymbol{\Sigma}} = \frac{1}{n-M} \sum_{m=1}^{M} \sum_{i:y_i=j} (\mathbf{x}_i - \widehat{\boldsymbol{\mu}}_m)(\mathbf{x}_i - \widehat{\boldsymbol{\mu}}_m)^{\mathsf{T}}.$$

The LDA classifier assigns a test input $\mathbf{x}_\star$ to class m with the maximum predicted probability $p(y = m \,|\, \mathbf{x}_\star)$. LDA is a **linear classifier**.

Summary of Lecture 4 (III/III)

Parametric models are specified using a fixed-dimensional vector of parameters.

Non-parametric models allow the flexibility of the model to grow with data.

One non-parametric model is the k-**nearest neighbor classifier**.

Given training data $\mathcal{T} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, for a test input $\mathbf{x}_\star$,

1. Identify the k training inputs $\mathbf{x}_i$ closest to $\mathbf{x}_\star$.
2. Classify $\mathbf{x}_\star$ according to a majority vote among these k training samples.

Evaluating a supervised machine learning method

Evaluating performance

Training/learning: Minimizes the average loss on training data. Loss functions differ between methods and the values can not directly be compared.

We would like to evaluate already trained models with respect to hyperparameters (eg. regularization parameter γ or k in k-NN) but also different methods (LDA, logistic regression, neural net).

Error function

$$E\big(\widehat{y}(\mathbf{x})\big), y) = \begin{cases} \mathbb{I}\{\widehat{y}(\mathbf{x}) = y\} & \text{(classification)} \\ \big(\widehat{y}(\mathbf{x}) - y\big)^2 & \text{(regression)} \end{cases}$$

Note: Error function not necessarily the same as loss function.

Evaluating performance

Let $\mathcal{T} = \{(x_i, y_i)\}_{i=1}^n$ be training data.

Training error

$$E_{\text{train}} = \frac{1}{n} \sum_{i=1}^{n} E\big(\widehat{y}(\mathbf{x}_i), y_i\big)$$

Measures how well a predictor $\widehat{y}$ performs on training data, but we are interested in new data. Let $p(\mathbf{x}, y)$ be the joint distribution over data.

Expected new data error

$$E_{\text{new}} = \mathbb{E}_\star \left[E\big(\widehat{y}(x_\star), y_\star\big) \right] = \int E\big(\widehat{y}(x_\star), y_\star\big) p(x_\star, y_\star) dx_\star dy_\star$$

Impossible to compute since $p(\mathbf{x}, y)$ is unknown, but minimizing E_{new} is our ultimate goal.

Maybe we can learn it?

Approximating integrals

By the law of large numbers, we can approximate integrals using samples:

$$\mathbb{E}\left[h(\mathbf{x})\right] = \int h(\mathbf{x})p(\mathbf{x})d\mathbf{x} \approx \frac{1}{n}\sum_{i=1}^{n} h(\mathbf{x}_i), \quad \mathbf{x}_i \overset{\text{i.i.d.}}{\sim} p(\mathbf{x}_i),\ i = 1, \ldots, n$$

With samples from $p(\mathbf{x}, y)$, we can estimate E_{new}!

Note: Important that samples come from real world, "in-production" distribution.

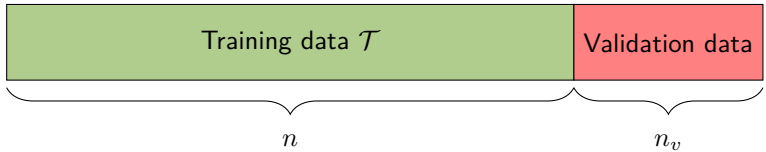
Training data are (or should be) n samples from $p(\mathbf{x}, y)$.

Approximate $E_{\text{new}} \overset{?}{\approx} E_{\text{train}}$? **NO!**

Training data is part of the predictor $\widehat{y}(\mathbf{x}; \mathcal{T})$!

Estimating E_{new}: hold-out validation data

Split data in **training data** and **validation data**.



Hold-out validation error:

$$E_{\text{new}} \approx E_{\text{hold-out}} = \frac{1}{n_v} \sum_{i=1}^{n_v} E\big(\widehat{y}(\mathbf{x}_i; \mathcal{T}), y_i\big)$$

\+ Simple
! Good estimate of E_{new} requires the validation set to be large
! Good predictor $\widehat{y}$ requires the training set to be large
– Not all data is used for learning

Always split **randomly** between training and validation data!

Estimating E_{new}: k-fold cross-validation

Split data in k batches and hold out batch ℓ when estimating model. Use batch ℓ to estimate E_{new} and average over all k estimates. Estimate final model using whole dataset.



k-fold cross-validation error

$$E_{\text{new}} \approx E_{\text{k-fold}} = \frac{1}{k} \sum_{\ell=1}^{k} E_{\text{hold-out}}^{(\ell)}$$

+ Gives a better estimate of E_{new}
– Computationally more demanding than the hold-out data approach

Using a test set

An important use of $E_{\text{k-fold}}$ is to choose between models or select hyperparameters (k in k-NN or λ in regularization). If a good estimate of E_{new} is important:

We can no longer use $E_{\text{k-fold}}$ to estimate E_{new}!

Set aside a test set and use **only** to estimate E_{new}.

Flavors of cross-validation

- k-fold cross-validation: Typically $k \approx 10$
- Leave-one-out cross-validation: k-fold cross-validation with $k = n$
- Monte Carlo cross-validation: Random selection of validation set at each iteration

Understanding E_{new}

Collecting data is a random process where $\mathcal{T}$ is sampled from $p(\mathbf{x}, y)$. Because $\mathcal{T}$ is random, so is our learned model $\widehat{y}(\mathbf{x}; \mathcal{T})$.

To better understand the behavior of E_{new}, we need to introduce

$$\bar{E}_{\text{train}} = \mathbb{E}_{\mathcal{T}}\left[E_{\text{train}}\right],$$
$$\bar{E}_{\text{new}} = \mathbb{E}_{\mathcal{T}}\left[E_{\text{new}}\right],$$

where $\mathbb{E}_{\mathcal{T}}[\cdot]$ is the average over **training data** $\mathcal{T}$.

Note: k-fold cross-validation estimates $\bar{E}_{new}$ rather than E_{new}.

It usually holds that

$$\bar{E}_{\text{train}} < \bar{E}_{\text{new}}.$$

On average, a method usually performs better on training data than new data.

Model complexity

Since $\bar{E}_{\text{train}} < \bar{E}_{\text{new}}$, define generalization gap:

$$\bar{E}_{\text{new}} = \bar{E}_{\text{train}} + \textbf{generalization gap}$$

A models ability to adapt to patterns in the data, we call the **model complexity**[1].

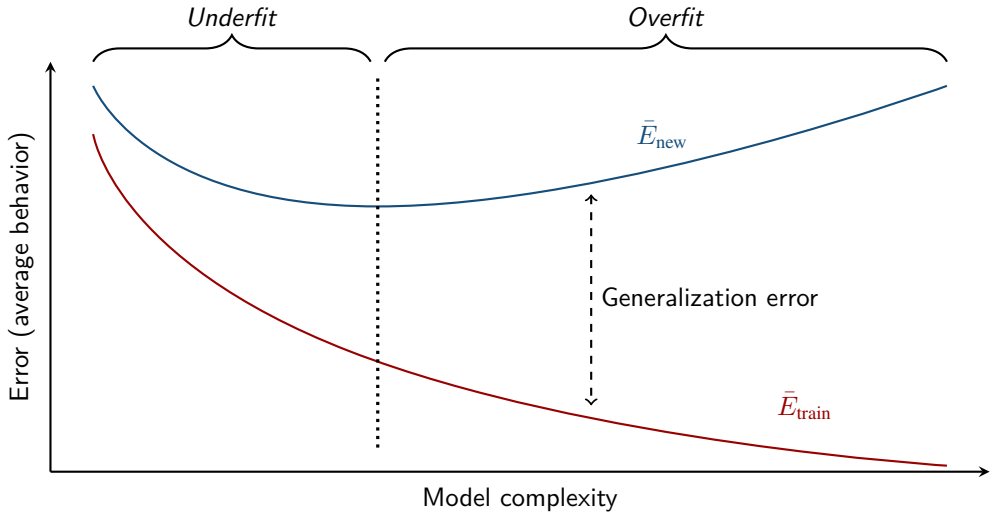
Model complexity ↗
- ↘ $\bar{E}_{\text{train}}$
- ↗ Generalization gap

Model complexity ↘
- ↗ $\bar{E}_{\text{train}}$
- ↘ Generalization gap

$\bar{E}_{\text{new}}$ usually attains a minimum at some intermediate complexity.

[1] A formal definition is possible. We will not do that in this course.

Model complexity, $\bar{E}_{\text{train}}$ and $\bar{E}_{\text{new}}$



Example: cross-validation for model selection



All models are wrong, but some are useful. — George Box

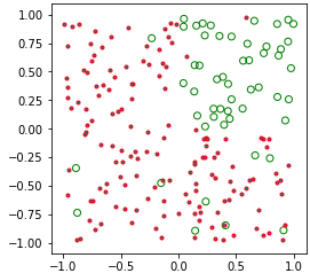
UPPSALA UNIVERSITET

Example: cross-validation for model selection

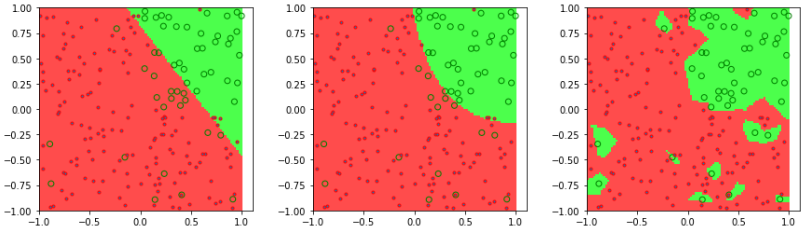
Binary classification, $p = 2$.
Evaluate the following methods

- logistic regression
- QDA
- k-NN

We would like to pick the method with lowest E_{new}.
Since we only have the data, we can only *estimate* E_{new}.



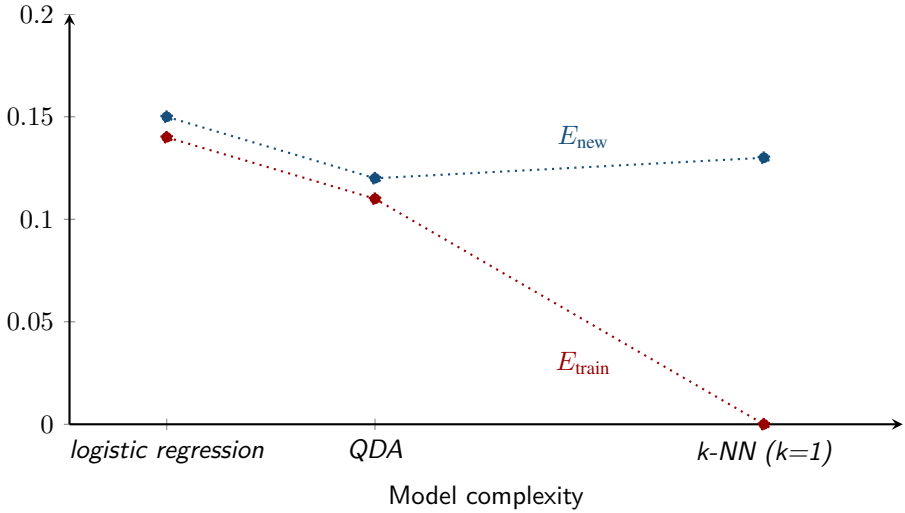
Example: cross-validation for model selection



	logistic regression	QDA	k-NN ($k = 1$)
E_{train}	0.14	0.11	0.0
E_{new}[2]	0.15	0.12	0.13

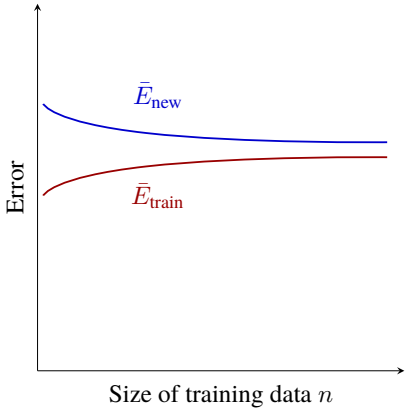
[2] Estimated using cross-validation

Example: cross-validation for model selection

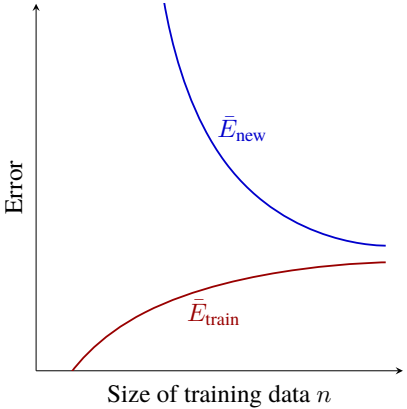


Generalization error and training data size

In general, more training data n decreases the generalization error.
However, $\bar{E}_{\text{train}}$ typically increases with n.



Low model complexity



High model complexity

Digging even deeper to understand E_{new}

Let us focus on the regression setting.
Assume that "the real world" works as

$$y = f_0(\mathbf{x}) + \epsilon, \text{ where } \begin{cases} \epsilon \text{ random, independent of } \mathbf{x} \\ \mathbb{E}[\epsilon] = 0, \\ \mathbb{E}\left[\epsilon^2\right] = \sigma^2. \end{cases}$$

Denote the **average trained model**

$$\bar{f}(\mathbf{x}) \triangleq \mathbb{E}_{\mathcal{T}}\left[\widehat{y}(\mathbf{x}; \mathcal{T})\right].$$

The average model if we could re-train the model on new data an infinite number of times.

Bias-variance decomposition

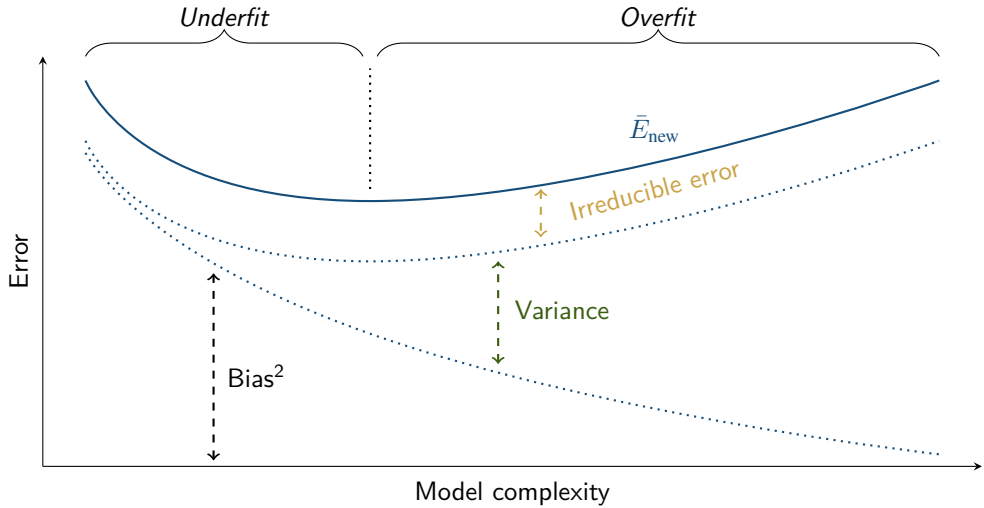
$$
\begin{aligned}
\bar{E}_{\text{new}} &= \mathbb{E}_{\mathcal{T}}\left[\mathbb{E}_{\star}\left[\left(\widehat{y}(\mathbf{x}_{\star};\mathcal{T}) - y_{\star}\right)^2\right]\right] = \mathbb{E}_{\star}\left[\mathbb{E}_{\mathcal{T}}\left[\left(\widehat{y}(\mathbf{x}_{\star};\mathcal{T}) - f_0(\mathbf{x}_{\star}) - \epsilon\right)^2\right]\right] \\
&= \mathbb{E}_{\star}\left[\mathbb{E}_{\mathcal{T}}\left[\left(\widehat{y}(\mathbf{x}_{\star};\mathcal{T}) - \bar{f}(\mathbf{x}_{\star}) + \bar{f}(\mathbf{x}_{\star}) - f_0(\mathbf{x}_{\star}) - \epsilon\right)^2\right]\right] \\
&= \underbrace{\mathbb{E}_{\star}\left[\mathbb{E}_{\mathcal{T}}\left[\left(\widehat{y}(\mathbf{x}_{\star};\mathcal{T}) - \bar{f}(\mathbf{x}_{\star})\right)^2\right]\right]}_{\text{Variance}} + \underbrace{\mathbb{E}_{\star}\left[\left(\bar{f}(\mathbf{x}_{\star}) - f_0(\mathbf{x}_{\star})\right)^2\right]}_{\text{Bias}^2} + \underbrace{\sigma^2}_{\substack{\text{Irreducible} \\ \text{error}}}
\end{aligned}
$$

Technical interpretation:

- **Bias²** $\mathbb{E}_{\star}\left[\left(\bar{f}(\mathbf{x}_{\star}) - f_0(\mathbf{x}_{\star})\right)^2\right]$: The part of $\bar{E}_{\text{new}}$ that is due to the fact that the model cannot represent the true f_0.
- **Variance** $\mathbb{E}_{\star}\left[\mathbb{E}_{\mathcal{T}}\left[\left(\widehat{y}(\mathbf{x}_{\star};\mathcal{T}) - \bar{f}(\mathbf{x}_{\star})\right)^2\right]\right]$: The part of $\bar{E}_{\text{new}}$ that is due to the variability in the training dataset.

Intuitive interpretation:

- **Bias**: The inability of a method to describe the complicated patterns we would like it to describe. Low model complexity.
- **Variance**: How sensitive a method is to the training data. High model complexity.



Finding a balanced fit (neither over- nor underfit) is called the **the bias-variance tradeoff**.

Regression example

The data ($n = 10$) comes from

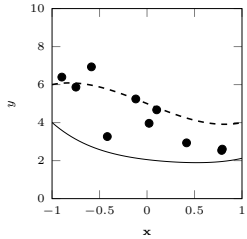
$$y = 5 - 2x + x^3 + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 1),$$

and our regression model is

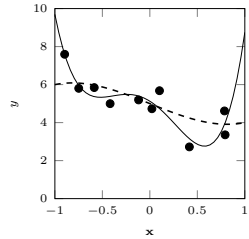
$$y = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4 + \varepsilon.$$

We use ridge regression to tune model complexity/bias-variance.

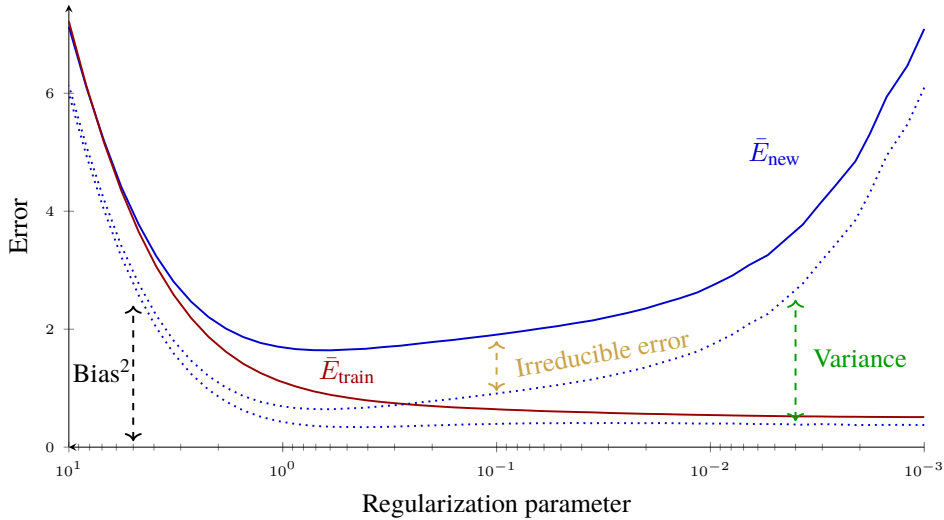
$\gamma = 10$



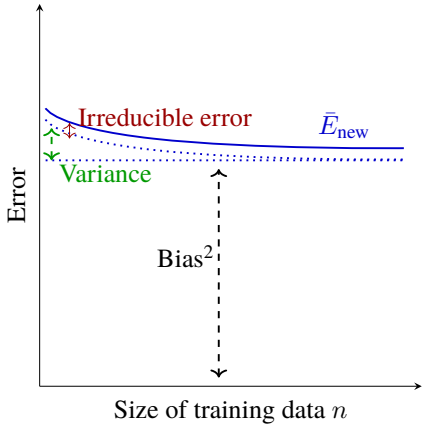
$\gamma = 0.001$



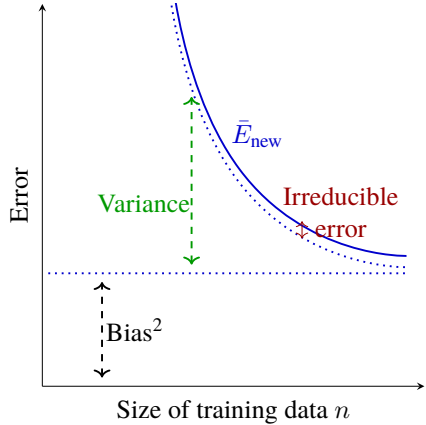
Regression example



Bias, variance and training data size



Low model complexity



High model complexity

Make the tradeoff

Some methods for **decreasing the model complexity/increasing the bias/ decreasing the variance**:

- Increase k in k-NN
- Regularization
- Bagging
- Early stopping (for methods trained using optimization, notably deep learning)
- Dropout (deep learning)

Warning! θ_0 in linear regression (and later deep learning) is sometimes called "bias term". That is **completely unrelated** to bias in this context.

A few concepts to summarize lecture 5

$E(y, \widehat{y})$**:** Error function which compares predictions $\widehat{y}$ to true output y: MSE for regression, misclassification for classification.

E_{train}**:** The training data error (E_{train} small $=$ the method fits the training data well).

E_{new}**:** The expected new data error; how well a method will perform when faced with an endless stream of new data.

Cross-validation: A method for estimating E_{new} using the training data.

Model complexity: How prone a method is to adapt to complicated patterns in the training data.

Overfitting: When a given method yields a smaller E_{train} and larger E_{new} than a model with lower model complexity would have done. That happens because the method/model is capturing patterns in the training data caused by random chance rather than true properties of the underlying function.

Bias: The inability of a method to describe the true patterns in the classification or regression problem. Low model complexity.

Variance: Sensitivity to random effects (noise) in the training data. High model complexity.

Lecture 6 – Tree-based methods: Bagging, Boosting and Random Forests



UPPSALA
UNIVERSITET

Jens Sjölund
https://jsjol.github.io/
Department of Information Technology
Uppsala University

Course webpage

Summary of Lecture 5 (I/IV)

When **choosing models/methods**, we are interested in how well they will perform when **faced with new unseen data**.

The new data error

$$E_{\text{new}} \triangleq \mathbb{E}_\star \left[E(\widehat{y}(\mathbf{x}_\star; \mathcal{T}), y_\star) \right]$$

describes how well a method (which is trained using data set $\mathcal{T}$) will perform "in production".

E is for instance mean squared error (regression) or misclassification (classification).

The overall goal in supervised machine learning is to achieve small E_{new}.

$E_{\text{train}} \triangleq \frac{1}{n} \sum_{i=1}^{n} E(\widehat{y}(\mathbf{x}_i; \mathcal{T}), y_i)$ is the training data error.
Not a good estimate of E_{new}.

Summary of Lecture 5 (II/IV)

Two methods for estimating E_{new}:

1. **Hold-out validation data approach:** Randomly split the data into a **training set** and a **hold-out validation set**. Learn the model using the training set. Estimate E_{new} using the hold-out validation set.
2. k**-fold cross-validation:** Randomly split the data into k parts (or **folds**) of roughly equal size.
   a) The first fold is kept aside as a validation set and the model is learned using only the remaining $k-1$ folds. E_{new} is estimated on the validation set.
   b) The procedure is repeated k times, each time a different fold is treated as the validation set.
   c) The average of all k estimates is taken as the final estimate of E_{new}.

Summary of Lecture 5 (III/IV)

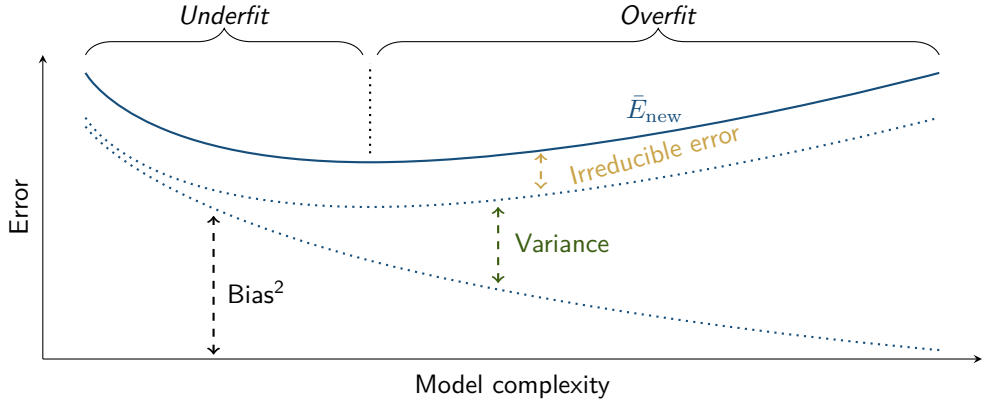
$$\bar{E}_{\text{new}} = \underbrace{\mathbb{E}_\star\left[\left(\bar{f}(\mathbf{x}_\star) - f_0(\mathbf{x}_\star)\right)^2\right]}_{\text{Bias}^2} + \underbrace{\mathbb{E}_\star\left[\mathbb{E}_{\mathcal{T}}\left[\left(\widehat{y}(\mathbf{x}_\star;\mathcal{T}) - \bar{f}(\mathbf{x}_\star)\right)^2\right]\right]}_{\text{Variance}} + \underbrace{\sigma^2}_{\substack{\text{Irreducible} \\ \text{error}}}$$

$$\text{where} \quad \bar{f}(\mathbf{x}) = \mathbb{E}_{\mathcal{T}}\left[\widehat{y}(\mathbf{x}_\star;\mathcal{T})\right]$$

- **Bias**: The inability of a method to describe the complicated patterns we would like it to describe.
- **Variance**: How sensitive a method is to the training data.

The more prone a model is to adapt to complicated pattern in the data, the higher the **model complexity** (or model flexibility).

Summary of Lecture 5 (IV/IV)



Finding a balanced fit (neither over- nor underfit) is called the **the bias-variance tradeoff**.

Contents – Lecture 6

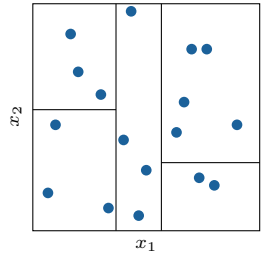
1. Tree-based methods
2. Bagging – *a general variance reduction technique*
3. Random forests
4. Boosting

Tree-based methods

The idea behind tree-based methods

In both regression and classification settings we seek a function $\widehat{y}(\mathbf{x})$ which maps the input $\mathbf{x}$ into a prediction.

One **flexible** way of designing this function is to partition the input space into disjoint regions and fit a simple model in each region.



- **Classification:** Majority vote within the region.
- **Regression:** Mean of training data within the region.

Finding the partition

The key challenge in using this strategy is to find a good partition.

Even if we restrict our attention to seemingly simple regions (e.g. "boxes"), finding an **optimal** partition w.r.t. minimizing the training error is **computationally infeasible!**

Instead, we use a "greedy" approach: **recursive binary splitting**.

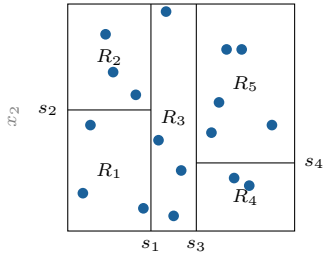
1. Select one input variable x_j and a cut-point s. Partition the input space into two half-spaces,

$$\{\mathbf{x} : x_j < s\} \qquad\qquad \{\mathbf{x} : x_j \geq s\}.$$

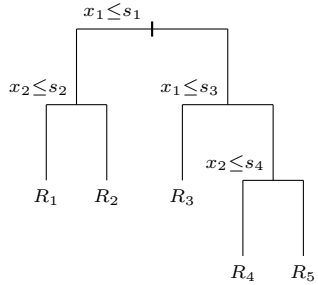
2. Repeat this splitting for each region until some stopping criterion is met (e.g., no region contains more than 5 training data points).

Recursive binary splitting

Partitioning of input space



Tree representation



Regression trees

Once the input space is partitioned into L regions, $R_1, R_2, \ldots, R_L$ the prediction model is

$$\widehat{y}_\star = \sum_{\ell=1}^{L} \widehat{y}_\ell \mathbb{I}\{\mathbf{x}_\star \in R_\ell\},$$

where $\mathbb{I}\{\mathbf{x}_\star \in R_\ell\}$ is the indicator function

$$\mathbb{I}\{\mathbf{x}_\star \in R_\ell\} = \begin{cases} 1 & \text{if } \mathbf{x}_\star \in R_\ell \\ 0 & \text{if } \mathbf{x}_\star \notin R_\ell \end{cases}$$

and $\widehat{y}_\ell$ is a constant prediction within each region.
For regression trees we use

$$\widehat{y}_\ell = \text{average}\{y_i : \mathbf{x}_i \in R_\ell\}$$

Recursive binary splitting for a regression tree

Recursive binary splitting is **greedy** - each split is made in order to minimize the loss **without looking ahead** at future splits.

For any j and s, define

$$R_1(j,s) = \{\mathbf{x} \,|\, x_j < s\} \qquad \text{and} \qquad R_2(j,s) = \{\mathbf{x} \,|\, x_j \geq s\}.$$

We then seek (j,s) that minimize

$$\sum_{i:\mathbf{x}_i \in R_1(j,s)} (y_i - \widehat{y}_1(j,s))^2 + \sum_{i:\mathbf{x}_i \in R_2(j,s)} (y_i - \widehat{y}_2(j,s))^2$$

where

$$\widehat{y}_1 = \text{average}\{y_i : \mathbf{x}_i \in R_1(j,s)\}$$
$$\widehat{y}_2 = \text{average}\{y_i : \mathbf{x}_i \in R_2(j,s)\}$$

This optimization problem is easily solved by "brute force".

Classification trees

Classification trees are constructed similarly to regression trees, but with *two differences*.

Firstly, the class prediction for each region is based on the proportion of data points from each class in that region. Let

$$\widehat{\pi}_{\ell m} = \frac{1}{n_\ell} \sum_{i:\mathbf{x}_i \in R_\ell} \mathbb{I}\{y_i = m\}$$

be the proportion of training data points in the lth region that belong to the mth class. Then we approximate

$$p(y = m \,|\, \mathbf{x}_\star) \approx \sum_{\ell=1}^{L} \widehat{\pi}_{\ell m} \mathbb{I}\{\mathbf{x}_\star \in R_\ell\}$$

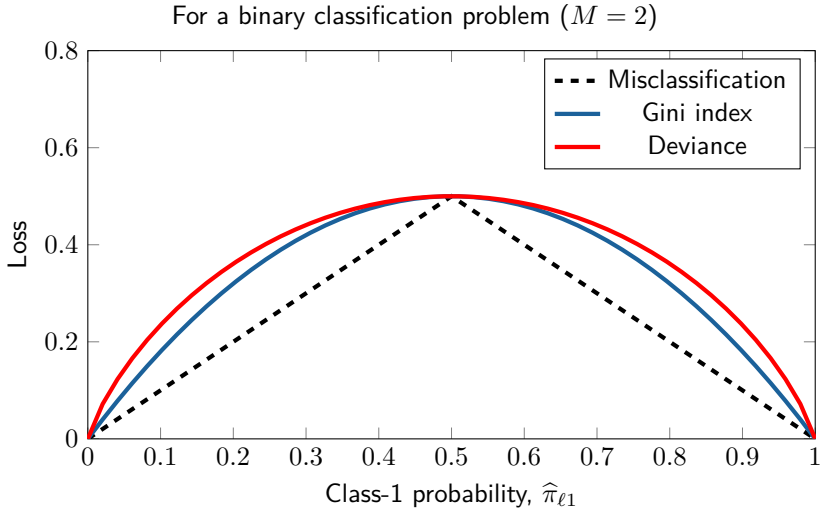
Classification trees

Secondly, the squared loss used to decide the splits needs to be replaced by a measure suitable to categorical outputs.

Three common error measures are,

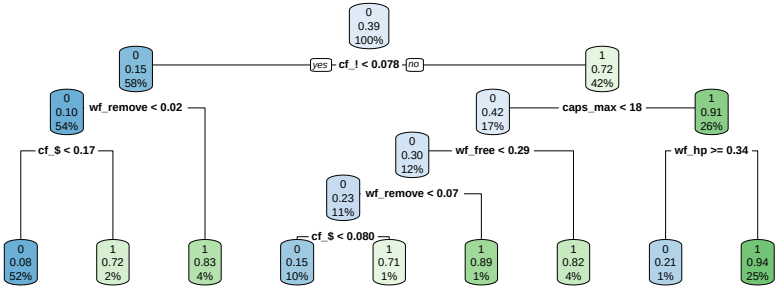
$$\begin{aligned} \text{Misclassification error:} \quad & 1 - \max_m \widehat{\pi}_{\ell m} \\ \text{Entropy/deviance:} \quad & -\sum_{m=1}^{M} \widehat{\pi}_{\ell m} \log \widehat{\pi}_{\ell m} \\ \text{Gini index:} \quad & \sum_{m=1}^{M} \widehat{\pi}_{\ell m}(1 - \widehat{\pi}_{\ell m}) \end{aligned}$$

Classification error measures



ex) Spam data

Classification tree for spam data:



	Tree	**LDA**
Test error:	11.3 %	10.9 %

Improving CART

The performance of (simple) CARTs is often unsatisfactory!

The flexibility/complexity of classification and regression trees (CART) is decided by the tree depth.

- ***!*** To obtain a **small bias** the tree need to be grown deep,
- ***!*** but this results in a **high variance!**

To improve the practical performance:

- **Pruning** – grow a deep tree (small bias) which is then pruned into a smaller one (reduce variance).
- **Ensemble methods** – average or combine multiple trees.

Bagging

Probability detour - Variance reduction by averaging

Let z_b, $b = 1, \ldots, B$ be identically distributed random variables with mean $\mathbb{E}[z_b] = \mu$ and variance $\mathrm{Var}[\sigma^2]$. Let ρ be the correlation between distinct variables.

Then,

$$\mathbb{E}\left[\frac{1}{B}\sum_{b=1}^{B} z_b\right] = \mu,$$

$$\mathrm{Var}\left[\frac{1}{B}\sum_{b=1}^{B} z_b\right] = \underbrace{\frac{1-\rho}{B}\sigma^2}_{\text{small for large } B} + \rho\sigma^2.$$

The variance is reduced by averaging (if $\rho < 1$) !

Bagging (I/II)

For now, assume that we have access to B **independent** datasets $\mathcal{T}^1, \ldots, \mathcal{T}^B$. We can then train a separate deep tree $\widehat{y}^b(\mathbf{x})$ for each dataset, $1, \ldots, B$.

- Each $\widehat{y}^b(\mathbf{x})$ has a **low bias** but **high variance**
- By averaging

$$\widehat{y}_{\text{bag}}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^{B} \widehat{y}^b(\mathbf{x})$$

the bias is kept small, but variance is reduced by a factor B!

Bagging (II/II)

Obvious problem. We only have access to one training dataset.

Solution Bootstrap the data!

- Sample n times with replacement from the original training data $\mathcal{T} = \{\mathbf{x}_i, y_i\}_{i=1}^n$
- Repeat B times to generate B "bootstrapped" training datasets $\widetilde{\mathcal{T}}^1, \ldots, \widetilde{\mathcal{T}}^B$

For each bootstrapped dataset $\widetilde{\mathcal{T}}^b$ we train a tree $\widetilde{y}^b(\mathbf{x})$. Averaging these,

$$\widetilde{y}^b_{\text{bag}}(\mathbf{x}) = \frac{1}{B}\sum_{b=1}^{B} \widetilde{y}^b(\mathbf{x})$$

is called "bootstrap aggregation", or bagging.

Bagging - Toy example

ex) Assume that we have a training set

$$\mathcal{T} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), (\mathbf{x}_3, y_3), (\mathbf{x}_4, y_4)\}.$$

We generate, say, $B = 3$ datasets by bootstrapping:

$$\widetilde{\mathcal{T}}^1 = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), (\mathbf{x}_3, y_3), (\mathbf{x}_3, y_3)\}$$
$$\widetilde{\mathcal{T}}^2 = \{(\mathbf{x}_1, y_1), (\mathbf{x}_4, y_4), (\mathbf{x}_4, y_4), (\mathbf{x}_4, y_4)\}$$
$$\widetilde{\mathcal{T}}^3 = \{(\mathbf{x}_1, y_1), (\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), (\mathbf{x}_2, y_2)\}$$

Compute $B = 3$ (deep) regression trees $\tilde{y}^1(\mathbf{x})$, $\tilde{y}^2(\mathbf{x})$ and $\tilde{y}^3(\mathbf{x})$, one for each dataset $\widetilde{\mathcal{T}}^1$, $\widetilde{\mathcal{T}}^2$, and $\widetilde{\mathcal{T}}^3$, and average

$$\tilde{y}_{\mathsf{bag}}(\mathbf{x}) = \frac{1}{3}\sum_{b=1}^{3} \tilde{y}^b(\mathbf{x})$$

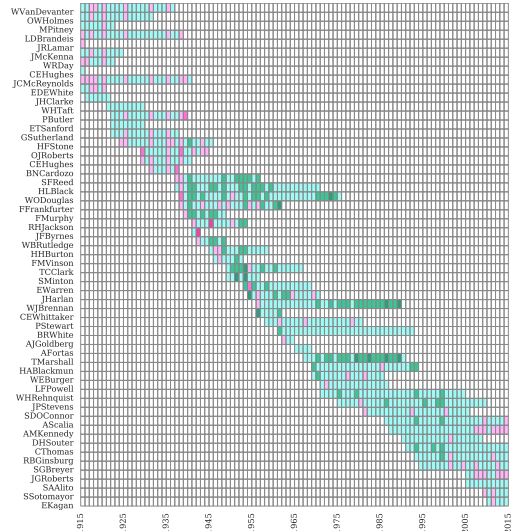
ex) Predicting US Supreme Court behavior

Random forest classifier built on SCDB data[1] to predict the votes of Supreme Court justices:

$$Y \in \{\texttt{affirm}, \texttt{reverse}, \texttt{other}\}$$

Result: 70% correct classifications

D. M. Katz, M. J. Bommarito II and J. Blackman. **A General Approach for Predicting the Behavior of the Supreme Court of the United States**. *arXiv.org*, arXiv:1612.03473v2, January 2017.



[1] `http://supremecourtdatabase.org`

ex) Predicting US Supreme Court behavior

Not only have random forests proven to be "unreasonably effective" in a wide array of supervised learning contexts, but in our testing, random forests outperformed other common approaches including support vector machines [. . .] and feedforward artificial neural network models such as multi-layer perceptron

— Katz, Bommarito II and Blackman (arXiv:1612.03473v2)

Random forests

Random forests

- ▲ Bagging can drastically improve the performance of CART!
- ▼ However, the B bootstrapped dataset are *correlated*
  $\Rightarrow$ the variance reduction due to averaging is diminished.

Idea: De-correlate the B trees by randomly perturbing each tree.

A **random forest** is constructed by bagging, but for each split in each tree only a ***random subset*** of $q \leq p$ inputs are considered as splitting variables.

Rule of thumb: $q = \sqrt{p}$ for classification trees and $q = p/3$ for regression trees.

Random forest pseudo-code

Algorithm Random forest for regression

1. For $b = 1$ to B *(can run in parallel)*
   (a) Draw a bootstrap data set $\widetilde{\mathcal{T}}$ of size n from $\mathcal{T}$.
   (b) Grow a regression tree by repeating the following steps until a minimum node size is reached:
      i. Select q out of the p input variables uniformly at random.
      ii. Find the variable x_j among the q selected, and the corresponding split point s, that minimizes the squared error.
      iii. Split the node into two children with $\{x_j \leq s\}$ and $\{x_j > s\}$.
2. Final model is the average of the B ensemble members,

$$\widehat{y}_\star^{\text{rf}} = \frac{1}{B} \sum_{b=1}^{B} \widetilde{y}_\star^b.$$

Random forests

Recall: For i.d. random variables $\{z_b\}_{b=1}^{B}$

$$\text{Var}\left(\frac{1}{B}\sum_{b=1}^{B} z_b\right) = \frac{1-\rho}{B}\sigma^2 + \rho\sigma^2$$

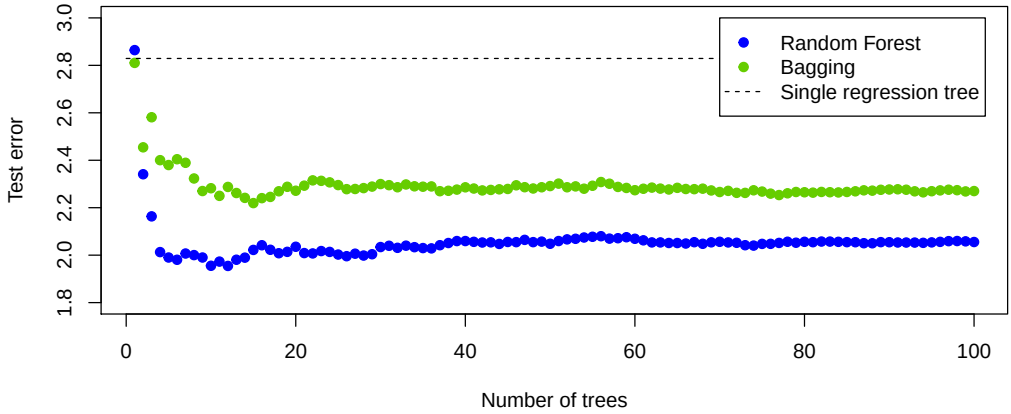
The random input selection used in random forests:

- ▼ increases the bias, but often very slowly
- ▼ adds to the variance (σ^2) of each tree
- ▲ reduces the correlation (ρ) of the trees

The reduction in correlation is typically the dominant effect $\Rightarrow$ there is an overall reduction in MSE!

ex) Toy regression model

For the toy model previously considered...



Overfitting?

The complexity of a bagging/random forest model increases with an increasing number of trees B.

Will this lead to overfitting as B increases?

No – more ensemble members ***does not*** increase the ***flexibility*** of the model!

Regression case:

$$\widehat{y}_\star^{\text{rf}} = \frac{1}{B}\sum_{b=1}^{B} \widehat{y}_\star^b \to \mathbb{E}\left[\widetilde{y}_\star \,|\, \mathcal{T}\right], \qquad \text{as } B \to \infty,$$

where the expectation is w.r.t. the randomness in the data bootstrapping and input selection.

Advantages of random forests

Random forests have several **computational advantages**:

- Embarrassingly parallelizable!
- Using $q < p$ potential split variables reduces the computational cost of each split.
- We ***could*** bootstrap fewer than n, say $\sqrt{n}$, data points when creating $\widetilde{\mathcal{T}}^b$ — very useful for "big data" problems.

. . . and they also come with some other benefits:

- Often works well off-the-shelf – few tuning parameters
- Requires little or no input preparation
- Implicit input selection

ex) Automatic music generation

ALYSIA: automated music generation using random forests.

- User specifies the lyrics
- ALYSIA generates accompanying music via
  - *rythm model*
  - *melody model*
- Trained on a corpus of pop songs.



https://www.youtube.com/watch?v=whgudcj82_I https://www.withalysia.com/

M. Ackerman and D. Loker. **Algorithmic Songwriting with ALYSIA**. *In: Correia J., Ciesielski V., Liapis A. (eds) Computational Intelligence in Music, Sound, Art and Design. EvoMUSART*, 2017.

Boosting

Boosting

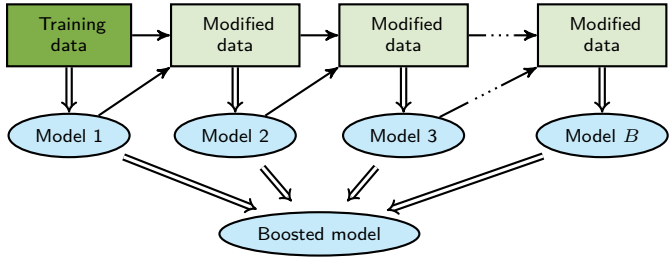
Even a simple (classification or regression) model can typically capture some aspects of the input-output relationship.

Can we then learn an ***ensemble*** of "weak models", each describing some part of this relationship, and combine these into one "strong model"?

Boosting:

- ***Sequentially*** learns an ensemble of ***weak models***.
- Combine these into one ***strong model***.
- General strategy – can in principle be used to improve any supervised learning algorithm.
- A very successful idea!

Boosting



The models are built **sequentially** such that each models tries to **correct the mistakes** made by the previous one!

Boosting vs. bagging

Bagging	Boosting
Learns base models in parallel	Learns base models sequentially
Uses bootstrapped datasets	Uses reweighted datasets
Does not overfit as B becomes large	Can overfit as B becomes large
Reduces variance but not bias (suitable for models with low bias)	Primarily reduces bias! (models with high bias are fine)

Boosting does **not** require each base model to have low bias. Thus, a shallow classification tree (say, 4-8 terminal nodes) or even a tree with a single split (2 terminal nodes, a "stump") is often sufficient.

Binary classification

We will restrict our attention to binary classification.

- Class labels are -1 and 1, i.e. $y \in \{-1, 1\}$.
- We have access to some (weak) base classifier, e.g. a classification tree.

Note. Using labels -1 and 1 is mathematically convenient as it allows us to express a majority vote between B classifiers $\widehat{y}^1(\mathbf{x}), \ldots, \widehat{y}^B(\mathbf{x})$ as

$$\text{sign}\left(\sum_{b=1}^{B} \widehat{y}^b(\mathbf{x})\right) = \begin{cases} +1 & \text{if more plus-votes than minus-votes,} \\ -1 & \text{if more minus-votes than plus-votes.} \end{cases}$$

Boosting procedure (for classification)

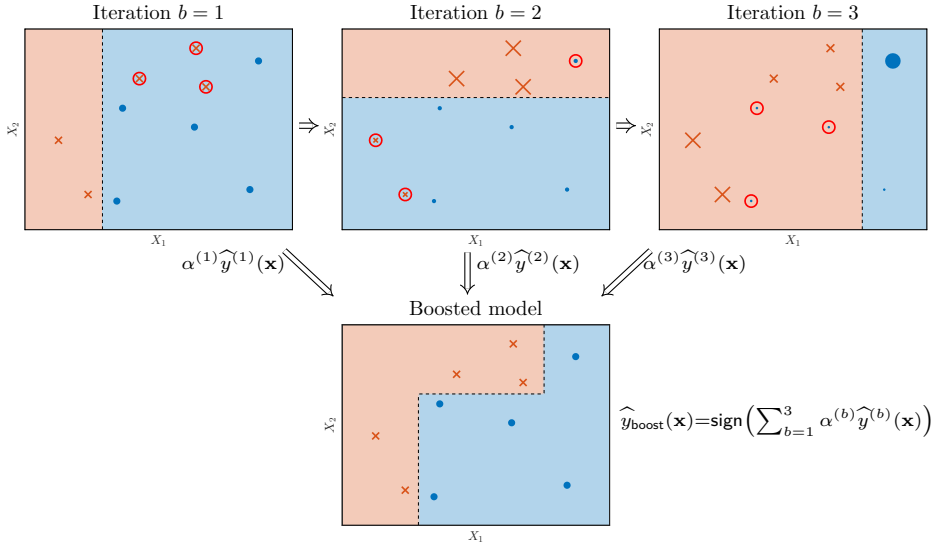
Boosting procedure:

1. Assign weights $w_i^1 = 1/n$ to all data points.
2. For $b = 1$ to B
   (a) Train a weak classifier $\widehat{y}^{(b)}(\mathbf{x})$ on the **weighted training data** $\{(\mathbf{x}_i, y_i, w_i^b)\}_{i=1}^n$.
   (b) *Update the weights* $\{w_i^{b+1}\}_{i=1}^n$ *from* $\{w_i^b\}_{i=1}^n$*:*
      i. Increase weights for all points misclassified by $\widehat{y}^{(b)}(\mathbf{x})$.
      ii. Decrease weights for all points correctly classified by $\widehat{y}^{(b)}(\mathbf{x})$.

The predictions of the B classifiers, $\widehat{y}^{(1)}(\mathbf{x}), \ldots, \widehat{y}^{(B)}(\mathbf{x})$, are combined using a **weighted** majority vote:

$$\widehat{y}^B_{\text{boost}}(\mathbf{x}) = \text{sign}\left(\sum_{b=1}^{B} \alpha^{(b)} \widehat{y}^{(b)}(\mathbf{x})\right).$$

ex) Boosting illustration



Important details

Q1: How do we reweight the data?

Q2: How are the coefficients $\alpha^{(1)}, \ldots, \alpha^{(B)}$ computed?

AdaBoost pseudo-code

1. Assign weights $w_i^1 = 1/n$ to all data points.
2. For $b = 1$ to B
   (a) Train a weak classifier $\widehat{y}^{(b)}(\mathbf{x})$ on the **weighted training data** $\{(\mathbf{x}_i, y_i, w_i^b)\}_{i=1}^n$.
   (b) *Update the weights* $\{w_i^{b+1}\}_{i=1}^n$ *from* $\{w_i^b\}_{i=1}^n$*:*
      i. Increase weights for all points misclassified by $\widehat{y}^{(b)}(\mathbf{x})$.
      ii. Decrease weights for all points correctly classified by $\widehat{y}^{(b)}(\mathbf{x})$.

      i. Compute weighted classification errorCompute $E_{\text{train}}^b = \sum_{i=1}^n w_i^b \mathbb{I}\{y_i \neq \widehat{y}^{(b)}(\mathbf{x}_i)\}$
      ii. Compute classifier "confidence"Compute $\alpha^b = 0.5 \log((1 - E_{\text{train}}^b)/E_{\text{train}}^b)$.
      iii. Compute new weightsCompute $w_i^{b+1} = w_i^b \exp(-\alpha^{(b)} y_i \widehat{y}^{(b)}(\mathbf{x}_i))$, $i = 1, \ldots, n$
      iv. *Normalize.* Set $w_i^{b+1} \leftarrow w_i^{b+1} / \sum_{j=1}^n w_j^{b+1}$, for $i = 1, \ldots, n$.
3. Output $\widehat{y}_{\text{boost}}^{(B)}(\mathbf{x}) = \text{sign}\left(\sum_{b=1}^B \alpha^{(b)} \widehat{y}^{(b)}(\mathbf{x})\right)$.

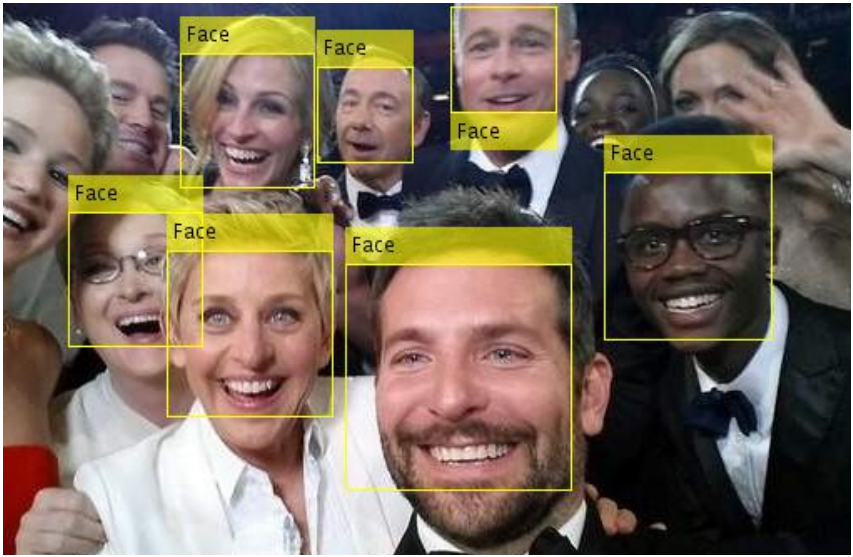
Y. Freund and R. E. Schapire. **Experiments with a New Boosting Algorithm**. *Proceedings of the 13th International Conference on Machine Learning (ICML)*. Bari, Italy, 1996.



2003 Gödel Prize

UPPSALA UNIVERSITET

ex) The Viola-Jones face detector



A few concepts to summarize lecture 6

CART: Classification and regression trees. A class of nonparametric methods based on partitioning the input space into regions and fitting a simple model for each region.

Recursive binary splitting: A greedy method for partitioning the input space into "boxes" aligned with the coordinate axes.

Gini index and deviance: Commonly used error measures for constructing classification trees.

Ensemble methods: Umbrella term for methods that average or combine multiple models.

Bagging: Bootstrap aggregating. An ensemble method based on the statistical bootstrap.

Random forests: Bagging of trees, combined with random feature selection for further variance reduction (and computational gains).

Boosting: Sequential ensemble method, where each consecutive model tries to correct the mistakes of the previous one.

AdaBoost: The first successful boosting algorithm. Designed for binary classification.

Lecture 7 – Deep neural networks



Sebastian Mair
https://smair.github.io/
Department of Information Technology
Uppsala University

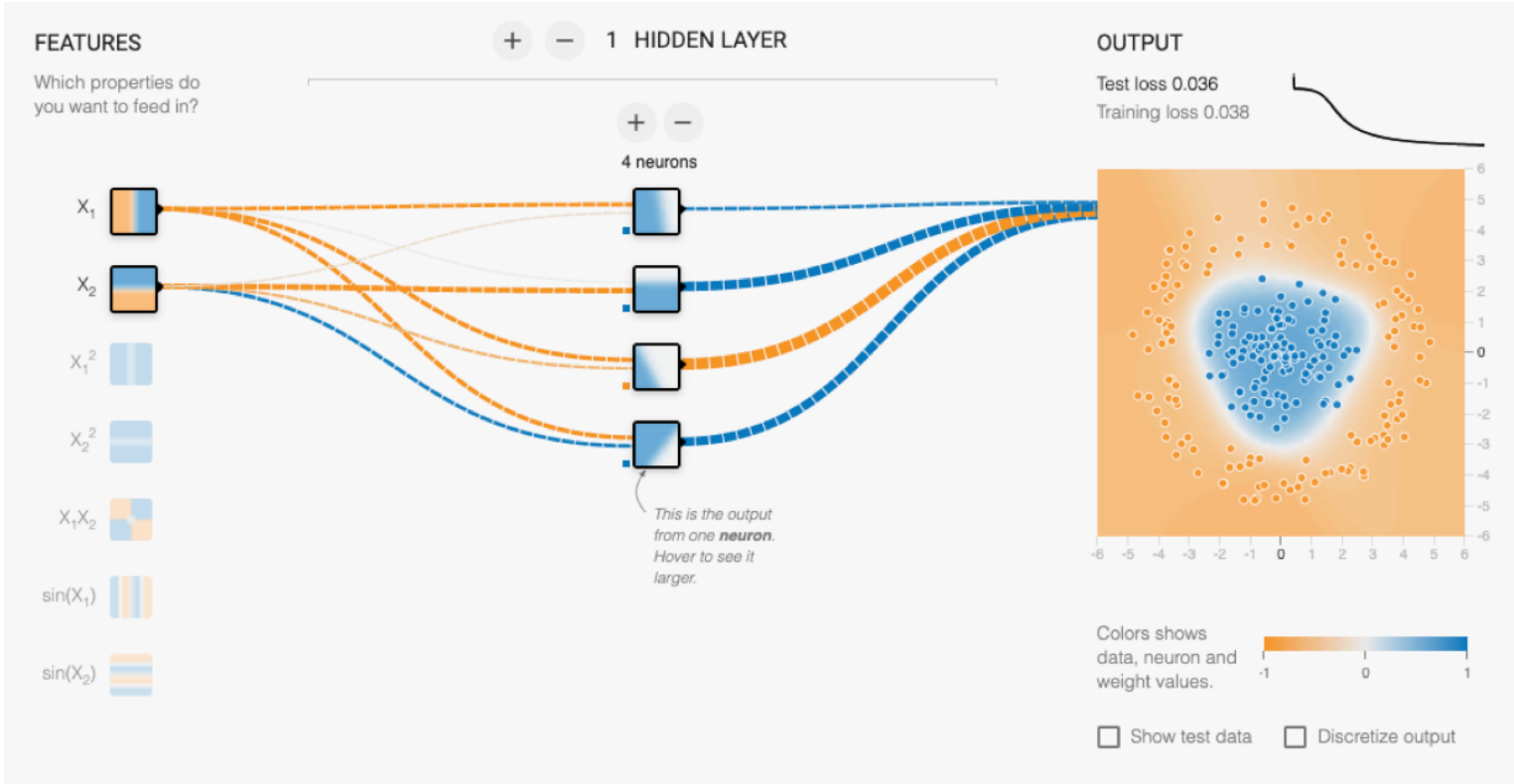
Course webpage

Contents – Lecture 7

1. **This lecture:** The neural network model
   - Neural network for regression
   - Neural network for classification
2. **Next lecture:**
   - Convolutional neural networks
   - How to train a neural network

UPPSALA UNIVERSITET

Some examples of neural networks

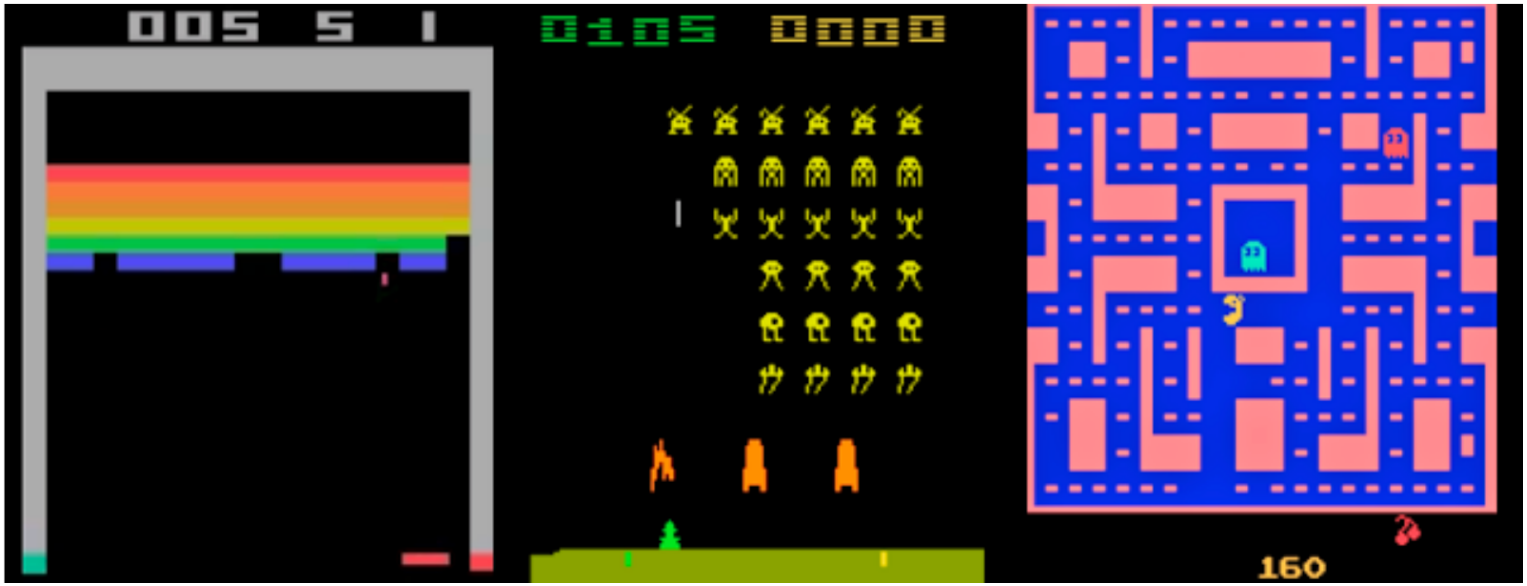


Some examples of neural networks



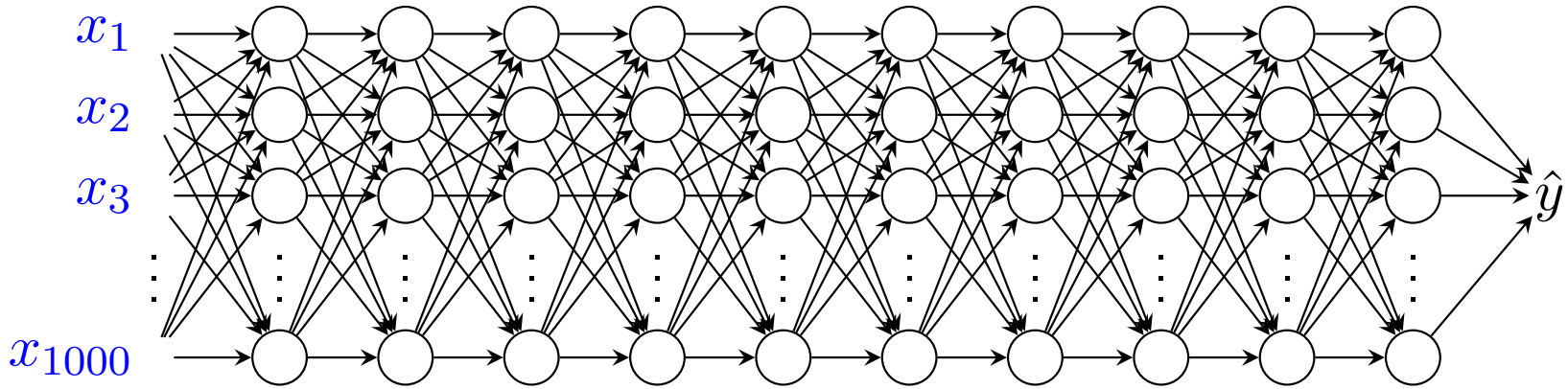
UPPSALA
UNIVERSITET

Some examples of neural networks



Where are we heading?

We will spend the next 30 minutes deriving the neural network, which graphically can be depicted as the one below



Constructing neural networks for regression

A **neural network (NN)** is a nonlinear function $\hat{y} = f(\mathbf{x}; \boldsymbol{\theta})$ from an input $\mathbf{x}$ to an output $\hat{y}$ parameterized by parameters $\boldsymbol{\theta}$.

Linear regression models the relationship between a continuous output y and a continuous input $\mathbf{x}$,

$$\hat{y} = \sum_{j=1}^{p} W_j x_j + b = \mathbf{W}\mathbf{x} + b,$$

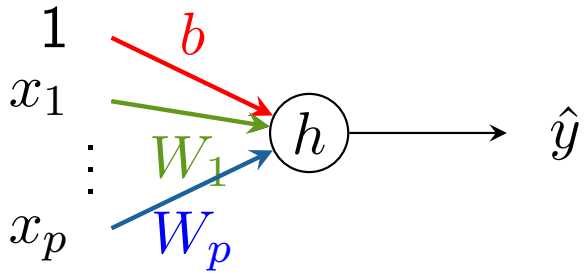
where the parameters $\boldsymbol{\theta}$ are the **weights** W_j, and the **offset** (aka bias/intercept) term b,

$$\boldsymbol{\theta} = \begin{bmatrix} b & \mathbf{W} \end{bmatrix}^{\mathsf{T}}, \qquad \mathbf{W} = \begin{bmatrix} W_1 & W_2 & \cdots & W_p \end{bmatrix}$$

Generalized linear regression

We can generalize this by introducing nonlinear transformations of the predictor $\mathbf{W}\mathbf{x} + b$,

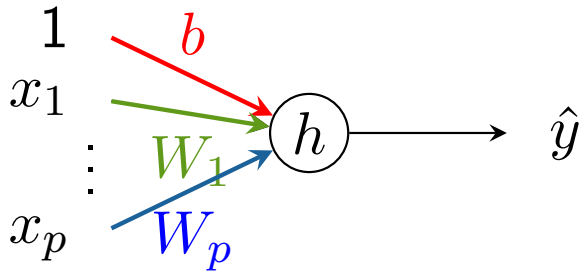
$$\hat{y} = h(\mathbf{W}\mathbf{x} + b),$$



Generalized linear regression

We can generalize this by introducing nonlinear transformations of the predictor $\mathbf{W}\mathbf{x} + b$,

$$\hat{y} = h(\mathbf{W}\mathbf{x} + b),$$

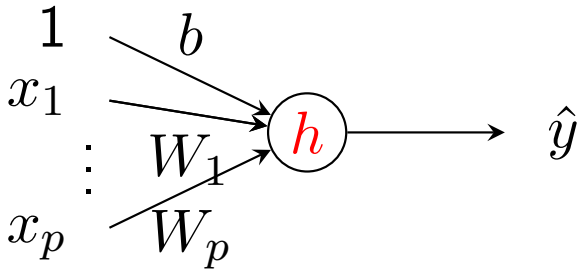


We call $h(x)$ the **activation function**. Two common choices are:

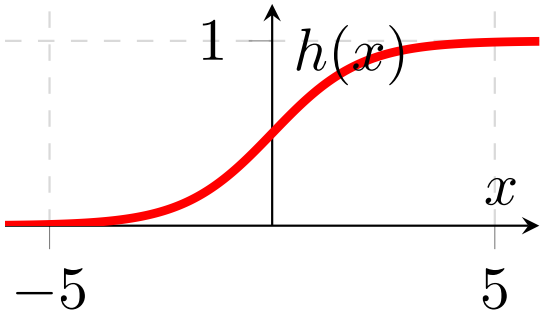
Generalized linear regression

We can generalize this by introducing nonlinear transformations of the predictor $\mathbf{W}\mathbf{x} + b$,

$$\hat{y} = h(\mathbf{W}\mathbf{x} + b),$$



We call $h(x)$ the **activation function**. Two common choices are:

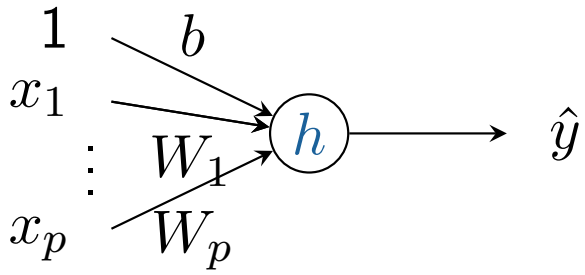


Sigmoid: $h(x) = \frac{e^x}{1+e^x}$

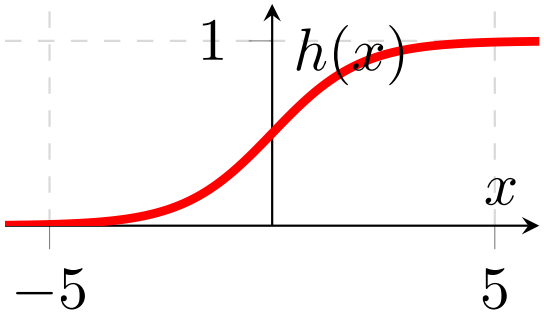
Generalized linear regression

We can generalize this by introducing nonlinear transformations of the predictor $\mathbf{W}\mathbf{x} + b$,

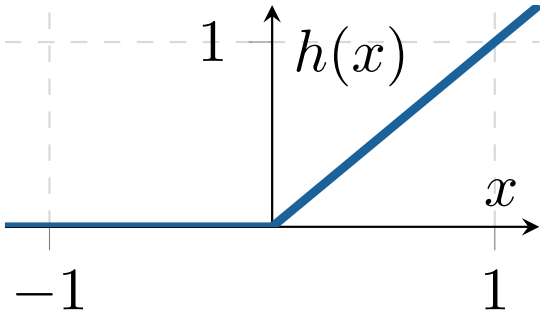
$$\hat{y} = h(\mathbf{W}\mathbf{x} + b),$$



We call $h(x)$ the **activation function**. Two common choices are:



Sigmoid: $h(x) = \frac{e^x}{1+e^x}$

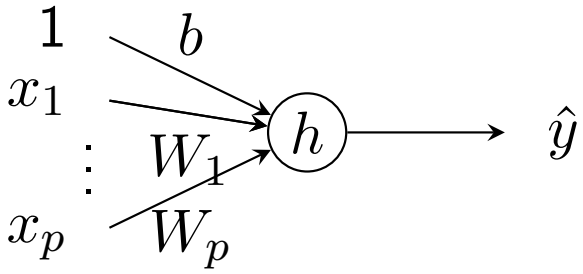


ReLU: $h(x) = \max(0, x)$

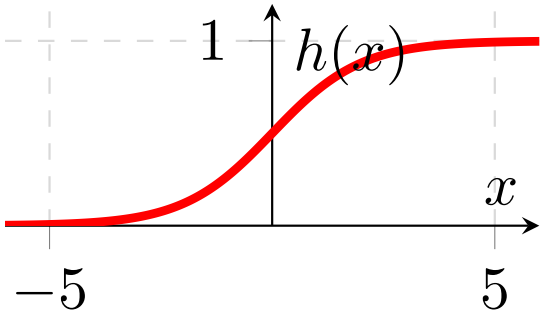
Generalized linear regression

We can generalize this by introducing nonlinear transformations of the predictor $\mathbf{W}\mathbf{x} + b$,

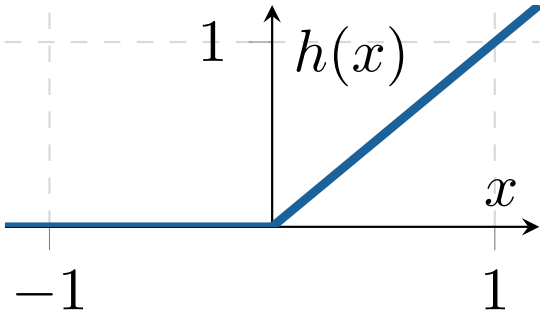
$$\hat{y} = h(\mathbf{W}\mathbf{x} + b),$$



We call $h(x)$ the **activation function**. Two common choices are:



Sigmoid: $h(x) = \frac{e^x}{1+e^x}$



ReLU: $h(x) = \max(0, x)$

Let us consider an example of a **neural network**.

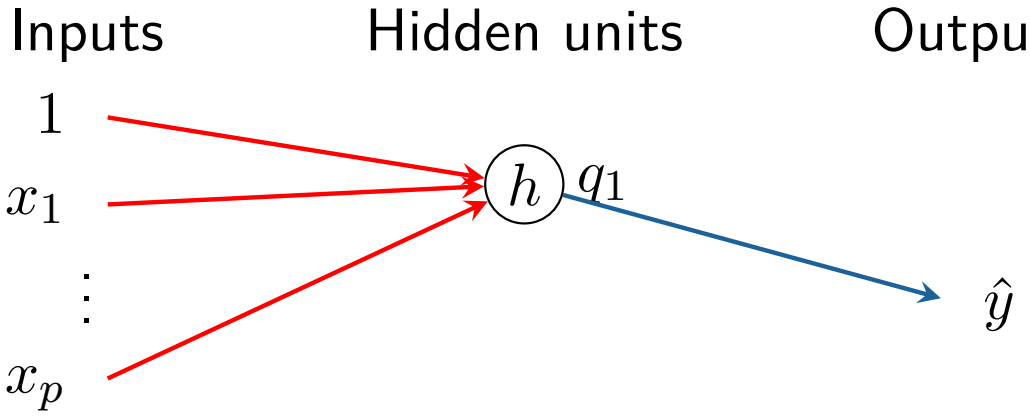
Neural network - construction

A neural network is a sequential construction of several generalized linear regression models.



Neural network - construction

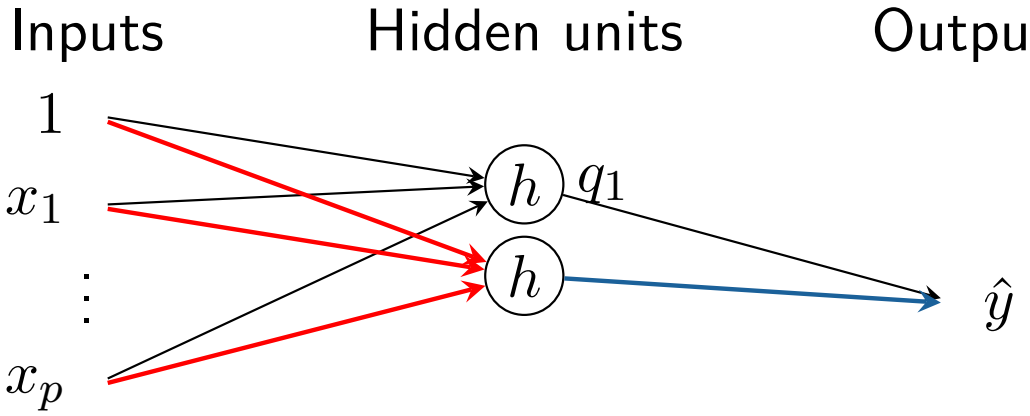
A neural network is a sequential construction of **several** generalized linear regression models.



$$q_1 = h\left(b_1^{(1)} + \sum_{j=1}^{p} W_{1j}^{(1)} x_j\right) \qquad \hat{y} = W_1^{(2)} q_1$$

Neural network - construction

A neural network is a sequential construction of **several** generalized linear regression models.



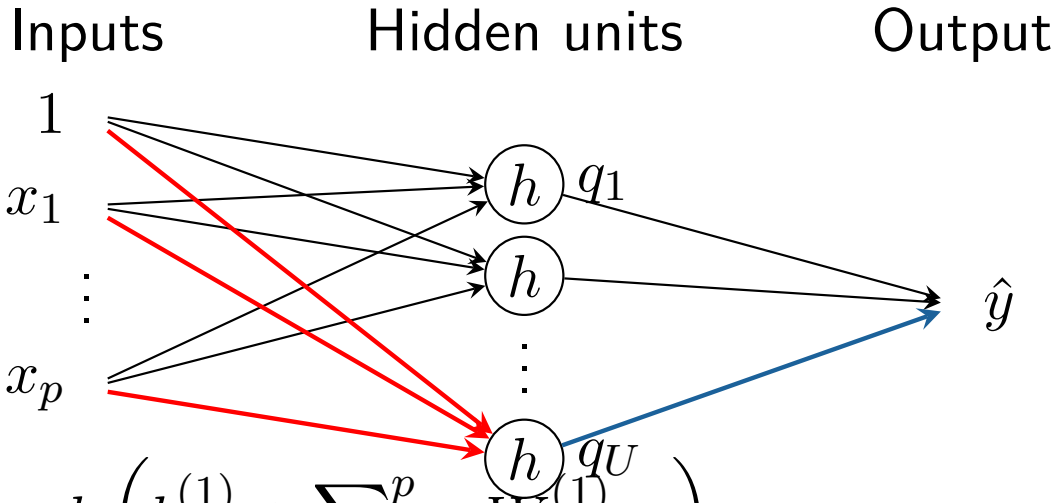
$$q_1 = h\left(b_1^{(1)} + \sum_{j=1}^{p} W_{1j}^{(1)} x_j\right)$$

$$q_2 = h\left(b_2^{(1)} + \sum_{j=1}^{p} W_{2j}^{(1)} x_j\right)$$

$$\hat{y} = \sum_{i=1}^{2} W_i^{(2)} q_i$$

Neural network - construction

A neural network is a sequential construction of **several** generalized linear regression models.



$$q_1 = h\left(b_1^{(1)} + \sum_{j=1}^{p} W_{1j}^{(1)} x_j\right)$$

$$q_2 = h\left(b_2^{(1)} + \sum_{j=1}^{p} W_{2j}^{(1)} x_j\right)$$

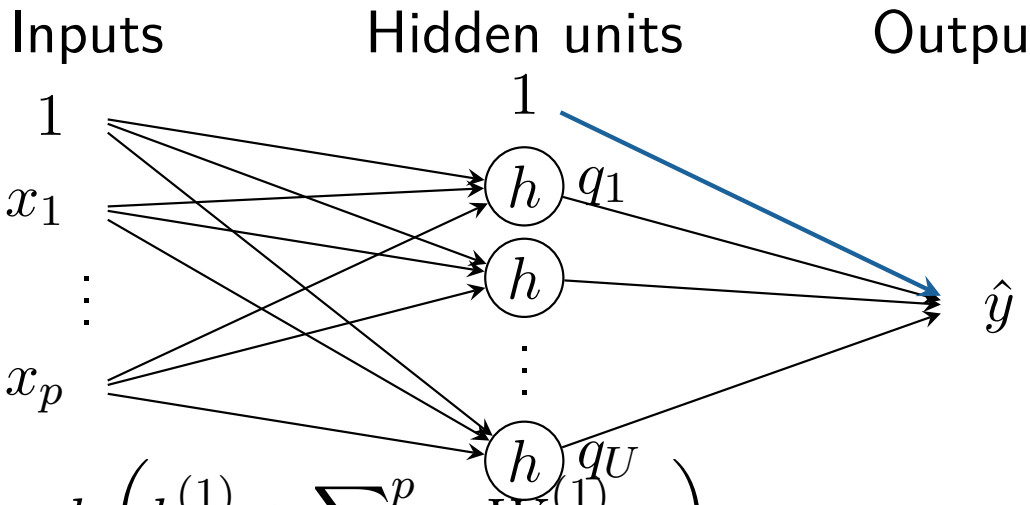
$$\vdots$$

$$q_U = h\left(b_U^{(1)} + \sum_{j=1}^{p} W_{Uj}^{(1)} x_j\right)$$

$$\hat{y} = \sum_{i=1}^{U} W_i^{(2)} q_i$$

Neural network - construction

A neural network is a sequential construction of **several** generalized linear regression models.



$$q_1 = h\left(b_1^{(1)} + \sum_{j=1}^{p} W_{1j}^{(1)} x_j\right)$$

$$q_2 = h\left(b_2^{(1)} + \sum_{j=1}^{p} W_{2j}^{(1)} x_j\right)$$

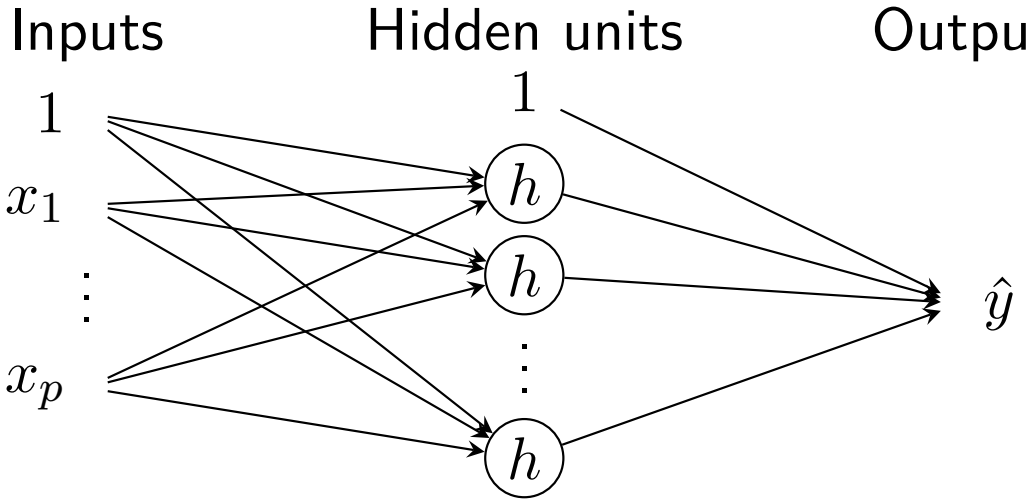
$$\vdots$$

$$q_U = h\left(b_U^{(1)} + \sum_{j=1}^{p} W_{Uj}^{(1)} x_j\right)$$

$$\hat{y} = b^{(2)} + \sum_{i=1}^{U} W_i^{(2)} q_i$$

Neural network - construction

A neural network is a sequential construction of **several** generalized linear regression models.



$$\mathbf{q} = h(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$\hat{y} = \mathbf{W}^{(2)}\mathbf{q} + \mathbf{b}^{(2)}$$

$$\mathbf{W}^{(1)} = \begin{bmatrix} W_{11}^{(1)} & \dots & W_{1p}^{(1)} \\ \vdots & & \vdots \\ W_{U1}^{(1)} & \dots & W_{Up}^{(1)} \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} b_1^{(1)} \\ \vdots \\ b_U^{(1)} \end{bmatrix}, \quad \mathbf{q} = \begin{bmatrix} q_1 \\ \vdots \\ q_U \end{bmatrix}$$

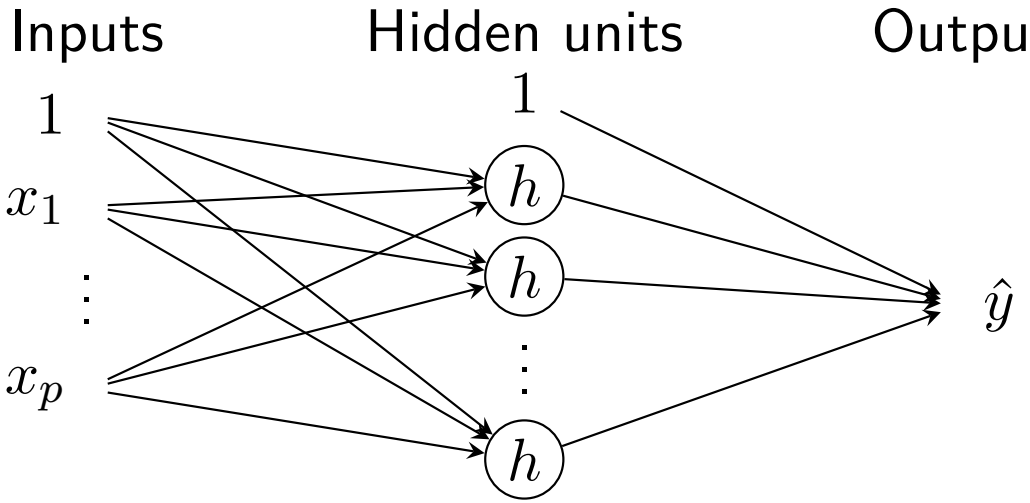
Weight matrix Offset vector Hidden units

$$\mathbf{b}^{(2)} = \begin{bmatrix} b^{(2)} \end{bmatrix}$$

$$\mathbf{W}^{(2)} = \begin{bmatrix} W_1^{(2)} & \dots & W_U^{(2)} \end{bmatrix}$$

Neural network - construction

A neural network is a sequential construction of **several** generalized linear regression models.



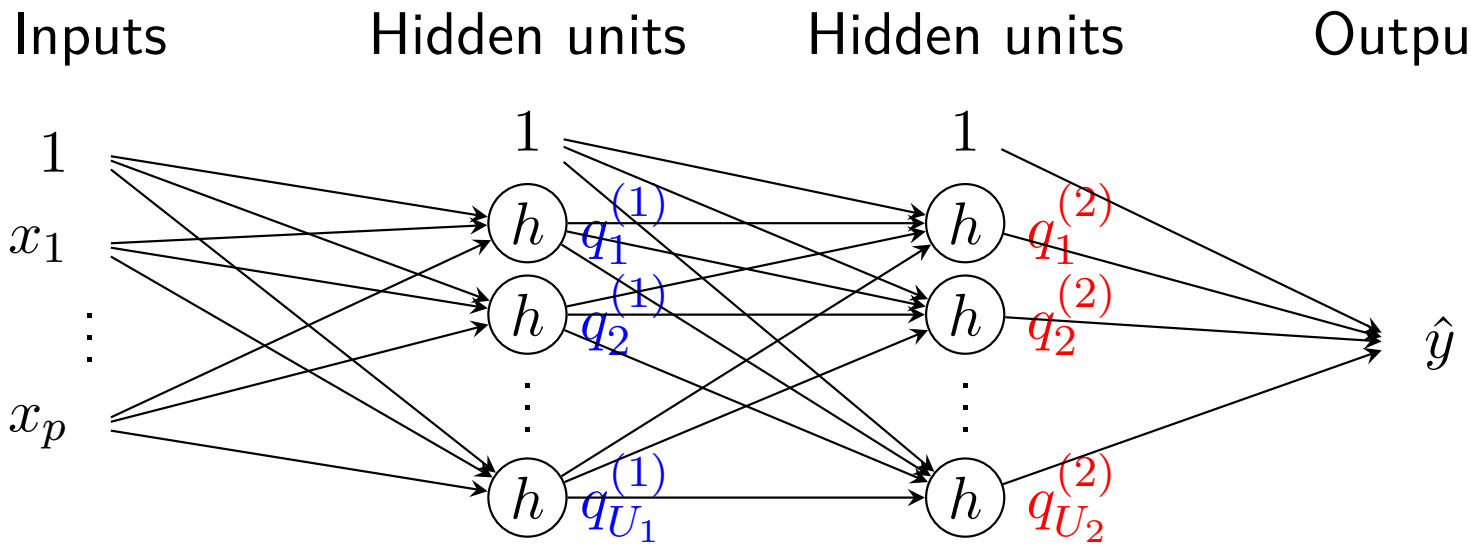
$$\mathbf{h} = h(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$\hat{y} = \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}$$

The non-linearity h acts element-wise.

Neural network - construction

A neural network is a **sequential** construction of several generalized linear regression models.



$$\mathbf{q}^{(1)} = h(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$
$$\mathbf{q}^{(2)} = h(\mathbf{W}^{(2)}\mathbf{q}^{(1)} + \mathbf{b}^{(2)})$$
$$\hat{y} = \mathbf{W}^{(3)}\mathbf{q}^{(2)} + \mathbf{b}^{(3)}$$

The model learns better using a deep network (several layers) instead of a wide and shallow network.

Deep learning

A neural network with L layers can be written as

$$\begin{aligned}
\mathbf{q}^{(0)} &= \mathbf{x} \\
\mathbf{q}^{(l)} &= h\left(\mathbf{W}^{(l)}\mathbf{q}^{(l-1)} + \mathbf{b}^{(l)}\right), \quad l = 1, \ldots, L-1 \\
\hat{y} &= \mathbf{W}^{(L)}\mathbf{q}^{(L-1)} + \mathbf{b}^{(L)}
\end{aligned}$$

Deep learning

A neural network with L layers can be written as

$$\begin{aligned}\mathbf{q}^{(0)} &= \mathbf{x} \\ \mathbf{q}^{(l)} &= h\left(\mathbf{W}^{(l)}\mathbf{q}^{(l-1)} + \mathbf{b}^{(l)}\right), \quad l = 1, \ldots, L-1 \\ \hat{y} &= \mathbf{W}^{(L)}\mathbf{q}^{(L-1)} + \mathbf{b}^{(L)}\end{aligned}$$

All weight matrices and offset vectors in all layers combined are the parameters of the network

$$\boldsymbol{\theta} = \left[\mathrm{vec}(\mathbf{W}^{(1)})^{\mathsf{T}}, \quad \mathbf{b}^{(1)\mathsf{T}}, \quad \ldots, \quad \mathrm{vec}(\mathbf{W}^{(L)})^{\mathsf{T}}, \quad \mathbf{b}^{(L)\mathsf{T}}\right]^{\mathsf{T}},$$

which constitutes the parametric model $\hat{y} = f(\mathbf{x}; \boldsymbol{\theta})$. If L is large we call it a deep neural network.

Deep learning

A neural network with L layers can be written as

$$\begin{aligned}\mathbf{q}^{(0)} &= \mathbf{x} \\ \mathbf{q}^{(l)} &= h\left(\mathbf{W}^{(l)}\mathbf{q}^{(l-1)} + \mathbf{b}^{(l)}\right), \quad l = 1, \ldots, L-1 \\ \hat{y} &= \mathbf{W}^{(L)}\mathbf{q}^{(L-1)} + \mathbf{b}^{(L)}\end{aligned}$$

All weight matrices and offset vectors in all layers combined are the parameters of the network

$$\boldsymbol{\theta} = \left[\text{vec}(\mathbf{W}^{(1)})^{\mathsf{T}}, \quad \mathbf{b}^{(1)\mathsf{T}}, \quad \ldots, \quad \text{vec}(\mathbf{W}^{(L)})^{\mathsf{T}}, \quad \mathbf{b}^{(L)\mathsf{T}}\right]^{\mathsf{T}},$$

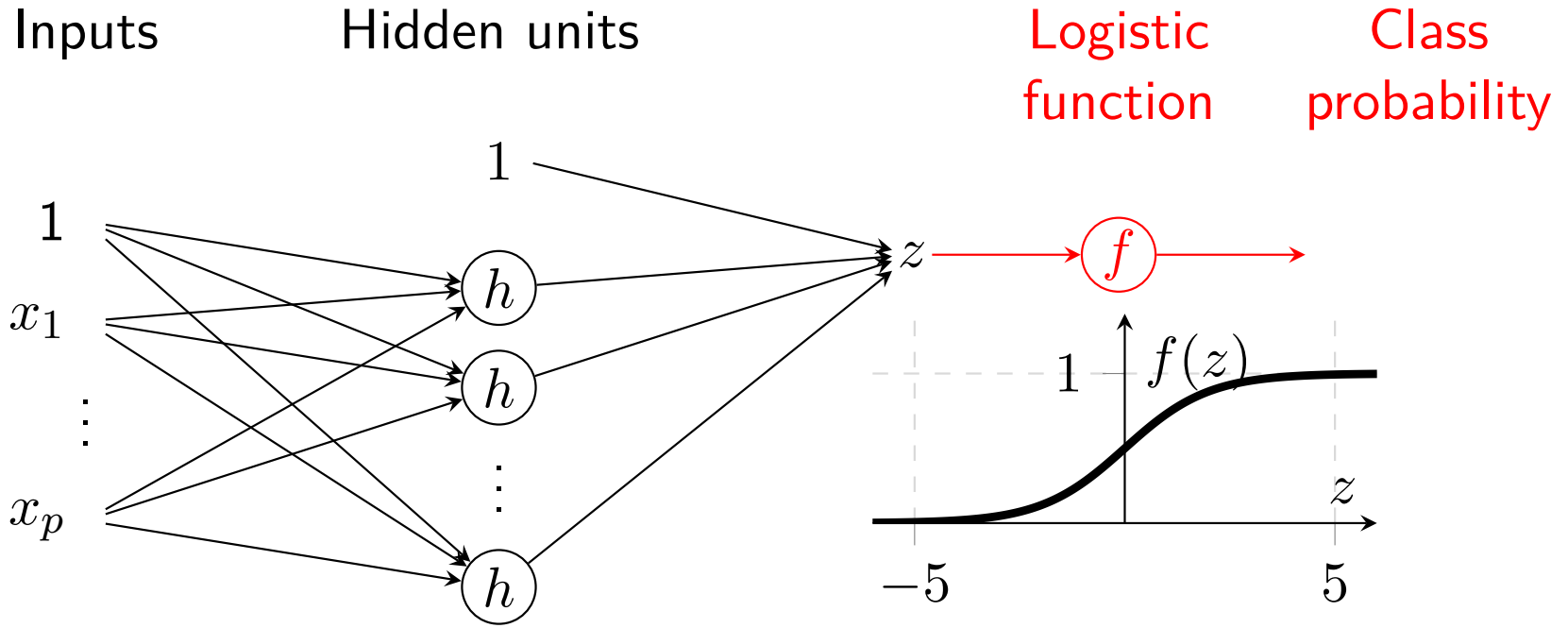
which constitutes the parametric model $\hat{y} = f(\mathbf{x}; \boldsymbol{\theta})$. If L is large we call it a deep neural network.

Deep learning is a class of machine learning models and algorithms that use a cascade of multiple layers, each of which is a nonlinear transformation.

NN for classification ($M = 2$ classes)

We can also use neural networks for classification. With $M = 2$ classes we squash the output through the logistic function to get a **class probability** $f(z) \in [0, 1]$

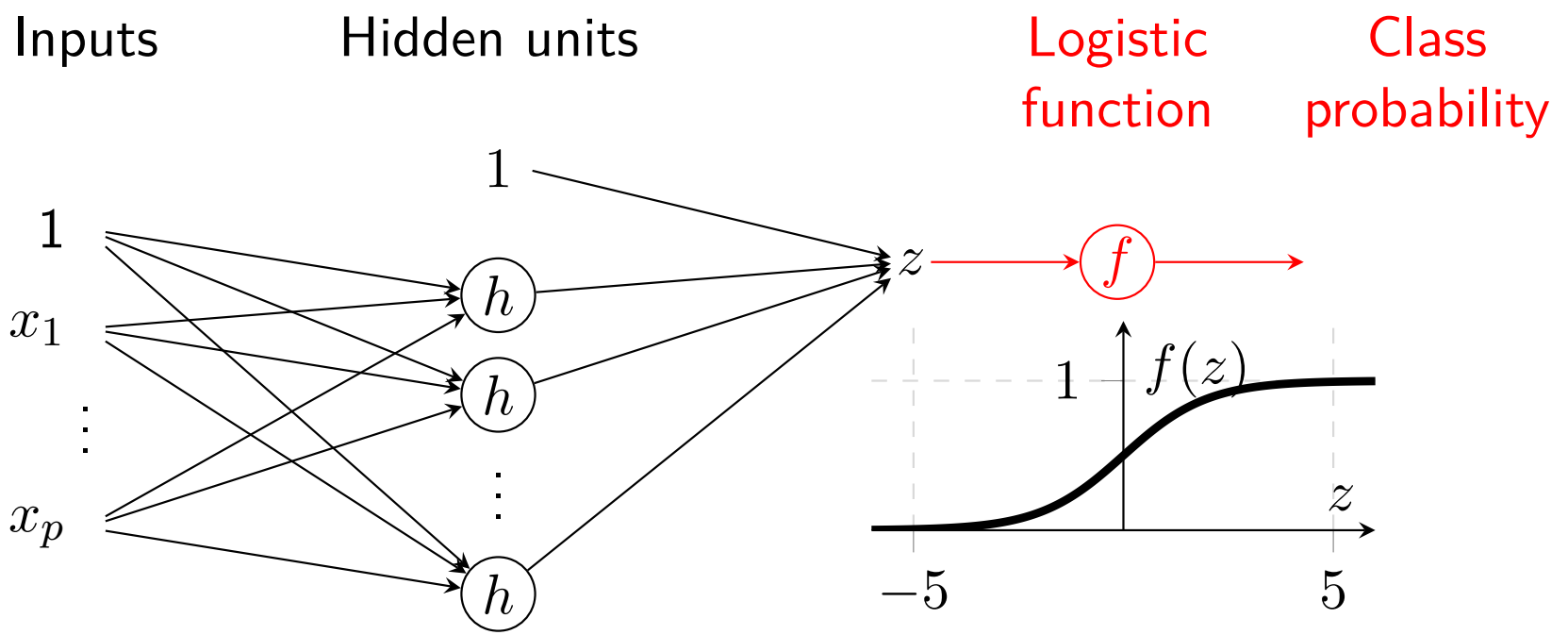
$$f(z) = p(y = 1|\mathbf{x}, \boldsymbol{\theta}) \quad \text{where} \quad f(z) = \frac{e^z}{1 + e^z}$$



NN for classification ($M = 2$ classes)

We can also use neural networks for classification. With $M = 2$ classes we squash the output through the logistic function to get a **class probability** $f(z) \in [0, 1]$

$$f(z) = p(y = 1|\mathbf{x}, \boldsymbol{\theta}) \quad \text{where} \quad f(z) = \frac{e^z}{1 + e^z}$$

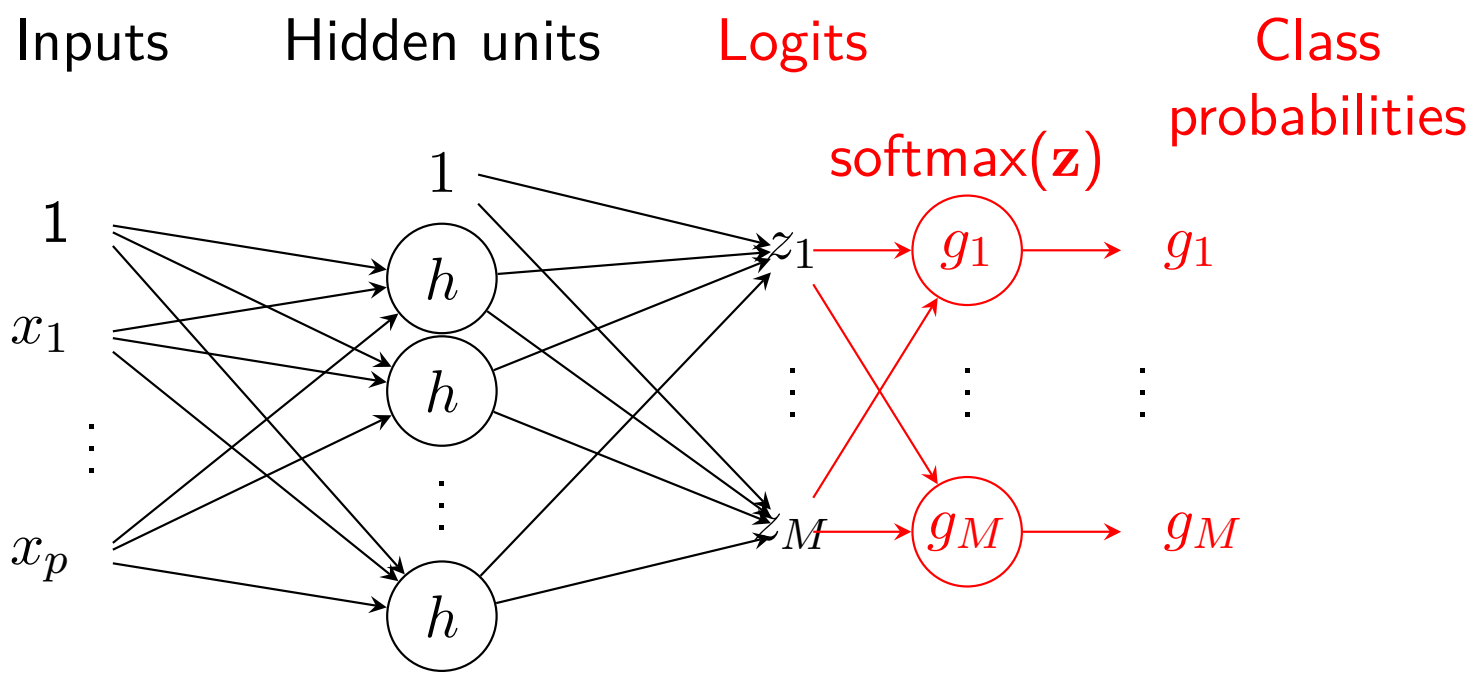


What if $M > 2$?

NN for classification ($M > 2$ classes)

For $M > 2$ classes we want to predict the class probability for all M classes $g_m(\mathbf{z}) = p(y = m|\mathbf{x}, \boldsymbol{\theta})$. We extend the logistic function to the **softmax function**

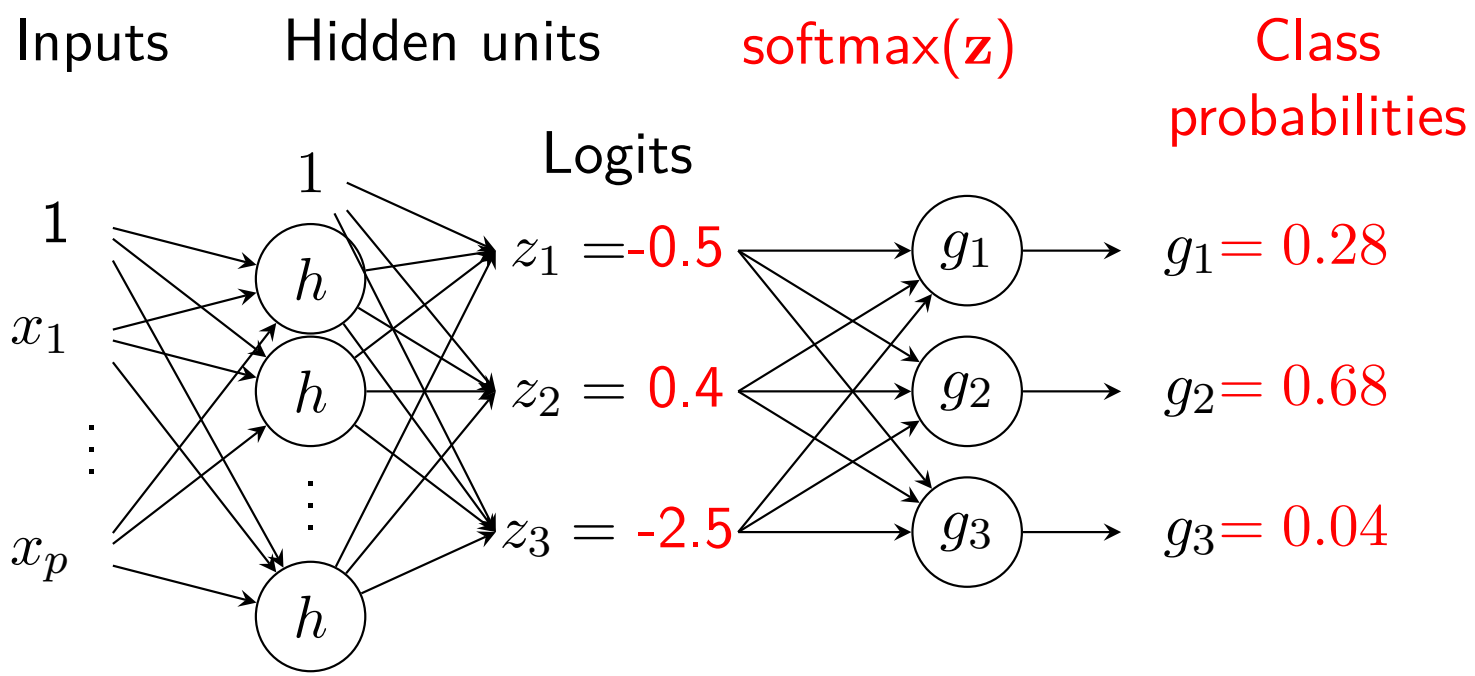
$$g_m(\mathbf{z}) = \frac{e^{z_m}}{\sum_{l=1}^{M} e^{z_l}}, \qquad m = 1, \ldots, M.$$



$$\text{softmax}(\mathbf{z}) = [g_1(\mathbf{z}), \ldots, g_M(\mathbf{z})]^\mathsf{T}$$

NN classification with $M = 3$ classes (I/II)

Consider an example with three classes $M = 3$.

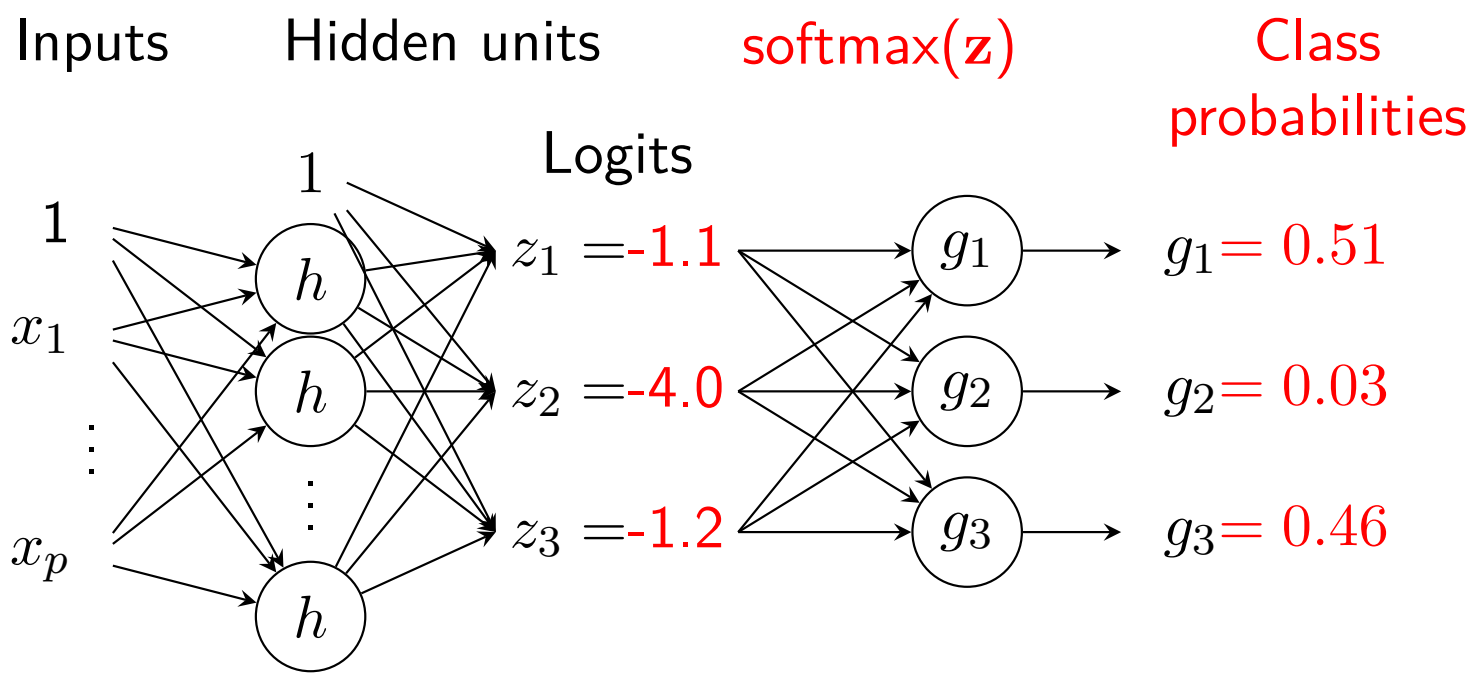


where

$$g_m(\mathbf{z}) = \frac{e^{z_m}}{\sum_{l=1}^{M} e^{z_l}}, \qquad \text{softmax}(\mathbf{z}) = [g_1(\mathbf{z}), \ldots, g_M(\mathbf{z})]^{\mathsf{T}}.$$

NN classification with $M = 3$ classes (I/II)

Consider an example with three classes $M = 3$.

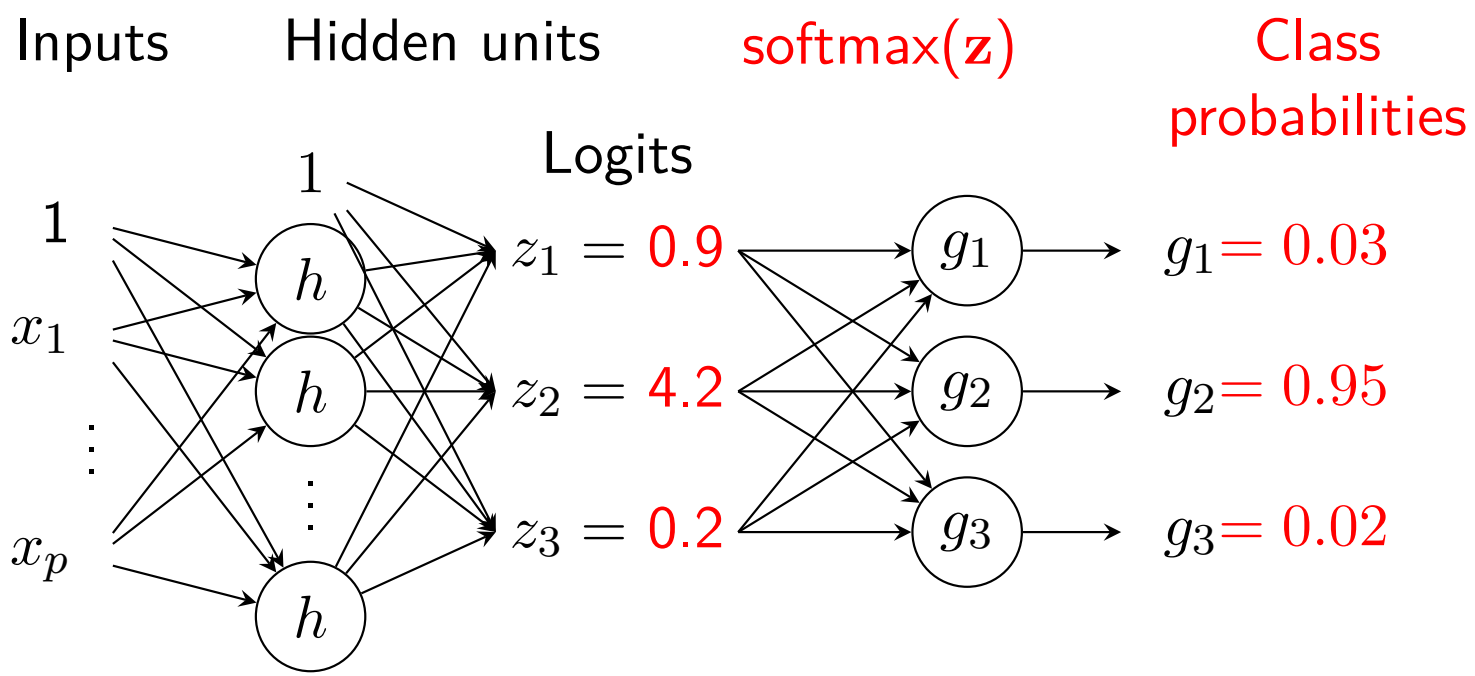


where

$$g_m(\mathbf{z}) = \frac{e^{z_m}}{\sum_{l=1}^{M} e^{z_l}}, \qquad \text{softmax}(\mathbf{z}) = [g_1(\mathbf{z}), \ldots, g_M(\mathbf{z})]^{\mathsf{T}}.$$

NN classification with $M = 3$ classes (I/II)

Consider an example with three classes $M = 3$.



where

$$g_m(\mathbf{z}) = \frac{e^{z_m}}{\sum_{l=1}^{M} e^{z_l}}, \qquad \text{softmax}(\mathbf{z}) = [g_1(\mathbf{z}), \ldots, g_M(\mathbf{z})]^\mathsf{T}.$$

One-hot encoding (for $M > 2$)

We use **one-hot encoding** for the output $\tilde{\mathbf{y}} = [\tilde{y}_1, \ldots, \tilde{y}_M]^\mathsf{T}$, which means

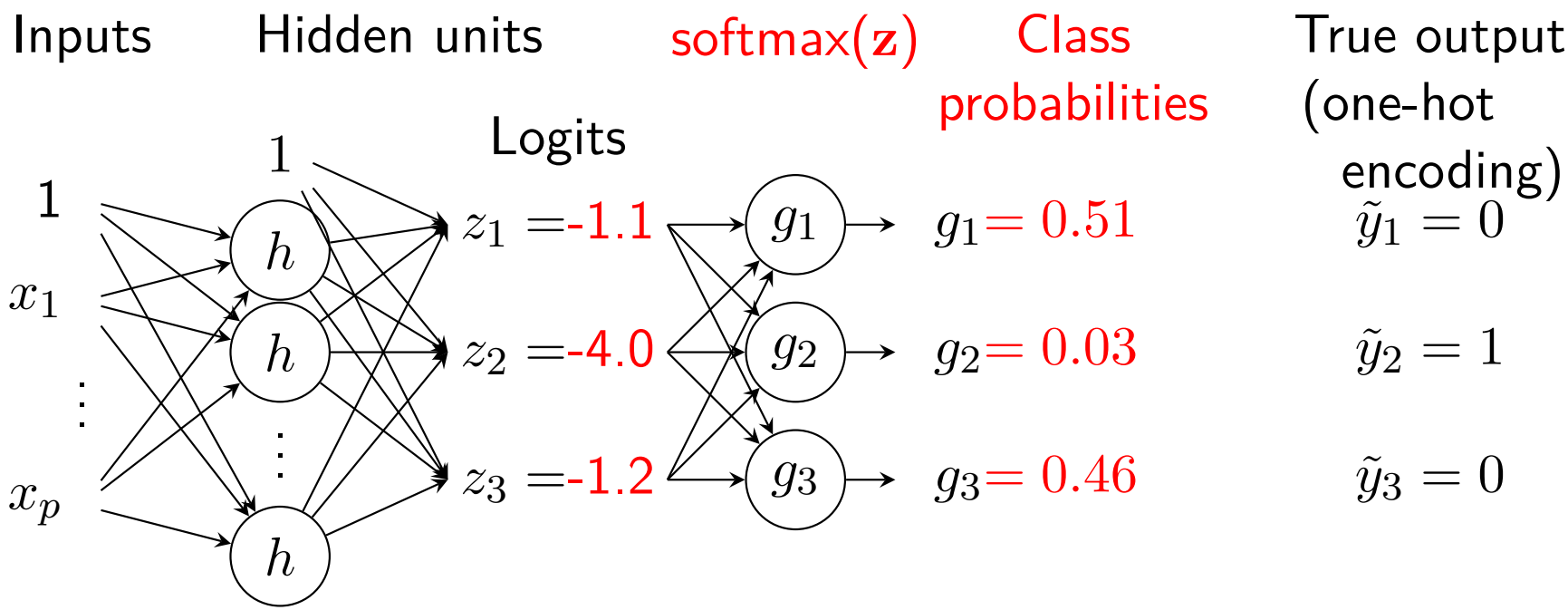
$$\tilde{y}_m = \begin{cases} 1 & \text{if } y = m \\ 0 & \text{if } y \neq m \end{cases}$$

Example: We have $M = 3$ classes and if belongs to ...

- ...$y = 1$, then $\tilde{\mathbf{y}} = [1,\ 0,\ 0]^\mathsf{T}$.
- ...$y = 2$, then $\tilde{\mathbf{y}} = [0,\ 1,\ 0]^\mathsf{T}$.
- ...$y = 3$, then $\tilde{\mathbf{y}} = [0,\ 0,\ 1]^\mathsf{T}$.

NN classification with $M = 3$ classes (II/II)

Consider an example with three classes $M = 3$ where $y = 2$.

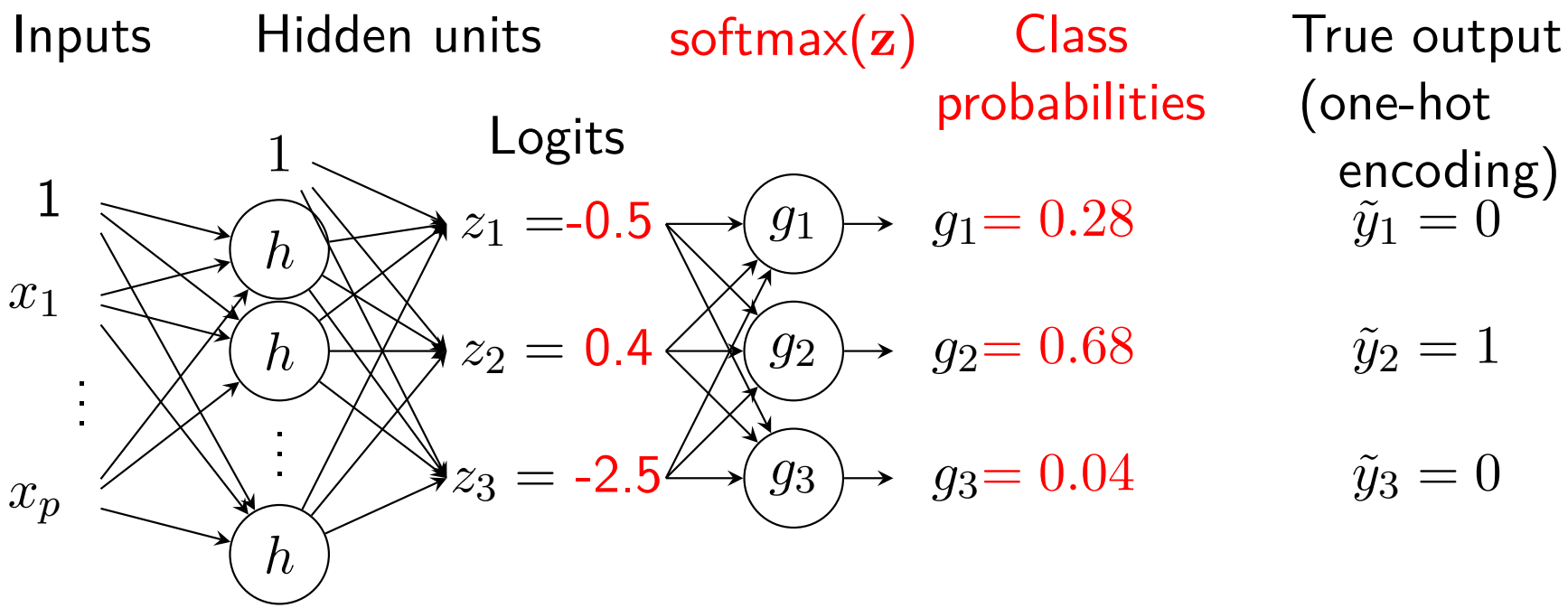


predicted class probabilities $\leftrightarrow$ true output



NN classification with $M = 3$ classes (II/II)

Consider an example with three classes $M = 3$ where $y = 2$.

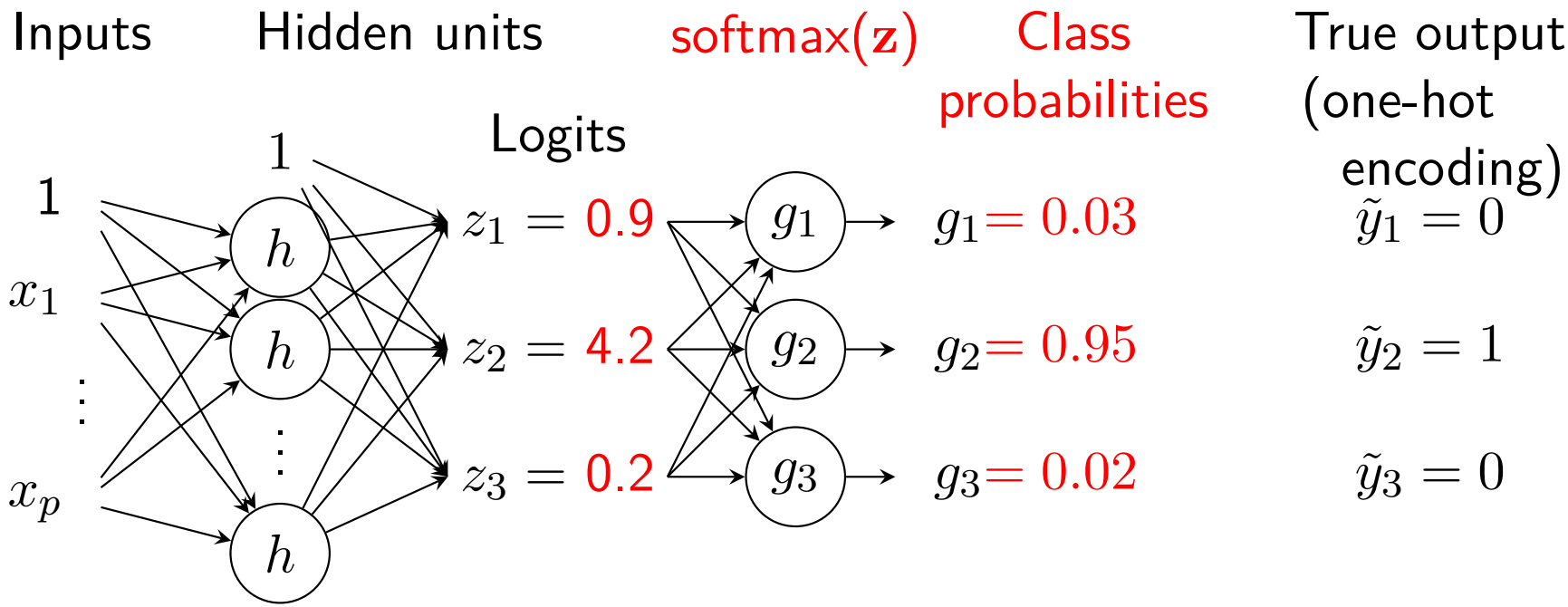


predicted class probabilities $\leftrightarrow$ true output



NN classification with $M = 3$ classes (II/II)

Consider an example with three classes $M = 3$ where $y = 2$.

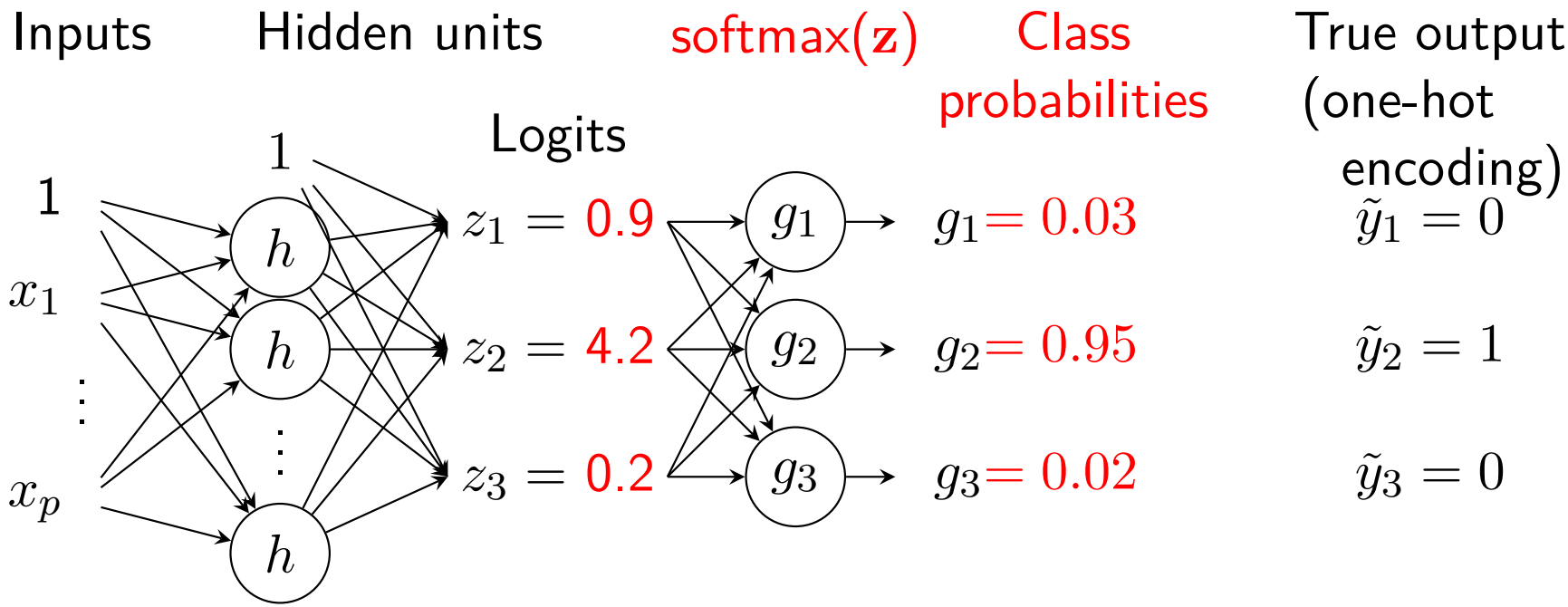


predicted class probabilities $\leftrightarrow$ true output



NN classification with $M = 3$ classes (II/II)

Consider an example with three classes $M = 3$ where $y = 2$.



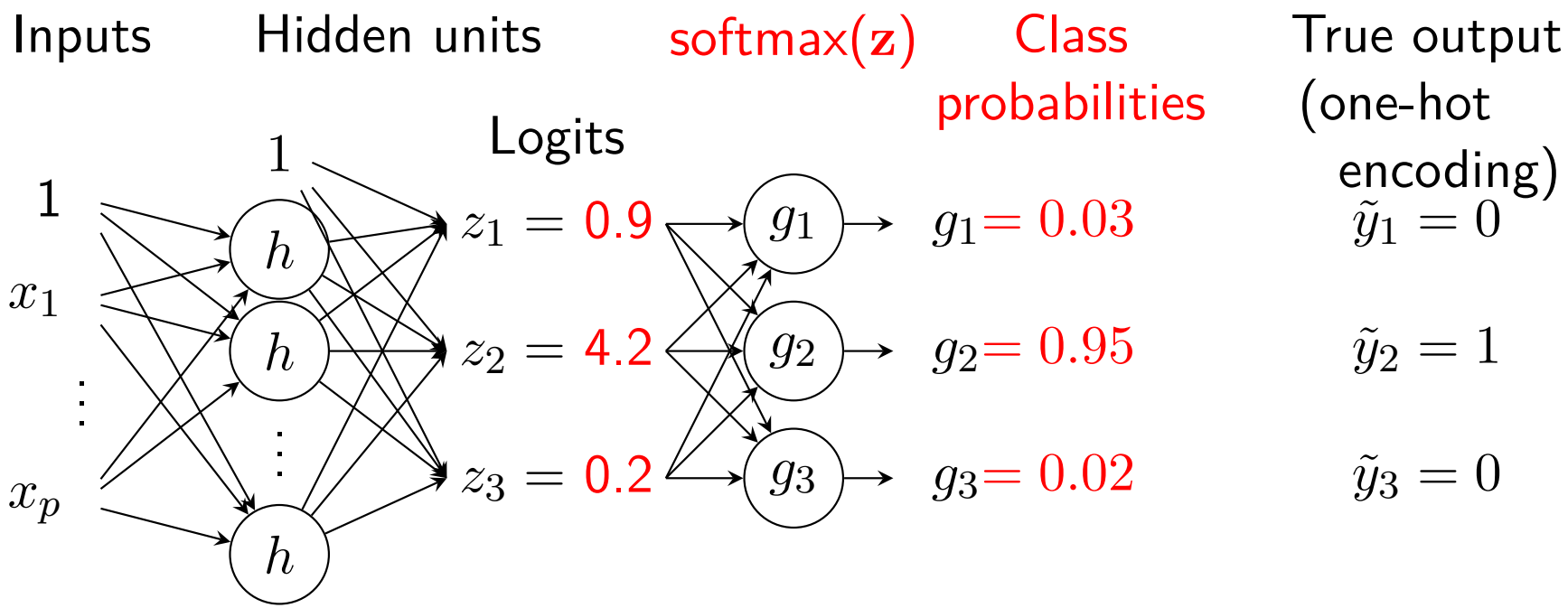
predicted class probabilities $\leftrightarrow$ true output



Q:Which loss do we use between g and $\tilde{y}$?

NN classification with $M = 3$ classes (II/II)

Consider an example with three classes $M = 3$ where $y = 2$.



predicted class probabilities $\leftrightarrow$ true output

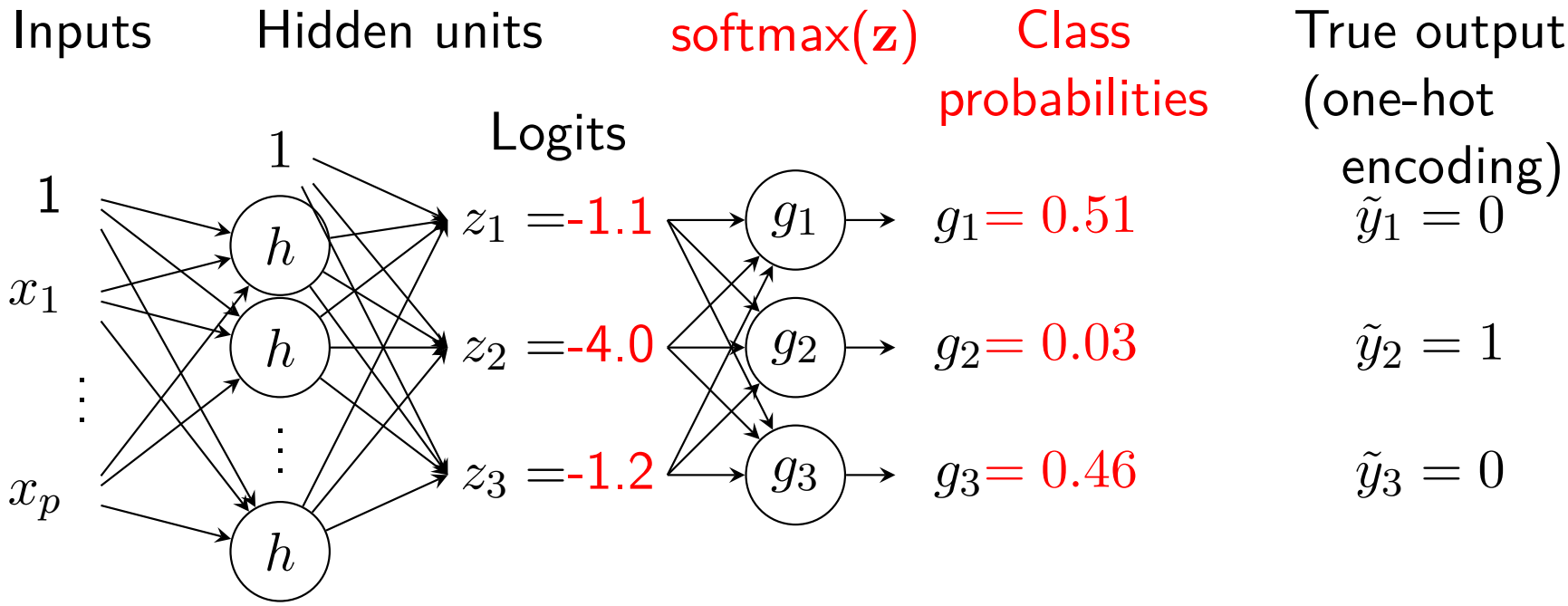


Q:Which loss do we use between g and $\tilde{y}$?

A: Derive using maximum likelihood! $\Rightarrow$ Cross-entropy loss

NN classification with $M = 3$ classes (II/II)

Consider an example with three classes $M = 3$ where $y = 2$.



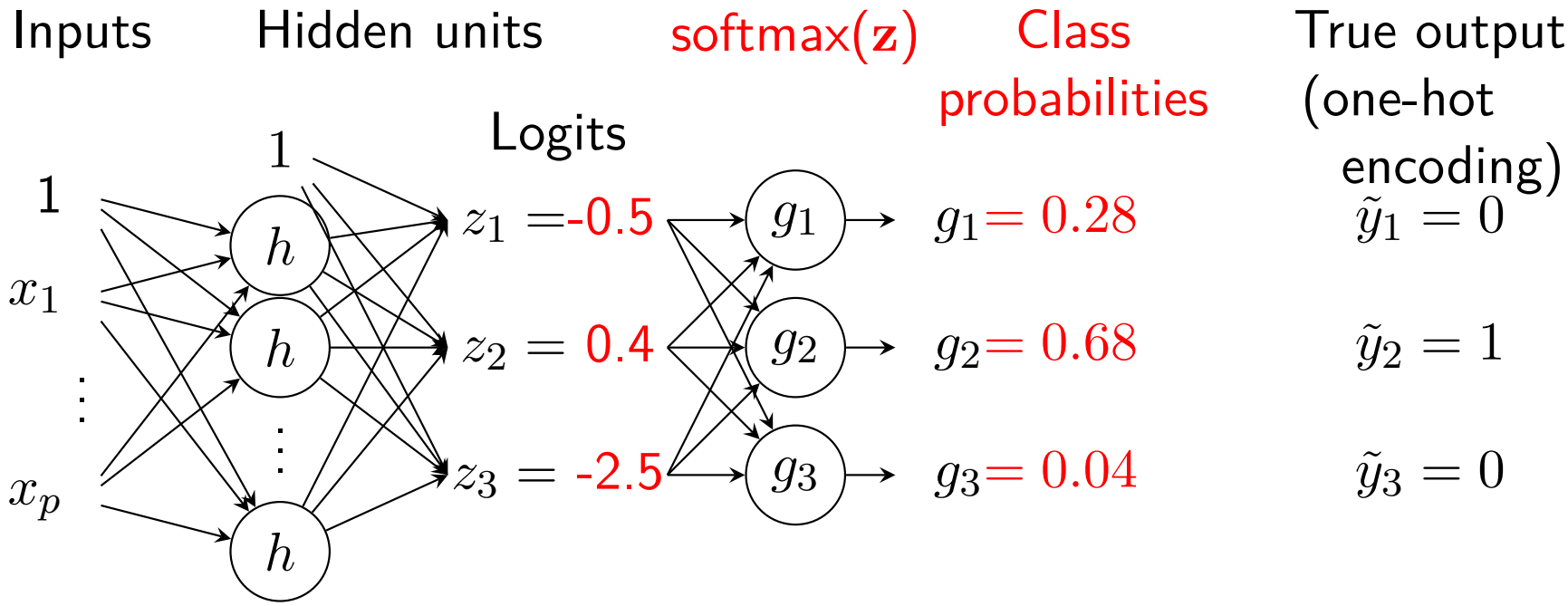
The network is trained by minimizing the **cross-entropy**

$$L(\tilde{\mathbf{y}}, \mathbf{g}) = -\sum_{m=1}^{M} \tilde{y}_m \ln(g_m) = -\ln 0.03 = 3.51$$



NN classification with $M = 3$ classes (II/II)

Consider an example with three classes $M = 3$ where $y = 2$.



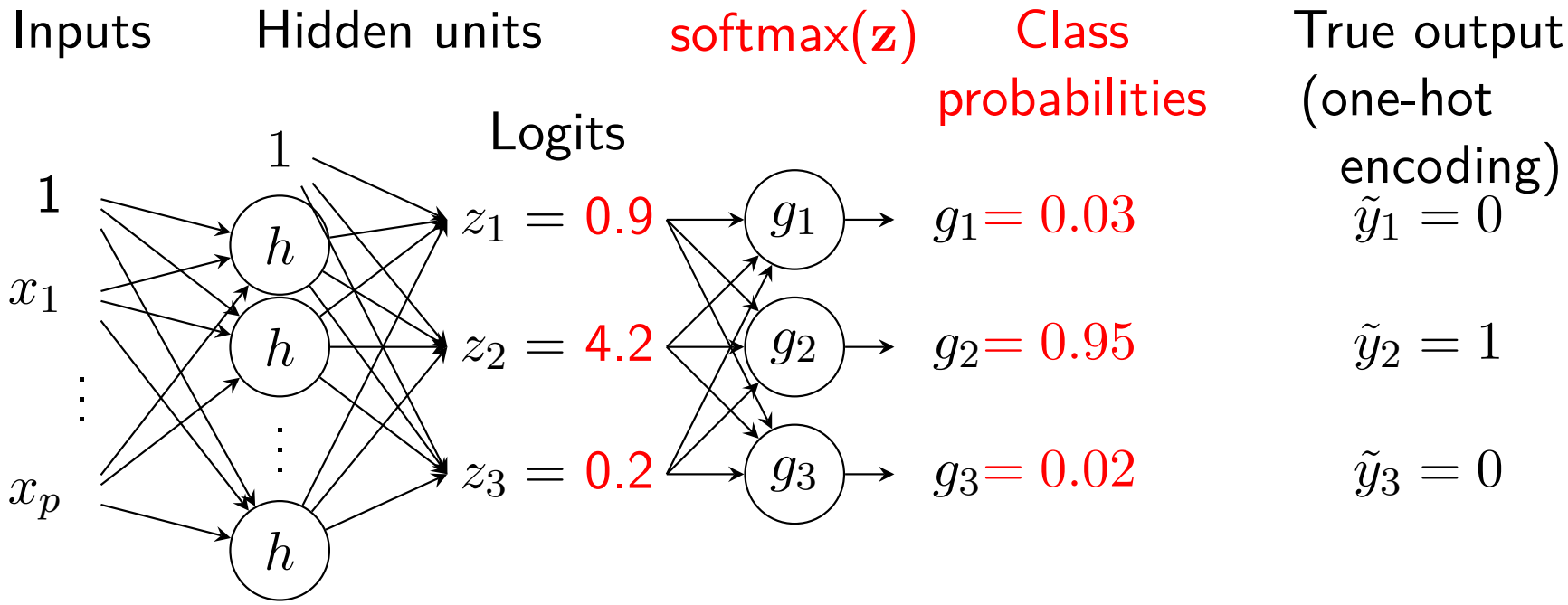
The network is trained by minimizing the **cross-entropy**

$$L(\tilde{\mathbf{y}}, \mathbf{g}) = -\sum_{m=1}^{M} \tilde{y}_m \ln(g_m) = -\ln 0.68 = 0.39$$



NN classification with $M = 3$ classes (II/II)

Consider an example with three classes $M = 3$ where $y = 2$.



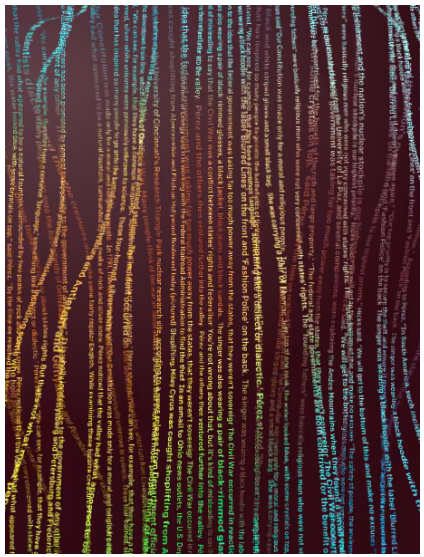
The network is trained by minimizing the **cross-entropy**

$$L(\tilde{\mathbf{y}}, \mathbf{g}) = -\sum_{m=1}^{M} \tilde{y}_m \ln(g_m) = -\ln 0.95 = 0.05$$



Example: Language models

One result on the use of deep learning for language modeling



Language Models - February 2019
Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I.
Language Models are Unsupervised Multitask Learners.,

February, 2019.

See also blog post:
`https://blog.openai.com/better-language-models/`

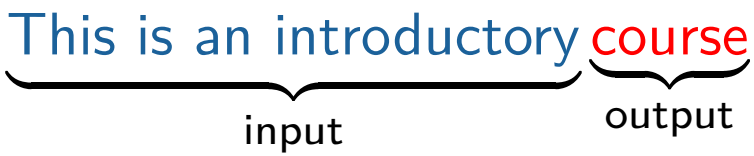
Language models - Background

A language model is a probabilistic model over sequences of words.

Language models - Background

A language model is a probabilistic model over sequences of words.

Training



Language models - Background

A language model is a probabilistic model over sequences of words.

Training

$$\underbrace{\text{This is an introductory course}}_{\text{input}} \underbrace{\text{to}}_{\text{output}}$$

Language models - Background

A language model is a probabilistic model over sequences of words.

Training

$$\underbrace{\text{This is an introductory course to}}_{\text{input}}\ \underbrace{\text{statistical}}_{\text{output}}$$

Language models - Background

A language model is a probabilistic model over sequences of words.

Training

$$\underbrace{\text{This is an introductory course to statistical}}_{\text{input}}\ \underbrace{\text{machine}}_{\text{output}}$$

Language models - Background

A language model is a probabilistic model over sequences of words.

Training

$$\underbrace{\text{This is an introductory course to statistical machine}}_{\text{input}}\ \underbrace{\text{learning}}_{\text{output}}$$

Language models - Background

A language model is a probabilistic model over sequences of words.

Training

$$\underbrace{\text{This is an introductory course to statistical machine}}_{\text{input}}\ \underbrace{\text{learning}}_{\text{output}}$$

The model can be used to hallucinate new texts.

Prediction

$$\underbrace{\text{Machine learning is the scientific}}_{\text{input}}$$

Language models - Background

A language model is a probabilistic model over sequences of words.

Training

$$\underbrace{\text{This is an introductory course to statistical machine}}_{\text{input}}\ \underbrace{\text{learning}}_{\text{output}}$$

The model can be used to hallucinate new texts.

Prediction

$$\underbrace{\text{Machine learning is the scientific}}_{\text{input}}\ \underbrace{\text{dicipline}}_{\text{prediction}}$$

Language models - Background

A language model is a probabilistic model over sequences of words.

Training

$$\underbrace{\text{This is an introductory course to statistical machine}}_{\text{input}}\ \underbrace{\text{learning}}_{\text{output}}$$

The model can be used to hallucinate new texts.

Prediction

$$\underbrace{\text{Machine learning is the scientific dicipline}}_{\text{input}}$$

Language models - Background

A language model is a probabilistic model over sequences of words.

Training

$$\underbrace{\text{This is an introductory course to statistical machine}}_{\text{input}}\ \underbrace{\text{learning}}_{\text{output}}$$

The model can be used to hallucinate new texts.

Prediction

$$\underbrace{\text{Machine learning is the scientific dicipline}}_{\text{input}}\ \underbrace{\text{that}}_{\text{prediction}}$$

Language models - Background

A language model is a probabilistic model over sequences of words.

Training

$$\underbrace{\text{This is an introductory course to statistical machine}}_{\text{input}}\ \underbrace{\text{learning}}_{\text{output}}$$

The model can be used to hallucinate new texts.

Prediction

$$\underbrace{\text{Machine learning is the scientific dicipline that}}_{\text{input}}$$

Language models - Background

A language model is a probabilistic model over sequences of words.

Training

$$\underbrace{\text{This is an introductory course to statistical machine}}_{\text{input}}\ \underbrace{\text{learning}}_{\text{output}}$$

The model can be used to hallucinate new texts.

Prediction

$$\underbrace{\text{Machine learning is the scientific dicipline that}}_{\text{input}}\ \underbrace{\text{explores}}_{\text{prediction}}$$

Language models - Background

A language model is a probabilistic model over sequences of words.

Training

$$\underbrace{\text{This is an introductory course to statistical machine}}_{\text{input}}\ \underbrace{\text{learning}}_{\text{output}}$$

The model can be used to hallucinate new texts.

Prediction

$$\underbrace{\text{Machine learning is the scientific dicipline that explores}}_{\text{input}}$$

Language models - Background

A language model is a probabilistic model over sequences of words.

Training

$$\underbrace{\text{This is an introductory course to statistical machine}}_{\text{input}}\ \underbrace{\text{learning}}_{\text{output}}$$

The model can be used to hallucinate new texts.

Prediction

$$\underbrace{\text{Machine learning is the scientific dicipline that explores}}_{\text{input}}\ \underbrace{\text{how}}_{\text{prediction}}$$

Language models - Background

A language model is a probabilistic model over sequences of words.

Training

$$\underbrace{\text{This is an introductory course to statistical machine}}_{\text{input}}\ \underbrace{\text{learning}}_{\text{output}}$$

The model can be used to hallucinate new texts.

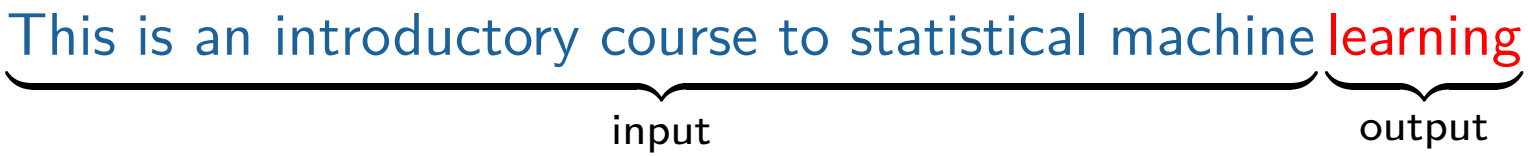
Prediction

$$\underbrace{\text{Machine learning is the scientific dicipline that explores how}}_{\text{input}}$$

Language models - Background

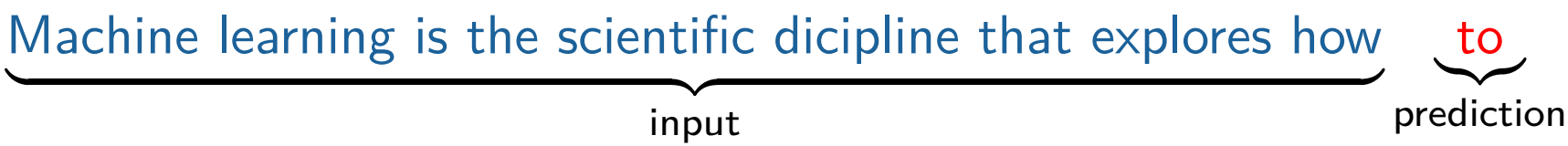
A language model is a probabilistic model over sequences of words.

Training



The model can be used to hallucinate new texts.

Prediction



Language models - Background

A language model is a probabilistic model over sequences of words.

Training

$$\underbrace{\text{This is an introductory course to statistical machine}}_{\text{input}}\ \underbrace{\text{learning}}_{\text{output}}$$

The model can be used to hallucinate new texts.

Prediction

$$\underbrace{\text{Machine learning is the scientific}}_{\text{input}}$$

Language models - Background

A language model is a probabilistic model over sequences of words.

Training

$$\underbrace{\text{This is an introductory course to statistical machine}}_{\text{input}}\ \underbrace{\text{learning}}_{\text{output}}$$

The model can be used to hallucinate new texts.

Prediction

$$\underbrace{\text{Machine learning is the scientific}}_{\text{input}}\ \underbrace{\text{study}}_{\text{prediction}}$$

Language models - Background

A language model is a probabilistic model over sequences of words.

Training

$$\underbrace{\text{This is an introductory course to statistical machine}}_{\text{input}}\ \underbrace{\text{learning}}_{\text{output}}$$

The model can be used to hallucinate new texts.

Prediction

$$\underbrace{\text{Machine learning is the scientific study}}_{\text{input}}$$

Language models - Background

A language model is a probabilistic model over sequences of words.

Training

$$\underbrace{\text{This is an introductory course to statistical machine}}_{\text{input}}\ \underbrace{\text{learning}}_{\text{output}}$$

The model can be used to hallucinate new texts.

Prediction

$$\underbrace{\text{Machine learning is the scientific study}}_{\text{input}}\ \underbrace{\text{of}}_{\text{prediction}}$$

Language models - Background

A language model is a probabilistic model over sequences of words.

Training

$$\underbrace{\text{This is an introductory course to statistical machine}}_{\text{input}}\ \underbrace{\text{learning}}_{\text{output}}$$

The model can be used to hallucinate new texts.

Prediction

$$\underbrace{\text{Machine learning is the scientific study of}}_{\text{input}}$$

Language models - Background

A language model is a probabilistic model over sequences of words.

Training

$$\underbrace{\text{This is an introductory course to statistical machine}}_{\text{input}}\ \underbrace{\text{learning}}_{\text{output}}$$

The model can be used to hallucinate new texts.

Prediction

$$\underbrace{\text{Machine learning is the scientific study of}}_{\text{input}}\ \underbrace{\text{algorithms}}_{\text{prediction}}$$

Language models - Background

A language model is a probabilistic model over sequences of words.

Training

$$\underbrace{\text{This is an introductory course to statistical machine}}_{\text{input}}\ \underbrace{\text{learning}}_{\text{output}}$$

The model can be used to hallucinate new texts.

Prediction

$$\underbrace{\text{Machine learning is the scientific study of algorithms}}_{\text{input}}$$

Language models - Background

A language model is a probabilistic model over sequences of words.

Training

$$\underbrace{\text{This is an introductory course to statistical machine}}_{\text{input}}\ \underbrace{\text{learning}}_{\text{output}}$$

The model can be used to hallucinate new texts.

Prediction

$$\underbrace{\text{Machine learning is the scientific study of algorithms}}_{\text{input}}\ \underbrace{\text{and}}_{\text{prediction}}$$

Language models - Background

A language model is a probabilistic model over sequences of words.

Training

$$\underbrace{\text{This is an introductory course to statistical machine}}_{\text{input}}\ \underbrace{\text{learning}}_{\text{output}}$$

The model can be used to hallucinate new texts.

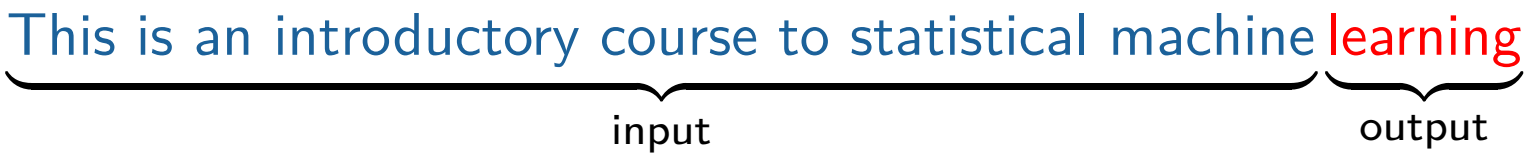
Prediction

$$\underbrace{\text{Machine learning is the scientific study of algorithms and}}_{\text{input}}$$

Language models - Background

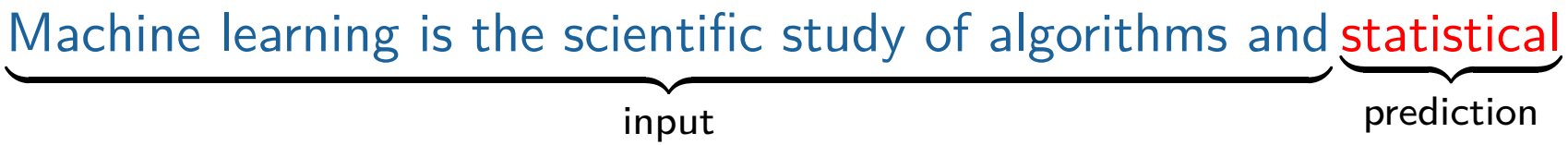
A language model is a probabilistic model over sequences of words.

Training



The model can be used to hallucinate new texts.

Prediction



Language models - Background

A language model is a probabilistic model over sequences of words.

Training

$$\underbrace{\text{This is an introductory course to statistical machine}}_{\text{input}}\ \underbrace{\text{learning}}_{\text{output}}$$

The model can be used to hallucinate new texts.

Prediction

$$\underbrace{\text{Machine learning is the scientific study of algorithms and}}_{\text{input}}\ \underbrace{\text{statistical}}_{\text{prediction}}$$

In the paper they trained a huge language model with 1.5 billion (!) parameters using 40 GB of text data from internet.

Language models - Background

A language model is a probabilistic model over sequences of words.

Training

$$\underbrace{\text{This is an introductory course to statistical machine}}_{\text{input}}\ \underbrace{\text{learning}}_{\text{output}}$$

The model can be used to hallucinate new texts.

Prediction

$$\underbrace{\text{Machine learning is the scientific study of algorithms and}}_{\text{input}}\ \underbrace{\text{statistical}}_{\text{prediction}}$$

In the paper they trained a huge language model with 1.5 billion (!) parameters using 40 GB of text data from internet.

Can also be used for Reading Comprehension, Language Translation, Question Answering etc...

Language models - Synthetic text (I/IV)

System Prompt (human-written) *In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.*

Model Completion (machine-written)
The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science.

Now, after almost two centuries, the mystery of what sparked this odd phenomenon is finally solved.

...

Language models - Synthetic text (II/IV)

...

Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans. Pérez noticed that the valley had what appeared to be a natural fountain, surrounded by two peaks of rock and silver snow.

Pérez and the others then ventured further into the valley. "By the time we reached the top of one peak, the water looked blue, with some crystals on top," said Pérez.

...

Language models - Synthetic text (III/IV)

...

Pérez and his friends were astonished to see the unicorn herd. These creatures could be seen from the air without having to move too much to see them - they were so close they could touch their horns.

While examining these bizarre creatures the scientists discovered that the creatures also spoke some fairly regular English. Pérez stated, "We can see, for example, that they have a common 'language,' something like a dialect or dialectic."

Dr. Pérez believes that the unicorns may have originated in Argentina, where the animals were believed to be descendants of a lost race of people who lived there before the arrival of humans in those parts of South America.

...

Language models - Synthetic text (IV/IV)

...
While their origins are still unclear, some believe that perhaps the creatures were created when a human and a unicorn met each other in a time before human civilization. According to Pérez, "In South America, such incidents seem to be quite common."

However, Pérez also pointed out that it is likely that the only way of knowing for sure if unicorns are indeed the descendants of a lost alien race is through DNA. "But they seem to be able to communicate in English quite well, which I believe is a sign of evolution, or at least a change in social organization," said the scientist.

Language models - policy implication

Could be used for

- AI writing assistants
- More capable dialogue agents
- Unsupervised translation between languages
- Better speech recognition systems

Language models - policy implication

Could be used for

- AI writing assistants
- More capable dialogue agents
- Unsupervised translation between languages
- Better speech recognition systems

Potential malicious purposes

- Generate misleading news articles
- Impersonate others online
- Automate the production of faked content on social media
- Automate the production of spam/phishing content

Language models - policy implication

Could be used for

- AI writing assistants
- More capable dialogue agents
- Unsupervised translation between languages
- Better speech recognition systems

Potential malicious purposes

- Generate misleading news articles
- Impersonate others online
- Automate the production of faked content on social media
- Automate the production of spam/phishing content

In May 2020 Open AI realised an improved model with 175 bilion (!!!) parameters!

Some comments - Why now?

Neural networks have been around for more than fifty years. Why have they become so popular now (again)?

To solve really interesting problems you need:

1. Efficient learning **algorithms**
2. Efficient computational **hardware**
3. A lot of labeled **data**!

These three factors have not been fulfilled to a satisfactory level until the last 5-10 years.

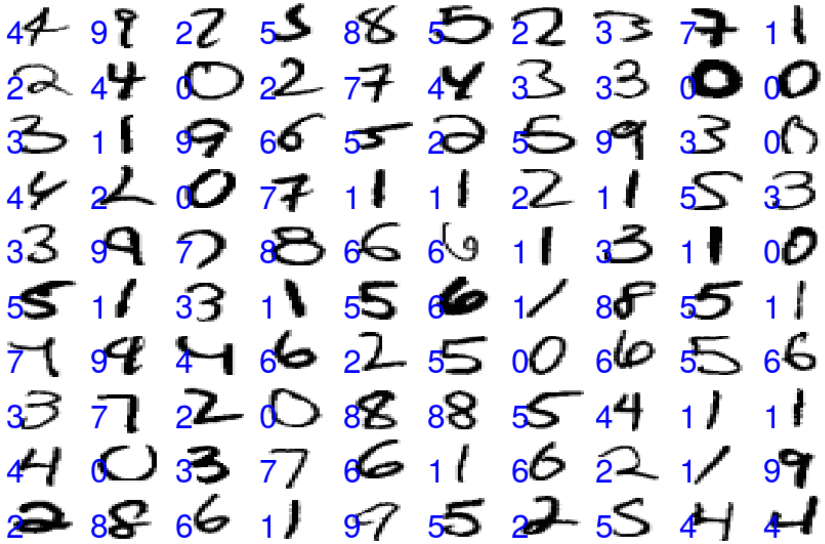
UPPSALA UNIVERSITET

The lab

Topic: Image classification with neural networks

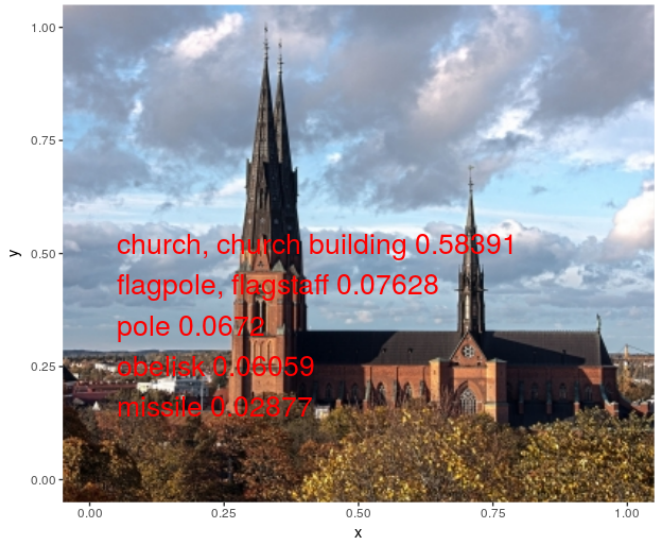
Task 1

Classification of hand-written digits



Task 2

Real world image classification



Summary – Lecture 7

1. **This lecture:** The neural network model
   - Neural network for regression
   - Neural network for classification

Summary – Lecture 7

1. **This lecture:** The neural network model
   - Neural network for regression
   - Neural network for classification
2. **Next lecture:**
   - Convolutional neural network
   - How to train a neural network

A few concepts to summarize lecture 7

Neural network (NN): A nonlinear parametric model constructed by stacking several linear models with intermediate nonlinear activation functions.

Activation function: (a.k.a squashing function) A nonlinear scalar function applied to each output element of the linear models in a NN.

Hidden units: Intermediate variables in the NN which are not observed, i.e., belongs neither to the input nor output data.

Softmax: A function used to transform the output of a NN to class probabilities.

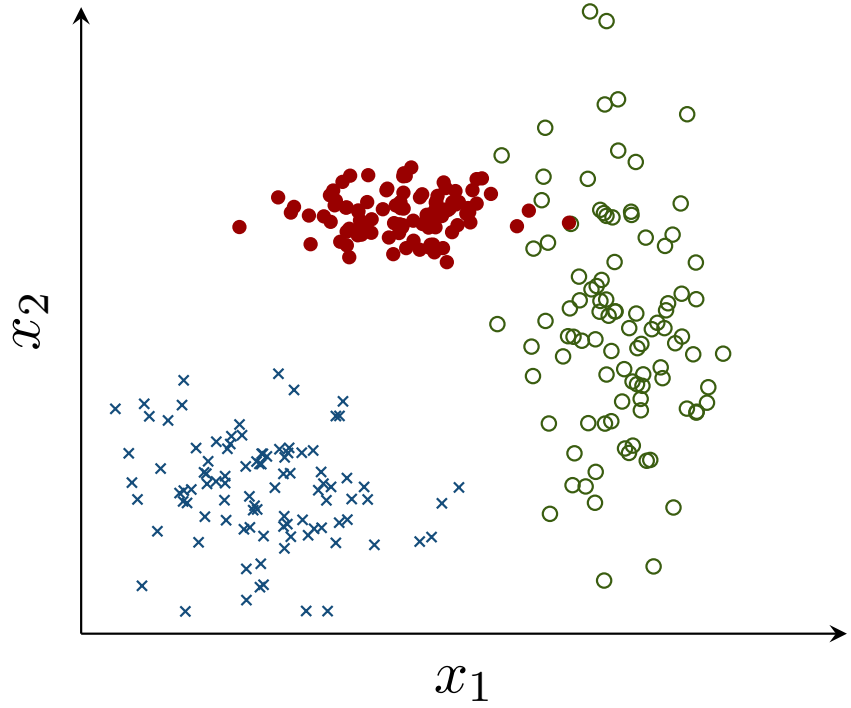
One-hot encoding: An encoding of the output for training NNs with softmax output.

Softmax regression - A multi-class problem

Consider a classification problem with 3 classes

Input: $\mathbf{x} = [x_1, x_2] \in \mathbb{R}^2$

Output: $y = \{$"red", "blue", "green"$\}$



Can we extend logistic regression to handle more than 2 classes?

Softmax regression

Consider a multi-class classification problem $y \in \{1, \ldots, M\}$.

$$g_{im} = p(y_i = m | \mathbf{x}_i)$$

Softmax regression

Consider a multi-class classification problem $y \in \{1, \ldots, M\}$.

$$g_{im} = p(y_i = m | \mathbf{x}_i)$$

Softmax regression Assume a linear model for each "log odds"

$$\ln \frac{g_{im}}{Z} = \mathbf{w}_m^{\mathsf{T}} \mathbf{x}_i + b_m, \qquad m = 1, \ldots, M$$

where Z is a **normalization factor** such that $\sum_{m=1}^{M} g_{im} = 1$.

Softmax regression

Consider a multi-class classification problem $y \in \{1, \ldots, M\}$.

$$g_{im} = p(y_i = m | \mathbf{x}_i)$$

Softmax regression Assume a linear model for each "log odds"

$$\ln \frac{g_{im}}{Z} = \mathbf{w}_m^{\mathsf{T}} \mathbf{x}_i + b_m, \qquad m = 1, \ldots, M$$

where Z is a **normalization factor** such that $\sum_{m=1}^{M} g_{im} = 1$. $\Rightarrow$

$$g_{im} = Z e^{z_{im}}, \quad z_{im} = \mathbf{w}_m^{\mathsf{T}} \mathbf{x}_i + b_m, \qquad m = 1, \ldots, M$$

Softmax regression

Consider a multi-class classification problem $y \in \{1, \ldots, M\}$.

$$g_{im} = p(y_i = m | \mathbf{x}_i)$$

Softmax regression Assume a linear model for each "log odds"

$$\ln \frac{g_{im}}{Z} = \mathbf{w}_m^{\mathsf{T}} \mathbf{x}_i + b_m, \qquad m = 1, \ldots, M$$

where Z is a **normalization factor** such that $\sum_{m=1}^{M} g_{im} = 1$. $\Rightarrow$

$$g_{im} = Z e^{z_{im}}, \quad z_{im} = \mathbf{w}_m^{\mathsf{T}} \mathbf{x}_i + b_m, \qquad m = 1, \ldots, M$$

The normalization factor Z can now be computed as

$$1 = \sum_{m=1}^{M} g_{im} = Z \sum_{m=1}^{M} e^{z_{im}} \quad \Rightarrow \quad Z = \frac{1}{\sum_{m=1}^{M} e^{z_{im}}}$$

Softmax regression

Consider a multi-class classification problem $y \in \{1, \ldots, M\}$.

$$g_{im} = p(y_i = m | \mathbf{x}_i)$$

Softmax regression Assume a linear model for each "log odds"

$$\ln \frac{g_{im}}{Z} = \mathbf{w}_m^\mathsf{T} \mathbf{x}_i + b_m, \qquad m = 1, \ldots, M$$

where Z is a **normalization factor** such that $\sum_{m=1}^{M} g_{im} = 1$. $\Rightarrow$

$$g_{im} = Z e^{z_{im}}, \quad z_{im} = \mathbf{w}_m^\mathsf{T} \mathbf{x}_i + b_m, \qquad m = 1, \ldots, M$$

The normalization factor Z can now be computed as

$$1 = \sum_{m=1}^{M} g_{im} = Z \sum_{m=1}^{M} e^{z_{im}} \quad \Rightarrow \quad Z = \frac{1}{\sum_{m=1}^{M} e^{z_{im}}}$$

Softmax regression model is then

$$g_{im} = \frac{e^{z_{im}}}{\sum_{l=1}^{M} e^{z_{il}}}, \qquad z_{im} = \mathbf{w}_m^\mathsf{T} \mathbf{x}_i + b_m, \qquad m = 1, \ldots, M$$

UPPSALA UNIVERSITET

Softmax regression using maximum limelihood

Pick $\mathbf{w}_m$ and b_m which make data as likely as possible

$$\widehat{\mathbf{w}}_{1:M}, \widehat{b}_{1:M} = \underset{\mathbf{w}_{1:M}, b_{1:M}}{\text{argmax}} \ \ln p(y_{1:n} | \mathbf{x}_{1:n}, \mathbf{w}_{1:M}, b_{1:M})$$

UPPSALA UNIVERSITET

Softmax regression using maximum limelihood

Pick $\mathbf{w}_m$ and b_m which make data as likely as possible

$$\widehat{\mathbf{w}}_{1:M}, \widehat{b}_{1:M} = \underset{\mathbf{w}_{1:M}, b_{1:M}}{\text{argmax}} \ \ln p(y_{1:n} | \mathbf{x}_{1:n}, \mathbf{w}_{1:M}, b_{1:M})$$

Softmax regression using maximum limelihood

Pick $\mathbf{w}_m$ and b_m which make data as likely as possible

$$\widehat{\mathbf{w}}_{1:M}, \widehat{b}_{1:M} = \underset{\mathbf{w}_{1:M}, b_{1:M}}{\text{argmax}} \ \ln p(y_{1:n} | \mathbf{x}_{1:n}, \mathbf{w}_{1:M}, b_{1:M})$$

Assume all y_i to be independent

$$\ln p(y_{1:n} | \mathbf{x}_{1:n}, \mathbf{w}_{1:M}, b_{1:M}) = \sum_{i=1}^{n} \ln p(y_i | \mathbf{x}_i, \mathbf{w}_{1:M}, b_{1:M})$$

Softmax regression using maximum limelihood

Pick $\mathbf{w}_m$ and b_m which make data as likely as possible

$$\widehat{\mathbf{w}}_{1:M}, \widehat{b}_{1:M} = \underset{\mathbf{w}_{1:M}, b_{1:M}}{\text{argmax}} \ \ln p(y_{1:n} | \mathbf{x}_{1:n}, \mathbf{w}_{1:M}, b_{1:M})$$

Assume all y_i to be independent

$$\ln p(y_{1:n} | \mathbf{x}_{1:n}, \mathbf{w}_{1:M}, b_{1:M}) = \sum_{i=1}^{n} \ln p(y_i | \mathbf{x}_i, \mathbf{w}_{1:M}, b_{1:M})$$

$$= \sum_{m=1}^{M} \sum_{\substack{i=1 \\ \text{where} \\ y_i = m}}^{n} \ln \underbrace{p(y_i = m | \mathbf{x}_i, \mathbf{w}_{1:M}, b_{1:M})}_{= g_{im}}$$

Softmax regression using maximum limelihood

Pick $\mathbf{w}_m$ and b_m which make data as likely as possible

$$\widehat{\mathbf{w}}_{1:M}, \widehat{b}_{1:M} = \underset{\mathbf{w}_{1:M}, b_{1:M}}{\text{argmax}} \ \ln p(y_{1:n} | \mathbf{x}_{1:n}, \mathbf{w}_{1:M}, b_{1:M})$$

Assume all y_i to be independent

$$\begin{aligned}\ln p(y_{1:n} | \mathbf{x}_{1:n}, \mathbf{w}_{1:M}, b_{1:M}) &= \sum_{i=1}^{n} \ln p(y_i | \mathbf{x}_i, \mathbf{w}_{1:M}, b_{1:M}) \\ &= \sum_{m=1}^{M} \sum_{\substack{i=1 \\ \text{where} \\ y_i = m}}^{n} \ln \underbrace{p(y_i = m | \mathbf{x}_i, \mathbf{w}_{1:M}, b_{1:M})}_{= g_{im}}\end{aligned}$$

This leads to the following optimization problem

$$\widehat{\mathbf{w}}_{1:M}, \widehat{b}_{1:M} = \underset{\mathbf{w}_{1:M}, b_{1:M}}{\text{argmin}} \ -\sum_{i=1}^{n} \sum_{m=1}^{M} \tilde{y}_{im} \ln(g_{im}),$$

Softmax regression using maximum limelihood

Pick $\mathbf{w}_m$ and b_m which make data as likely as possible

$$\widehat{\mathbf{w}}_{1:M}, \widehat{b}_{1:M} = \underset{\mathbf{w}_{1:M}, b_{1:M}}{\text{argmax}} \ \ln p(y_{1:n} | \mathbf{x}_{1:n}, \mathbf{w}_{1:M}, b_{1:M})$$

Assume all y_i to be independent

$$\ln p(y_{1:n} | \mathbf{x}_{1:n}, \mathbf{w}_{1:M}, b_{1:M}) = \sum_{i=1}^{n} \ln p(y_i | \mathbf{x}_i, \mathbf{w}_{1:M}, b_{1:M})$$

$$= \sum_{m=1}^{M} \sum_{\substack{i=1 \\ \text{where} \\ y_i = m}}^{n} \ln \underbrace{p(y_i = m | \mathbf{x}_i, \mathbf{w}_{1:M}, b_{1:M})}_{= g_{im}}$$

This leads to the following optimization problem

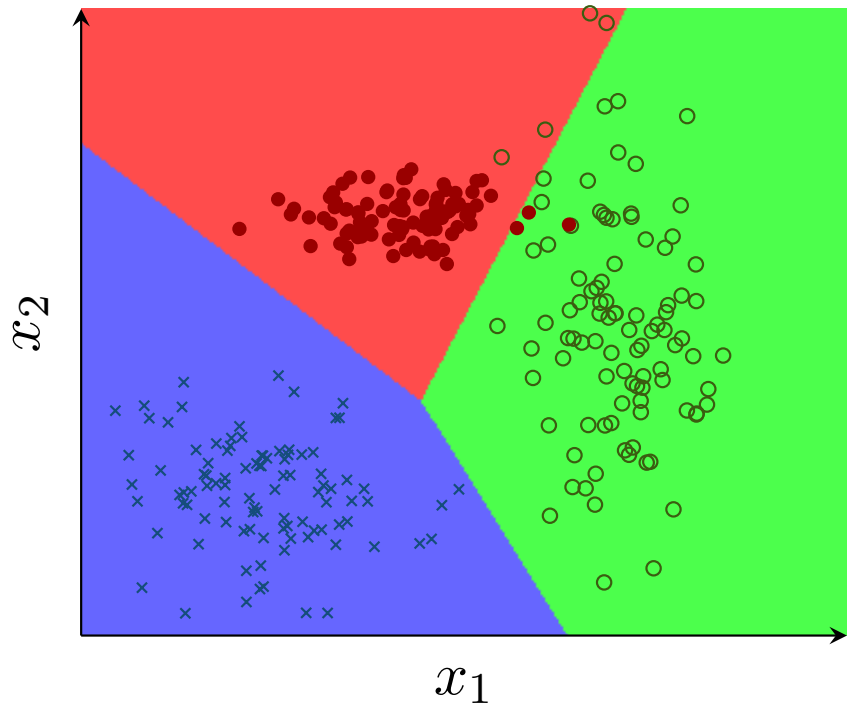
$$\widehat{\mathbf{w}}_{1:M}, \widehat{b}_{1:M} = \underset{\mathbf{w}_{1:M}, b_{1:M}}{\text{argmin}} \ -\sum_{i=1}^{n} \sum_{m=1}^{M} \tilde{y}_{im} \ln(g_{im}),$$

One-hot encoding of output

$$\tilde{y}_{im} = \begin{cases} 1, & \text{if} \quad y_i = m \\ 0, & \text{if} \quad y_i \neq m \end{cases}$$

Softmax regression on multi-class problem

Consider again the classification problem with 3 classes.



Softmax computes the probability for each of the three classes.

Here the result is visualized with **decision boundaries** for the class which has the highest probability.

Statistical Machine Learning

Lecture 8
Convolutional neural networks
Numerical optimization for neural network training



UPPSALA
UNIVERSITET

Paul Häusner
paul.hausner@it.uu.se
Department of Information Technology
Uppsala University

Course webpage

Summary of Lecture 7 (I/IV)
Neural network

A neural network is a sequential construction of several generalized linear regression models.

Inputs Hidden units Output

1

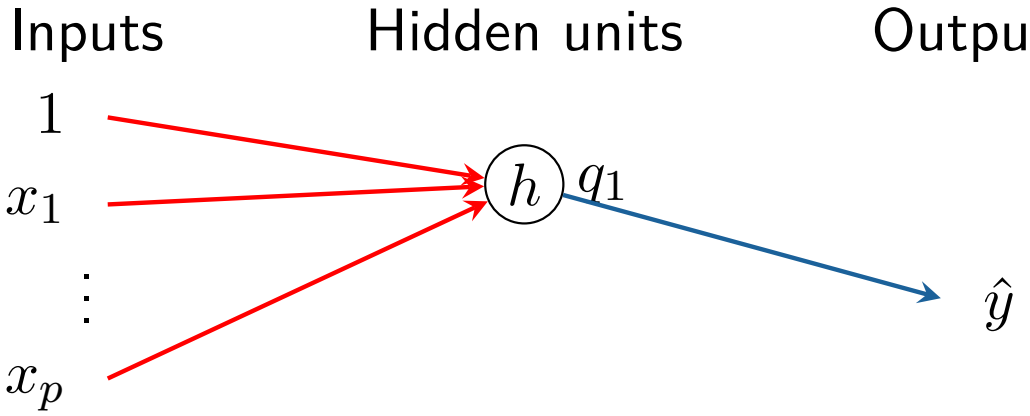
x_1

$\vdots$

x_p

Summary of Lecture 7 (I/IV)
Neural network

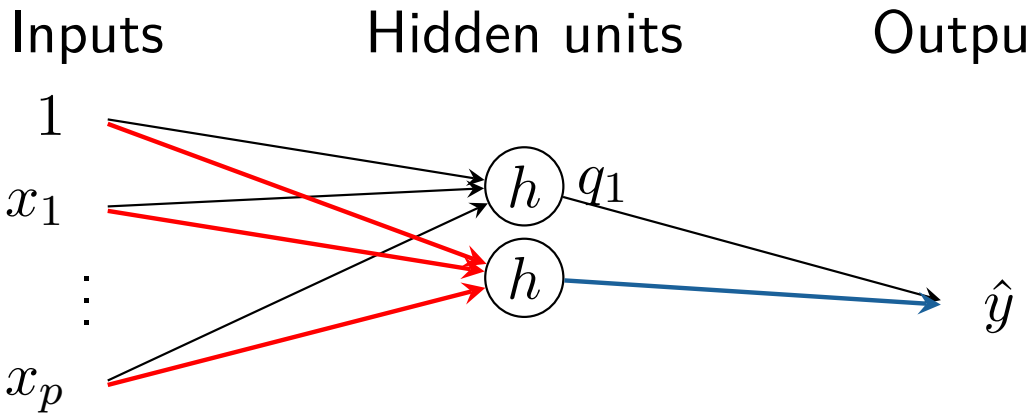
A neural network is a sequential construction of **several** generalized linear regression models.



$$q_1 = h\left(b_1^{(1)} + \sum_{j=1}^{p} w_{1j}^{(1)} x_j\right) \qquad \hat{y} = w_1^{(2)} q_1$$

Summary of Lecture 7 (I/IV)
Neural network

A neural network is a sequential construction of **several** generalized linear regression models.



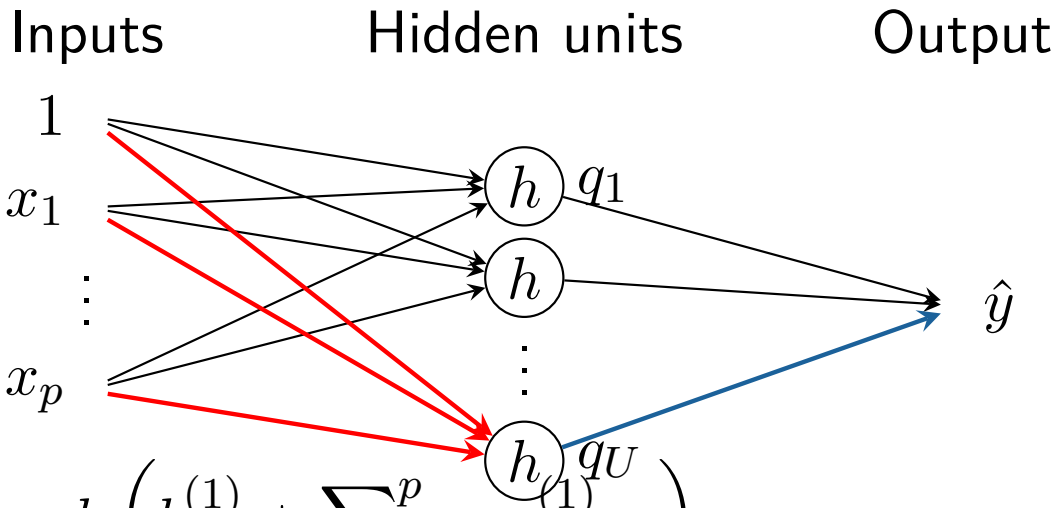
$$q_1 = h\left(b_1^{(1)} + \sum_{j=1}^{p} w_{1j}^{(1)} x_j\right)$$

$$q_2 = h\left(b_2^{(1)} + \sum_{j=1}^{p} w_{2j}^{(1)} x_j\right)$$

$$\hat{y} = \sum_{i=1}^{2} w_i^{(2)} q_i$$

Summary of Lecture 7 (I/IV)
Neural network

A neural network is a sequential construction of **several** generalized linear regression models.



$$q_1 = h\left(b_1^{(1)} + \sum_{j=1}^{p} w_{1j}^{(1)} x_j\right)$$

$$q_2 = h\left(b_2^{(1)} + \sum_{j=1}^{p} w_{2j}^{(1)} x_j\right)$$

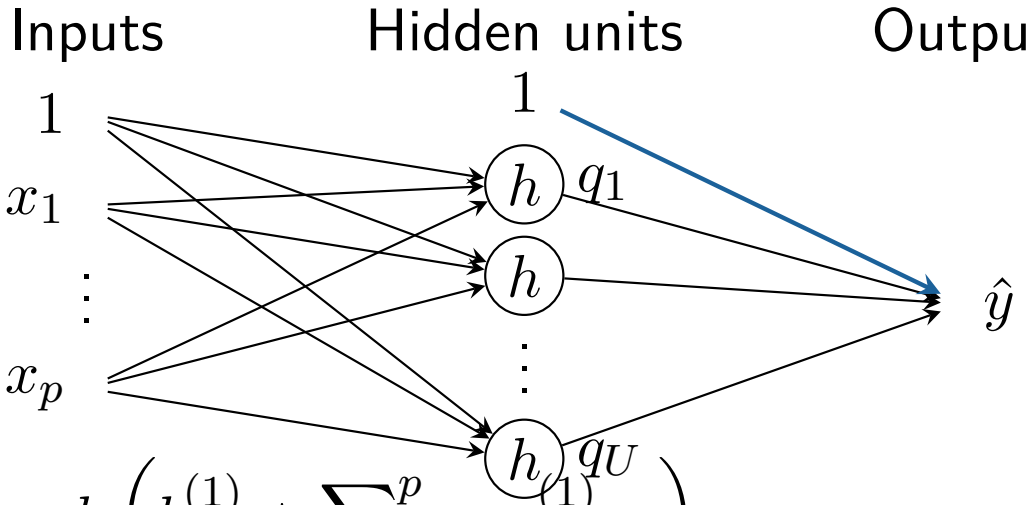
$$\vdots$$

$$q_U = h\left(b_U^{(1)} + \sum_{j=1}^{p} w_{Uj}^{(1)} x_j\right)$$

$$\hat{y} = \sum_{i=1}^{U} w_i^{(2)} q_i$$

Summary of Lecture 7 (I/IV)
Neural network

A neural network is a sequential construction of **several** generalized linear regression models.



$$q_1 = h\left(b_1^{(1)} + \sum_{j=1}^{p} w_{1j}^{(1)} x_j\right)$$

$$q_2 = h\left(b_2^{(1)} + \sum_{j=1}^{p} w_{2j}^{(1)} x_j\right)$$

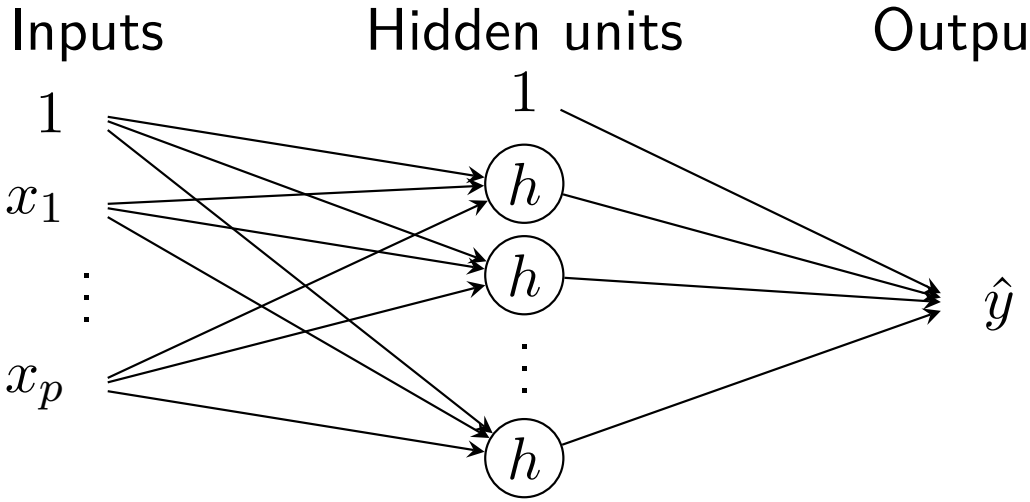
$$\vdots$$

$$q_U = h\left(b_U^{(1)} + \sum_{j=1}^{p} w_{Uj}^{(1)} x_j\right)$$

$$\hat{y} = b^{(2)} + \sum_{i=1}^{U} w_i^{(2)} q_i$$

Summary of Lecture 7 (I/IV)
Neural network

A neural network is a sequential construction of **several** generalized linear regression models.



$$\mathbf{q} = h(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$\hat{y} = \mathbf{W}^{(2)}\mathbf{q} + \mathbf{b}^{(2)}$$

$$\mathbf{W}^{(1)} = \begin{bmatrix} w_{11}^{(1)} & \dots & w_{1p}^{(1)} \\ \vdots & & \vdots \\ w_{U1}^{(1)} & \dots & w_{Up}^{(1)} \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} b_1^{(1)} \\ \vdots \\ b_U^{(1)} \end{bmatrix}, \quad \mathbf{q} = \begin{bmatrix} q_1 \\ \vdots \\ q_U \end{bmatrix}$$

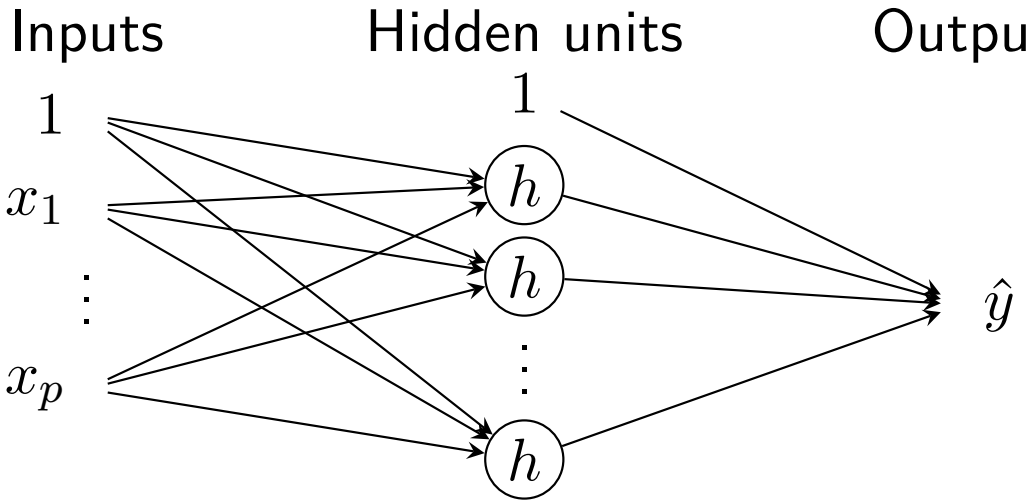
Weight matrix Offset vector Hidden units

$$\mathbf{b}^{(2)} = \begin{bmatrix} b^{(2)} \end{bmatrix}$$

$$\mathbf{W}^{(2)} = \begin{bmatrix} w_1^{(2)} & \dots & w_U^{(2)} \end{bmatrix}$$

Summary of Lecture 7 (I/IV)
Neural network

A neural network is a sequential construction of **several** generalized linear regression models.

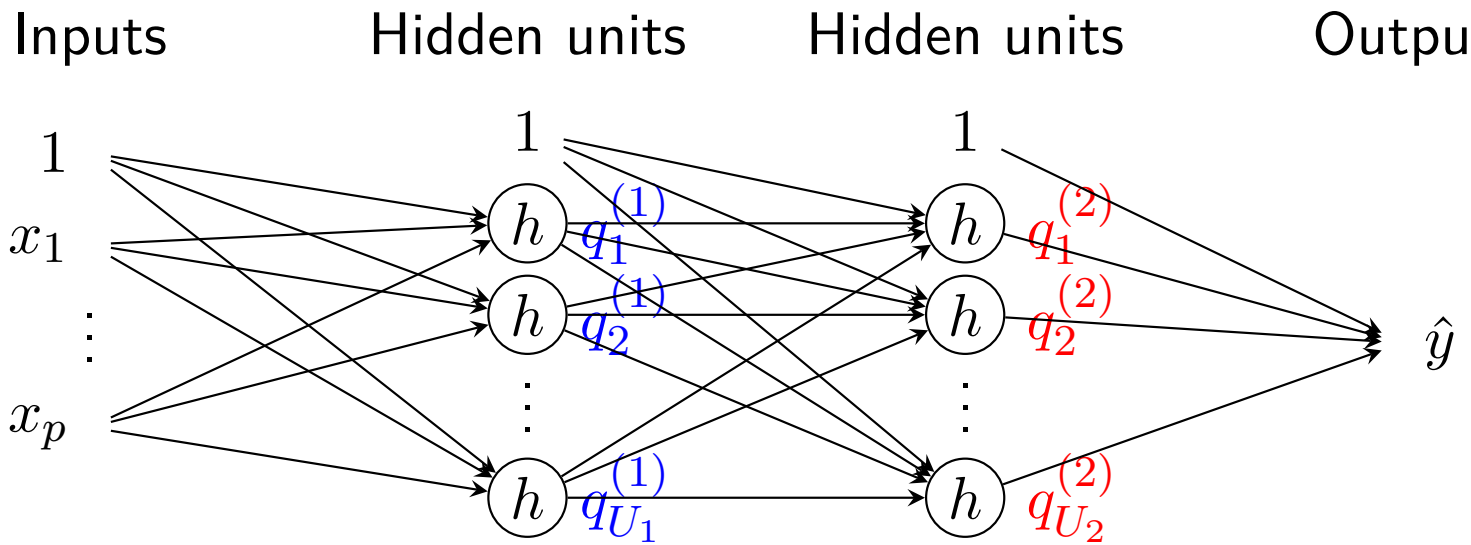


$$\mathbf{q} = h(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$
$$\hat{y} = \mathbf{W}^{(2)}\mathbf{q} + \mathbf{b}^{(2)}$$

The non-linearity h acts element-wise.

Summary of Lecture 7 (I/IV)
Neural network

A neural network is a **sequential** construction of several generalized linear regression models.



$$\mathbf{q}^{(1)} = h(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$
$$\mathbf{q}^{(2)} = h(\mathbf{W}^{(2)}\mathbf{q}^{(1)} + \mathbf{b}^{(2)})$$
$$\hat{y} = \mathbf{W}^{(3)}\mathbf{q}^{(2)} + \mathbf{b}^{(3)}$$

In dense (or fully-connected) layers all input units are connected to all output units.

Summary of Lecture 7 (II/IV)

Parameters = weight matrices and offset vectors

All weight matrices and offset vectors in all layers combined are the parameters of the network

$$\boldsymbol{\theta} = \left[\text{vec}(\mathbf{W}^{(1)})^\mathsf{T}, \quad \mathbf{b}^{(1)\mathsf{T}}, \quad \ldots, \quad \text{vec}(\mathbf{W}^{(L)})^\mathsf{T}, \quad \mathbf{b}^{(L)\mathsf{T}}\right]^\mathsf{T},$$

which constitutes the parametric model $\hat{y} = f(\mathbf{x}; \boldsymbol{\theta})$.

Summary of Lecture 7 (II/IV)

Parameters = weight matrices and offset vectors

All weight matrices and offset vectors in all layers combined are the parameters of the network

$$\boldsymbol{\theta} = \left[\text{vec}(\mathbf{W}^{(1)})^\mathsf{T}, \quad \mathbf{b}^{(1)\mathsf{T}}, \quad \ldots, \quad \text{vec}(\mathbf{W}^{(L)})^\mathsf{T}, \quad \mathbf{b}^{(L)\mathsf{T}}\right]^\mathsf{T},$$

which constitutes the parametric model $\hat{y} = f(\mathbf{x}; \boldsymbol{\theta})$.

Training

We train a network on training data $\{\mathbf{x}_i, y_i\}_{i=1}^n$ by considering the optimization problem

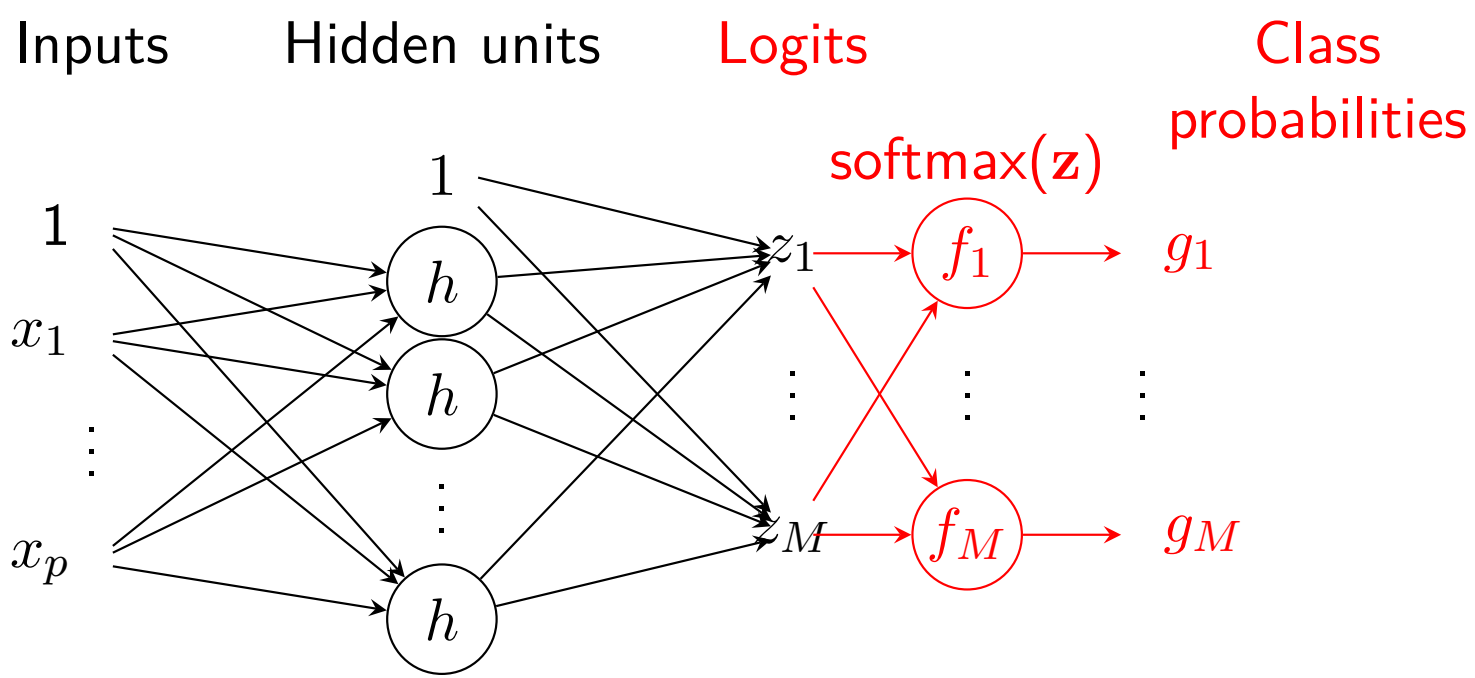
$$\widehat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta}), \qquad J(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} L(y_i, \hat{y}_i)$$

where $\hat{y}_i = f(\mathbf{x}_i; \boldsymbol{\theta})$

Summary of Lecture 7 (III/IV) NN for classification ($M > 2$ classes)

For $M > 2$ classes we want to predict the class probability for all M classes $g_m = p(y = m|\mathbf{x})$. We extend the logistic function to the **softmax activation function**

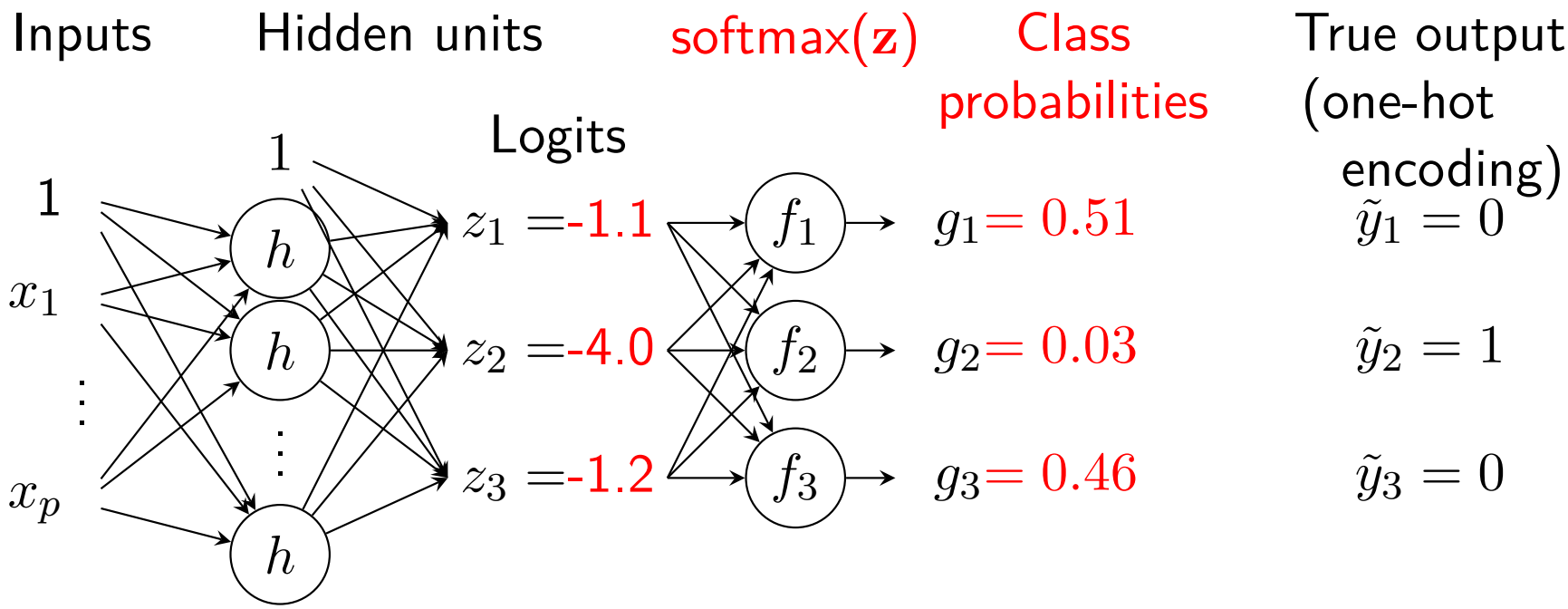
$$g_m = f_m(\mathbf{z}) = \frac{e^{z_m}}{\sum_{l=1}^{M} e^{z_l}}, \qquad m = 1, \ldots, M.$$



$$\text{softmax}(\mathbf{z}) = [f_1(\mathbf{z}), \ldots, f_M(\mathbf{z})]^\mathsf{T}$$

Summary of Lecture 7 (IV/IV)
Example $M = 3$ classes

Consider an example with three classes $M = 3$ and where $y = 2$.

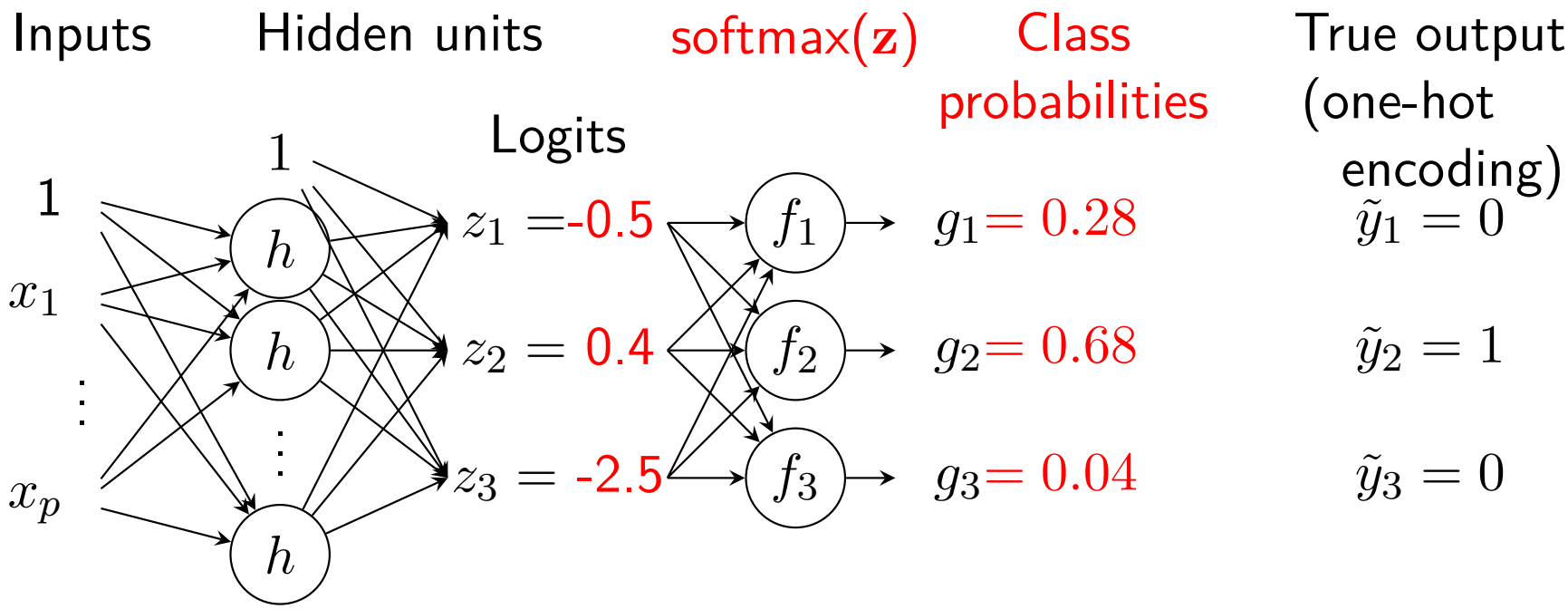


The network is trained by minimizing the **cross-entropy**

$$L(\tilde{\mathbf{y}}, \mathbf{g}) = -\sum_{m=1}^{M} \tilde{y}_m \ln(g_m) = -\ln 0.03 = 3.51$$

Summary of Lecture 7 (IV/IV)
Example $M = 3$ classes

Consider an example with three classes $M = 3$ and where $y = 2$.

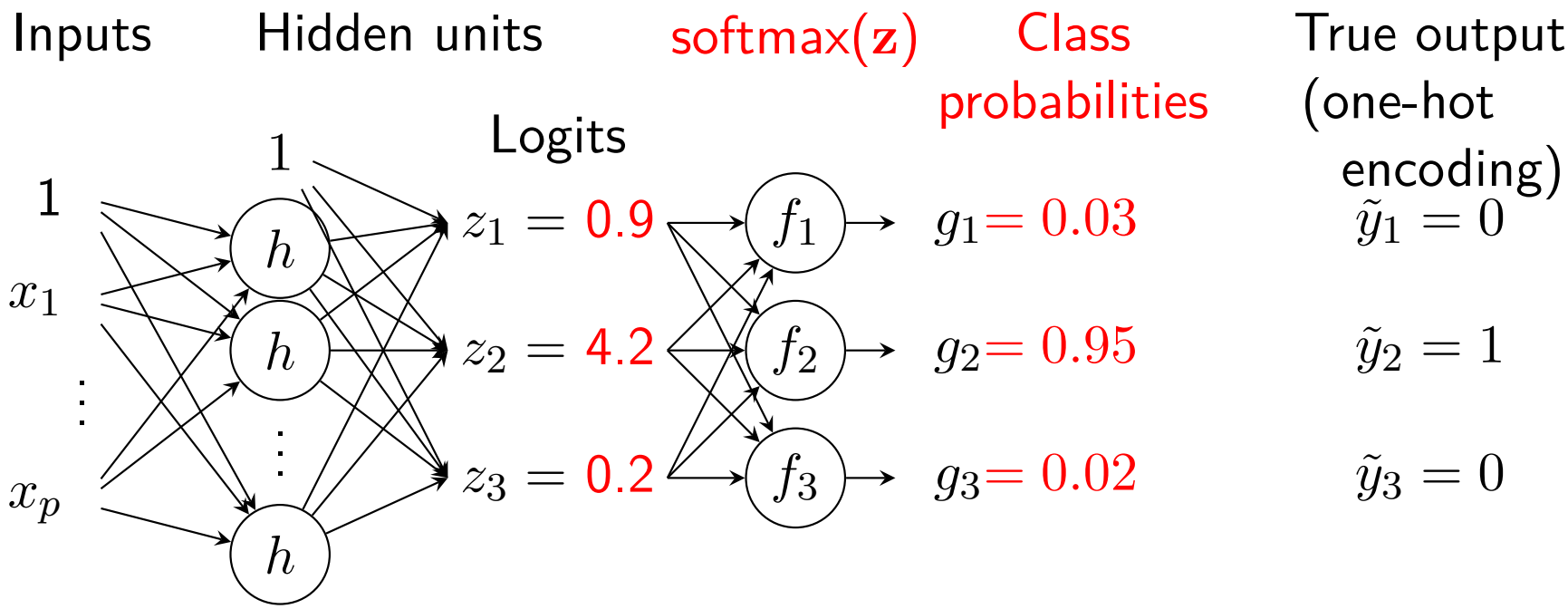


The network is trained by minimizing the **cross-entropy**

$$L(\tilde{\mathbf{y}}, \mathbf{g}) = -\sum_{m=1}^{M} \tilde{y}_m \ln(g_m) = -\ln 0.68 = 0.39$$

Summary of Lecture 7 (IV/IV)
Example $M = 3$ classes

Consider an example with three classes $M = 3$ and where $y = 2$.



The network is trained by minimizing the **cross-entropy**

$$L(\tilde{\mathbf{y}}, \mathbf{g}) = -\sum_{m=1}^{M} \tilde{y}_m \ln(g_m) = -\ln 0.95 = 0.05$$



UPPSALA
UNIVERSITET

Outline

1. **Previous lecture** The neural network model
   - Neural network for regression
   - Neural network for classification

Outline

1. **Previous lecture** The neural network model
   - Neural network for regression
   - Neural network for classification
2. **This lecture**
   - Convolutional neural network
   - How to train a neural network

Convolutional neural networks

Convolutional neural networks (CNN) are a special kind neural networks tailored for problems where the input data has a grid-like structure.

Examples

- Digital images (2D grid of pixels)
- Audio waveform data (1D grid, times series)
- Volumetric data e.g. CT scans (3D grid)

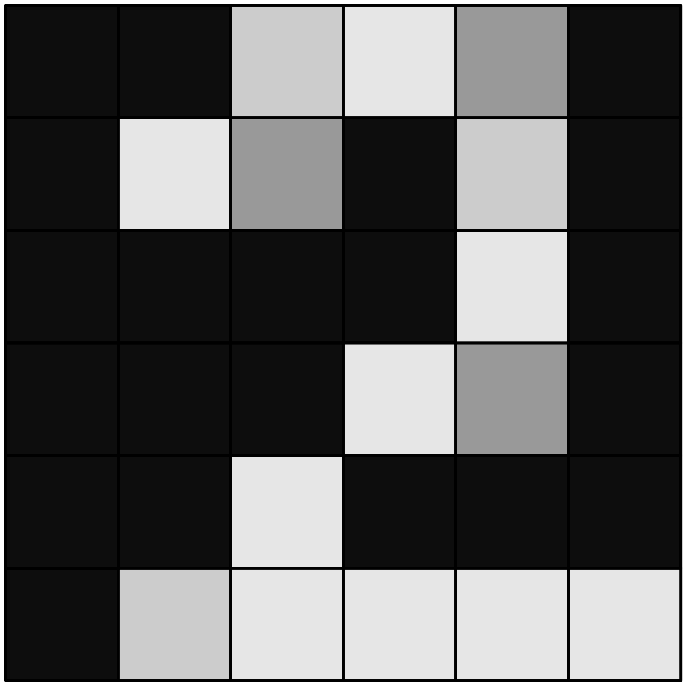
The description here will focus on images.

Data representation of images

Consider a grayscale image of 6×6 **pixels**.

- Each pixel value represents the color. The value ranges from 0 (total absence, black) to 1 (total presence, white)

Image



Data representation

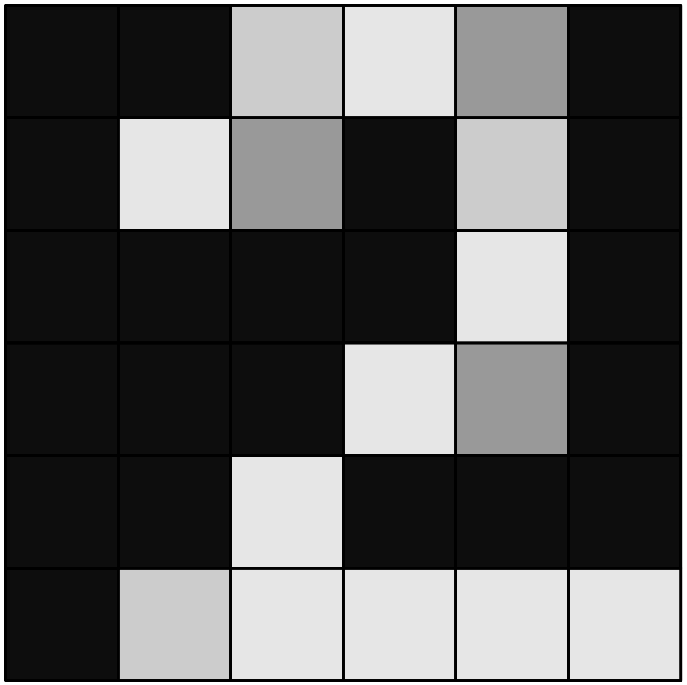
0.0	0.0	0.8	0.9	0.6	0.0
0.0	0.9	0.6	0.0	0.8	0.0
0.0	0.0	0.0	0.0	0.9	0.0
0.0	0.0	0.0	0.9	0.6	0.0
0.0	0.0	0.9	0.0	0.0	0.0
0.0	0.8	0.9	0.9	0.9	0.9

Data representation of images

Consider a grayscale image of 6×6 **pixels**.

- Each pixel value represents the color. The value ranges from 0 (total absence, black) to 1 (total presence, white)
- The pixels are the input variables $x_{1,1}, x_{1,2}, \ldots, x_{6,6}$.

Image

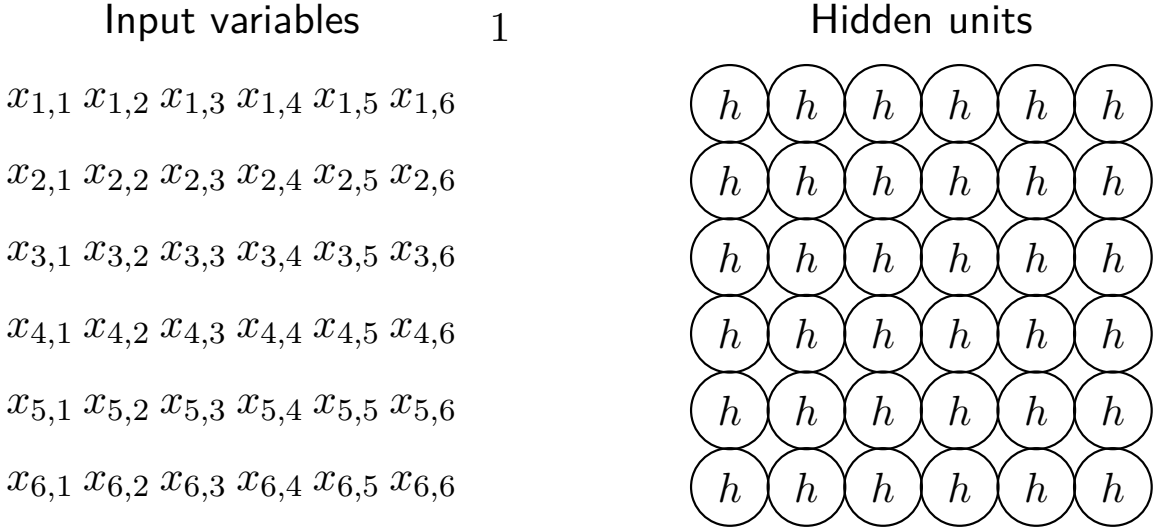


Input variables

$x_{1,1}$	$x_{1,2}$	$x_{1,3}$	$x_{1,4}$	$x_{1,5}$	$x_{1,6}$
$x_{2,1}$	$x_{2,2}$	$x_{2,3}$	$x_{2,4}$	$x_{2,5}$	$x_{2,6}$
$x_{3,1}$	$x_{3,2}$	$x_{3,3}$	$x_{3,4}$	$x_{3,5}$	$x_{3,6}$
$x_{4,1}$	$x_{4,2}$	$x_{4,3}$	$x_{4,4}$	$x_{4,5}$	$x_{4,6}$
$x_{5,1}$	$x_{5,2}$	$x_{5,3}$	$x_{5,4}$	$x_{5,5}$	$x_{5,6}$
$x_{6,1}$	$x_{6,2}$	$x_{6,3}$	$x_{6,4}$	$x_{6,5}$	$x_{6,6}$

The convolutional layer

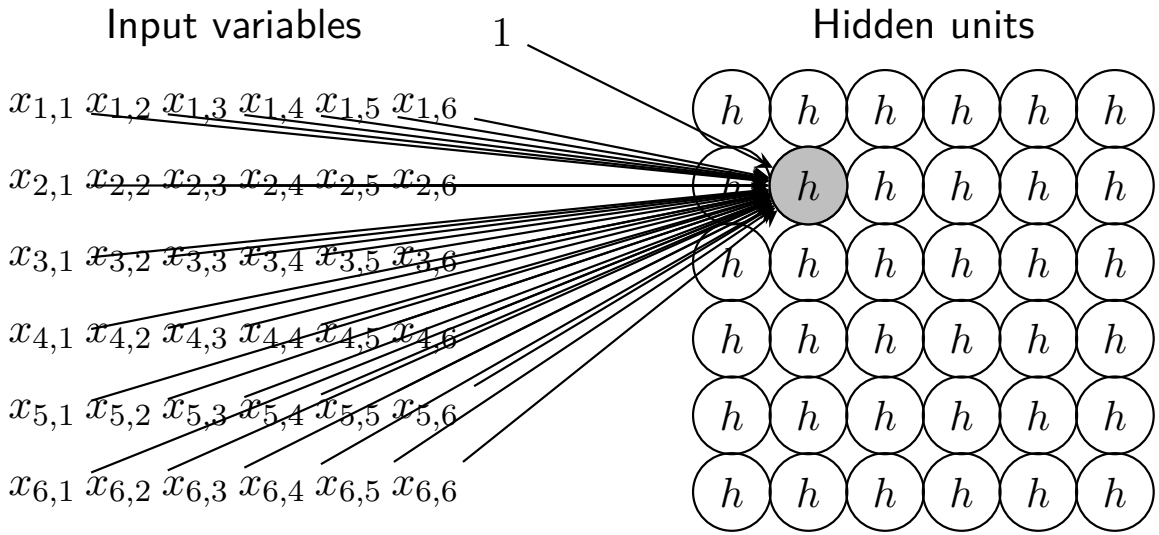
Consider a hidden layer with 6×6 hidden units.



The convolutional layer

Consider a hidden layer with 6×6 hidden units.

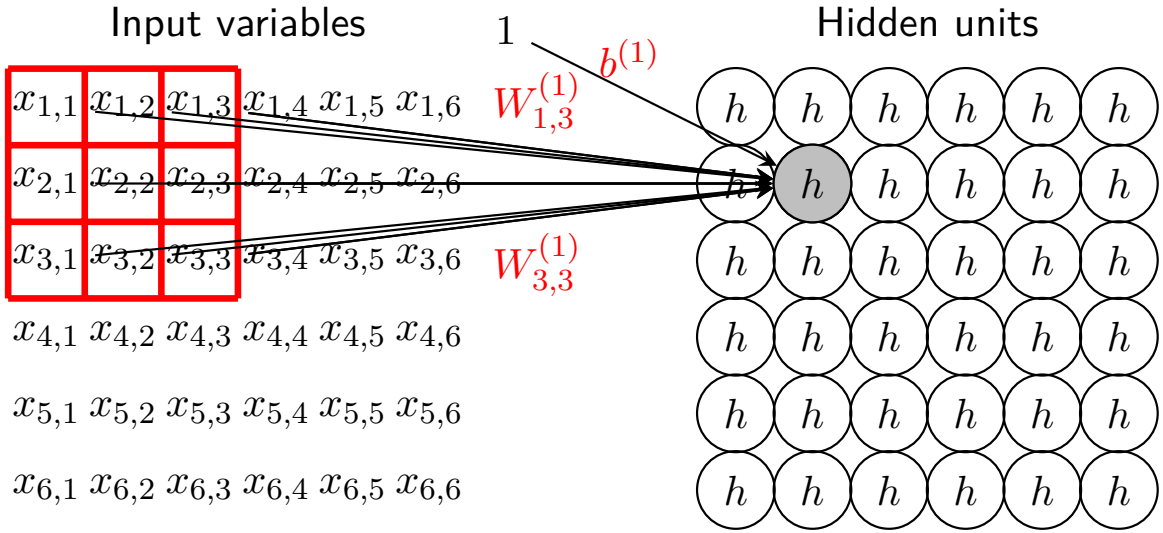
- **Dense layer** (previous lecture): Each hidden unit is connected with **all pixels**. Each pixel-hidden-unit-pair has its own **unique parameter**.



The convolutional layer

Consider a hidden layer with 6×6 hidden units.

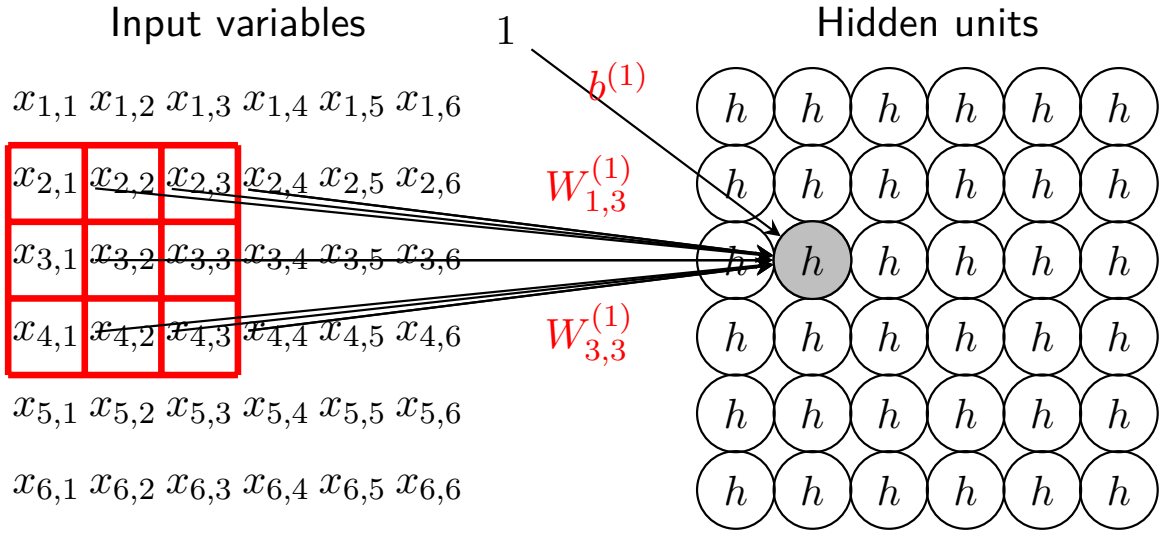
- **Dense layer** (previous lecture): Each hidden unit is connected with **all pixels**. Each pixel-hidden-unit-pair has its own **unique parameter**.
- **Convolutional layer**: Each hidden unit is connected with a **region of pixels** via a set of parameters, so-called **filter**. Different hidden units have the **same set of parameters**.



The convolutional layer

Consider a hidden layer with 6×6 hidden units.

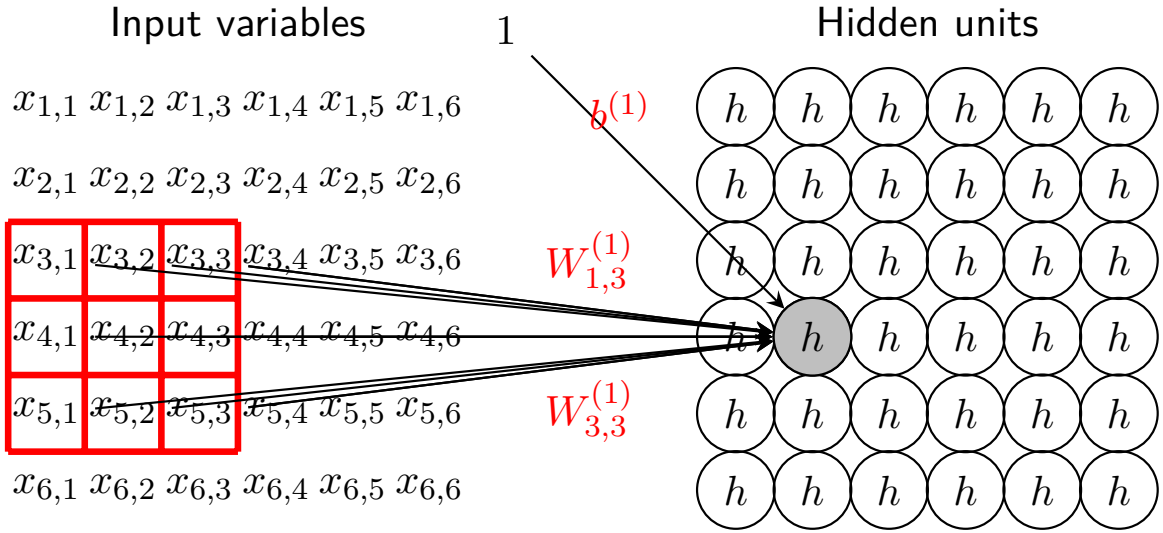
- **Dense layer** (previous lecture): Each hidden unit is connected with **all pixels**. Each pixel-hidden-unit-pair has its own **unique parameter**.
- **Convolutional layer**: Each hidden unit is connected with a **region of pixels** via a set of parameters, so-called **filter**. Different hidden units have the **same set of parameters**.



The convolutional layer

Consider a hidden layer with 6×6 hidden units.

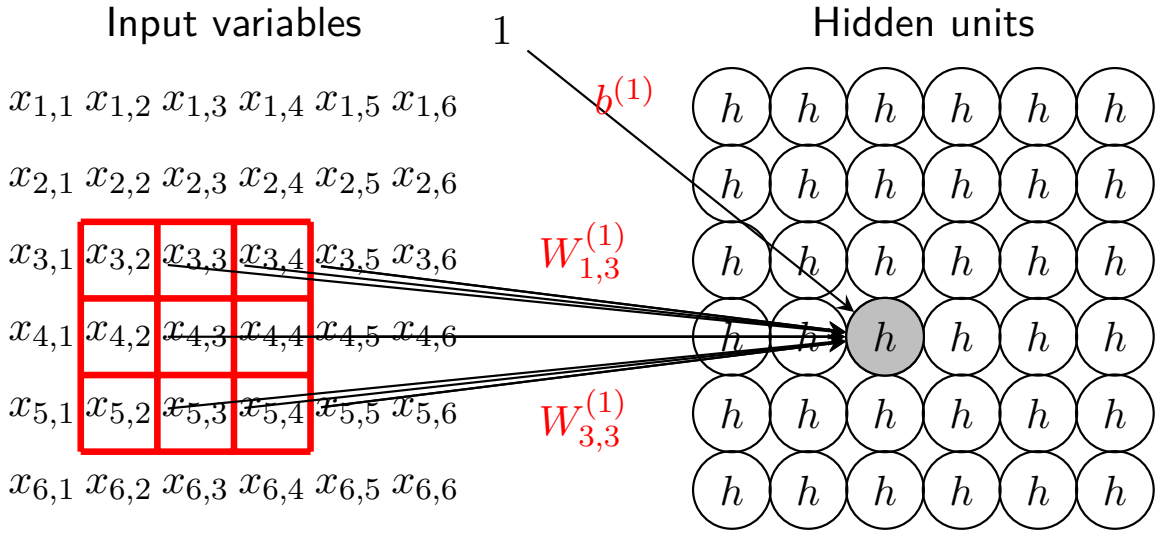
- **Dense layer** (previous lecture): Each hidden unit is connected with **all pixels**. Each pixel-hidden-unit-pair has its own **unique parameter**.
- **Convolutional layer**: Each hidden unit is connected with a **region of pixels** via a set of parameters, so-called **filter**. Different hidden units have the **same set of parameters**.



The convolutional layer

Consider a hidden layer with 6×6 hidden units.

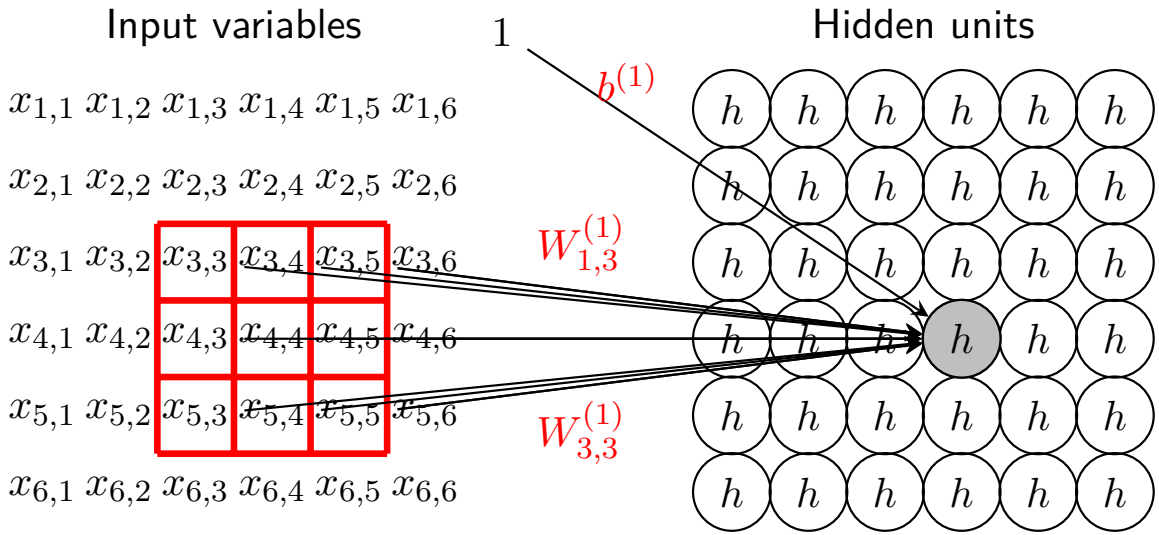
- **Dense layer** (previous lecture): Each hidden unit is connected with **all pixels**. Each pixel-hidden-unit-pair has its own **unique parameter**.
- **Convolutional layer**: Each hidden unit is connected with a **region of pixels** via a set of parameters, so-called **filter**. Different hidden units have the **same set of parameters**.



The convolutional layer

Consider a hidden layer with 6×6 hidden units.

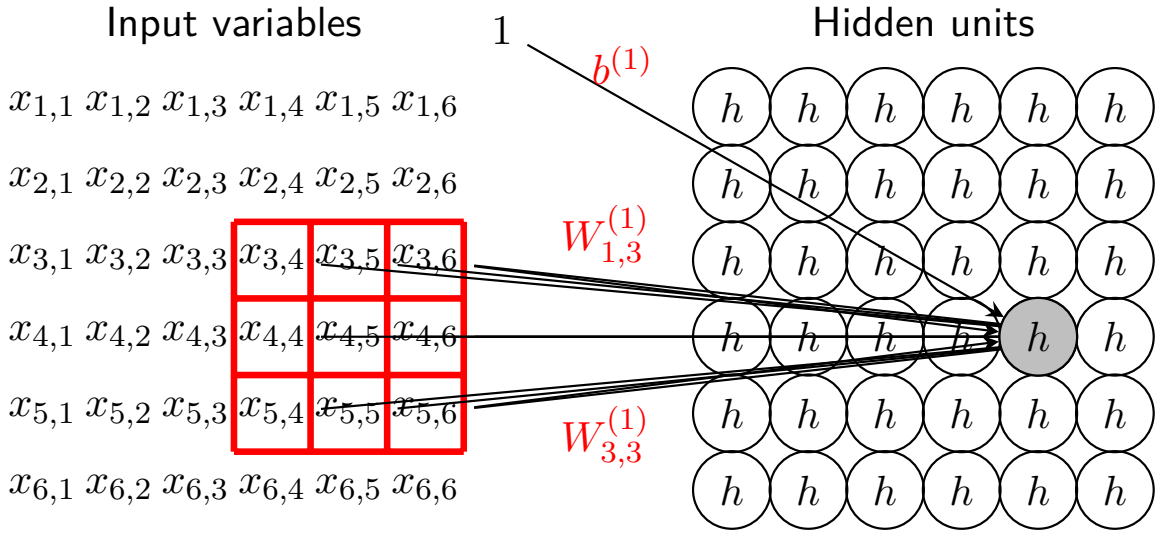
- **Dense layer** (previous lecture): Each hidden unit is connected with **all pixels**. Each pixel-hidden-unit-pair has its own **unique parameter**.
- **Convolutional layer**: Each hidden unit is connected with a **region of pixels** via a set of parameters, so-called **filter**. Different hidden units have the **same set of parameters**.



The convolutional layer

Consider a hidden layer with 6×6 hidden units.

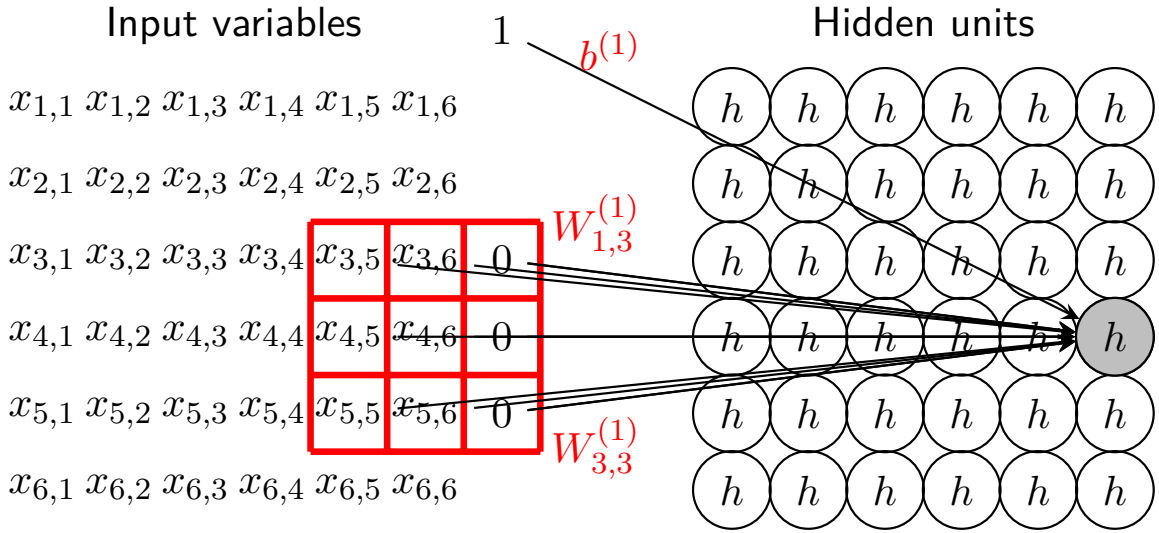
- **Dense layer** (previous lecture): Each hidden unit is connected with **all pixels**. Each pixel-hidden-unit-pair has its own **unique parameter**.
- **Convolutional layer**: Each hidden unit is connected with a **region of pixels** via a set of parameters, so-called **filter**. Different hidden units have the **same set of parameters**.



The convolutional layer

Consider a hidden layer with 6×6 hidden units.

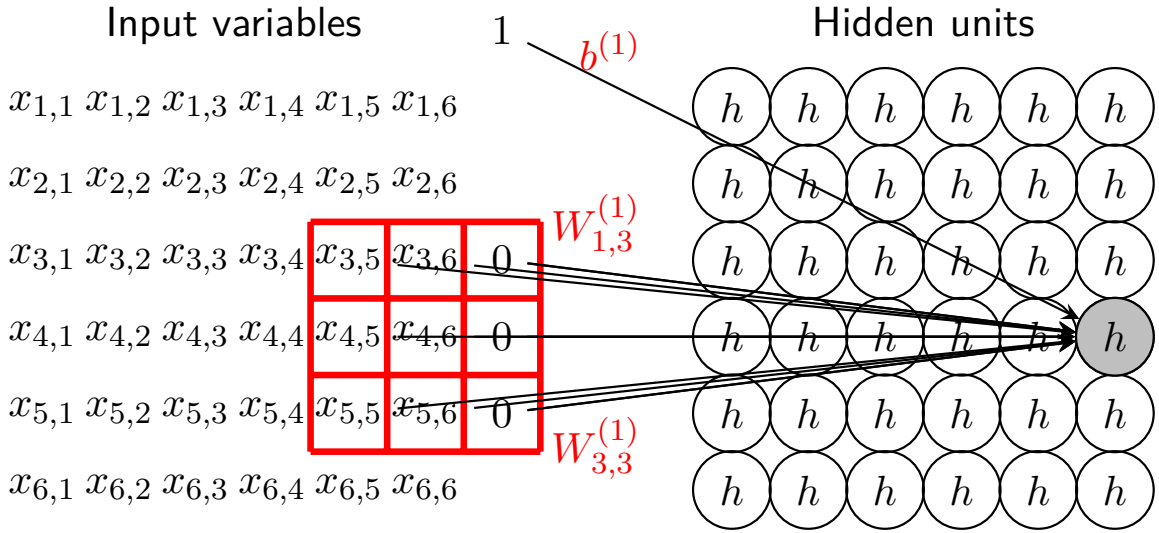
- **Dense layer** (previous lecture): Each hidden unit is connected with **all pixels**. Each pixel-hidden-unit-pair has its own **unique parameter**.
- **Convolutional layer**: Each hidden unit is connected with a **region of pixels** via a set of parameters, so-called **filter**. Different hidden units have the **same set of parameters**.



The convolutional layer

Consider a hidden layer with 6×6 hidden units.

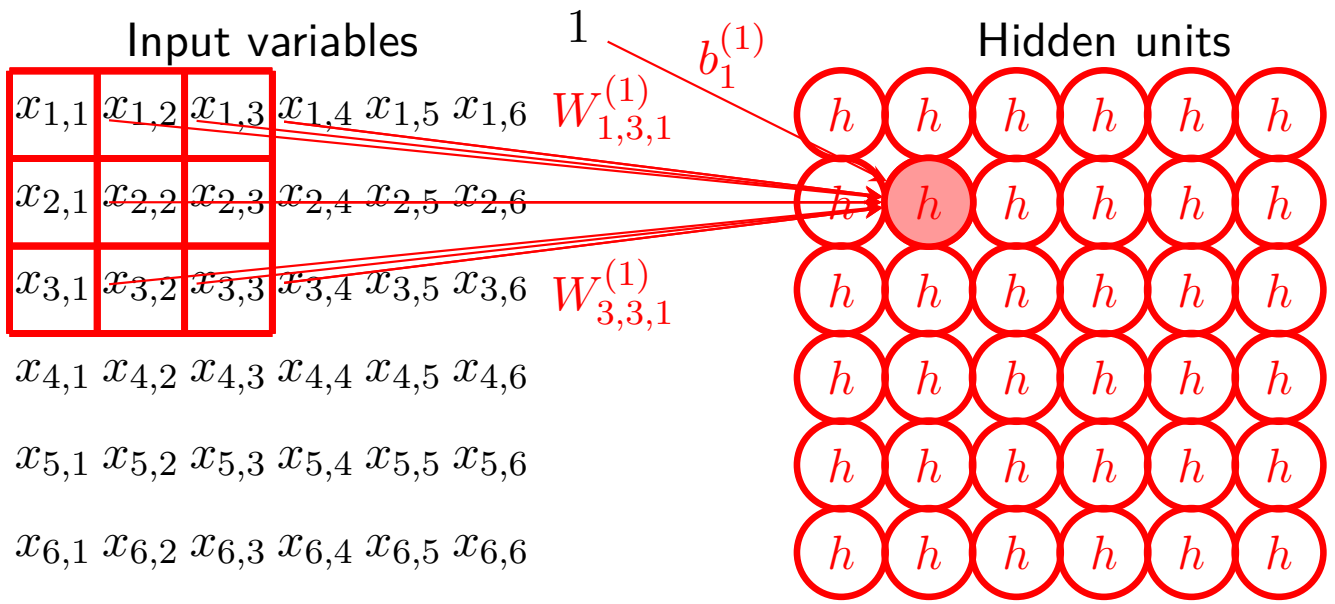
- **Dense layer** (previous lecture): Each hidden unit is connected with **all pixels**. Each pixel-hidden-unit-pair has its own **unique parameter**.
- **Convolutional layer**: Each hidden unit is connected with a **region of pixels** via a set of parameters, so-called **filter**. Different hidden units have the **same set of parameters**.



Conv. layer uses **sparse interactions** and **parameter sharing**

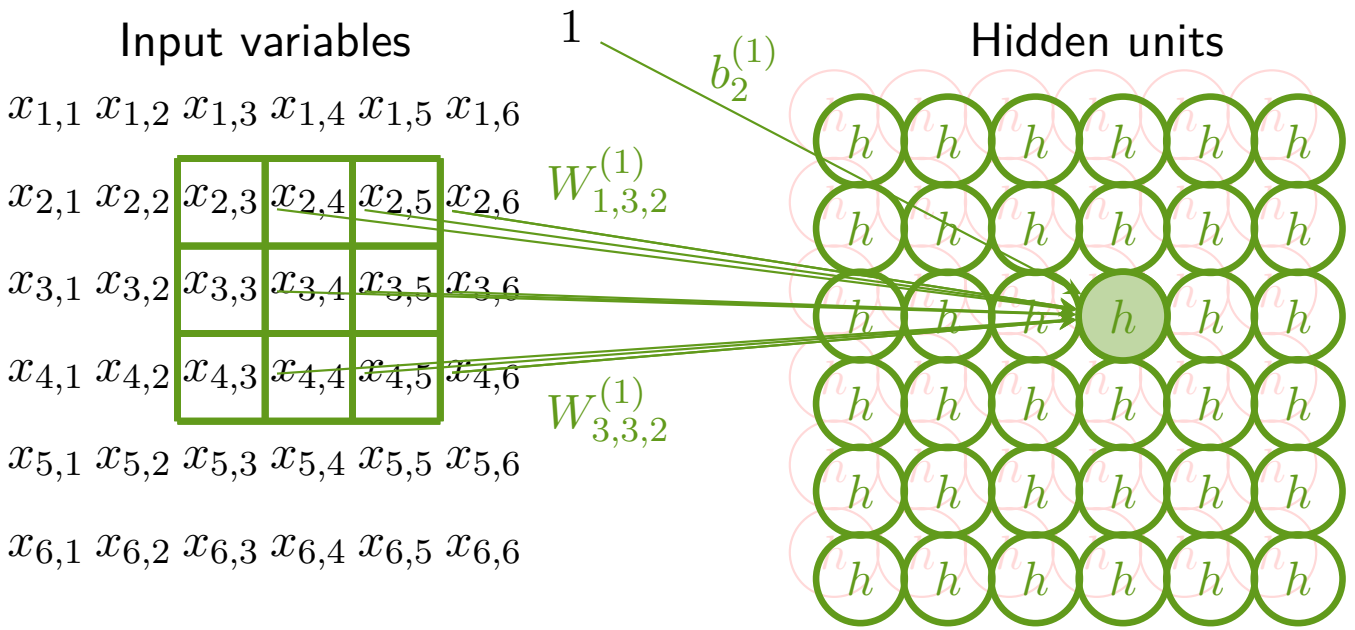
Multiple filters

- One filter per layer does not give enough flexibility. $\Rightarrow$
- We use **multiple filters** (visualized with different colors).
- Each filter produces its own set of hidden units – a **channel**.



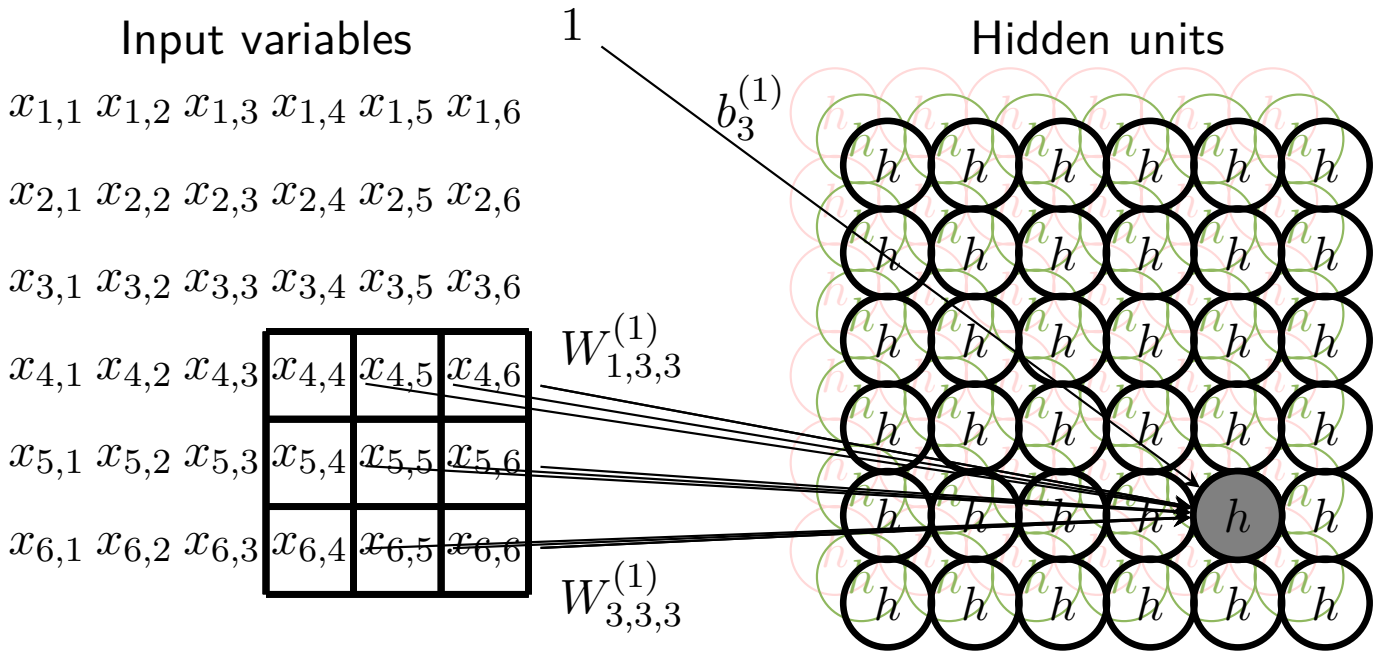
Multiple filters

- One filter per layer does not give enough flexibility. $\Rightarrow$
- We use **multiple filters** (visualized with different colors).
- Each filter produces its own set of hidden units – a **channel**.



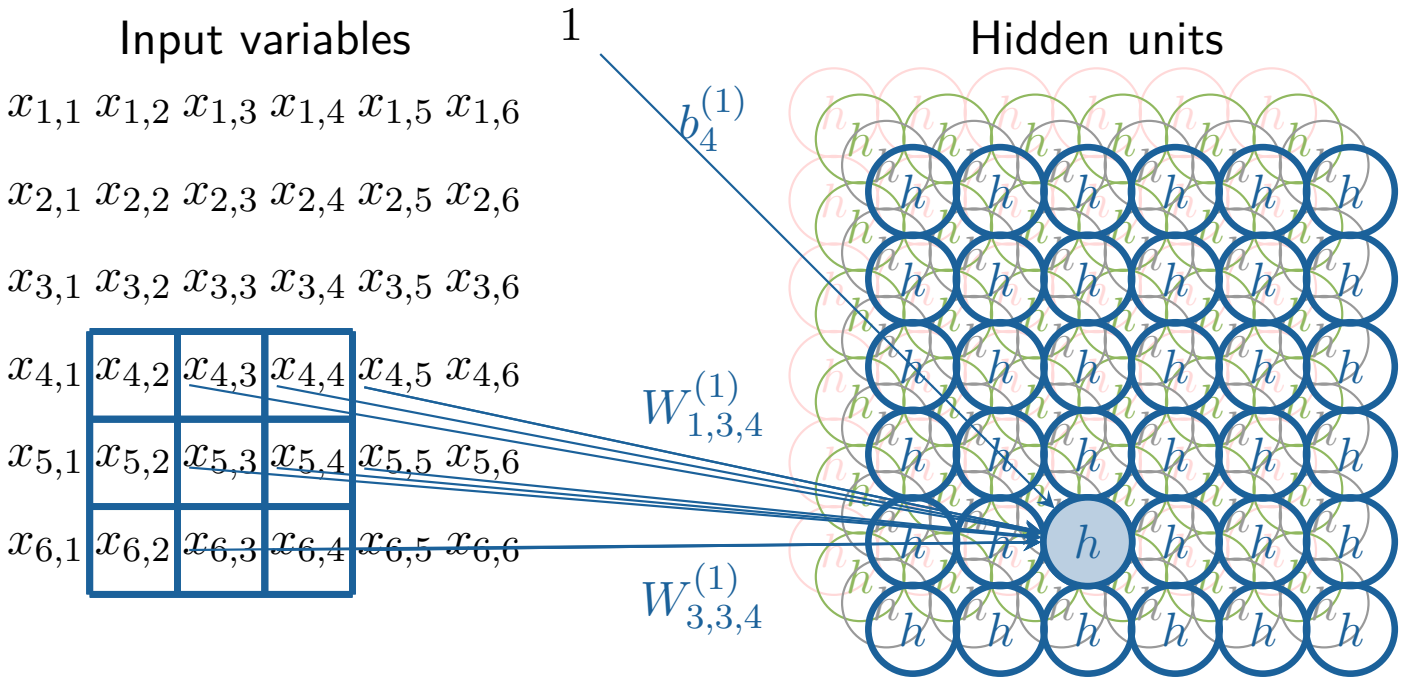
Multiple filters

- One filter per layer does not give enough flexibility. $\Rightarrow$
- We use **multiple filters** (visualized with different colors).
- Each filter produces its own set of hidden units – a **channel**.



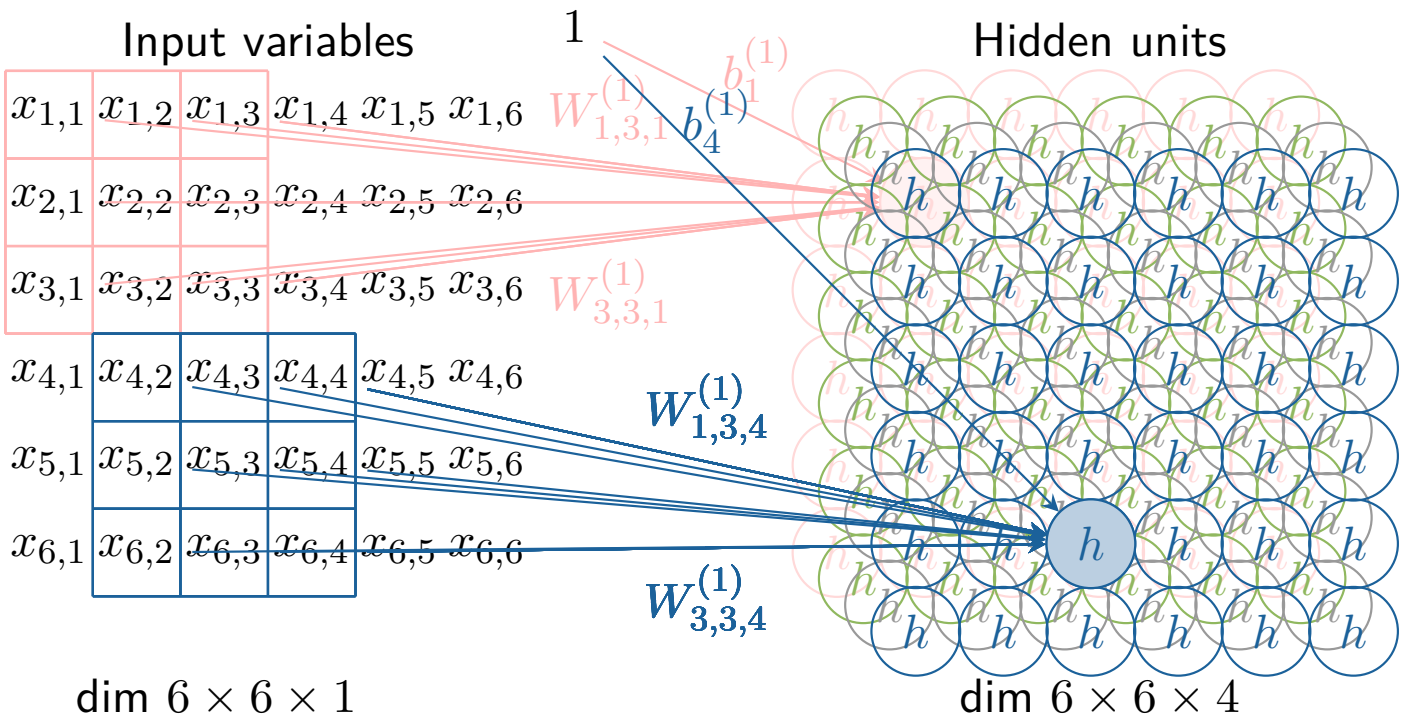
Multiple filters

- One filter per layer does not give enough flexibility. $\Rightarrow$
- We use **multiple filters** (visualized with different colors).
- Each filter produces its own set of hidden units – a **channel**.



Multiple filters

- One filter per layer does not give enough flexibility. $\Rightarrow$
- We use **multiple filters** (visualized with different colors).
- Each filter produces its own set of hidden units – a **channel**.



Hidden layers are organized in **tensors** of size (rows $\times$ columns $\times$ channels).

What is a tensor?

A **tensor** is a generalization of scalar, vector and matrix to arbitrary **order**.

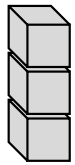
Scalar
order **0**

$$a = 3$$



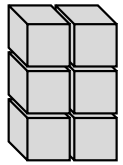
Vector
order **1**

$$\mathbf{b} = \begin{bmatrix} 3 \\ -2 \\ -1 \end{bmatrix}$$



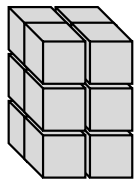
Matrix
order **2**

$$W = \begin{bmatrix} 3 & 2 \\ -2 & 1 \\ -1 & 2 \end{bmatrix}$$



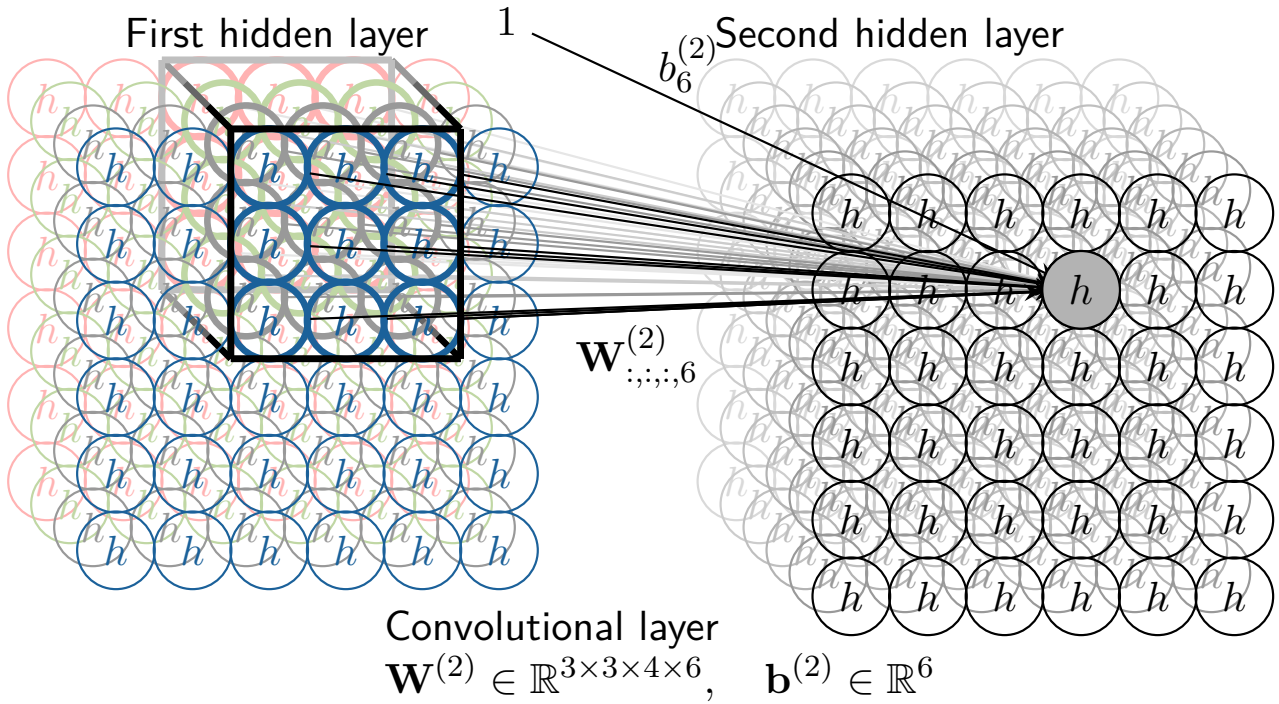
Tensor
any order
(here order **3**)

$$\mathbf{T}_{:,:,1} = \begin{bmatrix} 3 & 2 \\ -2 & 1 \\ -1 & 2 \end{bmatrix}, \quad \mathbf{T}_{:,:,2} = \begin{bmatrix} -1 & 4 \\ 1 & 2 \\ -5 & 3 \end{bmatrix}$$



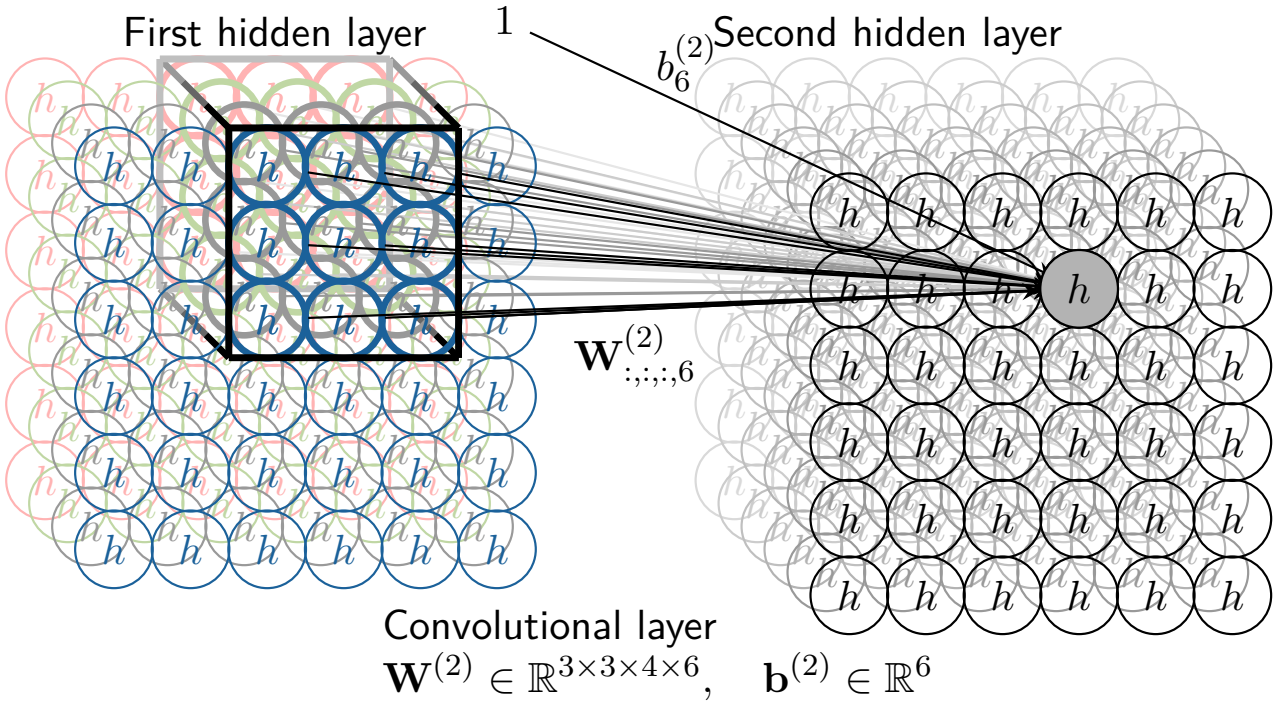
Multiple filters (cont.)

- A filter operates on **all channels** in a hidden layer.



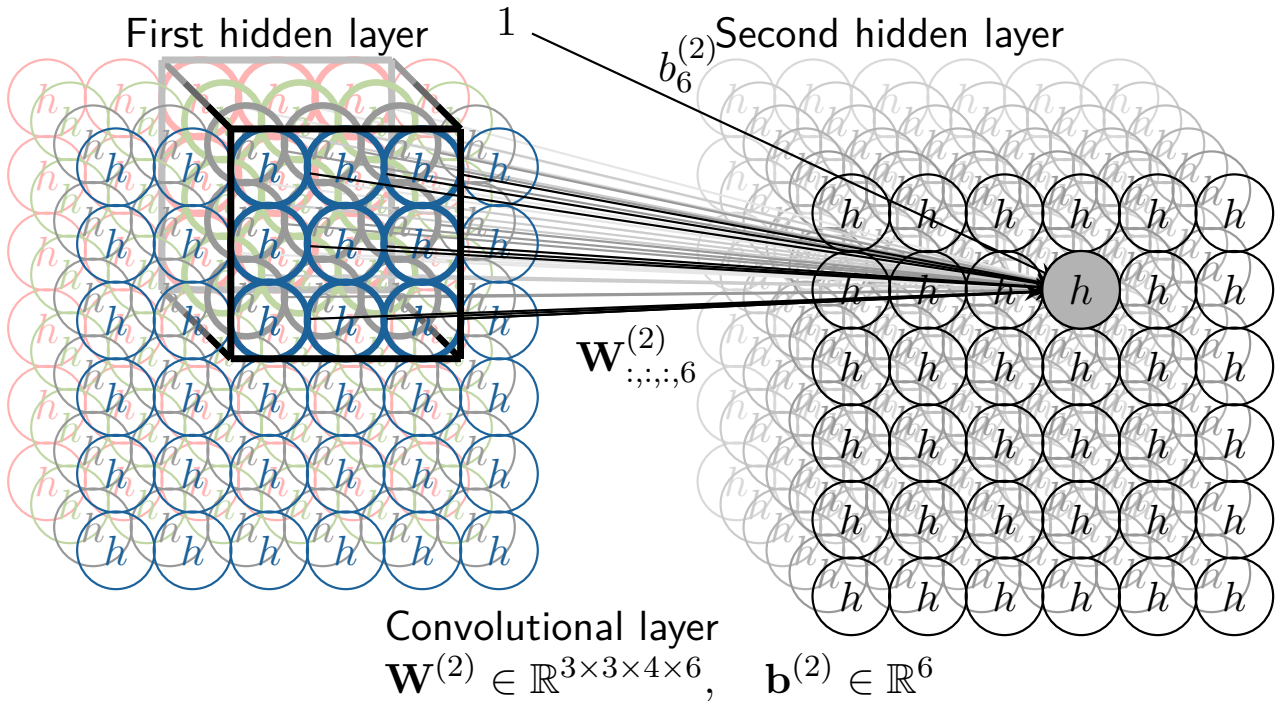
Multiple filters (cont.)

- A filter operates on **all channels** in a hidden layer.
- Each filter has the dimension (filter rows $\times$ filter colomns $\times$ input channels), here ($3 \times 3 \times 4$).



Multiple filters (cont.)

- A filter operates on **all channels** in a hidden layer.
- Each filter has the dimension (filter rows $\times$ filter colomns $\times$ input channels), here ($3 \times 3 \times 4$).
- We stack all filter parameters in a **weight tensor** with dimensions (filter rows $\times$ filter columns $\times$ input channels $\times$ output channels), here ($3 \times 3 \times 4 \times 6$)





UPPSALA
UNIVERSITET

Condensing information with strides

- **Problem**: As we proceed though the network we want to condense the information.



UPPSALA
UNIVERSITET

Condensing information with strides

- **Problem**: As we proceed though the network we want to condense the information.
- **Solution**: Apply the filter to every second pixel. We use a **stride** of 2 (instead of 1).



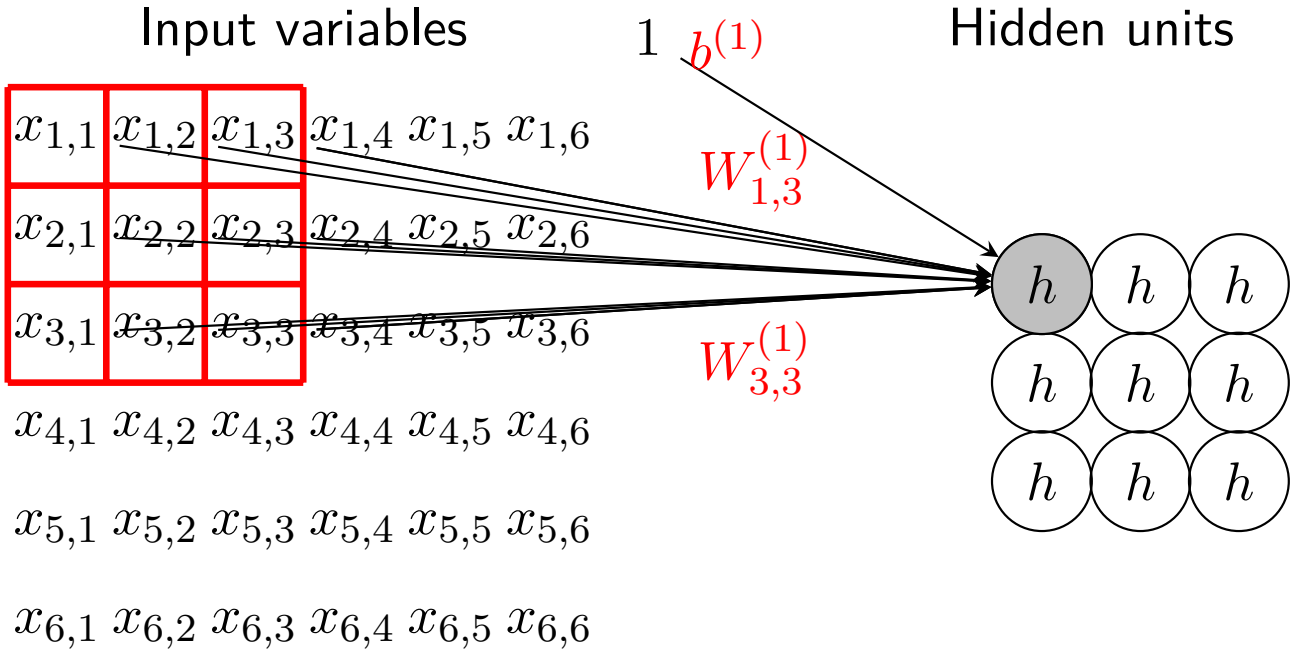
UPPSALA
UNIVERSITET

Condensing information with strides

- **Problem**: As we proceed though the network we want to condense the information.
- **Solution**: Apply the filter to every second pixel. We use a **stride** of 2 (instead of 1).

Condensing information with strides

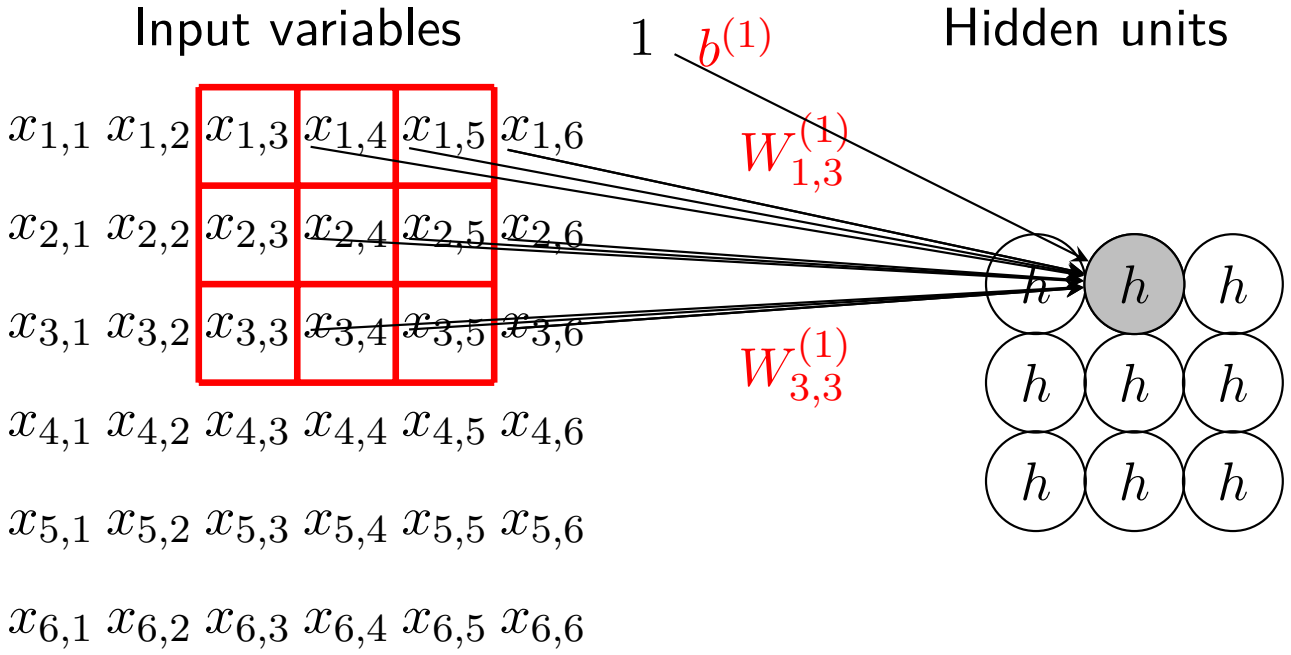
- **Problem**: As we proceed though the network we want to condense the information.
- **Solution**: Apply the filter to every second pixel. We use a **stride** of 2 (instead of 1).



With stride 2 we get half the number of rows and columns in the hidden layer.

Condensing information with strides

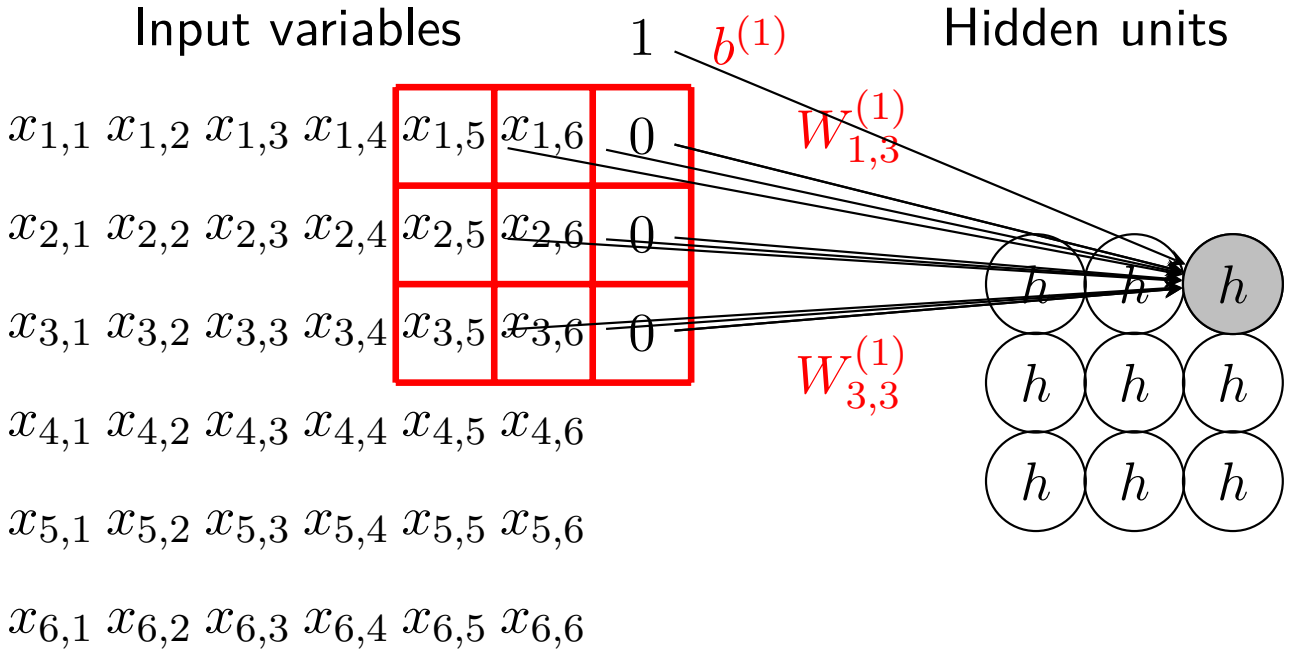
- **Problem**: As we proceed though the network we want to condense the information.
- **Solution**: Apply the filter to every second pixel. We use a **stride** of 2 (instead of 1).



With stride 2 we get half the number of rows and columns in the hidden layer.

Condensing information with strides

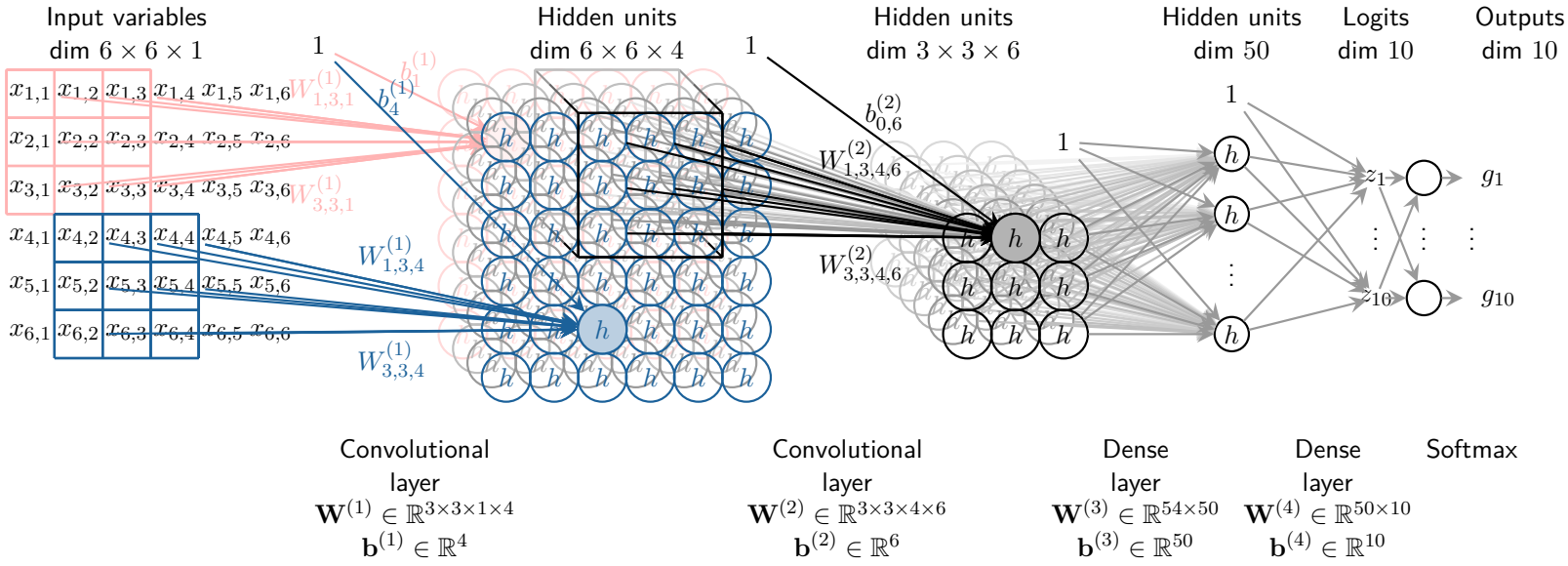
- **Problem**: As we proceed though the network we want to condense the information.
- **Solution**: Apply the filter to every second pixel. We use a **stride** of 2 (instead of 1).



With stride 2 we get half the number of rows and columns in the hidden layer.

Full CNN architecture

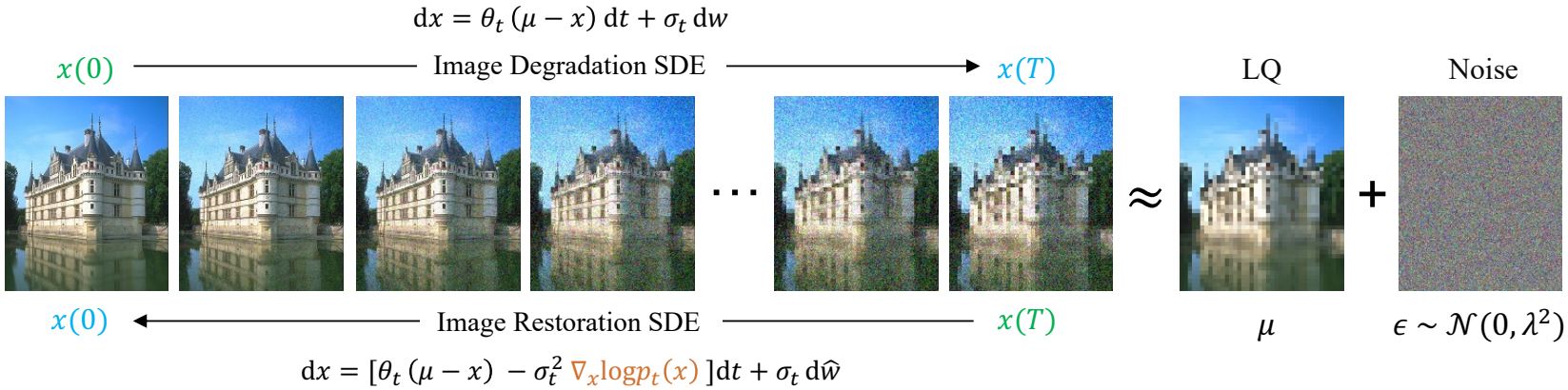
- A full CNN usually consist of multiple convolutional layers (here two) and a few final dense layers (here two).
- If we have a classification problem at hand, we end with a softmax activation function to produce class probabilities.



Here we use 50 hidden units in the last hidden layer and consider a classification problem with $M = 10$ classes.

CNN examples: image restoration

Using CNNs to remove degradations from images



Z. Luo, F.K. Gustafsson, Z. Zhao, J. Sjölund, T.B. Schön, *Image restoration with mean-reverting stochastic differential equations*, ICML, 2023.



UPPSALA
UNIVERSITET

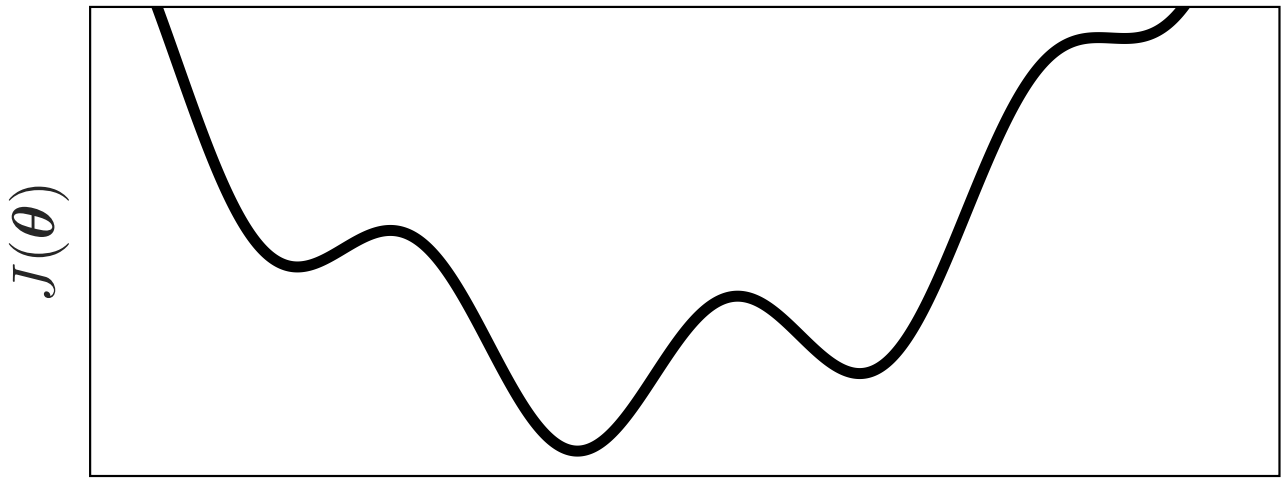
Numerical optimization

How to train a neural network

Unconstrained numerical optimization

We train a network by considering the optimization problem

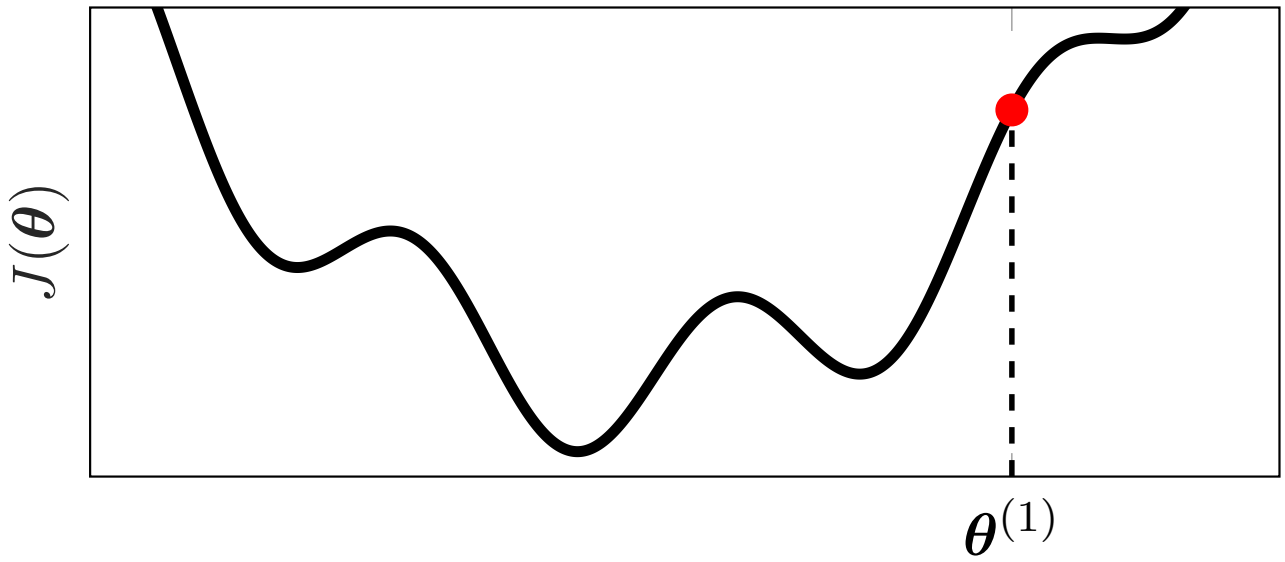
$$\widehat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta}), \qquad J(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} L(\mathbf{x}_i, \mathbf{y}_i, \boldsymbol{\theta})$$



Unconstrained numerical optimization

We train a network by considering the optimization problem

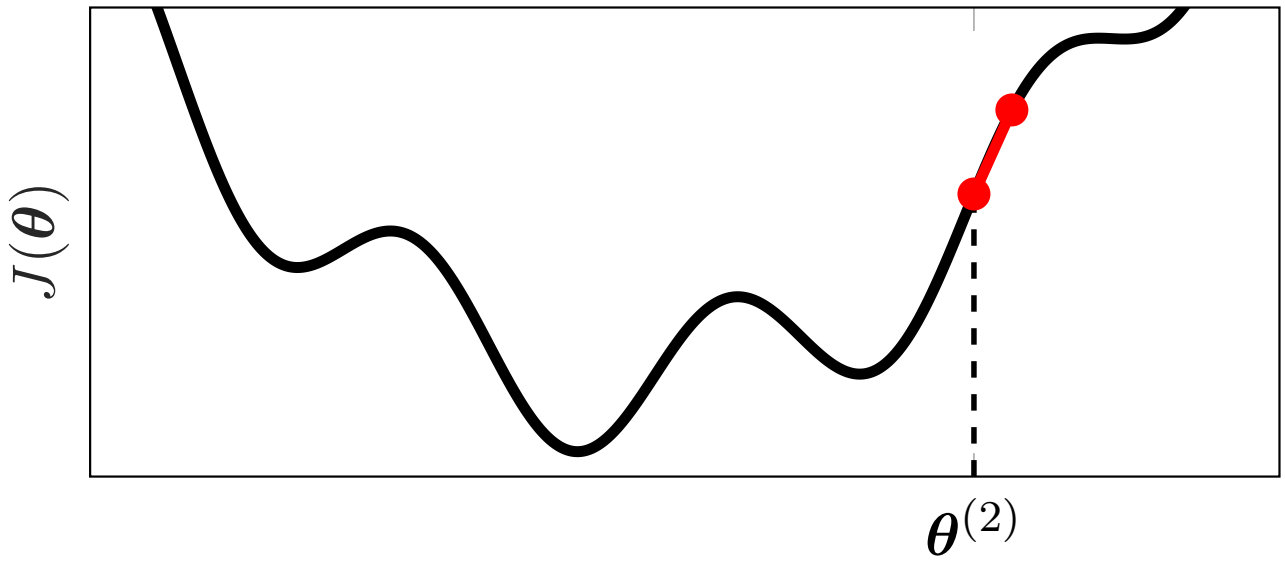
$$\widehat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta}), \qquad J(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} L(\mathbf{x}_i, \mathbf{y}_i, \boldsymbol{\theta})$$



Unconstrained numerical optimization

We train a network by considering the optimization problem

$$\widehat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta}), \qquad J(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} L(\mathbf{x}_i, \mathbf{y}_i, \boldsymbol{\theta})$$



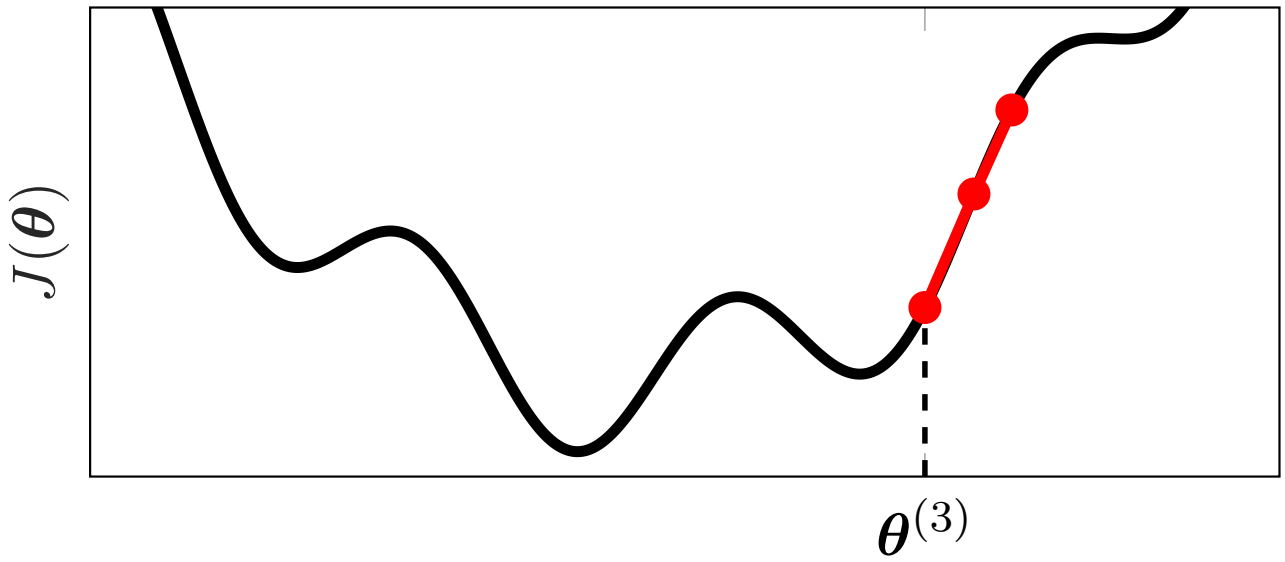
We solve the optimization problem by

- ... making an initial guess of $\boldsymbol{\theta}$...

Unconstrained numerical optimization

We train a network by considering the optimization problem

$$\widehat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta}), \qquad J(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} L(\mathbf{x}_i, \mathbf{y}_i, \boldsymbol{\theta})$$



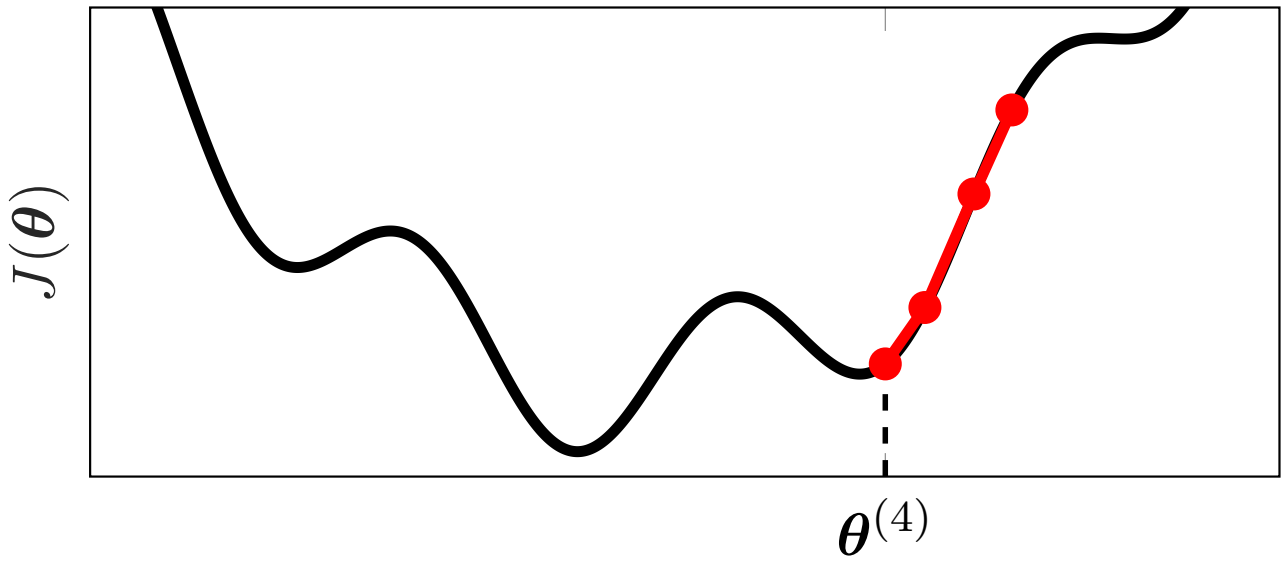
We solve the optimization problem by

- ... making an initial guess of $\boldsymbol{\theta}$...
- ... and updating $\boldsymbol{\theta}$ iteratively.

Unconstrained numerical optimization

We train a network by considering the optimization problem

$$\widehat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta}), \qquad J(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} L(\mathbf{x}_i, \mathbf{y}_i, \boldsymbol{\theta})$$



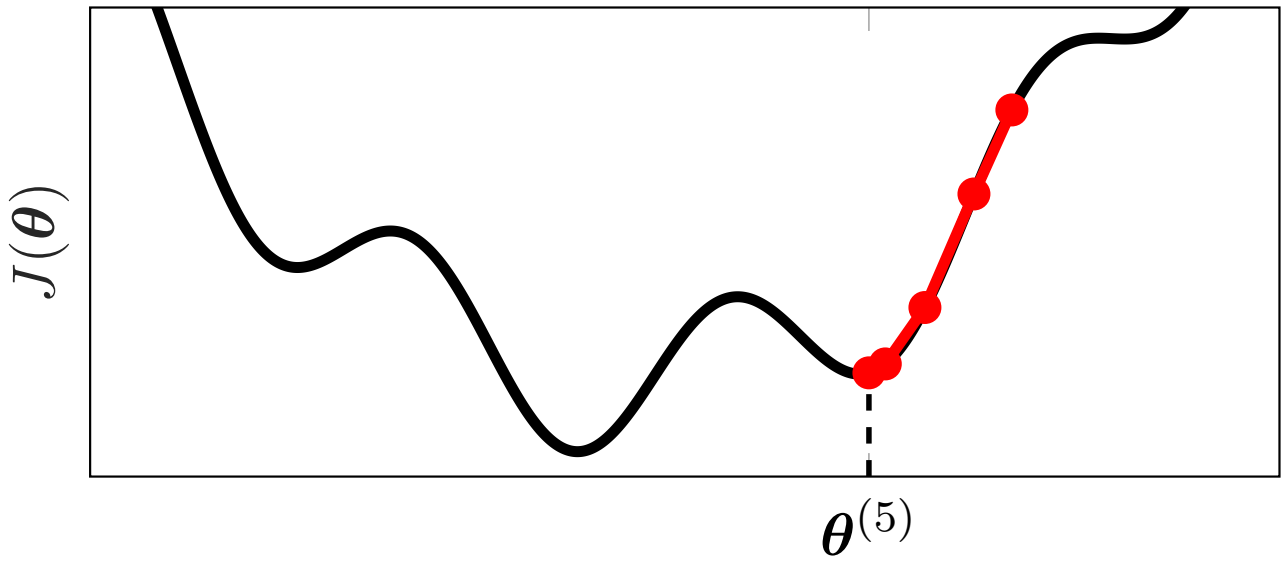
We solve the optimization problem by

- ... making an initial guess of $\boldsymbol{\theta}$...
- ... and updating $\boldsymbol{\theta}$ iteratively.

Unconstrained numerical optimization

We train a network by considering the optimization problem

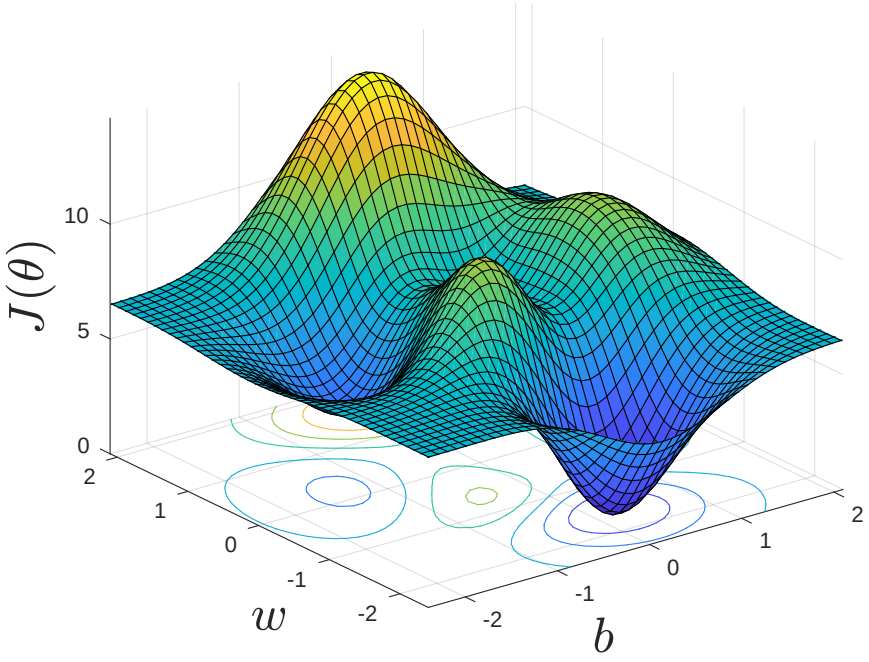
$$\widehat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta}), \qquad J(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} L(\mathbf{x}_i, \mathbf{y}_i, \boldsymbol{\theta})$$



We solve the optimization problem by

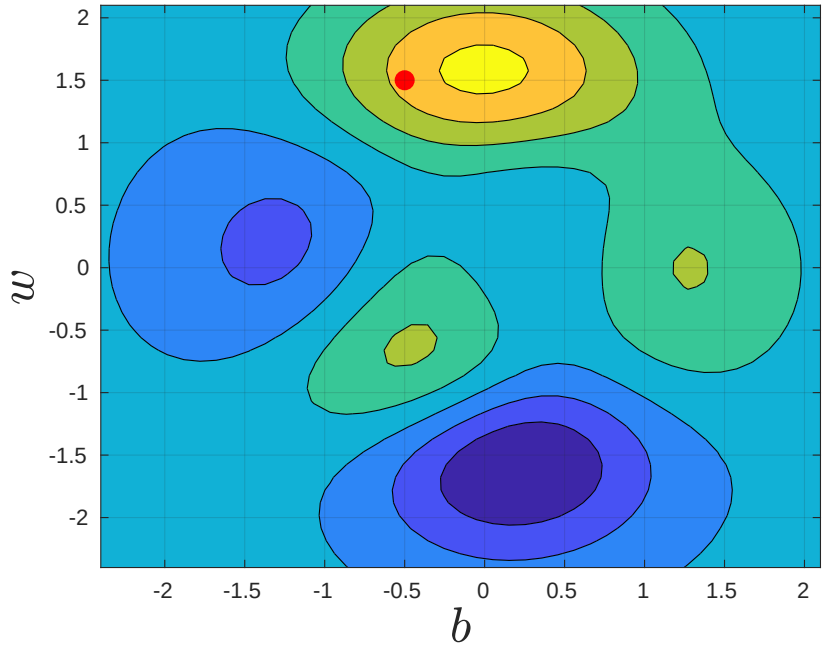
- ... making an initial guess of $\boldsymbol{\theta}$...
- ... and updating $\boldsymbol{\theta}$ iteratively.

Iterative solution (gradient descent) - Example 2D



$$\boldsymbol{\theta} = [b,\ w]^{\mathsf{T}} \in \mathbb{R}^2$$

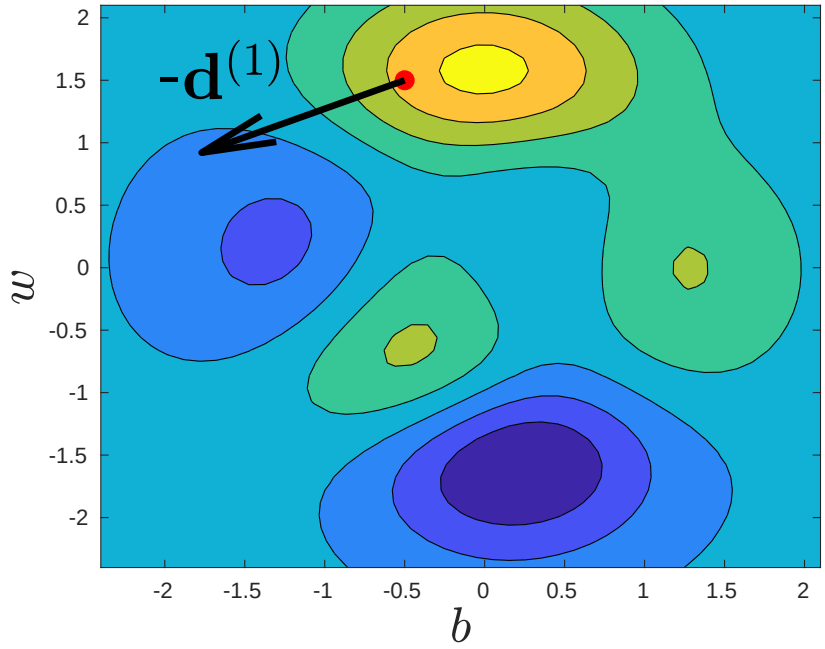
Iterative solution (gradient descent) - Example 2D



$$\boldsymbol{\theta} = [b,\ w]^{\mathsf{T}} \in \mathbb{R}^2$$

1. Pick a $\boldsymbol{\theta}^{(0)}$

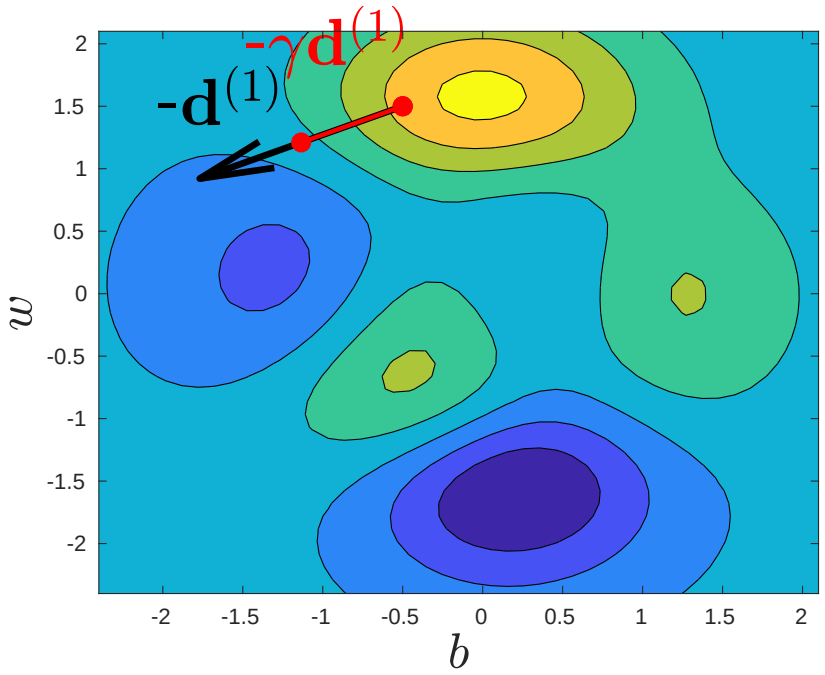
Iterative solution (gradient descent) - Example 2D



$$\boldsymbol{\theta} = [b,\ w]^{\mathsf{T}} \in \mathbb{R}^2$$

1. Pick a $\boldsymbol{\theta}^{(0)}$
2. while(*not converged*)
   - Update $\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \gamma \mathbf{d}^{(t)}$, where $\mathbf{d}^{(t)} = \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$
   - Update $t := t + 1$

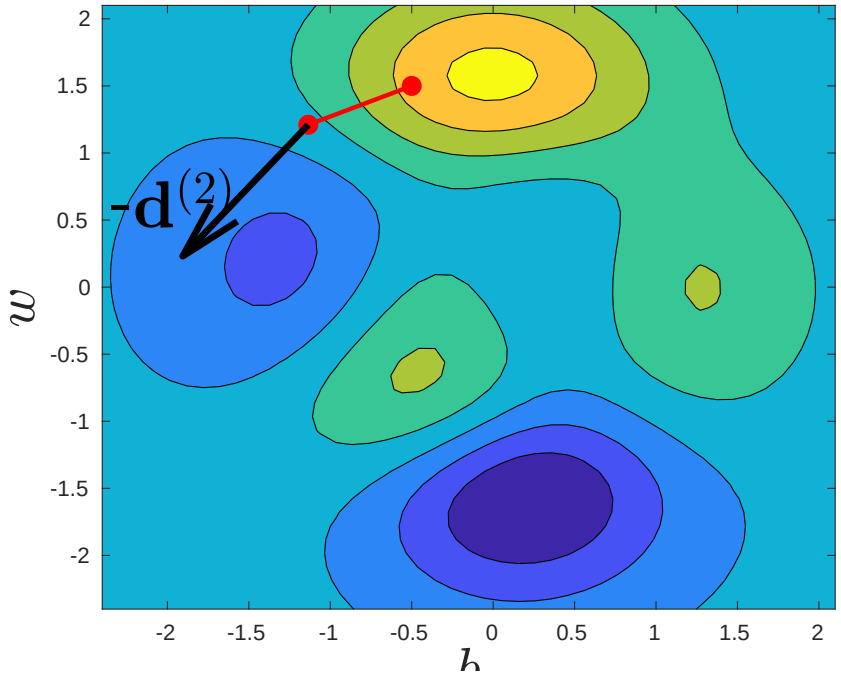
Iterative solution (gradient descent) - Example 2D



$$\boldsymbol{\theta} = [b,\ w]^{\mathsf{T}} \in \mathbb{R}^2$$

1. Pick a $\boldsymbol{\theta}^{(0)}$
2. while(*not converged*)
   - Update $\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \gamma \mathbf{d}^{(t)}$, where $\mathbf{d}^{(t)} = \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$
   - Update $t := t + 1$

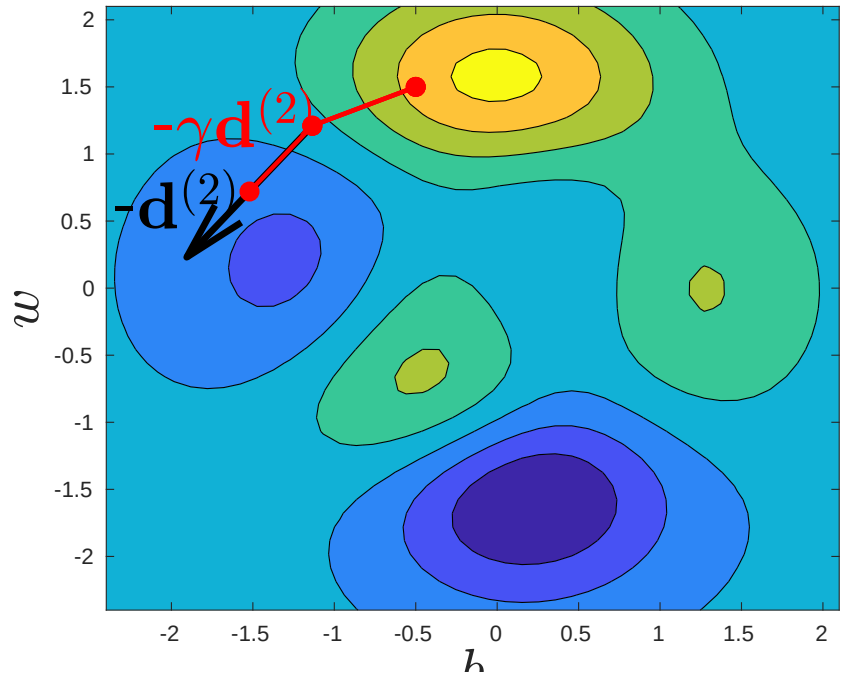
Iterative solution (gradient descent) - Example 2D



$$\boldsymbol{\theta} = [b,\ w]^{\mathsf{T}} \in \mathbb{R}^2$$

1. Pick a $\boldsymbol{\theta}^{(0)}$
2. while(*not converged*)
   - Update $\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \gamma \mathbf{d}^{(t)}$, where $\mathbf{d}^{(t)} = \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$
   - Update $t := t + 1$

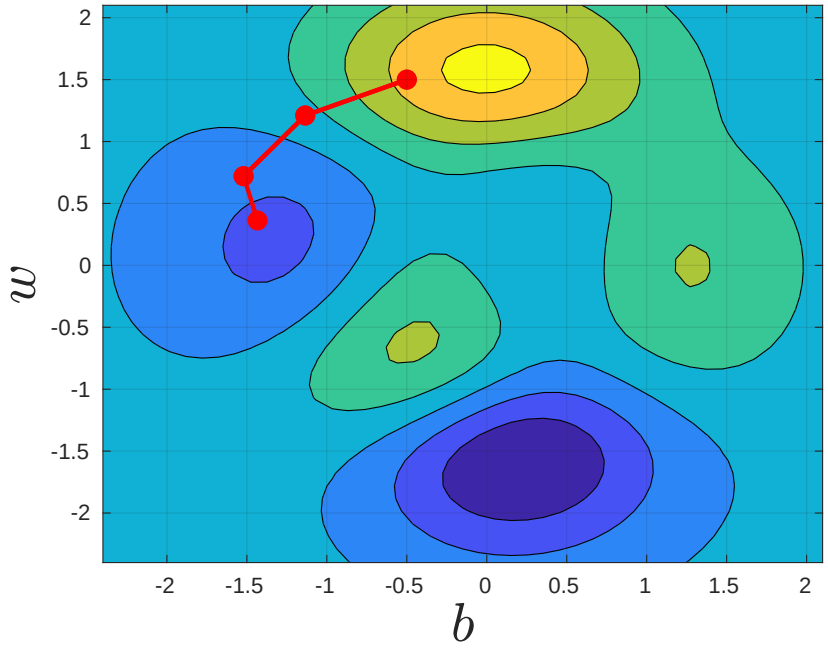
Iterative solution (gradient descent) - Example 2D



$$\boldsymbol{\theta} = [b,\ w]^{\mathsf{T}} \in \mathbb{R}^2$$

1. Pick a $\boldsymbol{\theta}^{(0)}$
2. while(*not converged*)
   - Update $\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \gamma \mathbf{d}^{(t)}$, where $\mathbf{d}^{(t)} = \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$
   - Update $t := t + 1$

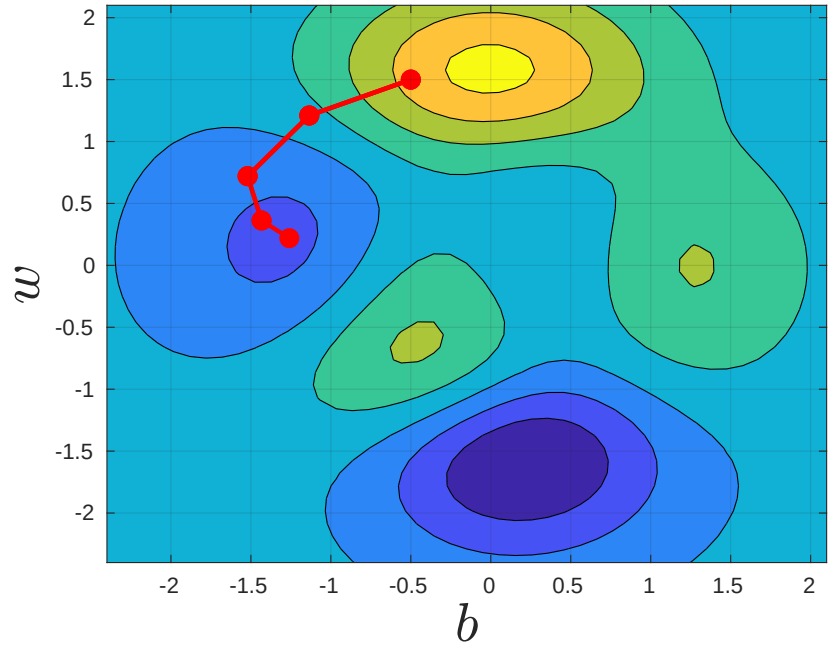
Iterative solution (gradient descent) - Example 2D



$$\boldsymbol{\theta} = [b,\ w]^{\mathsf{T}} \in \mathbb{R}^2$$

1. Pick a $\boldsymbol{\theta}^{(0)}$
2. while(*not converged*)
   - Update $\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \gamma \mathbf{d}^{(t)}$, where $\mathbf{d}^{(t)} = \boldsymbol{\nabla}_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$
   - Update $t := t + 1$

Iterative solution (gradient descent) - Example 2D



$$\boldsymbol{\theta} = [b,\ w]^{\mathsf{T}} \in \mathbb{R}^2$$

1. Pick a $\boldsymbol{\theta}^{(0)}$
2. while(*not converged*)
   - Update $\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \gamma \mathbf{d}^{(t)}$, where $\mathbf{d}^{(t)} = \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$
   - Update $t := t + 1$

We call $\gamma \in \mathbb{R}$ the **step length** or **learning rate**.

Computational challenge 1 - dim(θ) is big

At each optimization step we need to compute the gradient

$$\mathbf{d}^{(t)} = \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}^{(t)}) = \frac{1}{n} \sum_{i=1}^{n} \nabla_{\boldsymbol{\theta}} L(\mathbf{x}_i, \mathbf{y}_i, \boldsymbol{\theta}^{(t)}).$$

Computational challenge 1 - dim(θ) big: A neural network contains a lot of parameters. Computing the gradient is costly.

Solution: A NN is a composition of multiple layers. Hence, each term $\nabla_{\boldsymbol{\theta}} L(\mathbf{x}_i, \mathbf{y}_i, \boldsymbol{\theta})$ can be computed efficiently by repeatedly applying the chain rule. This is called the **back-propagation algorithm**. Not part of the course.

Computational challenge 2 - n is big

At each optimization step we need to compute the gradient

$$\mathbf{d}^{(t)} = \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}^{(t)}) = \frac{1}{n} \sum_{i=1}^{n} \nabla_{\boldsymbol{\theta}} L(\mathbf{x}_i, \mathbf{y}_i, \boldsymbol{\theta}^{(t)}).$$

Computational challenge 2 - n big: We typically use a lot of training data n for training the neural netowork. Computing the gradient is costly.

Solution: For each iteration, we only use a small part of the data set to compute the gradient $\mathbf{d}^{(t)}$. This is called the **stochastic gradient descent**.

Stochastic gradient descent

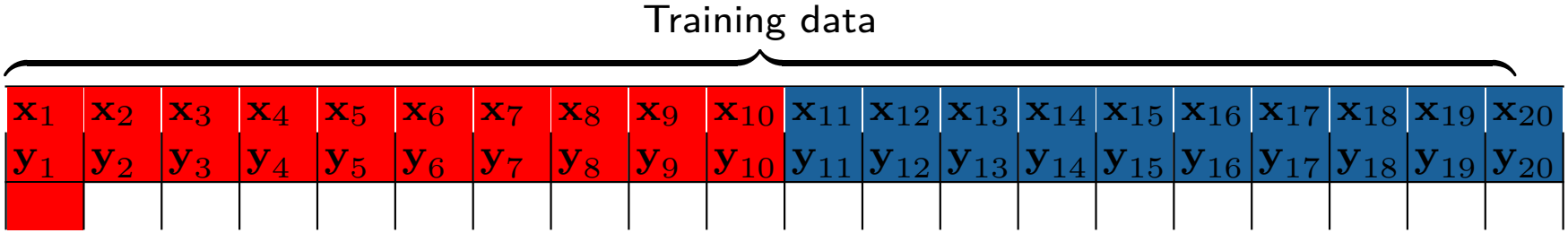
A big data set is often redundant = many data points are similar.

Training data

$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_4$	$\mathbf{x}_5$	$\mathbf{x}_6$	$\mathbf{x}_7$	$\mathbf{x}_8$	$\mathbf{x}_9$	$\mathbf{x}_{10}$	$\mathbf{x}_{11}$	$\mathbf{x}_{12}$	$\mathbf{x}_{13}$	$\mathbf{x}_{14}$	$\mathbf{x}_{15}$	$\mathbf{x}_{16}$	$\mathbf{x}_{17}$	$\mathbf{x}_{18}$	$\mathbf{x}_{19}$	$\mathbf{x}_{20}$
$\mathbf{y}_1$	$\mathbf{y}_2$	$\mathbf{y}_3$	$\mathbf{y}_4$	$\mathbf{y}_5$	$\mathbf{y}_6$	$\mathbf{y}_7$	$\mathbf{y}_8$	$\mathbf{y}_9$	$\mathbf{y}_{10}$	$\mathbf{y}_{11}$	$\mathbf{y}_{12}$	$\mathbf{y}_{13}$	$\mathbf{y}_{14}$	$\mathbf{y}_{15}$	$\mathbf{y}_{16}$	$\mathbf{y}_{17}$	$\mathbf{y}_{18}$	$\mathbf{y}_{19}$	$\mathbf{y}_{20}$

Stochastic gradient descent

A big data set is often redundant = many data points are similar.



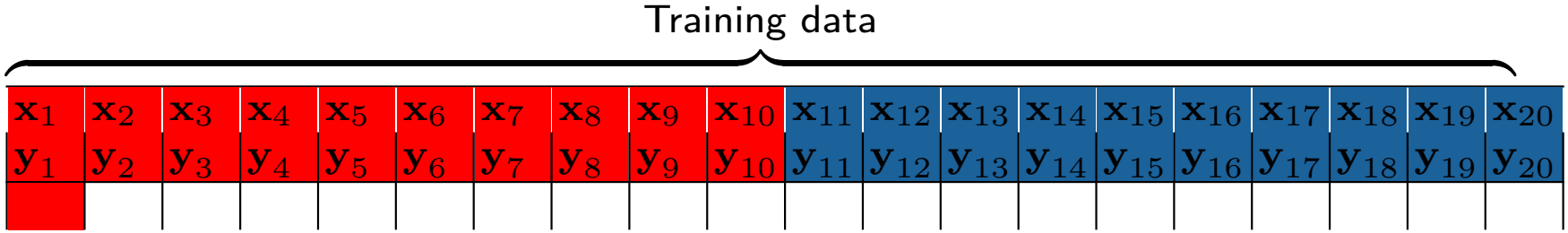
If the training data is big

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \approx \sum_{i=1}^{\frac{n}{2}} \nabla_{\boldsymbol{\theta}} L(\mathbf{x}_i, \mathbf{y}_i, \boldsymbol{\theta}) \qquad \text{and}$$

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \approx \sum_{i=\frac{n}{2}+1}^{n} \nabla_{\boldsymbol{\theta}} L(\mathbf{x}_i, \mathbf{y}_i, \boldsymbol{\theta}).$$

Stochastic gradient descent

A big data set is often redundant = many data points are similar.



If the training data is big

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \approx \sum_{i=1}^{\frac{n}{2}} \nabla_{\boldsymbol{\theta}} L(\mathbf{x}_i, \mathbf{y}_i, \boldsymbol{\theta}) \qquad \text{and}$$

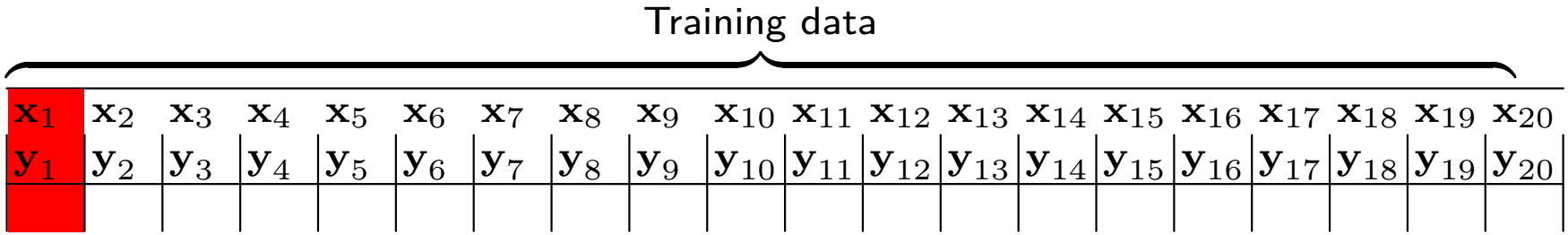
$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \approx \sum_{i=\frac{n}{2}+1}^{n} \nabla_{\boldsymbol{\theta}} L(\mathbf{x}_i, \mathbf{y}_i, \boldsymbol{\theta}).$$

We can do the update with only half the computation cost!

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \gamma \frac{1}{n/2} \sum_{i=1}^{\frac{n}{2}} \nabla_{\boldsymbol{\theta}} L(\mathbf{x}_i, \mathbf{y}_i, \boldsymbol{\theta}^{(t)}),$$

$$\boldsymbol{\theta}^{(t+2)} = \boldsymbol{\theta}^{(t+1)} - \gamma \frac{1}{n/2} \sum_{i=\frac{n}{2}+1}^{n} \nabla_{\boldsymbol{\theta}} L(\mathbf{x}_i, \mathbf{y}_i, \boldsymbol{\theta}^{(t+1)}).$$

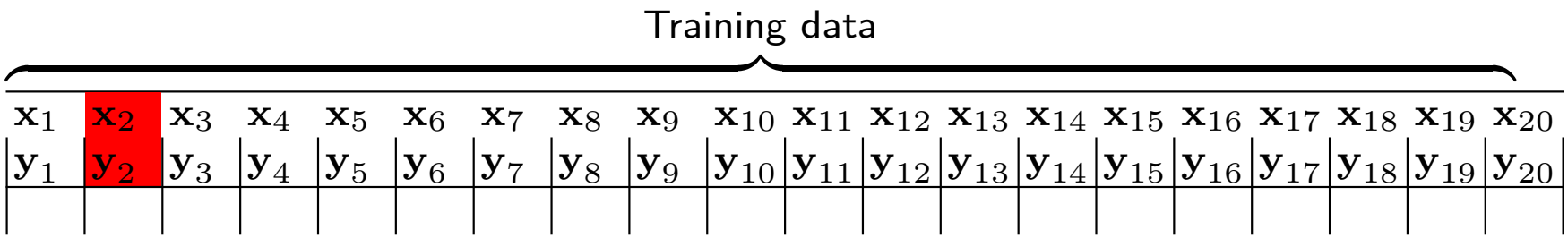
Stochastic gradient descent



$$\boldsymbol{\theta}^{(1)} = \boldsymbol{\theta}^{(0)} - \gamma \boldsymbol{\nabla}_{\boldsymbol{\theta}} L(\mathbf{x}_1, \mathbf{y}_1, \boldsymbol{\theta}^{(0)})$$

- The extreme version of this strategy is to use only one data point at each training step (called **online learning**)

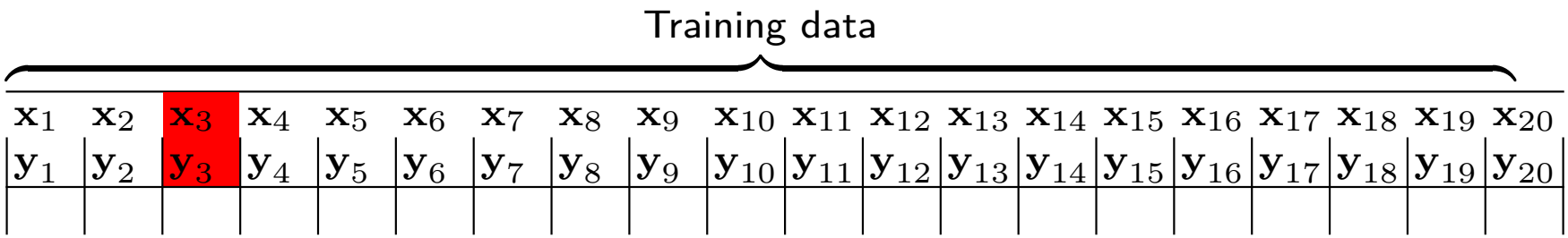
Stochastic gradient descent



$$\boldsymbol{\theta}^{(2)} = \boldsymbol{\theta}^{(1)} - \gamma \boldsymbol{\nabla}_{\boldsymbol{\theta}} L(\mathbf{x}_2, \mathbf{y}_2, \boldsymbol{\theta}^{(1)})$$

- The extreme version of this strategy is to use only one data point at each training step (called **online learning**)

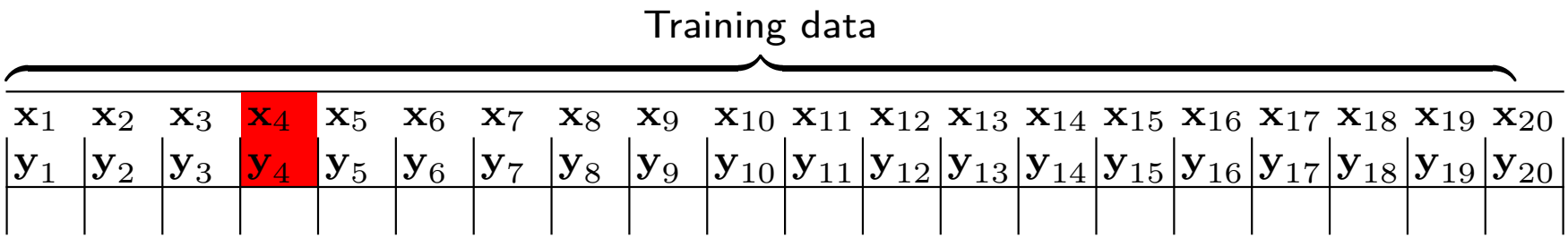
Stochastic gradient descent



$$\boldsymbol{\theta}^{(3)} = \boldsymbol{\theta}^{(2)} - \gamma \boldsymbol{\nabla}_{\boldsymbol{\theta}} L(\mathbf{x}_3, \mathbf{y}_3, \boldsymbol{\theta}^{(2)})$$

- The extreme version of this strategy is to use only one data point at each training step (called **online learning**)

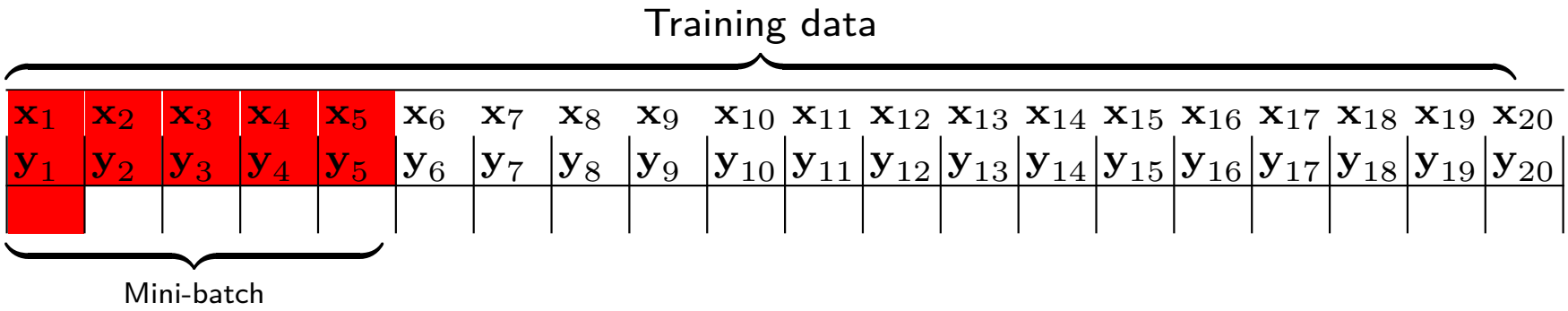
Stochastic gradient descent



$$\boldsymbol{\theta}^{(4)} = \boldsymbol{\theta}^{(3)} - \gamma \boldsymbol{\nabla}_{\boldsymbol{\theta}} L(\mathbf{x}_4, \mathbf{y}_4, \boldsymbol{\theta}^{(3)})$$

- The extreme version of this strategy is to use only one data point at each training step (called **online learning**)

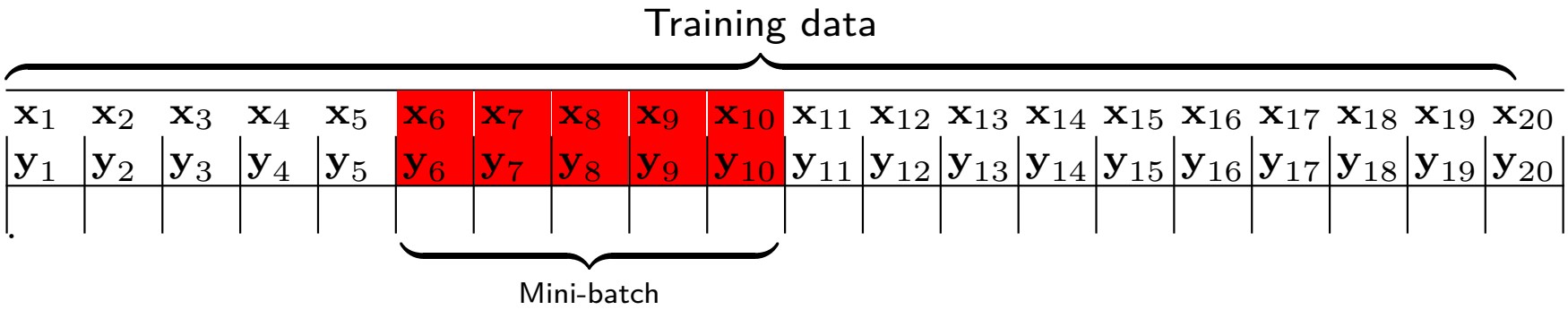
Stochastic gradient descent



$$\boldsymbol{\theta}^{(1)} = \boldsymbol{\theta}^{(0)} - \gamma \frac{1}{5} \sum_{i=1}^{5} \nabla_{\boldsymbol{\theta}} L(\mathbf{x}_i, \mathbf{y}_i, \boldsymbol{\theta}^{(0)})$$

- The extreme version of this strategy is to use only one data point at each training step (called **online learning**)
- We typically do something in between (not one data point, and not all data). We use a smaller set called **mini-batch**.

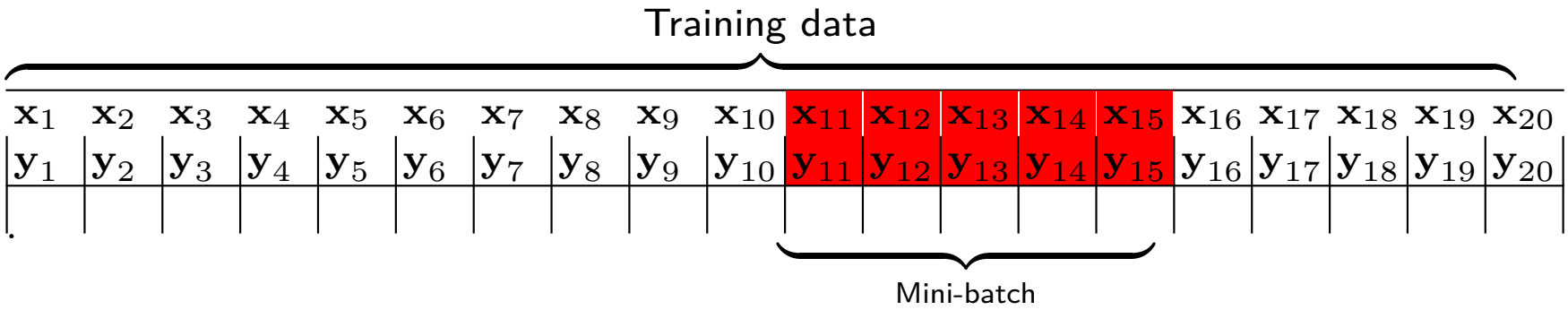
Stochastic gradient descent



$$\boldsymbol{\theta}^{(2)} = \boldsymbol{\theta}^{(1)} - \gamma \frac{1}{5} \sum_{i=6}^{10} \nabla_{\boldsymbol{\theta}} L(\mathbf{x}_i, \mathbf{y}_i, \boldsymbol{\theta}^{(1)})$$

- The extreme version of this strategy is to use only one data point at each training step (called **online learning**)
- We typically do something in between (not one data point, and not all data). We use a smaller set called **mini-batch**.

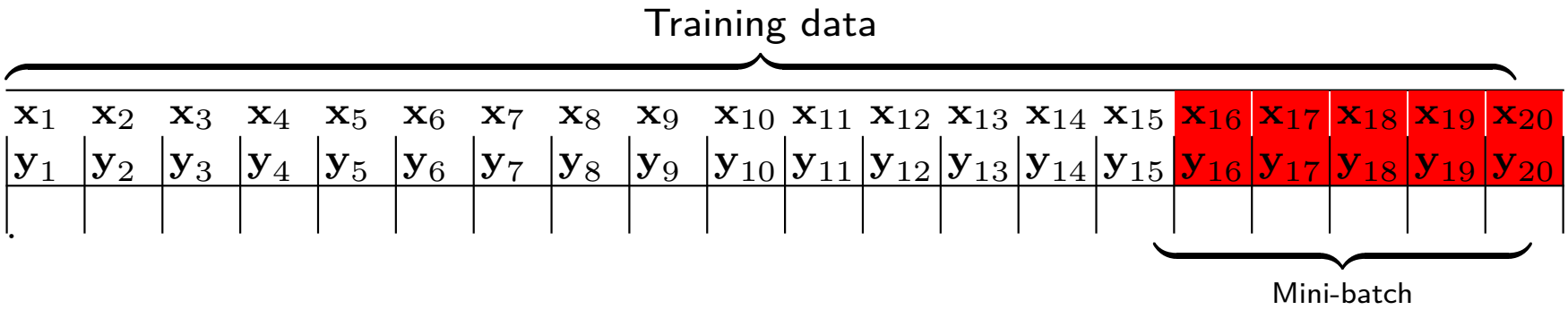
Stochastic gradient descent



$$\boldsymbol{\theta}^{(3)} = \boldsymbol{\theta}^{(2)} - \gamma \frac{1}{5} \sum_{i=11}^{15} \nabla_{\boldsymbol{\theta}} L(\mathbf{x}_i, \mathbf{y}_i, \boldsymbol{\theta}^{(2)})$$

- The extreme version of this strategy is to use only one data point at each training step (called **online learning**)
- We typically do something in between (not one data point, and not all data). We use a smaller set called **mini-batch**.

Stochastic gradient descent



$$\boldsymbol{\theta}^{(4)} = \boldsymbol{\theta}^{(3)} - \gamma \frac{1}{5} \sum_{i=16}^{20} \nabla_{\boldsymbol{\theta}} L(\mathbf{x}_i, \mathbf{y}_i, \boldsymbol{\theta}^{(3)})$$

- The extreme version of this strategy is to use only one data point at each training step (called **online learning**)
- We typically do something in between (not one data point, and not all data). We use a smaller set called **mini-batch**.
- One pass through the training data is called an **epoch**.

Stochastic gradient descent

Training data

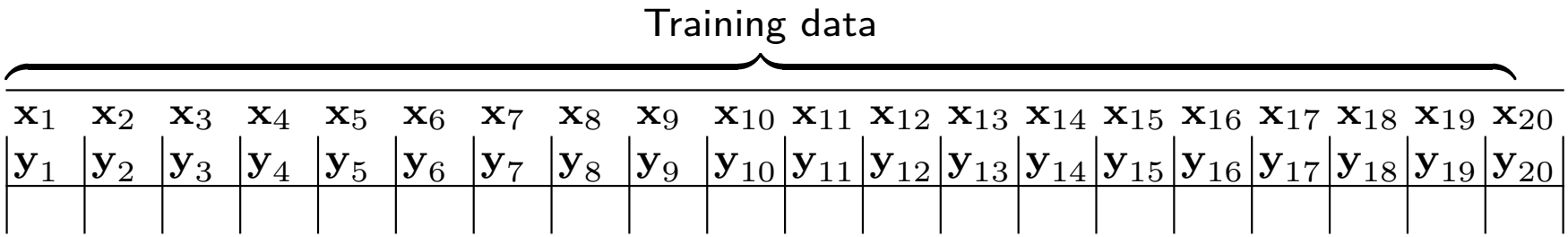
$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_4$	$\mathbf{x}_5$	$\mathbf{x}_6$	$\mathbf{x}_7$	$\mathbf{x}_8$	$\mathbf{x}_9$	$\mathbf{x}_{10}$	$\mathbf{x}_{11}$	$\mathbf{x}_{12}$	$\mathbf{x}_{13}$	$\mathbf{x}_{14}$	$\mathbf{x}_{15}$	$\mathbf{x}_{16}$	$\mathbf{x}_{17}$	$\mathbf{x}_{18}$	$\mathbf{x}_{19}$	$\mathbf{x}_{20}$
$\mathbf{y}_1$	$\mathbf{y}_2$	$\mathbf{y}_3$	$\mathbf{y}_4$	$\mathbf{y}_5$	$\mathbf{y}_6$	$\mathbf{y}_7$	$\mathbf{y}_8$	$\mathbf{y}_9$	$\mathbf{y}_{10}$	$\mathbf{y}_{11}$	$\mathbf{y}_{12}$	$\mathbf{y}_{13}$	$\mathbf{y}_{14}$	$\mathbf{y}_{15}$	$\mathbf{y}_{16}$	$\mathbf{y}_{17}$	$\mathbf{y}_{18}$	$\mathbf{y}_{19}$	$\mathbf{y}_{20}$

Iteration:

Epoch:

- If we pick the mini-batches in order, they might be unbalanced and not representative for the whole data set.

Stochastic gradient descent

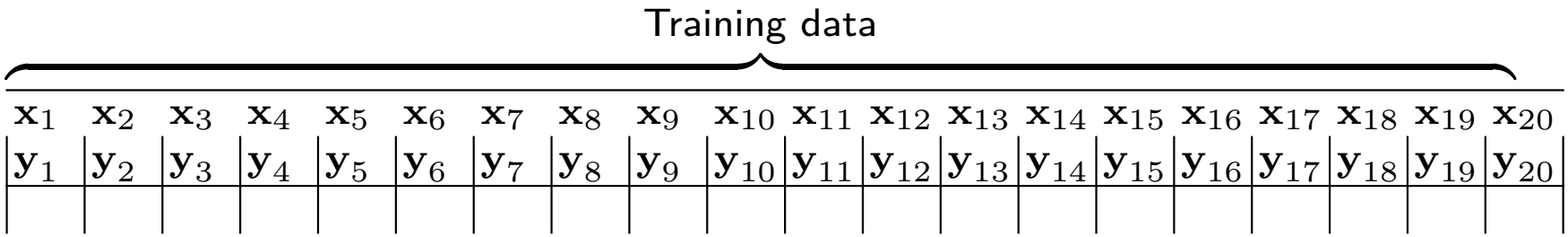


Iteration:

Epoch:

- If we pick the mini-batches in order, they might be unbalanced and not representative for the whole data set.
- Therefore, we pick data points **at random** from the training data to form a mini-batch.

Stochastic gradient descent

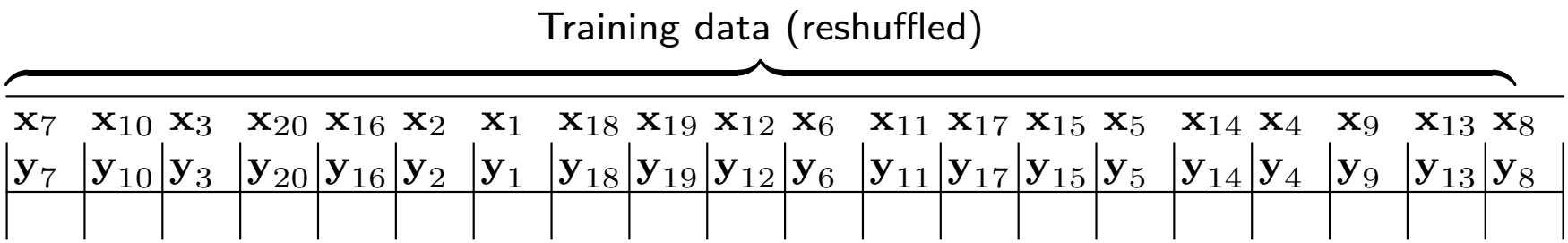


Iteration:

Epoch:

- If we pick the mini-batches in order, they might be unbalanced and not representative for the whole data set.
- Therefore, we pick data points **at random** from the training data to form a mini-batch.
- One implementation is to randomly reshuffle the data before dividing it into mini-batches.

Stochastic gradient descent

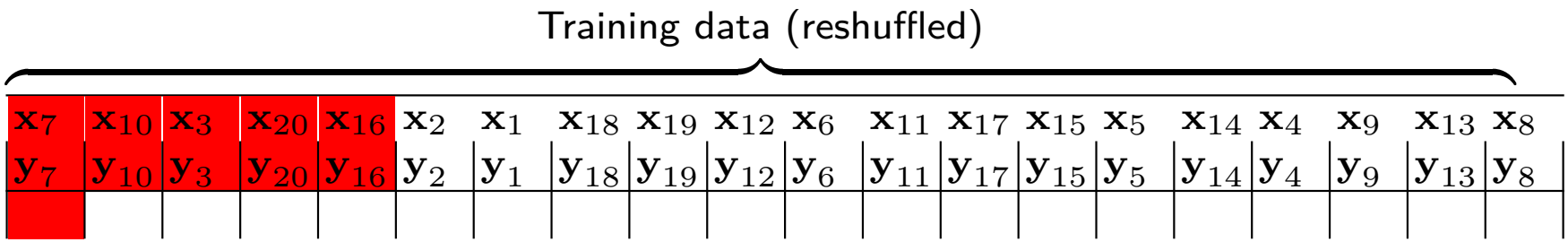


Iteration:

Epoch:

- If we pick the mini-batches in order, they might be unbalanced and not representative for the whole data set.
- Therefore, we pick data points **at random** from the training data to form a mini-batch.
- One implementation is to randomly reshuffle the data before dividing it into mini-batches.

Stochastic gradient descent

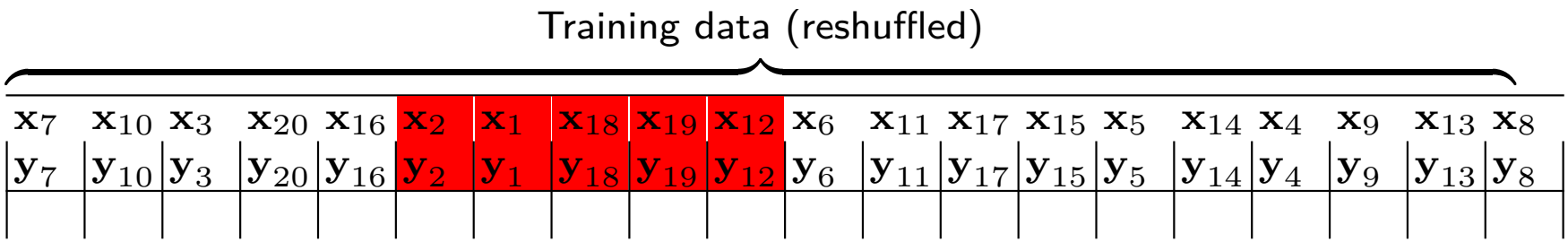


Iteration: 1

Epoch: 1

- If we pick the mini-batches in order, they might be unbalanced and not representative for the whole data set.
- Therefore, we pick data points **at random** from the training data to form a mini-batch.
- One implementation is to randomly reshuffle the data before dividing it into mini-batches.

Stochastic gradient descent

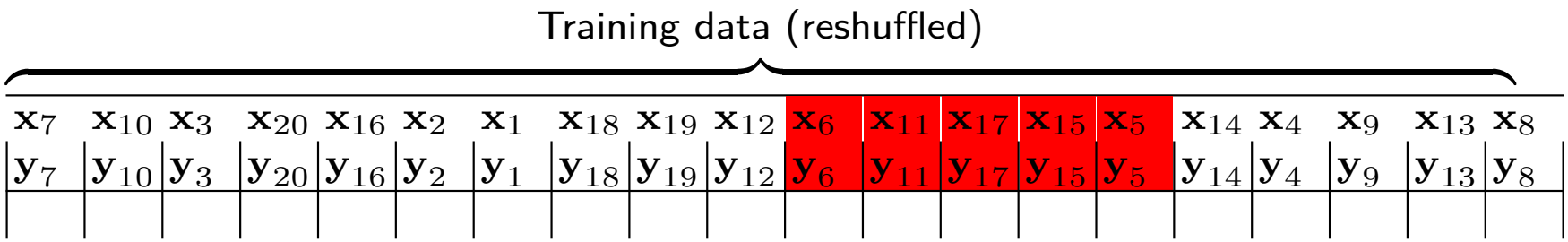


Iteration: 2

Epoch: 1

- If we pick the mini-batches in order, they might be unbalanced and not representative for the whole data set.
- Therefore, we pick data points **at random** from the training data to form a mini-batch.
- One implementation is to randomly reshuffle the data before dividing it into mini-batches.

Stochastic gradient descent

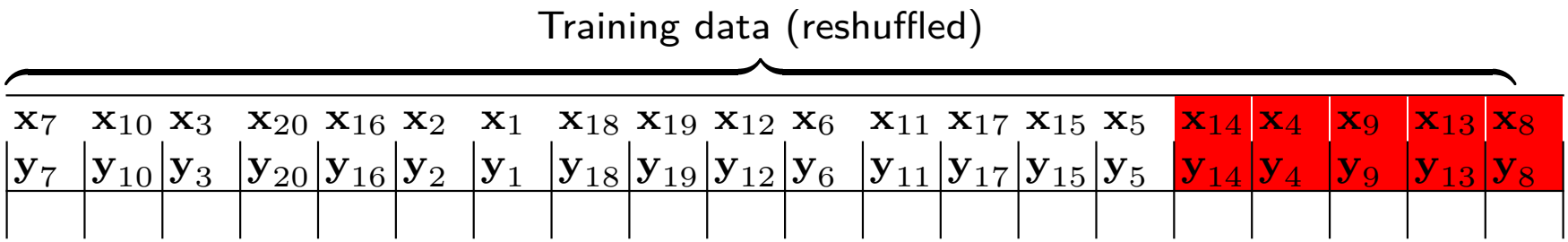


Iteration: 3

Epoch: 1

- If we pick the mini-batches in order, they might be unbalanced and not representative for the whole data set.
- Therefore, we pick data points **at random** from the training data to form a mini-batch.
- One implementation is to randomly reshuffle the data before dividing it into mini-batches.

Stochastic gradient descent

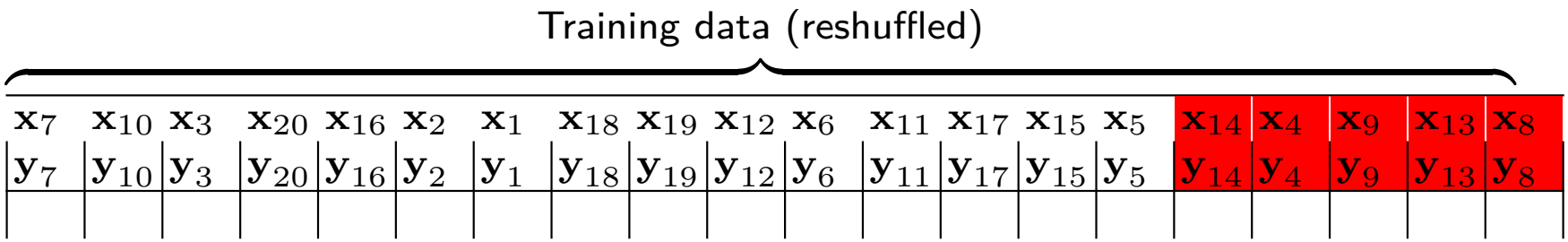


Iteration: 4

Epoch: 1

- If we pick the mini-batches in order, they might be unbalanced and not representative for the whole data set.
- Therefore, we pick data points **at random** from the training data to form a mini-batch.
- One implementation is to randomly reshuffle the data before dividing it into mini-batches.

Stochastic gradient descent



Iteration: 4

Epoch: 1

- If we pick the mini-batches in order, they might be unbalanced and not representative for the whole data set.
- Therefore, we pick data points **at random** from the training data to form a mini-batch.
- One implementation is to randomly reshuffle the data before dividing it into mini-batches.

Stochastic gradient descent

Training data (reshuffled)

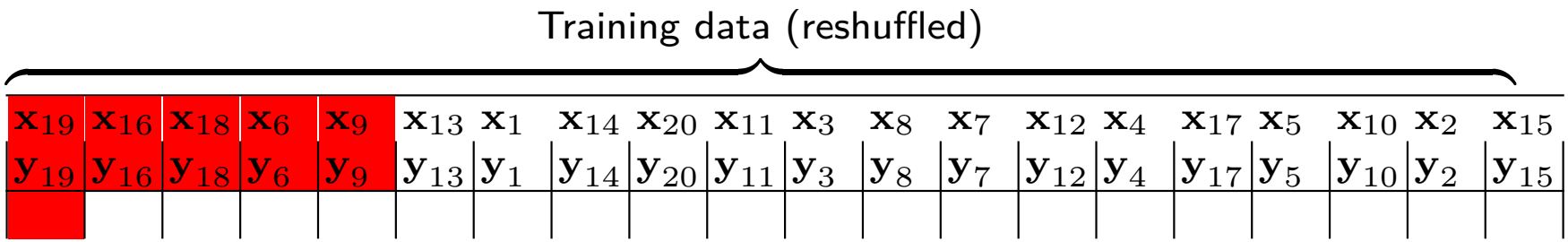
$\mathbf{x}_{19}$	$\mathbf{x}_{16}$	$\mathbf{x}_{18}$	$\mathbf{x}_{6}$	$\mathbf{x}_{9}$	$\mathbf{x}_{13}$	$\mathbf{x}_{1}$	$\mathbf{x}_{14}$	$\mathbf{x}_{20}$	$\mathbf{x}_{11}$	$\mathbf{x}_{3}$	$\mathbf{x}_{8}$	$\mathbf{x}_{7}$	$\mathbf{x}_{12}$	$\mathbf{x}_{4}$	$\mathbf{x}_{17}$	$\mathbf{x}_{5}$	$\mathbf{x}_{10}$	$\mathbf{x}_{2}$	$\mathbf{x}_{15}$
$\mathbf{y}_{19}$	$\mathbf{y}_{16}$	$\mathbf{y}_{18}$	$\mathbf{y}_{6}$	$\mathbf{y}_{9}$	$\mathbf{y}_{13}$	$\mathbf{y}_{1}$	$\mathbf{y}_{14}$	$\mathbf{y}_{20}$	$\mathbf{y}_{11}$	$\mathbf{y}_{3}$	$\mathbf{y}_{8}$	$\mathbf{y}_{7}$	$\mathbf{y}_{12}$	$\mathbf{y}_{4}$	$\mathbf{y}_{17}$	$\mathbf{y}_{5}$	$\mathbf{y}_{10}$	$\mathbf{y}_{2}$	$\mathbf{y}_{15}$

Iteration:

Epoch: 2

- If we pick the mini-batches in order, they might be unbalanced and not representative for the whole data set.
- Therefore, we pick data points **at random** from the training data to form a mini-batch.
- One implementation is to randomly reshuffle the data before dividing it into mini-batches.
- After each epoch we do another reshuffling and another pass through the data set.

Stochastic gradient descent

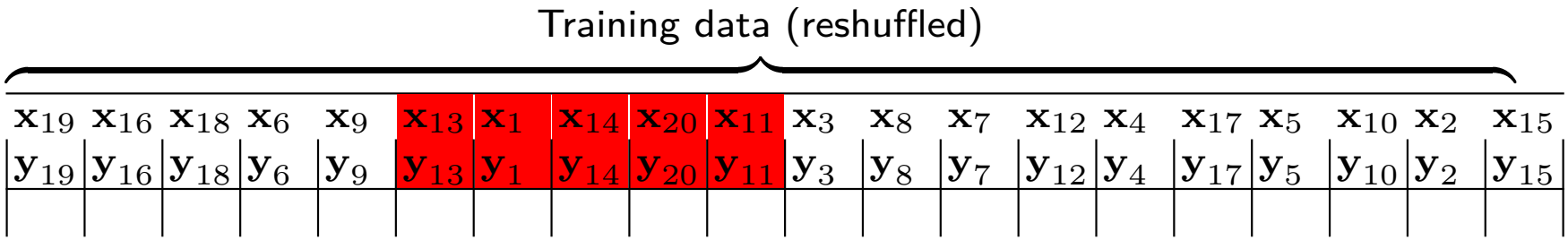


Iteration: 5

Epoch: 2

- If we pick the mini-batches in order, they might be unbalanced and not representative for the whole data set.
- Therefore, we pick data points **at random** from the training data to form a mini-batch.
- One implementation is to randomly reshuffle the data before dividing it into mini-batches.
- After each epoch we do another reshuffling and another pass through the data set.

Stochastic gradient descent

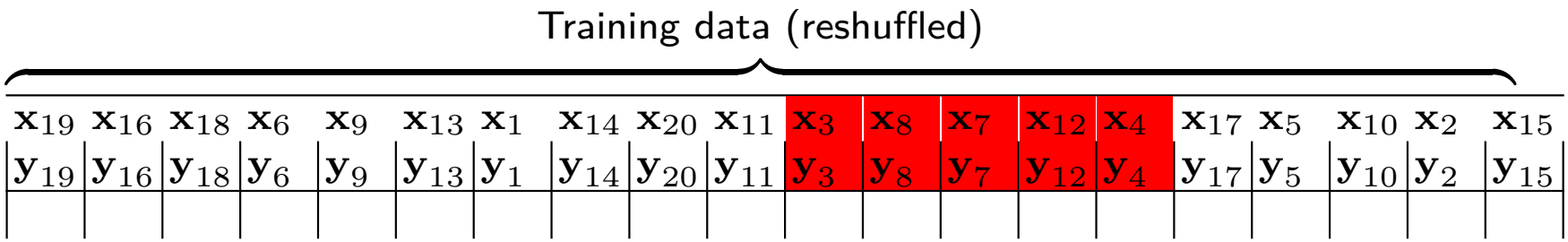


Iteration: 6

Epoch: 2

- If we pick the mini-batches in order, they might be unbalanced and not representative for the whole data set.
- Therefore, we pick data points **at random** from the training data to form a mini-batch.
- One implementation is to randomly reshuffle the data before dividing it into mini-batches.
- After each epoch we do another reshuffling and another pass through the data set.

Stochastic gradient descent

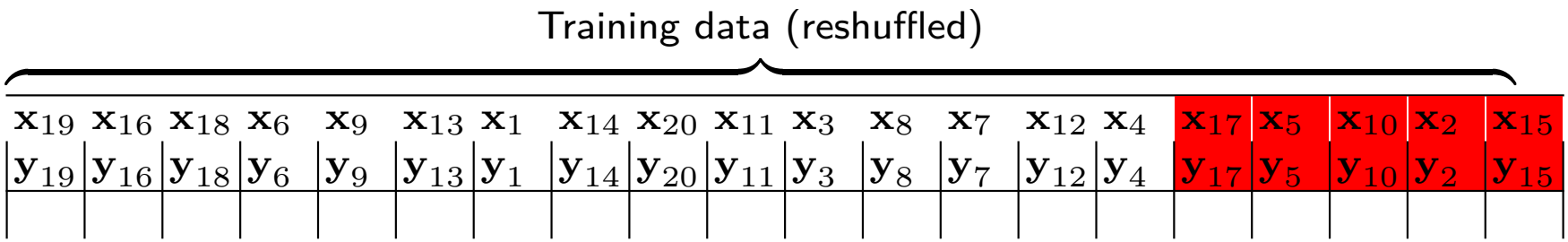


Iteration: 7

Epoch: 2

- If we pick the mini-batches in order, they might be unbalanced and not representative for the whole data set.
- Therefore, we pick data points **at random** from the training data to form a mini-batch.
- One implementation is to randomly reshuffle the data before dividing it into mini-batches.
- After each epoch we do another reshuffling and another pass through the data set.

Stochastic gradient descent



Iteration: 8

Epoch: 2

- If we pick the mini-batches in order, they might be unbalanced and not representative for the whole data set.
- Therefore, we pick data points **at random** from the training data to form a mini-batch.
- One implementation is to randomly reshuffle the data before dividing it into mini-batches.
- After each epoch we do another reshuffling and another pass through the data set.

Mini-batch gradient descent

The full **stochastic gradient descent** algorithm (a.k.a **mini-batch gradient descent**) is as follows

1. Initialize $\boldsymbol{\theta}^{(0)}$, set $t \leftarrow 1$, choose batch size n_b and number of epochs E.
2. For $i = 1$ to E
   (a) Randomly shuffle the training data $\{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^{n}$.
   (b) For $j = 1$ to $\frac{n}{n_b}$
      (i) Approximate the gradient of the loss function using the mini-batch $\{\mathbf{x}_i, \mathbf{y}_i\}_{i=(j-1)n_b+1}^{jn_b}$,
      $$\hat{\mathbf{d}}^{(t)} = \frac{1}{n_b} \sum_{i=(j-1)n_b+1}^{jn_b} \nabla_{\boldsymbol{\theta}} L(\mathbf{x}_i, \mathbf{y}_i, \boldsymbol{\theta}) \Big|_{\boldsymbol{\theta}=\boldsymbol{\theta}^{(t)}}.$$
      (ii) Do a gradient step $\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \gamma \hat{\mathbf{d}}^{(t)}$.
      (iii) Update the iteration index $t \leftarrow t + 1$.

At each time we get a stochastic approximation of the true gradient $\hat{\mathbf{d}}^{(t)} \approx \frac{1}{n} \sum_{i=1}^{n} \nabla_{\boldsymbol{\theta}} L(\mathbf{x}_i, \mathbf{y}_i, \boldsymbol{\theta}) \Big|_{\boldsymbol{\theta}=\boldsymbol{\theta}^{(t)}}$, hence the name.

Summary

1. **Previous lecture** The neural network model
   - Neural network for regression
   - Neural network for classification



UPPSALA
UNIVERSITET

Summary

1. **Previous lecture** The neural network model
   - Neural network for regression
   - Neural network for classification
2. **This lecture**
   - Convolutional neural network
   - How to train a neural network

A few concepts to summarize lecture 9

Convolutional neural network (CNN): A NN with a particular structure tailored for input data with a grid-like structure, like for example images.

Filter: (a.k.a kernel) A set of parameters that is convolved with a hidden layer. Each filter produces a new channel.

Channel: A set of hidden units produced by the same filter. Each hidden layer consists of one or more channels.

Stride: A positive integer deciding how many steps to move the filter during the convolution.

Tensor: A generalization of matrices to arbitrary order.

Gradient descent: An iterative optimization algorithm where we at iteration take a step proportional to the negative gradient.

Learning rate: (a.k.a step length). A scalar tuning parameter deciding the length of each gradient step in gradient descent.

Stochastic gradient descent (SGD): A version of gradient descent where we at each iteration only use a small part of the training data (a mini-batch).

Mini-batch: The group of training data that we use at each iteration in SG

Batch size: The number of data points in one mini-batch